

This file is a merged representation of a subset of the codebase, containing files not matching ignore patterns, combined into a single document by Repomix. The content has been processed where content has been compressed (code blocks are separated by `␣—` delimiter).

File Summary

Purpose

This file contains a packed representation of a subset of the repository's contents that is considered the most important context. It is designed to be easily consumable by AI systems for analysis, code review, or other automated processes.

File Format

The content is organized as follows: 1. This summary section 2. Repository information 3. Directory structure 4. Repository files (if enabled) 5. Multiple file entries, each consisting of: a. A header with the file path (`## File: path/to/file`) b. The full contents of the file in a code block

Usage Guidelines

- This file should be treated as read-only. Any changes should be made to the original repository files, not this packed version.
- When processing this file, use the file path to distinguish between different files in the repository.
- Be aware that this file may contain sensitive information. Handle it with the same level of security as you would the original repository.

Notes

- Some files may have been excluded based on .gitignore rules and Repomix's configuration
- Binary files are not included in this packed representation. Please refer to the Repository Structure section for a complete list of file paths, including binary files
- Files matching these patterns are excluded: `vendor/`, `parsers/`, `**/*.svg`
- Files matching patterns in .gitignore are excluded
- Files matching default ignore patterns are excluded
- Content has been compressed - code blocks are separated by `␣—` delimiter
- Files are sorted by Git change count (files with more changes are at the bottom)

Directory Structure

```
.iide/  
  lsp.json  
data/  
  icons/  
    meson.build  
  Adwaita-dark.xml  
  Adwaita.xml  
  meson.build  
  org.github.kai66673.iide.desktop.in  
  org.github.kai66673.iide.gschema.xml  
  org.github.kai66673.iide.metainfo.xml.in  
  org.github.kai66673.iide.service.in  
po/  
  LINGUAS  
  meson.build  
  POTFILES.in  
scripts/  
  gen-docs.sh  
  install-local-settings.sh.in  
src/  
  fonts/  
    SymbolsNerdFont-Regular.ttf  
    SymbolsNerdFontMono-Regular.ttf  
  Services/  
    Actions/  
      AppAction.vala  
      AppActionsManager.vala  
    LSP/  
      config/  
        CppLspConfig.vala  
        LspConfig.vala  
        LspRegistry.vala  
        PythonLspConfig.vala  
      index/  
        ClangTidy.vala  
      DiagnosticsService.vala  
      LspClient.vala  
      LspDiagnosticsMark.vala  
      LspDocumentClient.vala  
      LspService.vala  
      LspTypes.vala  
    TreeSitter/  
      cpp/
```

```

    CppHighlighter.vala
model/
    AstItem.vala
    AstModel.vala
python/
    highlights.scm
    PythonHighlighter.vala
    PythonIndenter.vala
rust/
    RustHighlighter.vala
vala/
    ValaHighlighter.vala
BaseTreeSitterHighlighter.vala
IndentBlock.vala
TreeSitterDocument.vala
TreeSitterManager.vala
DocumentManager.vala
ImageFactory.vala
JsonUtils.vala
LoggerService.vala
NavigationHistoryService.vala
PanelLayoutHelper.vala
PendingChange.vala
ProjectManager.vala
SettingsService.vala
SourceDocument.vala
SourceNodeItem.vala
StyleManager.vala
SymbIconProvider.vala
Utils.vala
Widgets/
Find/
    Engines/
        FzfSearchEngine.vala
        SymbolsSearchEngine.vala
        TextSearchEngine.vala
    SearchEngine.vala
    SearchPage.vala
    SearchResult.vala
    SearchResultsView.vala
    SearchWindow.vala
Panels/
    BasePanel.vala
    DiagnosticsPanel.vala
    LogPanel.vala
    ProjectPanel.vala

```

```

    TerminalPanel.vala
TextView/
    BreadcrumbFileNavigator.vala
    BreadcrumbsBar.vala
    BreadcrumbSymbolOutlineNavigator.vala
    BreadcrumbTreeSitterNavigator.vala
    DiagnosticsPopover.vala
    EditorStatusBar.vala
    FontZoomer.vala
    GutterMarkRenderer.vala
    LineNumbersGutter.vala
    LspCompletionProvider.vala
    SourceView.vala
    TextView.vala
    TreeSitterFoldingGutter.vala
ToolViews/
    LogView.vala
    ProjectView.vala
    TerminalView.vala
    PreferencesDialog.vala
application.vala
config.vapi
iide.gresource.xml
main.vala
meson.build
shortcuts-dialog.ui
style.css
window.ui
window.vala
symbols/
    fonts/
        SYMBOLIC.ttf
    export_glyph.py
    icons.json
    meson.build
    symbols.css
test_temp/
    test.c
vapi/
    libtreesitter.vapi
.gitignore
.gitmodules
COPYING
Doxyfile
meson.build
org.github.kai66673.iide.json

```

README.md
repomix-output.pdf

Files

File: .iide/lsp.json

```
{"languages": [{"id": "cpp", "server": ["clangd", "--header-insertion=never"], "patterns":
```

File: data/icons/meson.build

```
application_id = 'org.github.kai66673.iide'

scalable_dir = 'hicolor' / 'scalable' / 'apps'
install_data(
    scalable_dir / ('@00.svg').format(application_id),
    install_dir: get_option('datadir') / 'icons' / scalable_dir
)

symbolic_dir = 'hicolor' / 'symbolic' / 'apps'
install_data(
    symbolic_dir / ('@00-symbolic.svg').format(application_id),
    install_dir: get_option('datadir') / 'icons' / symbolic_dir
)
```

File: data/meson.build

```
desktop_file = i18n.merge_file(
    input: 'org.github.kai66673.iide.desktop.in',
    output: 'org.github.kai66673.iide.desktop',
    type: 'desktop',
    po_dir: '../po',
    install: true,
    install_dir: get_option('datadir') / 'applications'
)

desktop_utils = find_program('desktop-file-validate', required: false)
if desktop_utils.found()
    test('Validate desktop file', desktop_utils, args: [desktop_file])
endif

appstream_file = i18n.merge_file(
    input: 'org.github.kai66673.iide.metainfo.xml.in',
    output: 'org.github.kai66673.iide.metainfo.xml',
```

```

        po_dir: '../po',
        install: true,
        install_dir: get_option('datadir') / 'metainfo'
    )

    appstreamcli = find_program('appstreamcli', required: false, disabler: true)
    test('Validate appstream file', appstreamcli,
        args: ['validate', '--no-net', '--explain', appstream_file])

    install_data('org.github.kai66673.iide.gschema.xml',
        install_dir: get_option('datadir') / 'glib-2.0' / 'schemas'
    )

    compile_schemas = find_program('glib-compile-schemas', required: false, disabler: true)
    test('Validate schema file',
        compile_schemas,
        args: ['--strict', '--dry-run', meson.current_source_dir()])

    service_conf = configuration_data()
    service_conf.set('bindir', get_option('prefix') / get_option('bindir'))
    configure_file(
        input: 'org.github.kai66673.iide.service.in',
        output: 'org.github.kai66673.iide.service',
        configuration: service_conf,
        install_dir: get_option('datadir') / 'dbus-1' / 'services'
    )

    subdir('icons')

```

File: data/org.github.kai66673.iide.desktop.in

```

[Desktop Entry]
Name=iide
Exec=iide
Icon=org.github.kai66673.iide
Terminal=false
Type=Application
Categories=Utility;
Keywords=GTK;
StartupNotify=true
DBusActivatable=true

```

File: data/org.github.kai66673.iide.metainfo.xml.in

```
<?xml version="1.0" encoding="UTF-8"?>
<component type="desktop-application">
  <id>org.github.kai66673.iide</id>
  <metadata_license>CC0-1.0</metadata_license>
  <project_license>GPL-3.0-or-later</project_license>

  <name>Iide</name>
  <summary>Keep the summary shorter, between 10 and 35 characters</summary>
  <description>
    <p>No description</p>
  </description>

  <developer id="tld.vendor">
    <name>Developer name</name>
  </developer>

  <!-- Required: Should be a link to the upstream homepage for the component -->
  <url type="homepage">https://example.org</url>
  <!-- Recommended: It is highly recommended for open-source projects to display the source -->
  <url type="vcs-browser">https://example.org/repository</url>
  <!-- Should point to the software's bug tracking system, for users to report new bugs -->
  <url type="bugtracker">https://example.org/issues</url>
  <!-- Should link a FAQ page for this software, to answer some of the most-asked questions -->
  <!-- URLs of this type should point to a webpage where users can submit or modify translations -->
  <url type="translate">https://example.org/translate</url>
  <url type="faq">https://example.org/faq</url>
  <!-- Should provide a web link to an online user's reference, a software manual or help page -->
  <url type="help">https://example.org/help</url>
  <!-- URLs of this type should point to a webpage showing information on how to donate to the project -->
  <url type="donation">https://example.org/donate</url>
  <!-- This could for example be an HTTPS URL to an online form or a page describing how to contact the developers -->
  <url type="contact">https://example.org/contact</url>
  <!-- URLs of this type should point to a webpage showing information on how to contribute to the project -->
  <url type="contribute">https://example.org/contribute</url>

  <translation type="gettext">iide</translation>
  <!-- All graphical applications having a desktop file must have this tag in the MetaInfo.
       If this is present, appstreamcli compose will pull icons, keywords and categories from the desktop file -->
  <launchable type="desktop-id">org.github.kai66673.iide.desktop</launchable>
  <!-- Use the OARS website (https://hughsie.github.io/oars/generate.html) to generate these tags -->
  <content_rating type="oars-1.1" />

  <!-- Applications should set a brand color in both light and dark variants like so -->
  <branding>
```

```

        <color type="primary" scheme_preference="light">#ff00ff</color>
        <color type="primary" scheme_preference="dark">#993d3d</color>
    </branding>

    <screenshots>
        <screenshot type="default">
            <image>https://example.org/example1.png</image>
            <caption>A caption</caption>
        </screenshot>
        <screenshot>
            <image>https://example.org/example2.png</image>
            <caption>A caption</caption>
        </screenshot>
    </screenshots>

    <releases>
        <release version="1.0.1" date="2024-01-18">
            <url type="details">https://example.org/changelog.html#version_1.0.1</url>
            <description translate="no">
                <p>Release description</p>
                <ul>
                    <li>List of changes</li>
                    <li>List of changes</li>
                </ul>
            </description>
        </release>
    </releases>

</component>

```

File: data/org.github.kai66673.iide.service.in

```

[D-BUS Service]
Name=org.github.kai66673.iide
Exec=@bindir@/iide --gapapplication-service

```

File: po/LINGUAS

Please keep this file sorted alphabetically.

File: po/meson.build

```
i18n.gettext('iide', preset: 'glib')
```


File: po/POTFILES.in

```
# List of source files containing translatable strings.
# Please keep this file sorted alphabetically.
data/org.github.kai66673.iide.desktop.in
data/org.github.kai66673.iide.metainfo.xml.in
data/org.github.kai66673.iide.gschema.xml
src/main.vala
src/window.vala
src/window.ui
```

File: scripts/install-local-settings.sh.in

```
#!/bin/bash
set -e

PREFIX="@PREFIX@"
LOCAL_SCHEMA_DIR="$HOME/.local/share/glib-2.0/schemas"
LOCAL_ICON_DIR="$HOME/.local/share/icons/hicolor"
CURRENT_DIR="$(cd "$(dirname "$0")" && pwd)"

install_schemas() {
    mkdir -p "$LOCAL_SCHEMA_DIR"
    if [ -f "@PREFIX@/share/glib-2.0/schemas/org.github.kai66673.iide.gschema.xml" ]; then
        cp "@PREFIX@/share/glib-2.0/schemas/org.github.kai66673.iide.gschema.xml" "$LOCAL_SCHEMA_DIR/"
    elif [ -f "$CURRENT_DIR/data/org.github.kai66673.iide.gschema.xml" ]; then
        cp "$CURRENT_DIR/data/org.github.kai66673.iide.gschema.xml" "$LOCAL_SCHEMA_DIR/"
    fi
    glib-compile-schemas "$LOCAL_SCHEMA_DIR"
    echo "GSettings schemas installed to $LOCAL_SCHEMA_DIR"
}

install_icons() {
    if [ -d "$CURRENT_DIR/data/icons" ]; then
        mkdir -p "$LOCAL_ICON_DIR"
        cp -r "$CURRENT_DIR/data/icons/*" "$LOCAL_ICON_DIR/" 2>/dev/null || true
        gtk-update-icon-cache "$LOCAL_ICON_DIR" 2>/dev/null || true
        echo "Icons installed to $LOCAL_ICON_DIR"
    fi
}

echo "Installing iide settings locally for development..."
install_schemas
install_icons
echo "Done! If needed, set:"
echo "  export XDG_DATA_DIRS=\"\$HOME/.local/share:\$XDG_DATA_DIRS\""

```

File: src/Services/LSP/config/CppLspConfig.vala

```
public class Iide.CppLspConfig : Iide.LspConfig {
    public override string command () {
        return "clangd";
    }

    public override string[] args () {
        return {
            "--background-index", //
            "--clang-tidy", // (
            "--clang-tidy-checks=*", // ,
            "--background-index-priority=low", // ,
            "--all-scopes-completion", //
            "--header-insertion=never"
        };
    }

    public override Json.Node initialize_params (string? workspace_root, string? initial_uri) {
        message ("CPP: initialize_params: " + workspace_root);
        var root_uri = "file:///home/kai/kaigit/ktexteditor";
        var params = "{
            \"processId\": null,
            \"clientInfo\": {
                \"name\": \"Iide\",
                \"version\": \"1.0.1\"
            },
            \"rootPath\": \"%s\",
            \"rootUri\": \"%s\",
            \"capabilities\": {
                \"window\": {
                    \"workDoneProgress\": true
                },
                \"workspace\": {
                    \"applyEdit\": true,
                    \"diagnostic\": {
                        \"refreshSupport\": true
                    },
                    \"workspaceEdit\": {
                        \"documentChanges\": true
                    },
                    \"didChangeConfiguration\": {
                        \"dynamicRegistration\": true
                    },
                    \"didChangeWatchedFiles\": {
                        \"dynamicRegistration\": true
                    }
                }
            }
        }";
```

```

    },
    "symbol": {
      "dynamicRegistration": true
    },
    "executeCommand": {
      "dynamicRegistration": true
    }
  },
  "textDocument": {
    "diagnostic": {
      "dynamicRegistration": true
    },
    "synchronization": {
      "dynamicRegistration": true,
      "willSave": true,
      "willSaveWaitUntil": false,
      "didSave": true
    },
    "completion": {
      "dynamicRegistration": true,
      "contextSupport": true,
      "completionItem": {
        "snippetSupport": false,
        "commitCharactersSupport": true,
        "documentationFormat": ["markdown", "plaintext"],
        "deprecatedSupport": true,
        "labelDetailsSupport": true
      },
      "completionItemKind": {
        "valueSet": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17,
        ]
      },
    },
    "hover": {
      "dynamicRegistration": true,
      "contentFormat": ["markdown", "plaintext"]
    },
    "signatureHelp": {
      "dynamicRegistration": true,
      "signatureInformation": {
        "documentationFormat": ["markdown", "plaintext"],
        "parameterInformation": {
          "labelOffsetSupport": true
        }
      }
    },
    "definition": {

```

```

        "dynamicRegistration": true,
        "linkSupport": true
    },
    "references": {
        "dynamicRegistration": true
    },
    "documentSymbol": {
        "dynamicRegistration": true,
        "hierarchicalDocumentSymbolSupport": true
    },
    "codeAction": {
        "dynamicRegistration": true
    },
    "publishDiagnostics": {
        "relatedInformation": true,
        "tagSupport": {
            "valueSet": [1, 2]
        },
        "versionSupport": true
    }
}
},
"initializationOptions": {
    "clangdFileStatus": true,
    "backgroundIndex": true,
    "compilationDatabasePath": "build",
    "fallbackFlags": ["-std=c++17", "-Iinclude"]
}
}
"".printf (root_uri.replace ("file://", ""), root_uri);
var parser = new Json.Parser ();
try {
    parser.load_from_data (params);
} catch (GLib.Error e) {
    return new Json.Node (Json.NodeType.NULL);
}
return parser.get_root ();
}

public override Json.Node? server_response_result (Json.Object response) {
    return new Json.Node (Json.NodeType.NULL);
}
}

```

File: src/Services/LSP/config/LspConfig.vala

```
public abstract class Iide.LspConfig {
    public abstract string command ();

    public abstract string[] args ();

    public abstract Json.Node initialize_params (string? workspace_root, string? initial_uri);

    public abstract Json.Node? server_response_result (Json.Object response);
}
```

File: src/Services/LSP/config/LspRegistry.vala

```
public class Iide.LspRegistry {
    public static string ? get_lsp_id (string language) {
        switch (language) {
            case "python":
                return "basedpyright";
            case "cpp":
                return "clangd";
            default:
                return null;
        }
    }

    public static LspConfig ? get_config (string lsp_id) {
        switch (lsp_id) {
            case "basedpyright" :
                return new PythonLspConfig ();
            case "clangd":
                return new CppLspConfig ();
            default:
                return null;
        }
    }
}
```

File: src/Services/LSP/index/ClangTidy.vala

```
using GLib;

public class Iide.ClangTidyRunner : Object {
    public struct Diagnostic {
        public string file;
```

```

        public string line;
        public string type; // "error", "warning", "note"
        public string message;
    }

    //
    public signal void diagnostic_found(Diagnostic diag);
    public signal void finished(bool success, int exit_code, int total_errors, int total_war

    private Subprocess? current_process = null;
    private Cancellable? cancellable = null;

    //
    private int error_count = 0;
    private int warning_count = 0;

    public void run_async(string[] command) {
        this.cancellable = new Cancellable();
        this.error_count = 0;
        this.warning_count = 0;

        new Thread<int> ("clang-tidy-worker", () => {
            bool success = false;
            int exit_code = -1;

            try {
                this.current_process = new Subprocess.newv(
                                                                    command,
                                                                    SubprocessFlags.STDOUT_PIPE | Sub
                );

                var data_stream = new DataInputStream(current_process.get_stdout_pipe());

                string? line;
                string current_file = "unknown";
                var regex_full = new Regex(@"^(~:\n\\[\\+):(\d+):(\d+): (warning|error|not
                var regex_short = new Regex(@"^(warning|error|note): (.+?)(?: \\\\[^\n\\]+)\\

                while (!cancellable.is_cancelled()) {
                    line = data_stream.read_line(null, cancellable);
                    if (line == null)break;

                    process_line(line, regex_full, regex_short, ref current_file);
                }

                if (!cancellable.is_cancelled()) {

```

```

        current_process.wait(cancellable);
        success = current_process.get_successful();
        exit_code = current_process.get_exit_status();
    }
} catch (Error e) {
    if (!(e is IOError.CANCELLED))stderr.printf("Thread Error: %s\n", e.message);
} finally {
    kill_process();
    //
    Idle.add(() => {
        this.finished(success, exit_code, this.error_count, this.warning_count);
        return Source.REMOVE;
    });
}
return 0;
});
}

private void process_line(string line, Regex reg_f, Regex reg_s, ref string current_file)
{
    var sline = line.strip();
    if (sline.has_prefix("[")return;

    MatchInfo match;
    Diagnostic? diag = null;

    if (reg_f.match(sline, 0, out match)) {
        diag = Diagnostic() {
            file = match.fetch(1),
            line = match.fetch(2),
            type = match.fetch(4),
            message = match.fetch(5)
        };
        current_file = diag.file;
    } else if (reg_s.match(sline, 0, out match)) {
        diag = Diagnostic() {
            file = current_file,
            line = "0",
            type = match.fetch(1),
            message = match.fetch(2)
        };
    }

    if (diag != null) {
        //
        if (diag.type == "error")error_count++;
        else if (diag.type == "warning")warning_count++;
    }
}

```

```

        Idle.add(() => {
            this.diagnostic_found(diag);
            return Source.REMOVE;
        });
    }

    public void stop() {
        if (cancellable != null) cancellable.cancel();
        kill_process();
    }

    private void kill_process() {
        if (current_process != null) {
            current_process.force_exit();
            current_process = null;
        }
    }
}

//
// public static int main(string[] args) {
// var loop = new MainLoop();
// var runner = new ClangTidyRunner();

// runner.diagnostic_found.connect((d) => {
// stdout.printf("[%s] %s:%s -> %s\n", d.type.up(), d.file, d.line, d.message);
// });

// runner.finished.connect((ok, code, errors, warnings) => {
// stdout.printf("\n---      ---\n");
// stdout.printf("      : %d\n", errors);
// stdout.printf("      : %d\n", warnings);
// stdout.printf("      : %s ( %d)\n", ok ? "      " : "      /      ", code);
// loop.quit();
// });

// string[] cmd = { "run-clang-tidy", "-p", "build/", "-checks=*,clang-diagnostic-*" };
// runner.run_async(cmd);

// return loop.run();
// }

```


File: src/Services/TreeSitter/model/AstItem.vala

```
/*
 * Item for Abstract Syntax tree model
 *
 * Author: Alberto Fanjul <albfan@gnome.org>
 */
public class ASTItem : GLib.Object {
    public int start { get; set; }
    public int start_offset { get; set; }
    public int end { get; set; }
    public int end_offset { get; set; }
    public string node_type { get; set; }
    public bool fold { get; set; }
    public bool foldable { get; set; }

    public ASTItem(int start, int start_offset, int end, int end_offset,
        string node_type, bool fold) {
        this.start = start;
        this.start_offset = start_offset;
        this.end = end;
        this.end_offset = end_offset;
        this.node_type = node_type;
        this.fold = fold;
        this.foldable = false;
    }

    public bool contains_text_iter(Gtk.TextIter iter) {
        int line = iter.get_line();
        int offset = iter.get_line_offset();

        return (start < line || (start == line && start_offset <= offset))
            && (line < end || (line == end && offset <= end_offset));
    }
}
```

File: src/Services/TreeSitter/model/AstModel.vala

```
/*
 * Model for Abstract Syntax Tree
 *
 * Author: Alberto Fanjul <albfan@gnome.org>
 */
public class ASTModel : Gee.ArrayList<ASTItem> {

    public enum Style {
```

```

    INTELLIJ,
    ATOM;

    public string to_string() {
        switch (this) {
            case INTELLIJ:
                return "Intellij";
            case ATOM:
                return "Atom";
            default:
                assert_not_reached();
        }
    }
}

public Array<ASTItem> highlighted { get; set; default = new Array<ASTItem> (); }
public bool show_symbols { get; set; }
public string strategy { get; set; }
public Style style { get; set; }
public char whitespace { get; set; }
public int whitespace_count { get; set; }

public ASTModel() {
    whitespace = ' ';
    whitespace_count = 2;
}

public void add_item_from_iter(Gtk.TextIter start, Gtk.TextIter end,
                               string type, bool fold) {
    add_item(start.get_line(), start.get_line_offset(), end.get_line(),
             end.get_line_offset(), type, fold);
}

public void add_item(int start, int start_offset, int end, int end_offset,
                     string type, bool fold) {
    add(new ASTItem(start, start_offset, end, end_offset, type, fold));
}

public Array<ASTItem> get_folds(int line) {
    var result = new Array<ASTItem> ();
    foreach (ASTItem item in this) {
        if (line == item.start || (!item.fold && line == item.end)) {
            result.append_val(item);
        }
    }
    return result;
}

```

```

    }
}

```

File: src/Services/TreeSitter/python/highlights.scm

```

; Identifier naming conventions
(identifier) @variable

; Reset highlighting in f-string interpolations
(interpolation) @none

; Identifier naming conventions
((identifier) @type
  (#match? @type "[A-Z].*[a-z]"))

((identifier) @constant
  (#match? @constant "[A-Z][A-Z_0-9]*$"))

((identifier) @constant.builtin
  (#match? @constant.builtin "__[a-zA-Z0-9]*__$"))

((identifier) @constant.builtin
  (#any-of? @constant.builtin
    ; https://docs.python.org/3/library/constants.html
    "NotImplemented" "Ellipsis" "quit" "exit" "copyright" "credits" "license"))

"_" @character.special ; match wildcard

((assignment
  left: (identifier) @type.definition
  (type
    (identifier) @_annotation))
  (#eq? @_annotation "TypeAlias"))

((assignment
  left: (identifier) @type.definition
  right: (call
    function: (identifier) @_func))
  (#any-of? @_func "TypeVar" "NewType"))

; Function definitions
(function_definition
  name: (identifier) @function)

(type

```

```

(identifier) @type)

(type
  (subscript
    (identifier) @type)) ; type subscript: Tuple[int]

((call
  function: (identifier) @_isinstance
  arguments: (argument_list
    (
      (identifier) @type))
  (#eq? @_isinstance "isinstance"))

; Literals
(none) @constant.builtin

[
  (true)
  (false)
] @boolean

(integer) @number

(float) @number.float

(comment) @comment @spell

((module
  .
  (comment) @keyword.directive @nospell)
  (#match? @keyword.directive "~#!/"))

(string) @string

[
  (escape_sequence)
  (escape_interpolation)
] @string.escape

; doc-strings
(expression_statement
  (string
    (string_content) @spell) @string.documentation)

; Tokens
[

```

```

"_"
"_"=
":="
"!="
"*"
"**"
"**="
"*="
"/"
"//"
"/="
"/="
"&"
"&="
"%"
"%="
"^"
"^="
"+"
"+="
"<"
"<<"
"<<="
"<="
"<>"
"="
"=="
">"
">="
">>"
">>="
"@ "
"@="
"| "
"|="
"~"
"->"
] @operator

; Keywords
[
"and"
"in"
"is"
"not"
"or"

```

```

    "is not"
    "not in"
    "del"
] @keyword.operator

[
    "def"
    "lambda"
] @keyword.function

[
    "assert"
    "exec"
    "global"
    "nonlocal"
    "pass"
    "print"
    "with"
    "as"
] @keyword

[
    "type"
    "class"
] @keyword.type

[
    "async"
    "await"
] @keyword.coroutine

[
    "return"
    "yield"
] @keyword.return

(yield
    "from" @keyword.return)

(future_import_statement
    "from" @keyword.import
    "__future__" @module.builtin)

(import_from_statement
    "from" @keyword.import)

```

```

"import" @keyword.import

(aliased_import
  "as" @keyword.import)

(wildcard_import
  "*" @character.special)

(import_statement
  name: (dotted_name
    (identifier) @module))

(import_statement
  name: (aliasd_import
    name: (dotted_name
      (identifier) @module)
    alias: (identifier) @module))

(import_from_statement
  module_name: (dotted_name
    (identifier) @module))

(import_from_statement
  module_name: (relative_import
    (dotted_name
      (identifier) @module)))

[
  "if"
  "elif"
  "else"
  "match"
  "case"
] @keyword.conditional

[
  "for"
  "while"
  "break"
  "continue"
] @keyword.repeat

[
  "try"
  "except"
  "raise"

```

```

    "finally"
] @keyword.exception

(raise_statement
  "from" @keyword.exception)

(try_statement
  (else_clause
    "else" @keyword.exception))

[
  "("
  ")"
  "["
  "]"
  "{"
  "}"
] @punctuation.bracket

(interpolation
  "{" @punctuation.special
  "}" @punctuation.special)

(type_conversion) @function.macro

[
  ","
  "."
  ":"
  ";"
  (ellipsis)
] @punctuation.delimiter

((identifier) @type.builtin
  (#any-of? @type.builtin
    ; https://docs.python.org/3/library/exceptions.html
    "BaseException" "Exception" "ArithmeticError" "BufferError" "LookupError" "AssertionError"
    "AttributeError" "EOFError" "FloatingPointError" "GeneratorExit" "ImportError"
    "ModuleNotFoundError" "IndexError" "KeyError" "KeyboardInterrupt" "MemoryError" "NameError"
    "NotImplementedError" "OSError" "OverflowError" "RecursionError" "ReferenceError" "RuntimeError"
    "StopIteration" "StopAsyncIteration" "SyntaxError" "IndentationError" "TabError" "SystemError"
    "SystemExit" "TypeError" "UnboundLocalError" "UnicodeError" "UnicodeEncodeError"
    "UnicodeDecodeError" "UnicodeTranslateError" "ValueError" "ZeroDivisionError" "EnvironmentError"
    "IOError" "WindowsError" "BlockingIOError" "ChildProcessError" "ConnectionError"
    "BrokenPipeError" "ConnectionAbortedError" "ConnectionRefusedError" "ConnectionResetError"
    "FileExistsError" "FileNotFoundError" "InterruptedError" "IsADirectoryError"

```



```

    "NotADirectoryError" "PermissionError" "ProcessLookupError" "TimeoutError" "Warning"
    "UserWarning" "DeprecationWarning" "PendingDeprecationWarning" "SyntaxWarning" "RuntimeWarning"
    "FutureWarning" "ImportWarning" "UnicodeWarning" "BytesWarning" "ResourceWarning"
    ; https://docs.python.org/3/library/stdtypes.html
    "bool" "int" "float" "complex" "list" "tuple" "range" "str" "bytes" "bytearray" "memoryview"
    "set" "frozenset" "dict" "type" "object"))

; Normal parameters
(parameters
  (identifier) @variable.parameter)

; Lambda parameters
(lambda_parameters
  (identifier) @variable.parameter)

(lambda_parameters
  (tuple_pattern
    (identifier) @variable.parameter))

; Default parameters
(keyword_argument
  name: (identifier) @variable.parameter)

; Naming parameters on call-site
(default_parameter
  name: (identifier) @variable.parameter)

(typed_parameter
  (identifier) @variable.parameter)

(typed_default_parameter
  name: (identifier) @variable.parameter)

; Variadic parameters *args, **kwargs
(parameters
  (list_splat_pattern ; *args
    (identifier) @variable.parameter))

(parameters
  (dictionary_splat_pattern ; **kwargs
    (identifier) @variable.parameter))

; Typed variadic parameters
(parameters
  (typed_parameter
    (list_splat_pattern ; *args: type

```

```

        (identifier) @variable.parameter)))

(parameters
  (typed_parameter
    (dictionary_splat_pattern ; *kwargs: type
      (identifier) @variable.parameter)))

; Lambda parameters
(lambda_parameters
  (list_splat_pattern
    (identifier) @variable.parameter))

(lambda_parameters
  (dictionary_splat_pattern
    (identifier) @variable.parameter))

((identifier) @variable.builtin
  (#eq? @variable.builtin "self"))

((identifier) @variable.builtin
  (#eq? @variable.builtin "cls"))

; After @type.builtin because builtins (such as `type`) are valid as attribute name
((attribute
  attribute: (identifier) @variable.member)
  (#match? @variable.member "^[%1_].*$"))

; Class definitions
(class_definition
  name: (identifier) @type)

(class_definition
  body: (block
    (function_definition
      name: (identifier) @function.method)))

(class_definition
  superclasses: (argument_list
    (identifier) @type))

((class_definition
  body: (block
    (expression_statement
      (assignment
        left: (identifier) @variable.member))))
  (#match? @variable.member "^[%1_].*$"))

```

```

((class_definition
  body: (block
    (expression_statement
      (assignment
        left: (
          (identifier) @variable.member))))))
(#match? @variable.member "^[%l_].*$"))

((class_definition
  (block
    (function_definition
      name: (identifier) @constructor)))
  (#any-of? @constructor "__new__" "__init__"))

; Function calls
(call
  function: (identifier) @function.call)

(call
  function: (attribute
    attribute: (identifier) @function.method.call))

((call
  function: (identifier) @constructor)
  (#match? @constructor "%u"))

((call
  function: (attribute
    attribute: (identifier) @constructor))
  (#match? @constructor "%u"))

; Builtin functions
((call
  function: (identifier) @function.builtin)
  (#any-of? @function.builtin
    "abs" "all" "any" "ascii" "bin" "bool" "breakpoint" "bytearray" "bytes" "callable" "chr"
    "classmethod" "compile" "complex" "delattr" "dict" "dir" "divmod" "enumerate" "eval" "ex"
    "filter" "float" "format" "frozenset" "getattr" "globals" "hasattr" "hash" "help" "hex"
    "input" "int" "isinstance" "issubclass" "iter" "len" "list" "locals" "map" "max" "memory"
    "min" "next" "object" "oct" "open" "ord" "pow" "print" "property" "range" "repr" "revers"
    "round" "set" "setattr" "slice" "sorted" "staticmethod" "str" "sum" "super" "tuple" "typ"
    "vars" "zip" "__import__"))

; Regex from the `re` module
(call

```

```

function: (attribute
  object: (identifier) @_re)
arguments: (argument_list
  .
  (string
    (string_content) @string.regexp))
(#eq? @_re "re"))

; Decorators
((decorator
  "@" @attribute)
  (#set! priority 101))

(decorator
  (identifier) @attribute)

(decorator
  (attribute
    attribute: (identifier) @attribute))

(decorator
  (call
    (identifier) @attribute))

(decorator
  (call
    (attribute
      attribute: (identifier) @attribute)))

((decorator
  (identifier) @attribute.builtin)
  (#any-of? @attribute.builtin "classmethod" "property" "staticmethod"))

```

File: src/Services/TreeSitter/TreeSitterManager.vala

```

// [CCode (cname = "tree_sitter_rust")]
// extern unowned TreeSitter.Language ? get_language_rust ();
// [CCode (cname = "tree_sitter_c")]
// extern unowned TreeSitter.Language ? get_language_c ();
// [CCode (cname = "tree_sitter_javascript")]
// extern unowned TreeSitter.Language ? get_language_javascript ();
// [CCode (cname = "tree_sitter_python")]
// extern unowned TreeSitter.Language ? get_language_python ();
// [CCode (cname = "tree_sitter_ruby")]
// extern unowned TreeSitter.Language ? get_language_ruby ();

```

```

// [CCode(cname = "tree_sitter_cpp")]
// extern unowned TreeSitter.Language ? get_language_cpp();
// [CCode(cname = "tree_sitter_vala")]
// extern unowned TreeSitter.Language ? get_language_vala();
// [CCode (cname = "tree_sitter_go")]
// extern unowned TreeSitter.Language ? get_language_go ();
// [CCode (cname = "tree_sitter_bash")]
// extern unowned TreeSitter.Language ? get_language_bash ();
// [CCode (cname = "tree_sitter_json")]
// extern unowned TreeSitter.Language ? get_language_json ();
// [CCode (cname = "tree_sitter_php")]
// extern unowned TreeSitter.Language ? get_language_php ();
// [CCode (cname = "tree_sitter_html")]
// extern unowned TreeSitter.Language ? get_language_html ();
// [CCode (cname = "tree_sitter_xml")]
// extern unowned TreeSitter.Language ? get_language_xml ();
// [CCode (cname = "tree_sitter_typescript")]
// extern unowned TreeSitter.Language ? get_language_typescript ();
// [CCode (cname = "tree_sitter_yaml")]
// extern unowned TreeSitter.Language ? get_language_yaml ();

```

```

class Ide.TreeSitterManager : GLib.Object {
    public BaseTreeSitterHighlighter ? get_ts_highlighter(SourceView view) {
        var language_name = ((GtkSource.Buffer) view.buffer).language.name.down();
        message("LANG Detected: " + language_name);
        switch (language_name) {
            case "cpp" :
                return new CppHighlighter(view);
            case "python":
                return new PythonHighlighter(view);
            case "rust":
                return new RustHighlighter(view);
            case "vala":
                return new ValaHighlighter(view);
        }
        return null;
    }

    // public unowned TreeSitter.Language? get_ts_language (GtkSource.Buffer buffer) {
    // var language_name = buffer.language.name.down ();
    // unowned TreeSitter.Language? language = null;
    // switch (language_name) {
    // case "bash" :
    // language = get_language_bash ();

```

```

// break;
// case "c" :
// language = get_language_c ();
// break;
// case "cpp" :
// language = get_language_cpp ();
// break;
// case "go" :
// language = get_language_go ();
// break;
// case "javascript" :
// language = get_language_javascript ();
// break;
// case "json" :
// language = get_language_json ();
// break;
// case "html" :
// language = get_language_html ();
// break;
// case "php" :
// language = get_language_php ();
// break;
// case "python" :
// language = get_language_python ();
// break;
// case "ruby" :
// language = get_language_ruby ();
// break;
// case "rust" :
// language = get_language_rust ();
// break;
// case "typescript" :
// language = get_language_typescript ();
// break;
// case "xml" :
// language = get_language_xml ();
// break;
// case "yaml" :
// language = get_language_yaml ();
// break;
// }
// return language;
// }
}

```

File: src/Services/JsonUtils.vala

```
namespace Iide {
    public string json_node_to_string (Json.Node node) {
        var generator = new Json.Generator ();
        generator.set_root (node);
        return generator.to_data (null);
    }

    public string json_object_to_string (Json.Object? obj) {
        if (obj == null) return "<<null>>";
        var node = new Json.Node (Json.NodeType.OBJECT);
        node.set_object (obj);
        return json_node_to_string (node);
    }
}
```

File: src/Services/LoggerService.vala

```
/*
 * loggerservice.vala
 *
 * Copyright 2026 kai
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program. If not, see <https://www.gnu.org/licenses/>.
 *
 * SPDX-License-Identifier: GPL-3.0-or-later
 */

using GLib;

public enum Iide.LogLevel {
    DEBUG,
    INFO,
    WARNING,
```

```

    ERROR,
    CRITICAL;

    public string to_string () {
        switch (this) {
            case DEBUG:      return "DEBUG";
            case INFO:       return "INFO";
            case WARNING:    return "WARN";
            case ERROR:      return "ERROR";
            case CRITICAL:   return "CRITICAL";
            default:         return "UNKNOWN";
        }
    }

    public string get_color () {
        switch (this) {
            case DEBUG:      return "#888888";
            case INFO:       return "#4a9eff";
            case WARNING:    return "#f5a623";
            case ERROR:      return "#e74c3c";
            case CRITICAL:   return "#ff4757";
            default:         return "#ffffff";
        }
    }
}

public class Iide.LogEntry : Object {
    public int64 timestamp { get; set; }
    public Iide.LogLevel level { get; set; }
    public string domain { get; set; }
    public string message { get; set; }
    public string? details { get; set; }

    public string get_timestamp_string () {
        var time = new DateTime.from_unix_utc (timestamp / 1000000);
        var local = time.to_timezone (new TimeZone.local ());
        var msec = (int) ((timestamp / 1000) % 1000);
        return "%s.%03d".printf (local.format ("%H:%M:%S"), msec);
    }
}

public class Iide.LoggerService : Object {
    private static LoggerService? _instance;
    private Gee.ArrayList<Iide.LogEntry> entries;
    private int max_entries = 1000;
    private bool enable_logging = true;

```



```

private File log_file;

public signal void log_added (Iide.LogEntry entry);
public signal void log_cleared ();

public static LoggerService get_instance () {
    if (_instance == null) {
        _instance = new LoggerService ();
    }
    return _instance;
}

private LoggerService () {
    entries = new Gee.ArrayList<Iide.LogEntry> ();
    log_file = File.new_for_path (
        Path.build_filename (Environment.get_user_data_dir ());
    );
    try {
        var dir = log_file.get_parent ();
        if (dir != null && !dir.query_exists (null)) {
            dir.make_directory_with_parents (null);
        }
    } catch (Error e) {
        stderr.printf ("Failed to create log directory: %s\n", e.message);
    }
}

public void log (LogLevel level, string domain, string message, string? details = null)
    if (!enable_logging) return;

    var entry = new Iide.LogEntry () {
        timestamp = (int64) get_real_time (),
        level = level,
        domain = domain,
        message = message,
        details = details
    };

    entries.add (entry);

    if (entries.size > max_entries) {
        entries.remove_at (0);
    }

    log_to_file (entry);
    log_added (entry);

```

```

}

public void debug (string domain, string message, string? details = null) {
    log (LogLevel.DEBUG, domain, message, details);
}

public void info (string domain, string message, string? details = null) {
    log (LogLevel.INFO, domain, message, details);
}

public void warning (string domain, string message, string? details = null) {
    log (LogLevel.WARNING, domain, message, details);
}

public void error (string domain, string message, string? details = null) {
    log (LogLevel.ERROR, domain, message, details);
}

public void critical (string domain, string message, string? details = null) {
    log (LogLevel.CRITICAL, domain, message, details);
}

private void log_to_file (LogEntry entry) {
    try {
        var time = new DateTime.from_unix_utc (entry.timestamp / 1000000);
        var local = time.to_timezone (new TimeZone.local ());
        var timestamp = local.format ("%Y-%m-%d %H:%M:%S");
        var details_str = entry.details != null ? " | " + entry.details : "";
        var line = "[%s] [%s] [%s] %s%s\n".printf (timestamp, entry.level.to_string (),
            details_str);

        if (log_file.query_exists (null)) {
            log_file.append_to (FileCreateFlags.NONE, null).write (line.data);
        } else {
            log_file.create (FileCreateFlags.REPLACE_DESTINATION, null).write (line.data);
        }
    } catch (Error e) {
        stderr.printf ("Failed to write to log file: %s\n", e.message);
    }
}

public Gee.ArrayList<Iide.LogEntry> get_entries () {
    return entries;
}

public Gee.ArrayList<Iide.LogEntry> get_entries_by_level (LogLevel level) {
    var result = new Gee.ArrayList<Iide.LogEntry> ();

```

```

        foreach (var entry in entries) {
            if (entry.level == level) {
                result.add (entry);
            }
        }
        return result;
    }

    public void clear () {
        entries.clear ();
        log_cleared ();
    }

    public void set_max_entries (int max) {
        max_entries = max;
        while (entries.size > max_entries) {
            entries.remove_at (0);
        }
    }

    public void set_logging_enabled (bool enabled) {
        enable_logging = enabled;
    }
}

```

File: src/Services/PanelLayoutHelper.vala

```

/*
 * panellayouthelper.vala
 *
 * Copyright 2026 kai
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program. If not, see <https://www.gnu.org/licenses/>.
 */

```

```

* SPDX-License-Identifier: GPL-3.0-or-later
*/

using Adw;

public class Ide.PanelLayoutHelper : Object {

    public class WidgetInfo {
        public string panel_id { get; set; }
        public int area { get; set; }
        public uint column { get; set; }
        public uint row { get; set; }
        public uint depth { get; set; }

        public Panel.Position to_pos () {
            var pos = new Panel.Position ();
            pos.area = (Panel.Area) this.area;
            pos.column = this.column;
            pos.row = this.row;
            pos.depth = this.depth;
            return pos;
        }
    }

    public class DocumentInfo {
        public string uri { get; set; }
        public uint column { get; set; }
        public uint row { get; set; }
    }

    public static string serialize_dock (Panel.Dock dock) {
        var builder = new Json.Builder ();
        builder.begin_object ();

        builder.set_member_name ("reveal_start");
        builder.add_boolean_value (dock.reveal_start);
        builder.set_member_name ("reveal_end");
        builder.add_boolean_value (dock.reveal_end);
        builder.set_member_name ("reveal_bottom");
        builder.add_boolean_value (dock.reveal_bottom);
        builder.set_member_name ("start_width");
        builder.add_int_value (dock.start_width);
        builder.set_member_name ("end_width");
        builder.add_int_value (dock.end_width);
        builder.set_member_name ("bottom_height");
        builder.add_int_value (dock.bottom_height);
    }
}

```

```

builder.set_member_name ("widgets");
builder.begin_array ();

dock.foreach_frame ((frame) => {
    var pages = frame.get_pages ();
    var position = frame.get_position ();

    for (uint i = 0; i < pages.get_n_items (); i++) {
        var item = pages.get_item (i) as Adw.TabPage;
        if (item == null) {
            continue;
        }
        var child = item.get_child ();
        if (child == null) {
            continue;
        }
        var widget = child as BasePanel;
        if (widget == null) {
            continue;
        }

        builder.begin_object ();
        builder.set_member_name ("panel_id");
        builder.add_string_value (widget.panel_id ());

        if (position != null) {
            builder.set_member_name ("area");
            builder.add_int_value ((int) position.get_area ());
            builder.set_member_name ("column");
            builder.add_int_value ((int) position.get_column ());
            builder.set_member_name ("row");
            builder.add_int_value ((int) position.get_row ());
            builder.set_member_name ("depth");
            builder.add_int_value ((int) position.get_depth ());
        }

        builder.end_object ();
    }
});

builder.end_array ();
builder.end_object ();

var generator = new Json.Generator ();
generator.root = builder.get_root ();

```

```

        return generator.to_data (null);
    }

    public static Gee.HashMap<string, WidgetInfo> parse_widgets (string data) {
        var result = new Gee.HashMap<string, WidgetInfo> ();

        if (data == null || data == "") {
            return result;
        }

        try {
            var parser = new Json.Parser ();
            parser.load_from_data (data);
            var root = parser.get_root ().get_object ();

            if (root.has_member ("widgets")) {
                var widgets_array = root.get_array_member ("widgets");
                foreach (var node in widgets_array.get_elements ()) {
                    var obj = node.get_object ();
                    var info = new WidgetInfo ();
                    info.panel_id = obj.has_member ("panel_id") ? obj.get_string_member ("panel_id") : "";
                    info.area = obj.has_member ("area") ? (int) obj.get_int_member ("area") : 0;
                    info.column = obj.has_member ("column") ? (uint) obj.get_int_member ("column") : 0;
                    info.row = obj.has_member ("row") ? (uint) obj.get_int_member ("row") : 0;
                    info.depth = obj.has_member ("depth") ? (uint) obj.get_int_member ("depth") : 0;
                    result.set (info.panel_id, info);
                }
            }
        } catch (Error e) {
            warning ("Failed to parse widgets: %s", e.message);
        }

        return result;
    }

    public static void deserialize_dock (string data, Panel.Dock dock) {
        if (data == null || data == "") {
            return;
        }

        try {
            var parser = new Json.Parser ();
            parser.load_from_data (data);
            var root = parser.get_root ().get_object ();

            if (root.has_member ("reveal_start")) {

```

```

        dock.reveal_start = root.get_boolean_member ("reveal_start");
    }
    if (root.has_member ("reveal_end")) {
        dock.reveal_end = root.get_boolean_member ("reveal_end");
    }
    if (root.has_member ("reveal_bottom")) {
        dock.reveal_bottom = root.get_boolean_member ("reveal_bottom");
    }
    if (root.has_member ("start_width")) {
        dock.start_width = (int) root.get_int_member ("start_width");
    }
    if (root.has_member ("end_width")) {
        dock.end_width = (int) root.get_int_member ("end_width");
    }
    if (root.has_member ("bottom_height")) {
        dock.bottom_height = (int) root.get_int_member ("bottom_height");
    }
} catch (Error e) {
    warning ("Failed to parse dock layout: %s", e.message);
}
}

public static string serialize_grid (Panel.Grid grid) {
    var builder = new Json.Builder ();
    builder.begin_object ();

    builder.set_member_name ("n_columns");
    builder.add_int_value ((int) grid.get_n_columns ());

    builder.set_member_name ("documents");
    builder.begin_array ();

    grid.foreach_frame ((frame) => {
        var pages = frame.get_pages ();
        var position = frame.get_position ();

        for (uint i = 0; i < pages.get_n_items (); i++) {
            var item = pages.get_item (i) as Adw.TabPage;
            if (item == null) {
                continue;
            }
            var child = item.get_child ();
            if (child == null) {
                continue;
            }
            var widget = child as Panel.Widget;

```

```

        if (widget == null) {
            continue;
        }
        var text_view = widget as Iide.TextView;
        if (text_view == null) {
            continue;
        }

        builder.begin_object ();
        builder.set_member_name ("uri");
        builder.add_string_value (text_view.uri);
        if (position != null) {
            builder.set_member_name ("column");
            builder.add_int_value ((int) position.get_column ());
            builder.set_member_name ("row");
            builder.add_int_value ((int) position.get_row ());
        }
        builder.end_object ();
    }
});

builder.end_array ();
builder.end_object ();

var generator = new Json.Generator ();
generator.root = builder.get_root ();
return generator.to_data (null);
}

public static Gee.ArrayList<DocumentInfo> parse_grid_documents (string data) {
    var result = new Gee.ArrayList<DocumentInfo> ();

    if (data == null || data == "") {
        return result;
    }

    try {
        var parser = new Json.Parser ();
        parser.load_from_data (data);
        var root = parser.get_root ().get_object ();

        if (root.has_member ("documents")) {
            var docs_array = root.get_array_member ("documents");
            foreach (var node in docs_array.get_elements ()) {
                var obj = node.get_object ();
                var info = new DocumentInfo ();
            }
        }
    }
}

```



```

        info.uri = obj.get_string_member ("uri");
        info.column = obj.has_member ("column") ? (uint) obj.get_int_member ("column") : 0;
        info.row = obj.has_member ("row") ? (uint) obj.get_int_member ("row") : 0;
        result.add (info);
    }
}
} catch (Error e) {
    warning ("Failed to parse grid documents: %s", e.message);
}

return result;
}
}

```

File: src/Widgets/Panels/BasePanel.vala

```

public abstract class Ide.BasePanel : Panel.Widget {
    protected BasePanel (string title, string icon_name) {
        Object (title: title, icon_name: icon_name);
    }

    public abstract Panel.Position initial_pos ();
    public abstract string panel_id ();
}

```

File: src/Widgets/ToolViews/TerminalView.vala

```

/*
 * SPDX-License-Identifier: LGPL-3.0-or-later
 * SPDX-FileCopyrightText: 2022 elementary, Inc. (https://elementary.io)
 *
 * 2011-2013 Mario Guerriero <mario@elementaryos.org>
 */

public class Ide.Terminal : Gtk.Box {
    private const string ACTION_GROUP = "term";
    private const string ACTION_PREFIX = ACTION_GROUP + ".";
    private const string ACTION_COPY = "action-copy";
    private const string ACTION_PASTE = "action-paste";
    private const string ACTION_INCREASE_FONT = "increase-font";
    private const string ACTION_DECREASE_FONT = "decrease-font";
    private const string ACTION_RESET_FONT = "reset-font";

    private const double MAX_SCALE = 5.0;
    private const double MIN_SCALE = 0.2;
    private const string LEGACY_SETTINGS_SCHEMA = "org.pantheon.terminal.settings";
}

```

```

private const string SETTINGS_SCHEMA = "io.elementary.terminal.settings";
private const string GNOME_DESKTOP_INTERFACE_SCHEMA = "org.gnome.desktop.interface";
private const string GNOME_DESKTOP_WM_PREFERENCES_SCHEMA = "org.gnome.desktop.wm.preferences";
private const string TERMINAL_FONT_KEY = "font";
private const string TERMINAL_BELL_KEY = "audible-bell";
private const string TERMINAL_CURSOR_KEY = "cursor-shape";
private const string TERMINAL_FOREGROUND_KEY = "foreground";
private const string TERMINAL_BACKGROUND_KEY = "background";
private const string TERMINAL_PALETTE_KEY = "palette";
private const string GNOME_FONT_KEY = "monospace-font-name";
private const string GNOME_BELL_KEY = "audible-bell";

public Vte.Terminal terminal { get; construct; }
private Gtk.EventControllerKey key_controller;
private Gtk.GestureClick gesture_click;
private Gtk.EventControllerScroll scroll_controller;
private Gdk.Clipboard current_clipboard;
private Gtk.ScrolledWindow scrolled_window;

private const double FONT_SCALE_STEP = 0.1;

private Settings? terminal_settings = null;
private Settings? gnome_interface_settings = null;
private Settings? gnome_wm_settings = null;

public SimpleActionGroup actions { get; construct; }

private GLib.Pid child_pid;
// private Gtk.Clipboard current_clipboard;

construct {
    terminal = new Vte.Terminal () {
        hexpand = true,
        vexpand = true,
        scrollbar_lines = -1,
        cursor_blink_mode = SYSTEM // There is no Terminal setting so follow Gnome
    };

    // Set font, allow-bold, audible-bell, background, foreground, and palette of panthe
    var schema_source = SettingsSchemaSource.get_default ();
    var terminal_schema = schema_source.lookup (SETTINGS_SCHEMA, true);
    if (terminal_schema == null) {
        terminal_schema = schema_source.lookup (LEGACY_SETTINGS_SCHEMA, true);
    }

    if (terminal_schema != null) {

```

```

terminal_settings = new Settings.full (terminal_schema, null, null);
terminal_settings.changed.connect ((key) => {
    switch (key) {
        case TERMINAL_FONT_KEY :
            update_font ();
            break;
        case TERMINAL_BELL_KEY :
            update_audible_bell ();
            break;
        case TERMINAL_CURSOR_KEY :
            update_cursor ();
            break;
        case TERMINAL_FOREGROUND_KEY:
        case TERMINAL_BACKGROUND_KEY:
        case TERMINAL_PALETTE_KEY:
            update_colors ();
            break;
        default:
            // TODO Handle other relevant terminal settings?
            // "theme"
            // "prefer-dark-style"
            break;
    }
});
} else {
    var gnome_wm_settings_schema = schema_source.lookup (GNOME_DESKTOP_WM_PREFERENCE_SCHEMA);
    if (gnome_wm_settings_schema != null) {
        gnome_wm_settings = new Settings.full (gnome_wm_settings_schema, null, null);
        gnome_wm_settings.changed.connect ((key) => {
            switch (key) {
                case GNOME_BELL_KEY:
                    update_audible_bell ();
                    break;
                default:
                    break;
            }
        });
    }
    // TODO monitor changes in relevant system settings?
    // "org.gnome.desktop.interface.color-scheme"
}

// Always monitor changes in default font as that is what Terminal usually follows
var gnome_interface_settings_schema = schema_source.lookup (GNOME_DESKTOP_INTERFACE_SCHEMA);
if (gnome_interface_settings_schema != null) {
    gnome_interface_settings = new Settings.full (gnome_interface_settings_schema, null, null);
}

```

```

        gnome_interface_settings.changed.connect ((key) => {
            switch (key) {
                case GNOME_FONT_KEY:
                    update_font ();
                    break;
                default:
                    break;
            }
        });
    }

    update_font ();
    update_audible_bell ();
    update_cursor ();
    update_colors ();

    var adw_style_manager = Adw.StyleManager.get_default ();
    adw_style_manager.notify["color-scheme"].connect (() => {
        update_colors ();
    });

    terminal.child_exited.connect (() => {
        // Hide the exited terminal
        // var win_group = get_action_group (Scratch.MainWindow.ACTION_GROUP);
        // win_group.activate_action (Scratch.MainWindow.ACTION_TOGGLE_TERMINAL, null);
        //// Get ready to resume at last saved location
        // spawn_shell (Scratch.saved_state.get_string ("last-opened-path"));
        //// Clear screen of new shell
        terminal.feed_child ("clear -x\n".data);
    });

    var copy_action = new SimpleAction (ACTION_COPY, null);
    copy_action.set_enabled (false);
    copy_action.activate.connect (() => terminal.copy_clipboard ());

    var paste_action = new SimpleAction (ACTION_PASTE, null);
    paste_action.activate.connect (() => terminal.paste_clipboard ());

    var increase_font_action = new SimpleAction (ACTION_INCREASE_FONT, null);
    increase_font_action.activate.connect (() => increment_size ());

    var decrease_font_action = new SimpleAction (ACTION_DECREASE_FONT, null);
    decrease_font_action.activate.connect (() => decrement_size ());

    var reset_font_action = new SimpleAction (ACTION_RESET_FONT, null);
    reset_font_action.activate.connect (() => set_default_font_size ());

```

```

actions = new SimpleActionGroup ();
actions.add_action (copy_action);
actions.add_action (paste_action);
actions.add_action (increase_font_action);
actions.add_action (decrease_font_action);
actions.add_action (reset_font_action);

var menu_model = new GLib.Menu ();
var clipboard_section = new GLib.Menu ();
clipboard_section.append (_("Copy"), ACTION_PREFIX + ACTION_COPY);
clipboard_section.append (_("Paste"), ACTION_PREFIX + ACTION_PASTE);
menu_model.append_section (null, clipboard_section);
var zoom_section = new GLib.Menu ();
zoom_section.append (_("Zoom In"), ACTION_PREFIX + ACTION_INCREASE_FONT);
zoom_section.append (_("Zoom Out"), ACTION_PREFIX + ACTION_DECREASE_FONT);
zoom_section.append (_("Reset Zoom"), ACTION_PREFIX + ACTION_RESET_FONT);
menu_model.append_section (null, zoom_section);

scrolled_window = new Gtk.ScrolledWindow ();
scrolled_window.hexpand = true;
scrolled_window.set_child (terminal);

var popover_menu = new Gtk.PopoverMenu.from_model (menu_model);
popover_menu.insert_action_group (ACTION_GROUP, actions);
popover_menu.set_parent (scrolled_window);
// menu.show_all ();

key_controller = new Gtk.EventControllerKey ();
key_controller.propagation_phase = BUBBLE;
terminal.add_controller (key_controller);
key_controller.key_pressed.connect (key_pressed);

gesture_click = new Gtk.GestureClick ();
gesture_click.button = 3; // Right click
scrolled_window.add_controller (gesture_click);
gesture_click.pressed.connect ((n_press, x, y) => {
    if (n_press == 1) {
        paste_action.set_enabled (current_clipboard.content != null);
        var rect = Gdk.Rectangle () { x = (int) x, y = (int) y, width = 1, height =
        popover_menu.set_pointing_to (rect);
        popover_menu.popup ();
    }
});

scroll_controller = new Gtk.EventControllerScroll (Gtk.EventControllerScrollFlags.VI

```

```

scroll_controller.propagation_phase = CAPTURE;
scrolled_window.add_controller (scroll_controller);
scroll_controller.scroll.connect ((dx, dy) => {
    var state = scroll_controller.get_current_event_state ();
    if (Gdk.ModifierType.CONTROL_MASK in state) {
        if (dy > 0) {
            decrement_size ();
        } else {
            increment_size ();
        }
        return Gdk.EVENT_STOP;
    }
    return Gdk.EVENT_PROPAGATE;
});

// terminal.button_press_event.connect ((event) => {
// if (event.button == 3) {
// paste_action.set_enabled (current_clipboard.wait_is_text_available ());
// menu.select_first (false);
// menu.popup_at_pointer (event);
// }
// return false;
// });

// realize.connect (() => {
// current_clipboard = terminal.get_clipboard (Gdk.SELECTION_CLIPBOARD);
// copy_action.set_enabled (terminal.get_has_selection ());
// });

spawn_shell ();

terminal.selection_changed.connect (() => {
    copy_action.set_enabled (terminal.get_has_selection ());
});

realize.connect (() => {
    current_clipboard = scrolled_window.get_clipboard ();
    copy_action.set_enabled (terminal.get_has_selection ());
});

append (scrolled_window);
}

private void spawn_shell (string dir = GLib.Environment.get_current_dir ()) {
    terminal.spawn_async (
        Vte.PtyFlags.DEFAULT,

```

```

        dir,
        { Vte.get_user_shell () },
        null,
        SpawnFlags.SEARCH_PATH,
        null,
        100,
        null,
        (source, pid, error) => {
    if (error != null) {
        stderr.printf ("                : %s\n", error.message);
    } else {
        this.child_pid = pid;
        stdout.printf ("                . PID: %d\n", pid);
    }
    });
}

// public void change_location (string dir) {
//   Posix.kill (child_pid, Posix.Signal.TERM);
//   terminal.reset (true, true);
//   spawn_shell (dir);
//   Scratch.saved_state.set_string ("last-opened-path", dir);
// }

// private string get_shell_location () {
//   int pid = (!) (this.child_pid);
//   try {
//     return GLib.FileUtils.read_link ("/proc/%d/cwd".printf (pid));
//   } catch (GLib.FileError error) {
//     warning ("An error occurred while fetching the current dir of shell: %s", error.message);
//     return "";
//   }
// }

private void update_font () {
    var font_name = "";
    if (terminal_settings != null) {
        font_name = terminal_settings.get_string (TERMINAL_FONT_KEY);
    }

    if (font_name == "" && gnome_interface_settings != null) {
        font_name = gnome_interface_settings.get_string (GNOME_FONT_KEY);
    }

    var fd = Pango.FontDescription.from_string (font_name);
    terminal.set_font (fd);

```

```

}

private void update_audible_bell () {
    var audible_bell = false;
    if (terminal_settings != null) {
        audible_bell = terminal_settings.get_boolean (TERMINAL_BELL_KEY);
    } else if (gnome_wm_settings != null) {
        audible_bell = gnome_wm_settings.get_boolean (GNOME_BELL_KEY);
    }

    terminal.set_audible_bell (audible_bell);
}

private void update_cursor () {
    if (terminal_settings != null) {
        var cursor_shape_setting = terminal_settings.get_string (TERMINAL_CURSOR_KEY);
        switch (cursor_shape_setting) {
            case "Block":
                terminal.cursor_shape = Vte.CursorShape.BLOCK;
                break;
            case "I-Beam":
                terminal.cursor_shape = Vte.CursorShape.IBEAM;
                break;
            case "Underline":
                terminal.cursor_shape = Vte.CursorShape.UNDERLINE;
                break;
        }
    } // No corresponding system keymap
}

private void update_colors () {
    if (terminal_settings != null) {
        string background_setting = terminal_settings.get_string (TERMINAL_BACKGROUND_KEY);
        Gdk.RGBA background_color = Gdk.RGBA ();
        background_color.parse (background_setting);

        string foreground_setting = terminal_settings.get_string (TERMINAL_FOREGROUND_KEY);
        Gdk.RGBA foreground_color = Gdk.RGBA ();
        foreground_color.parse (foreground_setting);

        string palette_setting = terminal_settings.get_string (TERMINAL_PALETTE_KEY);

        string[] hex_palette = { "#000000", "#FF6C60", "#A8FF60", "#FFFFCC", "#96CBFE",
            "#FF73FE", "#C6C5FE", "#EEEEEE", "#000000", "#FF6C60",
            "#A8FF60", "#FFFFB6", "#96CBFE", "#FF73FE", "#C6C5FE",
            "#EEEEEE" };
    }
}

```



```

        string current_string = "";
        int current_color = 0;
        for (var i = 0; i < palette_setting.length; i++) {
            if (palette_setting[i] == ':') {
                hex_palette[current_color] = current_string;
                current_string = "";
                current_color++;
            } else {
                current_string += palette_setting[i].to_string ();
            }
        }

        Gdk.RGBA[] palette = new Gdk.RGBA[16];

        for (int i = 0; i < hex_palette.length; i++) {
            Gdk.RGBA new_color = Gdk.RGBA ();
            new_color.parse (hex_palette[i]);
            palette[i] = new_color;
        }

        terminal.set_colors (foreground_color, background_color, palette);
    } else {
        update_colors_for_theme (Adw.StyleManager.get_default ().color_scheme);
    }
}

private void update_colors_for_theme (Adw.ColorScheme scheme) {
    Gdk.RGBA foreground_color;
    Gdk.RGBA background_color;
    string[] hex_palette = { "#000000", "#cc0000", "#4e9a06", "#c4a000", "#3465a4", "#7f7f7f", "#000000", "#cc0000", "#4e9a06", "#c4a000", "#3465a4", "#7f7f7f", "#000000", "#cc0000", "#4e9a06", "#c4a000", "#3465a4", "#7f7f7f" };

    Gdk.RGBA[] palette = new Gdk.RGBA[16];

    for (int i = 0; i < hex_palette.length; i++) {
        Gdk.RGBA new_color = Gdk.RGBA ();
        new_color.parse (hex_palette[i]);
        palette[i] = new_color;
    }

    if (scheme == Adw.ColorScheme.FORCE_LIGHT) {
        foreground_color = { 0.0f, 0.0f, 0.0f, 1.0f };
        background_color = { 1.0f, 1.0f, 1.0f, 1.0f };
    } else {
        foreground_color = { 1.0f, 1.0f, 1.0f, 1.0f };
        background_color = { 0.0f, 0.0f, 0.0f, 1.0f };
    }
}

```

```

    }

    terminal.set_colors (foreground_color, background_color, palette);
}

public void save_settings () {
    // Scratch.saved_state.set_string ("last-opened-path", get_shell_location ());
}

private void increment_size () {
    terminal.font_scale = (terminal.font_scale + FONT_SCALE_STEP).clamp (MIN_SCALE, MAX_SCALE);
}

private void decrement_size () {
    terminal.font_scale = (terminal.font_scale - FONT_SCALE_STEP).clamp (MIN_SCALE, MAX_SCALE);
}

public void set_default_font_size () {
    terminal.font_scale = 1.0;
}

private bool key_pressed (uint keyval, uint keycode, Gdk.ModifierType modifiers) {
    if (Gdk.ModifierType.CONTROL_MASK in modifiers && keyval == Gdk.Key.d) {
        terminal.reset (true, true);
        return Gdk.EVENT_STOP;
    }

    if (Gdk.ModifierType.CONTROL_MASK in modifiers &&
        (Gdk.ModifierType.SHIFT_MASK in modifiers ||
         terminal_settings != null && terminal_settings.get_boolean ("natural-copy-paste"))) {
        if (keyval == Gdk.Key.c && terminal.get_has_selection ()) {
            actions.activate_action (ACTION_COPY, null);
            return Gdk.EVENT_STOP;
        }
        // } else if (keyval == Gdk.Key.v && current_clipboard.wait_is_text_available ()) {
        // actions.activate_action (ACTION_PASTE, null);
        // return Gdk.EVENT_STOP;
    }
}

return Gdk.EVENT_PROPAGATE;
}
}

```

File: src/config.vapi

```
[CCode (cprefix = "", lower_case_cprefix = "", cheader_filename = "config.h")]
namespace Config {
    public const string GETTEXT_PACKAGE;
    public const string LOCALEDIR;
    public const string PACKAGE_VERSION;
}
```

File: src/shortcuts-dialog.ui

```
<?xml version="1.0" encoding="UTF-8"?>
<interface>
  <object class="AdwShortcutsDialog" id="shortcuts_dialog">
    <child>
      <object class="AdwShortcutsSection">
        <property name="title" translatable="yes">Shortcuts</property>
        <child>
          <object class="AdwShortcutsItem">
            <property name="title" translatable="yes" context="shortcut window">Show Shortcuts</property>
            <property name="action-name">app.shortcuts</property>
          </object>
        </child>
        <child>
          <object class="AdwShortcutsItem">
            <property name="title" translatable="yes" context="shortcut window">Quit</property>
            <property name="action-name">app.quit</property>
          </object>
        </child>
      </object>
    </child>
  </interface>
```

File: src/window.ui

```
<?xml version="1.0" encoding="UTF-8" ?>
<interface>
  <template class="IideWindow" parent="PanelDocumentWorkspace">
    <child type="titlebar">
      <object class="AdwHeaderBar">
        <child type="title">
          <object class="AdwWindowTitle" id="window_title">
            <property name="title">IIDE Application</property>
          </object>
        </child>
      </object>
    </child>
  </template>
</interface>
```

```

</child>
<child type="start">
  <object class="PanelToggleButton">
    <property name="area">start</property>
    <property name="dock">dock</property>
  </object>
</child>
<child type="start">
  <object class="GtkButton">
    <property name="icon-name">list-add-symbolic</property>
    <property name="action-name">workspace.add-page</property>
    <property name="tooltip-text">Add Page</property>
  </object>
</child>
<child type="end">
  <object class="PanelToggleButton">
    <property name="area">end</property>
    <property name="dock">dock</property>
  </object>
</child>
<child type="end">
  <object class="GtkMenuButton">
    <property name="icon-name">open-menu-symbolic</property>
    <property name="menu-model">primary_menu</property>
    <property name="tooltip-text">Primary Menu</property>
  </object>
</child>
</object>
</child>
<child internal-child="dock">
  <object class="PanelDock" id="dock">
  </object>
</child>
<child internal-child="start_area">
  <object class="PanelPaned" id="start_area">
    <child>
      <object class="PanelFrame">
        <child>
          <object class="PanelWidget">
            <property name="title">XXX</property>
            <property name="icon-name">folder-documents-symbolic</property>
            <property name="vexpand">true</property>
          </object>
          <object class="GtkScrolledWindow">
            <property name="hscrollbar-policy">never</property>
          </object>
        </child>
      </object>
    </child>
  </object>
</child>

```

```

        <object class="GtkListView">
        </object>
    </child>
</object>
</child>
</object>
</child>
</object>
</child>
</object>
</child>
</object>
<object class="PanelPaned">
    <child>
        <object class="PanelFrame">
            <child>
                <object class="PanelWidget">
                    <property name="title">ZZZ ZZZ</property>
                    <property name="icon-name">folder-symbolic</property>
                    <property name="vexpand">true</property>
                <child>
                    <object class="GtkScrolledWindow">
                        <property name="hscrollbar-policy">never</property>
                        <child>
                            <object class="GtkListView">
                            </object>
                        </child>
                    </object>
                </child>
            </object>
        </child>
    </object>
    <child internal-child="bottom_area">
        <object class="PanelPaned">
            <child>
                <object class="PanelFrame">
                    <child>
                        <object class="PanelWidget">
                            <property name="title">123</property>
                            <property name="icon-name">folder-symbolic</property>
                            <property name="vexpand">true</property>
                        <child>
                            <object class="GtkScrolledWindow">
                                <property name="hscrollbar-policy">never</property>
                                <child>

```

```

        <object class="GtkListView">
        </object>
    </child>
</object>
</child>
</object>
</child>
</object>
</child>
</object>
</child>
</object>
<object class="PanelPaned">
    <child>
        <object class="PanelFrame">
            <child>
                <object class="PanelWidget">
                    <property name="title">00P</property>
                    <property name="icon-name">folder-symbolic</property>
                    <property name="vexpand">true</property>
                    <child>
                        <object class="GtkScrolledWindow">
                            <property name="hscrollbar-policy">never</property>
                            <child>
                                <object class="GtkListView">
                                </object>
                            </child>
                        </object>
                    </child>
                </object>
            </child>
        </object>
    </child>
    <child internal-child="end_area">
        <object class="PanelPaned" id="end_area">
        </object>
    </child>
    <child internal-child="statusbar">
        <object class="PanelStatusbar">
            <child type="suffix">
                <object class="PanelToggleButton">
                    <property name="area">bottom</property>
                    <property name="dock">dock</property>
                </object>
            </child>
        </object>
    </child>
</object>

```

```

        <object class="PanelStatusbar">
            <child type="suffix">
                <object class="PanelToggleButton">
                    <property name="area">top</property>
                    <property name="dock">dock</property>
                </object>
            </child>
        </object>
    </child>
</template>
<menu id="primary_menu">
    <section>
        <item>
            <attribute name="label">Quit</attribute>
            <attribute name="action">app.quit</attribute>
        </item>
    </section>
</menu>
</interface>

```

File: test_temp/test.c

```
int main() {
```

File: .gitmodules

```

[submodule "vendor/tree-sitter"]
    path = vendor/tree-sitter
    url = https://github.com/albfan/tree-sitter
[submodule "parsers/c"]
    path = parsers/c
    url = https://github.com/tree-sitter/tree-sitter-c
[submodule "parsers/bash"]
    path = parsers/bash
    url = https://github.com/tree-sitter/tree-sitter-bash
[submodule "parsers/json"]
    path = parsers/json
    url = https://github.com/tree-sitter/tree-sitter-json
[submodule "parsers/python"]
    path = parsers/python
    url = https://github.com/tree-sitter/tree-sitter-python
[submodule "parsers/php"]
    path = parsers/php
    url = https://github.com/tree-sitter/tree-sitter-php
[submodule "parsers/cpp"]

```

```

    path = parsers/cpp
    url = https://github.com/tree-sitter/tree-sitter-cpp
[submodule "parsers/go"]
    path = parsers/go
    url = https://github.com/tree-sitter/tree-sitter-go
[submodule "parsers/html"]
    path = parsers/html
    url = https://github.com/tree-sitter/tree-sitter-html
[submodule "parsers/javascript"]
    path = parsers/javascript
    url = https://github.com/tree-sitter/tree-sitter-javascript
[submodule "parsers/ruby"]
    path = parsers/ruby
    url = https://github.com/tree-sitter/tree-sitter-ruby
[submodule "parsers/rust"]
    path = parsers/rust
    url = https://github.com/tree-sitter/tree-sitter-rust
[submodule "parsers/typescript"]
    path = parsers/typescript
    url = https://github.com/tree-sitter/tree-sitter-typescript
[submodule "parsers/yaml"]
    path = parsers/yaml
    url = https://github.com/tree-sitter-grammars/tree-sitter-yaml
[submodule "parsers/xml"]
    path = parsers/xml
    url = https://github.com/tree-sitter-grammars/tree-sitter-xml
[submodule "parsers/vala"]
    path = parsers/vala
    url = https://github.com/vala-lang/tree-sitter-vala

```

File: COPYING

GNU GENERAL PUBLIC LICENSE

Version 3, 29 June 2007

Copyright (C) 2007 Free Software Foundation, Inc. <<https://fsf.org/>>
 Everyone is permitted to copy and distribute verbatim copies
 of this license document, but changing it is not allowed.

Preamble

The GNU General Public License is a free, copyleft license for
 software and other kinds of works.

The licenses for most software and other practical works are designed

to take away your freedom to share and change the works. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change all versions of a program--to make sure it remains free software for all its users. We, the Free Software Foundation, use the GNU General Public License for most of our software; it applies also to any other work released this way by its authors. You can apply it to your programs, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for them if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs, and that you know you can do these things.

To protect your rights, we need to prevent others from denying you these rights or asking you to surrender the rights. Therefore, you have certain responsibilities if you distribute copies of the software, or if you modify it: responsibilities to respect the freedom of others.

For example, if you distribute copies of such a program, whether gratis or for a fee, you must pass on to the recipients the same freedoms that you received. You must make sure that they, too, receive or can get the source code. And you must show them these terms so they know their rights.

Developers that use the GNU GPL protect your rights with two steps: (1) assert copyright on the software, and (2) offer you this License giving you legal permission to copy, distribute and/or modify it.

For the developers' and authors' protection, the GPL clearly explains that there is no warranty for this free software. For both users' and authors' sake, the GPL requires that modified versions be marked as changed, so that their problems will not be attributed erroneously to authors of previous versions.

Some devices are designed to deny users access to install or run modified versions of the software inside them, although the manufacturer can do so. This is fundamentally incompatible with the aim of protecting users' freedom to change the software. The systematic pattern of such abuse occurs in the area of products for individuals to use, which is precisely where it is most unacceptable. Therefore, we have designed this version of the GPL to prohibit the practice for those products. If such problems arise substantially in other domains, we stand ready to extend this provision to those domains in future versions of the GPL, as needed to protect the freedom of users.

Finally, every program is threatened constantly by software patents. States should not allow patents to restrict development and use of software on general-purpose computers, but in those that do, we wish to avoid the special danger that patents applied to a free program could make it effectively proprietary. To prevent this, the GPL assures that patents cannot be used to render the program non-free.

The precise terms and conditions for copying, distribution and modification follow.

TERMS AND CONDITIONS

0. Definitions.

"This License" refers to version 3 of the GNU General Public License.

"Copyright" also means copyright-like laws that apply to other kinds of works, such as semiconductor masks.

"The Program" refers to any copyrightable work licensed under this License. Each licensee is addressed as "you". "Licensees" and "recipients" may be individuals or organizations.

To "modify" a work means to copy from or adapt all or part of the work in a fashion requiring copyright permission, other than the making of an exact copy. The resulting work is called a "modified version" of the earlier work or a work "based on" the earlier work.

A "covered work" means either the unmodified Program or a work based on the Program.

To "propagate" a work means to do anything with it that, without permission, would make you directly or secondarily liable for infringement under applicable copyright law, except executing it on a computer or modifying a private copy. Propagation includes copying, distribution (with or without modification), making available to the public, and in some countries other activities as well.

To "convey" a work means any kind of propagation that enables other parties to make or receive copies. Mere interaction with a user through a computer network, with no transfer of a copy, is not conveying.

An interactive user interface displays "Appropriate Legal Notices" to the extent that it includes a convenient and prominently visible feature that (1) displays an appropriate copyright notice, and (2)

tells the user that there is no warranty for the work (except to the extent that warranties are provided), that licensees may convey the work under this License, and how to view a copy of this License. If the interface presents a list of user commands or options, such as a menu, a prominent item in the list meets this criterion.

1. Source Code.

The "source code" for a work means the preferred form of the work for making modifications to it. "Object code" means any non-source form of a work.

A "Standard Interface" means an interface that either is an official standard defined by a recognized standards body, or, in the case of interfaces specified for a particular programming language, one that is widely used among developers working in that language.

The "System Libraries" of an executable work include anything, other than the work as a whole, that (a) is included in the normal form of packaging a Major Component, but which is not part of that Major Component, and (b) serves only to enable use of the work with that Major Component, or to implement a Standard Interface for which an implementation is available to the public in source code form. A "Major Component", in this context, means a major essential component (kernel, window system, and so on) of the specific operating system (if any) on which the executable work runs, or a compiler used to produce the work, or an object code interpreter used to run it.

The "Corresponding Source" for a work in object code form means all the source code needed to generate, install, and (for an executable work) run the object code and to modify the work, including scripts to control those activities. However, it does not include the work's System Libraries, or general-purpose tools or generally available free programs which are used unmodified in performing those activities but which are not part of the work. For example, Corresponding Source includes interface definition files associated with source files for the work, and the source code for shared libraries and dynamically linked subprograms that the work is specifically designed to require, such as by intimate data communication or control flow between those subprograms and other parts of the work.

The Corresponding Source need not include anything that users can regenerate automatically from other parts of the Corresponding Source.

The Corresponding Source for a work in source code form is that

same work.

2. Basic Permissions.

All rights granted under this License are granted for the term of copyright on the Program, and are irrevocable provided the stated conditions are met. This License explicitly affirms your unlimited permission to run the unmodified Program. The output from running a covered work is covered by this License only if the output, given its content, constitutes a covered work. This License acknowledges your rights of fair use or other equivalent, as provided by copyright law.

You may make, run and propagate covered works that you do not convey, without conditions so long as your license otherwise remains in force. You may convey covered works to others for the sole purpose of having them make modifications exclusively for you, or provide you with facilities for running those works, provided that you comply with the terms of this License in conveying all material for which you do not control copyright. Those thus making or running the covered works for you must do so exclusively on your behalf, under your direction and control, on terms that prohibit them from making any copies of your copyrighted material outside their relationship with you.

Conveying under any other circumstances is permitted solely under the conditions stated below. Sublicensing is not allowed; section 10 makes it unnecessary.

3. Protecting Users' Legal Rights From Anti-Circumvention Law.

No covered work shall be deemed part of an effective technological measure under any applicable law fulfilling obligations under article 11 of the WIPO copyright treaty adopted on 20 December 1996, or similar laws prohibiting or restricting circumvention of such measures.

When you convey a covered work, you waive any legal power to forbid circumvention of technological measures to the extent such circumvention is effected by exercising rights under this License with respect to the covered work, and you disclaim any intention to limit operation or modification of the work as a means of enforcing, against the work's users, your or third parties' legal rights to forbid circumvention of technological measures.

4. Conveying Verbatim Copies.

You may convey verbatim copies of the Program's source code as you

receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice; keep intact all notices stating that this License and any non-permissive terms added in accord with section 7 apply to the code; keep intact all notices of the absence of any warranty; and give all recipients a copy of this License along with the Program.

You may charge any price or no price for each copy that you convey, and you may offer support or warranty protection for a fee.

5. Conveying Modified Source Versions.

You may convey a work based on the Program, or the modifications to produce it from the Program, in the form of source code under the terms of section 4, provided that you also meet all of these conditions:

a) The work must carry prominent notices stating that you modified it, and giving a relevant date.

b) The work must carry prominent notices stating that it is released under this License and any conditions added under section 7. This requirement modifies the requirement in section 4 to "keep intact all notices".

c) You must license the entire work, as a whole, under this License to anyone who comes into possession of a copy. This License will therefore apply, along with any applicable section 7 additional terms, to the whole of the work, and all its parts, regardless of how they are packaged. This License gives no permission to license the work in any other way, but it does not invalidate such permission if you have separately received it.

d) If the work has interactive user interfaces, each must display Appropriate Legal Notices; however, if the Program has interactive interfaces that do not display Appropriate Legal Notices, your work need not make them do so.

A compilation of a covered work with other separate and independent works, which are not by their nature extensions of the covered work, and which are not combined with it such as to form a larger program, in or on a volume of a storage or distribution medium, is called an "aggregate" if the compilation and its resulting copyright are not used to limit the access or legal rights of the compilation's users beyond what the individual works permit. Inclusion of a covered work in an aggregate does not cause this License to apply to the other parts of the aggregate.

6. Conveying Non-Source Forms.

You may convey a covered work in object code form under the terms of sections 4 and 5, provided that you also convey the machine-readable Corresponding Source under the terms of this License, in one of these ways:

- a) Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by the Corresponding Source fixed on a durable physical medium customarily used for software interchange.
- b) Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by a written offer, valid for at least three years and valid for as long as you offer spare parts or customer support for that product model, to give anyone who possesses the object code either (1) a copy of the Corresponding Source for all the software in the product that is covered by this License, on a durable physical medium customarily used for software interchange, for a price no more than your reasonable cost of physically performing this conveying of source, or (2) access to copy the Corresponding Source from a network server at no charge.
- c) Convey individual copies of the object code with a copy of the written offer to provide the Corresponding Source. This alternative is allowed only occasionally and noncommercially, and only if you received the object code with such an offer, in accord with subsection 6b.
- d) Convey the object code by offering access from a designated place (gratis or for a charge), and offer equivalent access to the Corresponding Source in the same way through the same place at no further charge. You need not require recipients to copy the Corresponding Source along with the object code. If the place to copy the object code is a network server, the Corresponding Source may be on a different server (operated by you or a third party) that supports equivalent copying facilities, provided you maintain clear directions next to the object code saying where to find the Corresponding Source. Regardless of what server hosts the Corresponding Source, you remain obligated to ensure that it is available for as long as needed to satisfy these requirements.
- e) Convey the object code using peer-to-peer transmission, provided you inform other peers where the object code and Corresponding

Source of the work are being offered to the general public at no charge under subsection 6d.

A separable portion of the object code, whose source code is excluded from the Corresponding Source as a System Library, need not be included in conveying the object code work.

A "User Product" is either (1) a "consumer product", which means any tangible personal property which is normally used for personal, family, or household purposes, or (2) anything designed or sold for incorporation into a dwelling. In determining whether a product is a consumer product, doubtful cases shall be resolved in favor of coverage. For a particular product received by a particular user, "normally used" refers to a typical or common use of that class of product, regardless of the status of the particular user or of the way in which the particular user actually uses, or expects or is expected to use, the product. A product is a consumer product regardless of whether the product has substantial commercial, industrial or non-consumer uses, unless such uses represent the only significant mode of use of the product.

"Installation Information" for a User Product means any methods, procedures, authorization keys, or other information required to install and execute modified versions of a covered work in that User Product from a modified version of its Corresponding Source. The information must suffice to ensure that the continued functioning of the modified object code is in no case prevented or interfered with solely because modification has been made.

If you convey an object code work under this section in, or with, or specifically for use in, a User Product, and the conveying occurs as part of a transaction in which the right of possession and use of the User Product is transferred to the recipient in perpetuity or for a fixed term (regardless of how the transaction is characterized), the Corresponding Source conveyed under this section must be accompanied by the Installation Information. But this requirement does not apply if neither you nor any third party retains the ability to install modified object code on the User Product (for example, the work has been installed in ROM).

The requirement to provide Installation Information does not include a requirement to continue to provide support service, warranty, or updates for a work that has been modified or installed by the recipient, or for the User Product in which it has been modified or installed. Access to a network may be denied when the modification itself materially and adversely affects the operation of the network or violates the rules and protocols for communication across the network.

Corresponding Source conveyed, and Installation Information provided, in accord with this section must be in a format that is publicly documented (and with an implementation available to the public in source code form), and must require no special password or key for unpacking, reading or copying.

7. Additional Terms.

"Additional permissions" are terms that supplement the terms of this License by making exceptions from one or more of its conditions. Additional permissions that are applicable to the entire Program shall be treated as though they were included in this License, to the extent that they are valid under applicable law. If additional permissions apply only to part of the Program, that part may be used separately under those permissions, but the entire Program remains governed by this License without regard to the additional permissions.

When you convey a copy of a covered work, you may at your option remove any additional permissions from that copy, or from any part of it. (Additional permissions may be written to require their own removal in certain cases when you modify the work.) You may place additional permissions on material, added by you to a covered work, for which you have or can give appropriate copyright permission.

Notwithstanding any other provision of this License, for material you add to a covered work, you may (if authorized by the copyright holders of that material) supplement the terms of this License with terms:

- a) Disclaiming warranty or limiting liability differently from the terms of sections 15 and 16 of this License; or
- b) Requiring preservation of specified reasonable legal notices or author attributions in that material or in the Appropriate Legal Notices displayed by works containing it; or
- c) Prohibiting misrepresentation of the origin of that material, or requiring that modified versions of such material be marked in reasonable ways as different from the original version; or
- d) Limiting the use for publicity purposes of names of licensors or authors of the material; or
- e) Declining to grant rights under trademark law for use of some trade names, trademarks, or service marks; or

f) Requiring indemnification of licensors and authors of that material by anyone who conveys the material (or modified versions of it) with contractual assumptions of liability to the recipient, for any liability that these contractual assumptions directly impose on those licensors and authors.

All other non-permissive additional terms are considered "further restrictions" within the meaning of section 10. If the Program as you received it, or any part of it, contains a notice stating that it is governed by this License along with a term that is a further restriction, you may remove that term. If a license document contains a further restriction but permits relicensing or conveying under this License, you may add to a covered work material governed by the terms of that license document, provided that the further restriction does not survive such relicensing or conveying.

If you add terms to a covered work in accord with this section, you must place, in the relevant source files, a statement of the additional terms that apply to those files, or a notice indicating where to find the applicable terms.

Additional terms, permissive or non-permissive, may be stated in the form of a separately written license, or stated as exceptions; the above requirements apply either way.

8. Termination.

You may not propagate or modify a covered work except as expressly provided under this License. Any attempt otherwise to propagate or modify it is void, and will automatically terminate your rights under this License (including any patent licenses granted under the third paragraph of section 11).

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, you do not qualify to receive new licenses for the same material under section 10.

9. Acceptance Not Required for Having Copies.

You are not required to accept this License in order to receive or run a copy of the Program. Ancillary propagation of a covered work occurring solely as a consequence of using peer-to-peer transmission to receive a copy likewise does not require acceptance. However, nothing other than this License grants you permission to propagate or modify any covered work. These actions infringe copyright if you do not accept this License. Therefore, by modifying or propagating a covered work, you indicate your acceptance of this License to do so.

10. Automatic Licensing of Downstream Recipients.

Each time you convey a covered work, the recipient automatically receives a license from the original licensors, to run, modify and propagate that work, subject to this License. You are not responsible for enforcing compliance by third parties with this License.

An "entity transaction" is a transaction transferring control of an organization, or substantially all assets of one, or subdividing an organization, or merging organizations. If propagation of a covered work results from an entity transaction, each party to that transaction who receives a copy of the work also receives whatever licenses to the work the party's predecessor in interest had or could give under the previous paragraph, plus a right to possession of the Corresponding Source of the work from the predecessor in interest, if the predecessor has it or can get it with reasonable efforts.

You may not impose any further restrictions on the exercise of the rights granted or affirmed under this License. For example, you may not impose a license fee, royalty, or other charge for exercise of rights granted under this License, and you may not initiate litigation (including a cross-claim or counterclaim in a lawsuit) alleging that any patent claim is infringed by making, using, selling, offering for sale, or importing the Program or any portion of it.

11. Patents.

A "contributor" is a copyright holder who authorizes use under this

License of the Program or a work on which the Program is based. The work thus licensed is called the contributor's "contributor version".

A contributor's "essential patent claims" are all patent claims owned or controlled by the contributor, whether already acquired or hereafter acquired, that would be infringed by some manner, permitted by this License, of making, using, or selling its contributor version, but do not include claims that would be infringed only as a consequence of further modification of the contributor version. For purposes of this definition, "control" includes the right to grant patent sublicenses in a manner consistent with the requirements of this License.

Each contributor grants you a non-exclusive, worldwide, royalty-free patent license under the contributor's essential patent claims, to make, use, sell, offer for sale, import and otherwise run, modify and propagate the contents of its contributor version.

In the following three paragraphs, a "patent license" is any express agreement or commitment, however denominated, not to enforce a patent (such as an express permission to practice a patent or covenant not to sue for patent infringement). To "grant" such a patent license to a party means to make such an agreement or commitment not to enforce a patent against the party.

If you convey a covered work, knowingly relying on a patent license, and the Corresponding Source of the work is not available for anyone to copy, free of charge and under the terms of this License, through a publicly available network server or other readily accessible means, then you must either (1) cause the Corresponding Source to be so available, or (2) arrange to deprive yourself of the benefit of the patent license for this particular work, or (3) arrange, in a manner consistent with the requirements of this License, to extend the patent license to downstream recipients. "Knowingly relying" means you have actual knowledge that, but for the patent license, your conveying the covered work in a country, or your recipient's use of the covered work in a country, would infringe one or more identifiable patents in that country that you have reason to believe are valid.

If, pursuant to or in connection with a single transaction or arrangement, you convey, or propagate by procuring conveyance of, a covered work, and grant a patent license to some of the parties receiving the covered work authorizing them to use, propagate, modify or convey a specific copy of the covered work, then the patent license you grant is automatically extended to all recipients of the covered work and works based on it.

A patent license is "discriminatory" if it does not include within the scope of its coverage, prohibits the exercise of, or is conditioned on the non-exercise of one or more of the rights that are specifically granted under this License. You may not convey a covered work if you are a party to an arrangement with a third party that is in the business of distributing software, under which you make payment to the third party based on the extent of your activity of conveying the work, and under which the third party grants, to any of the parties who would receive the covered work from you, a discriminatory patent license (a) in connection with copies of the covered work conveyed by you (or copies made from those copies), or (b) primarily for and in connection with specific products or compilations that contain the covered work, unless you entered into that arrangement, or that patent license was granted, prior to 28 March 2007.

Nothing in this License shall be construed as excluding or limiting any implied license or other defenses to infringement that may otherwise be available to you under applicable patent law.

12. No Surrender of Others' Freedom.

If conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot convey a covered work so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not convey it at all. For example, if you agree to terms that obligate you to collect a royalty for further conveying from those to whom you convey the Program, the only way you could satisfy both those terms and this License would be to refrain entirely from conveying the Program.

13. Use with the GNU Affero General Public License.

Notwithstanding any other provision of this License, you have permission to link or combine any covered work with a work licensed under version 3 of the GNU Affero General Public License into a single combined work, and to convey the resulting work. The terms of this License will continue to apply to the part which is the covered work, but the special requirements of the GNU Affero General Public License, section 13, concerning interaction through a network will apply to the combination as such.

14. Revised Versions of this License.

The Free Software Foundation may publish revised and/or new versions of

the GNU General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Program specifies that a certain numbered version of the GNU General Public License "or any later version" applies to it, you have the option of following the terms and conditions either of that numbered version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of the GNU General Public License, you may choose any version ever published by the Free Software Foundation.

If the Program specifies that a proxy can decide which future versions of the GNU General Public License can be used, that proxy's public statement of acceptance of a version permanently authorizes you to choose that version for the Program.

Later license versions may give you additional or different permissions. However, no additional obligations are imposed on any author or copyright holder as a result of your choosing to follow a later version.

15. Disclaimer of Warranty.

THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

16. Limitation of Liability.

IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MODIFIES AND/OR CONVEYS THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

17. Interpretation of Sections 15 and 16.

If the disclaimer of warranty and limitation of liability provided above cannot be given local legal effect according to their terms, reviewing courts shall apply local law that most closely approximates an absolute waiver of all civil liability in connection with the Program, unless a warranty or assumption of liability accompanies a copy of the Program in return for a fee.

END OF TERMS AND CONDITIONS

How to Apply These Terms to Your New Programs

If you develop a new program, and you want it to be of the greatest possible use to the public, the best way to achieve this is to make it free software which everyone can redistribute and change under these terms.

To do so, attach the following notices to the program. It is safest to attach them to the start of each source file to most effectively state the exclusion of warranty; and each file should have at least the "copyright" line and a pointer to where the full notice is found.

```
<one line to give the program's name and a brief idea of what it does.>
Copyright (C) <year> <name of author>
```

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see [<https://www.gnu.org/licenses/>](https://www.gnu.org/licenses/).

Also add information on how to contact you by electronic and paper mail.

If the program does terminal interaction, make it output a short notice like this when it starts in an interactive mode:

```
<program> Copyright (C) <year> <name of author>
This program comes with ABSOLUTELY NO WARRANTY; for details type `show w'.
This is free software, and you are welcome to redistribute it
```

under certain conditions; type ``show c'` for details.

The hypothetical commands ``show w'` and ``show c'` should show the appropriate parts of the General Public License. Of course, your program's commands might be different; for a GUI interface, you would use an "about box".

You should also get your employer (if you work as a programmer) or school, if any, to sign a "copyright disclaimer" for the program, if necessary. For more information on this, and how to apply and follow the GNU GPL, see <https://www.gnu.org/licenses/>.

The GNU General Public License does not permit incorporating your program into proprietary programs. If your program is a subroutine library, you may consider it more useful to permit linking proprietary applications with the library. If this is what you want to do, use the GNU Lesser General Public License instead of this License. But first, please read <https://www.gnu.org/philosophy/why-not-lgpl.html>.

File: org.github.kai66673.iide.json

```
{
  "id" : "org.github.kai66673.iide",
  "runtime" : "org.gnome.Platform",
  "runtime-version" : "master",
  "sdk" : "org.gnome.Sdk",
  "sdk-extensions" : [
    "org.freedesktop.Sdk.Extension.vala"
  ],
  "command" : "iide",
  "finish-args" : [
    "--share=network",
    "--share=ipc",
    "--socket=fallback-x11",
    "--device=dri",
    "--socket=wayland"
  ],
  "build-options" : {
    "append-path" : "/usr/lib/sdk/vala/bin",
    "prepend-ld-library-path" : "/usr/lib/sdk/vala/lib"
  },
  "cleanup" : [
    "/include",
    "/lib/pkgconfig",
    "/man",
    "/share/doc",
  ]
}
```

```

        "/share/gtk-doc",
        "/share/man",
        "/share/pkgconfig",
        "/share/vala",
        "*.la",
        "*.a"
    ],
    "modules" : [
        {
            "name" : "iide",
            "builddir" : true,
            "buildsystem" : "meson",
            "sources" : [
                {
                    "type" : "git",
                    "url" : "file:///home/kai/Projects"
                }
            ]
        }
    ]
}

```

File: README.md

```
# iide
```

A description of this project.

File: data/Adwaita-dark.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<!--
```

Copyright 2020 Christian Hergert <christian@hergert.me>

GtkSourceView is free software; you can redistribute it and/or modify it under the terms of the GNU Lesser General Public License as published by the Free Software Foundation; either version 2.1 of the License, or (at your option) any later version.

GtkSourceView is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Lesser General Public License for more details.

You should have received a copy of the GNU Lesser General Public License along with this library; if not, see <<http://www.gnu.org/licenses/>>.

```
-->
<style-scheme id="Adwaita-dark" _name="Adwaita Dark" version="1.0"><author
>Christian Hergert</author><_description
>An style scheme for Adwaita</_description><metadata><property
  name="variant"
>dark</property><property
  name="light-variant"
>Adwaita</property></metadata><!-- Named Colors --><color
name="blue_1"
value="#99C1F1"
/><color name="blue_2" value="#62A0EA" /><color
name="blue_3"
value="#28ef5a"
/><color name="blue_4" value="#1C71D8" /><color
name="blue_5"
value="#1A5FB4"
/><color name="blue_6" value="#1B497E" /><color
name="blue_7"
value="#193D66"
/><color name="brown_1" value="#CDAB8F" /><color
name="brown_2"
value="#B5835A"
/><color name="brown_3" value="#986A44" /><color
name="brown_4"
value="#865E3C"
/><color name="brown_5" value="#63452C" /><color
name="chameleon_3"
value="#4E9A06"
/><color name="dark_1" value="#777777" /><color
name="dark_2"
value="#5E5E5E"
/><color name="dark_3" value="#505050" /><color
name="dark_4"
value="#3D3D3D"
/><color name="dark_5" value="#242424" /><color
name="dark_6"
value="#121212"
/><color name="dark_7" value="#000000" /><color
name="green_1"
value="#8FF0A4"
/><color name="green_2" value="#57E389" /><color
name="green_3"
value="#33D17A"
```

```

/><color name="green_4" value="#2EC27E" /><color
    name="green_5"
    value="#26A269"
/><color name="green_6" value="#1F7F56" /><color
    name="green_7"
    value="#1C6849"
/><color name="libadwaita-dark" value="#1d1d20" /><color
    name="libadwaita-dark-alt"
    value="#242428"
/><color name="light_1" value="FFFFFF" /><color
    name="light_2"
    value="FCFCFC"
/><color name="light_3" value="F6F5F4" /><color
    name="light_4"
    value="DEDDDA"
/><color name="light_5" value="COBFBC" /><color
    name="light_6"
    value="BOAFAC"
/><color name="light_7" value="9A9996" /><color
    name="orange_1"
    value="FFBE6F"
/><color name="orange_2" value="FFA348" /><color
    name="orange_3"
    value="FF7800"
/><color name="orange_4" value="E66100" /><color
    name="orange_5"
    value="C64600"
/><color name="purple_1" value="DC8ADD" /><color
    name="purple_2"
    value="C061CB"
/><color name="purple_3" value="9141AC" /><color
    name="purple_4"
    value="813D9C"
/><color name="purple_5" value="613583" /><color
    name="red_1"
    value="F66151"
/><color name="red_2" value="ED333B" /><color
    name="red_3"
    value="E01B24"
/><color name="red_4" value="C01C28" /><color
    name="red_5"
    value="A51D2D"
/><color name="teal_1" value="93DDC2" /><color
    name="teal_2"
    value="5BC8AF"
/><color name="teal_3" value="33B2A4" /><color

```

```

        name="teal_4"
        value="#26A1A2"
/><color name="teal_5" value="#218787" /><color
        name="violet_2"
        value="#7D8AC7"
/><color name="violet_3" value="#6362C8" /><color
        name="violet_4"
        value="#4E57BA"
/><color name="yellow_1" value="#F9F06B" /><color
        name="yellow_2"
        value="#F8E45C"
/><color name="yellow_3" value="#F6D32D" /><color
        name="yellow_4"
        value="#F5C211"
/><color name="yellow_5" value="#E5A50A" /><color
        name="yellow_6"
        value="#D38B09"
/><!-- Global Styles --><style
        name="background-pattern"
        background="#141414"
/><style name="bracket-match" bold="true" /><style
        name="current-line"
        background="libadwaita-dark-alt"
/><style
        name="current-line-number"
        background="libadwaita-dark-alt"
        foreground="dark_1"
/><style name="cursor" foreground="light_5" /><style
        name="draw-spaces"
        foreground="dark_3"
/><style
        name="line-numbers"
        background="libadwaita-dark"
        foreground="dark_2"
/><style name="map-overlay" background="dark_1" /><style
        name="right-margin"
        background="dark_1"
        foreground="dark_1"
/><style
        name="search-match"
        background="#rgba(246,211,45,.5)"
        foreground="dark_5"
/><style
        name="text"
        background="libadwaita-dark"
        foreground="light_5"

```

```

/><!-- Defaults --><style
    name="def:base-n-integer"
    foreground="violet_2"
/><style name="def:boolean" foreground="violet_2" /><style
    name="def:comment"
    foreground="dark_1"
/><style name="def:constant" foreground="violet_2" /><style
    name="def:decimal"
    foreground="violet_2"
/><style name="def:deletion" strikethrough="true" /><style
    name="def:doc-comment-element"
    foreground="light_7"
/><style name="def:emphasis" italic="true" /><style
    name="def:error"
    underline="error"
    underline-color="red_4"
/><style name="def:floating-point" foreground="violet_2" /><style
    name="def:function"
    foreground="blue_2"
/><style name="def:heading" foreground="teal_3" bold="true" /><style
    name="def:identifier"
    foreground="chameleon_3"
/><style name="def:inline-code" foreground="violet_2" /><style
    name="def:link-destination"
    foreground="blue_2"
    italic="true"
    underline="low"
/><style name="def:link-text" foreground="red_2" /><style
    name="def:list-marker"
    foreground="orange_4"
    bold="true"
/><style name="def:net-address" foreground="blue_2" underline="low" /><style
    name="def:note"
    foreground="dark_4"
    background="yellow_4"
    bold="true"
/><style name="def:number" foreground="violet_2" /><style
    name="def:preformatted-section"
    foreground="violet_2"
/><style name="def:preprocessor" foreground="orange_4" /><style
    name="def:shebang"
    foreground="light_7"
    bold="true"
/><style name="def:special-char" foreground="red_1" bold="false" /><style
    name="def:statement"
    foreground="orange_2"

```

```

        bold="true"
/><style name="def:string" foreground="teal_2" /><style
    name="def:strong-emphasis"
    bold="true"
/><style name="def:type" foreground="teal_2" bold="true" /><style
    name="def:underlined"
    underline="single"
/><style
    name="def:warning"
    underline="error"
    underline-color="yellow_4"
/><!-- C# --><style name="c-sharp:format" foreground="violet_4" /><style
    name="c-sharp:preprocessor"
    foreground="dark_2"
/><!-- C --><style name="c:printf" foreground="violet_2" /><style
    name="c:signal-name"
    foreground="red_1"
/><style name="c:storage-class" foreground="teal_2" bold="true" /><style
    name="c:type-keyword"
    foreground="teal_2"
    bold="true"
/><!-- CSS --><style
    name="css:id-selector"
    foreground="teal_3"
    bold="true"
/><style name="css:property-name" foreground="orange_3" /><style
    name="css:pseudo-selector"
    foreground="violet_2"
    bold="true"
/><style
    name="css:selector-symbol"
    foreground="orange_3"
    bold="true"
/><style name="css:type-selector" foreground="teal_3" bold="true" /><style
    name="css:vendor-specific"
    foreground="yellow_5"
/><!-- Diff --><style name="diff:added-line" foreground="teal_3" /><style
    name="diff:changed-line"
    foreground="orange_3"
/><style name="diff:diff-file" foreground="violet_2" /><style
    name="diff:location"
    foreground="yellow_4"
/><style name="diff:removed-line" foreground="red_1" /><!-- Go --><style
    name="go:printf"
    foreground="violet_4"
/><!-- Python 2 --><style

```

```

        name="python:builtin-function"
        foreground="blue_2"
/><style name="python:class-name" foreground="teal_2" bold="true" /><style
name="python:module-handler"
foreground="red_1"
/><!-- Rust --><style name="rust:attribute" foreground="violet_2" /><style
name="rust:lifetime"
foreground="orange_2"
bold="false"
italic="false"
/><style name="rust:macro" foreground="violet_2" bold="false" /><style
name="rust:scope"
foreground="orange_2"
/><!-- Vala --><style
name="vala:attributes"
foreground="light_5"
bold="false"
/><!-- XML --><style
name="xml:attribute-name"
foreground="orange_3"
/><style name="xml:attribute-value" foreground="violet_2" /><style
name="xml:element-name"
foreground="teal_3"
/><style name="xml:namespace" foreground="yellow_4" /><style
name="xml:processing-instruction"
foreground="yellow_4"
bold="true"
/></style-scheme>

```

File: data/Adwaita.xml

```

<?xml version="1.0" encoding="UTF-8" ?>
<!--

```

Copyright 2020 Christian Hergert <christian@hergert.me>

GtkSourceView is free software; you can redistribute it and/or modify it under the terms of the GNU Lesser General Public License as published by the Free Software Foundation; either version 2.1 of the License, or (at your option) any later version.

GtkSourceView is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Lesser General Public License for more details.

You should have received a copy of the GNU Lesser General Public License along with this library; if not, see <<http://www.gnu.org/licenses/>>.

```
-->
<style-scheme id="Adwaita" _name="Adwaita" version="1.0"><author
>Christian Hergert</author><_description
>An style scheme for Adwaita</_description><metadata><property
  name="variant"
  >light</property><property
    name="dark-variant"
    >Adwaita-dark</property></metadata><!-- Named Colors --><color
  name="blue_1"
  value="#99C1F1"
/><color name="blue_2" value="#62A0EA" /><color
  name="blue_3"
  value="#09203c"
/><color name="blue_4" value="#1C71D8" /><color
  name="blue_5"
  value="#1A5FB4"
/><color name="blue_6" value="#1B497E" /><color
  name="blue_7"
  value="#193D66"
/><color name="brown_1" value="#CDAB8F" /><color
  name="brown_2"
  value="#B5835A"
/><color name="brown_3" value="#986A44" /><color
  name="brown_4"
  value="#865E3C"
/><color name="brown_5" value="#63452C" /><color
  name="chameleon_3"
  value="#4E9A06"
/><color name="dark_1" value="#77767B" /><color
  name="dark_2"
  value="#5E5C64"
/><color name="dark_3" value="#504E55" /><color
  name="dark_4"
  value="#3D3846"
/><color name="dark_5" value="#241F31" /><color
  name="dark_6"
  value="#000000"
/><color name="green_1" value="#8FF0A4" /><color
  name="green_2"
  value="#57E389"
/><color name="green_3" value="#33D17A" /><color
  name="green_4"
```

```

        value="#2EC27E"
/><color name="green_5" value="#26A269" /><color
name="green_6"
value="#1F7F56"
/><color name="green_7" value="#1C6849" /><color
name="light_1"
value="#FFFFFF"
/><color name="light_2" value="#FCFCFC" /><color
name="light_3"
value="#F6F5F4"
/><color name="light_4" value="#DEDDDA" /><color
name="light_5"
value="#C0BFBC"
/><color name="light_6" value="#BOAFAC" /><color
name="light_7"
value="#9A9996"
/><color name="orange_1" value="#FFBE6F" /><color
name="orange_2"
value="#FFA348"
/><color name="orange_3" value="#FF7800" /><color
name="orange_4"
value="#E66100"
/><color name="orange_5" value="#C64600" /><color
name="purple_1"
value="#DC8ADD"
/><color name="purple_2" value="#C061CB" /><color
name="purple_3"
value="#9141AC"
/><color name="purple_4" value="#813D9C" /><color
name="purple_5"
value="#613583"
/><color name="red_1" value="#F66151" /><color
name="red_2"
value="#ED333B"
/><color name="red_3" value="#E01B24" /><color
name="red_4"
value="#C01C28"
/><color name="red_5" value="#A51D2D" /><color
name="teal_1"
value="#93DDC2"
/><color name="teal_2" value="#5BC8AF" /><color
name="teal_3"
value="#33B2A4"
/><color name="teal_4" value="#26A1A2" /><color
name="teal_5"
value="#218787"

```



```

/><color name="violet_2" value="#7D8AC7" /><color
    name="violet_3"
    value="#6362C8"
/><color name="violet_4" value="#4E57BA" /><color
    name="yellow_1"
    value="#F9F06B"
/><color name="yellow_2" value="#F8E45C" /><color
    name="yellow_3"
    value="#F6D32D"
/><color name="yellow_4" value="#F5C211" /><color
    name="yellow_5"
    value="#E5A50A"
/><color name="yellow_6" value="#D38B09" /><!-- Global Styles --><style
    name="background-pattern"
    background="#FAFAFA"
/><style name="bracket-match" bold="true" /><style
    name="current-line"
    background="light_3"
/><style
    name="current-line-number"
    background="light_3"
    foreground="light_7"
/><style name="cursor" foreground="dark_1" /><style
    name="draw-spaces"
    foreground="light_6"
/><style
    name="line-numbers"
    background="light_1"
    foreground="light_6"
/><style name="map-overlay" background="dark_1" /><style
    name="right-margin"
    background="dark_1"
    foreground="dark_1"
/><style
    name="search-match"
    background="yellow_2"
    foreground="dark_4"
/><style
    name="text"
    background="light_1"
    foreground="dark_3"
/><!-- Defaults --><style
    name="def:base-n-integer"
    foreground="violet_4"
/><style name="def:boolean" foreground="violet_4" /><style
    name="def:comment"

```

```

        foreground="dark_1"
/><style name="def:constant" foreground="violet_4" /><style
    name="def:decimal"
    foreground="violet_4"
/><style name="def:deletion" strikethrough="true" /><style
    name="def:doc-comment-element"
    foreground="dark_3"
/><style name="def:emphasis" italic="true" /><style
    name="def:error"
    underline="error"
    underline-color="red_4"
/><style name="def:floating-point" foreground="violet_4" /><style
    name="def:function"
    foreground="blue_4"
/><style name="def:heading" foreground="teal_5" bold="true" /><style
    name="def:identifier"
    foreground="chameleon_3"
/><style name="def:inline-code" foreground="violet_4" /><style
    name="def:link-destination"
    foreground="blue_3"
    italic="true"
    underline="low"
/><style name="def:link-text" foreground="red_3" /><style
    name="def:list-marker"
    foreground="orange_5"
    bold="true"
/><style name="def:net-address" foreground="blue_3" underline="low" /><style
    name="def:note"
    foreground="dark_4"
    background="#FCF7B5"
    bold="true"
/><style name="def:number" foreground="violet_4" /><style
    name="def:preformatted-section"
    foreground="violet_4"
/><style name="def:preprocessor" foreground="orange_5" /><style
    name="def:shebang"
    foreground="dark_1"
    bold="true"
/><style name="def:special-char" foreground="red_2" bold="false" /><style
    name="def:statement"
    foreground="orange_5"
    bold="true"
/><style name="def:string" foreground="teal_5" /><style
    name="def:strong-emphasis"
    bold="true"
/><style name="def:type" foreground="teal_5" bold="true" /><style

```

```

        name="def:underlined"
        underline="single"
/><style
    name="def:warning"
    underline="error"
    underline-color="yellow_4"
/><!-- C# --><style name="c-sharp:format" foreground="violet_4" /><style
    name="c-sharp:preprocessor"
    foreground="dark_2"
/><!-- C --><style name="c:printf" foreground="violet_4" /><style
    name="c:signal-name"
    foreground="red_4"
/><style name="c:storage-class" foreground="teal_5" bold="true" /><style
    name="c:type-keyword"
    foreground="teal_5"
    bold="true"
/><!-- CSS --><style
    name="css:id-selector"
    foreground="teal_5"
    bold="true"
/><style name="css:property-name" foreground="orange_5" /><style
    name="css:pseudo-selector"
    foreground="violet_4"
    bold="true"
/><style
    name="css:selector-symbol"
    foreground="orange_5"
    bold="true"
/><style name="css:type-selector" foreground="teal_5" bold="true" /><style
    name="css:vendor-specific"
    foreground="yellow_6"
/><!-- Diff --><style name="diff:added-line" foreground="teal_4" /><style
    name="diff:changed-line"
    foreground="orange_4"
/><style name="diff:diff-file" foreground="violet_4" /><style
    name="diff:location"
    foreground="yellow_6"
/><style name="diff:removed-line" foreground="red_1" /><!-- Go --><style
    name="go:printf"
    foreground="violet_4"
/><!-- Python 2 --><style
    name="python:builtin-function"
    foreground="blue_4"
/><style name="python:class-name" foreground="teal_5" bold="true" /><style
    name="python:module-handler"
    foreground="red_3"

```

```

/><!-- Rust --><style name="rust:attribute" foreground="violet_4" /><style
  name="rust:lifetime"
  foreground="orange_5"
  bold="false"
  italic="false"
/><style name="rust:macro" foreground="violet_4" bold="false" /><style
  name="rust:scope"
  foreground="orange_5"
/><!-- Vala --><style
  name="vala:attributes"
  foreground="dark_2"
  bold="false"
/><!-- XML --><style
  name="xml:attribute-name"
  foreground="orange_5"
/><style name="xml:attribute-value" foreground="violet_4" /><style
  name="xml:element-name"
  foreground="teal_5"
/><style name="xml:namespace" foreground="yellow_6" /><style
  name="xml:processing-instruction"
  foreground="yellow_6"
  bold="true"
/></style-scheme>

```

File: data/org.github.kai66673.iide.gschema.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<schemalist gettext-domain="iide">
  <schema id="org.github.kai66673.iide" path="/org/github/kai66673/iide/">
    <key name="color-scheme" type="s">
      <default>"system"</default>
      <summary>Color scheme</summary>
      <description>Color scheme: light, dark, or system</description>
    </key>

    <key name="editor-font-size" type="d">
      <default>6</default>
      <summary>Editor zoom level</summary>
      <description>Editor zoom level (1-15)</description>
    </key>

    <key name="show-minimap" type="b">
      <default>true</default>
      <summary>Show minimap</summary>
      <description>Whether to show the minimap in text editors</description>
    </key>
  </schema>
</schemalist>

```

```

</key>

<key name="show-line-numbers" type="b">
  <default>true</default>
  <summary>Show line numbers</summary>
  <description>Whether to show line numbers in text editors</description>
</key>

<key name="show-diagnostics-marks" type="b">
  <default>true</default>
  <summary>Show diagnostics marks</summary>
  <description>Whether to show diagnostics marks in text editors</description>
</key>

<key name="show-folding-gutter" type="b">
  <default>true</default>
  <summary>Show folding gutter</summary>
  <description>Whether to show folding gutter in text editors</description>
</key>

<key name="highlight-current-line" type="b">
  <default>true</default>
  <summary>Highlight current line</summary>
  <description>Whether to highlight the current line in text editors</description>
</key>

<key name="auto-indent" type="b">
  <default>true</default>
  <summary>Auto indent</summary>
  <description>Whether to enable auto indentation</description>
</key>

<key name="panel-start-width" type="d">
  <default>250</default>
  <summary>Start panel width</summary>
  <description>Width of the start panel (file tree)</description>
</key>

<key name="panel-end-width" type="d">
  <default>250</default>
  <summary>End panel width</summary>
  <description>Width of the end panel</description>
</key>

<key name="panel-bottom-height" type="d">
  <default>200</default>

```

```

    <summary>Bottom panel height</summary>
    <description>Height of the bottom panel (terminal)</description>
</key>

<key name="panel-bottom-width" type="d">
    <default>500</default>
    <summary>Bottom panel width</summary>
    <description>Width of the bottom panel split between terminal and log</description>
</key>

<key name="reveal-start-panel" type="b">
    <default>true</default>
    <summary>Reveal start panel</summary>
    <description>Whether the start panel is visible</description>
</key>

<key name="reveal-end-panel" type="b">
    <default>false</default>
    <summary>Reveal end panel</summary>
    <description>Whether the end panel is visible</description>
</key>

<key name="reveal-bottom-panel" type="b">
    <default>false</default>
    <summary>Reveal bottom panel</summary>
    <description>Whether the bottom panel is visible</description>
</key>

<key name="recent-projects" type="as">
    <default>[]</default>
    <summary>Recent projects</summary>
    <description>List of recently opened project paths</description>
</key>

<key name="max-recent-projects" type="d">
    <default>10</default>
    <summary>Maximum recent projects</summary>
    <description>Maximum number of recent projects to remember</description>
</key>

<key name="last-open-directory" type="s">
    <default>""</default>
    <summary>Last open directory</summary>
    <description>Last directory used for opening files or projects</description>
</key>

```

```

<key name="current-project-path" type="s">
  <default>""</default>
  <summary>Current project path</summary>
  <description>Path to the currently open project</description>
</key>

<key name="open-documents" type="as">
  <default>[]</default>
  <summary>Open documents</summary>
  <description>List of currently open document URIs</description>
</key>

<key name="panel-layout" type="s">
  <default>""</default>
  <summary>Panel layout</summary>
  <description>JSON string describing panel dock layout</description>
</key>

<key name="grid-layout" type="s">
  <default>""</default>
  <summary>Grid layout</summary>
  <description>JSON string describing grid layout</description>
</key>

<key name="window-width" type="d">
  <default>1200</default>
  <summary>Window width</summary>
  <description>Width of the main window</description>
</key>

<key name="window-height" type="d">
  <default>800</default>
  <summary>Window height</summary>
  <description>Height of the main window</description>
</key>

<key name="window-maximized" type="b">
  <default>false</default>
  <summary>Window maximized</summary>
  <description>Whether the window is maximized</description>
</key>

<key name="shortcuts" type="s">
  <default>""</default>
  <summary>Custom shortcuts</summary>
  <description>JSON string mapping action IDs to keyboard shortcuts</description>

```

```

        </key>
    </schema>
</schemalist>

```

File: scripts/gen-docs.sh

```

#!/bin/bash

#
#
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"

#
# (scripts)
cd "$SCRIPT_DIR/.." || exit

echo "          Doxygen: $(pwd)"

# 1.
doxygen Doxyfile

# 2.
if [ $? -eq 0 ]; then
    echo "          ."

    #      HTML (
    #      Doxyfile OUTPUT_DIRECTORY = build/docs,
    DOC_PATH="./build/docs/html"

    if [ -d "$DOC_PATH" ]; then
        echo "          ..."
        xdg-open "http://localhost:8080/index.html" &
        cd "$DOC_PATH" && python3 -m http.server 8080
    else
        echo "          :          $DOC_PATH          .          OUTPUT_DIRECTORY Doxyfile."
    fi
else
    echo "          Doxygen."
    exit 1
fi

```

File: src/Services/Actions/AppAction.vala

```

/*
*/

public abstract class Iide.AppAction : Object {

```



```

private string? _current_shortcut = null;

public abstract string id { get; }
public abstract string name { get; }
public abstract string? description { get; }
public abstract string? icon_name { get; }
public virtual string? category { get; default = null; }

public abstract string? default_shortcut { get; default = null; }
public string? shortcut { get { return _current_shortcut ?? default_shortcut; } }
public string? current_shortcut { get { return _current_shortcut; } }
public bool is_default_shortcut { get { return _current_shortcut == null || _current_sho

public signal void shortcut_changed (string? new_shortcut);
public signal void state_changed (bool new_state);

public virtual bool is_toggle { get; default = false; }
public virtual bool state { get; protected set; default = false; }

public void set_shortcut(string? new_shortcut) {
    _current_shortcut = new_shortcut;
    shortcut_changed(shortcut);
}

public void update_state (bool new_state) {
    state = new_state;
    state_changed (new_state);
}

public virtual bool can_execute () { return true; }
public virtual void execute () {}
}

```

File: src/Services/LSP/config/PythonLspConfig.vala

```

public class Iide.PythonLspConfig : Iide.LspConfig {
    public override string command () {
        return "basedpyright-langserver";
    }

    public override string[] args () {
        return { "--stdio" };
    }

    public override Json.Node initialize_params (string? workspace_root, string? initial_ur

```

```

var root_uri = workspace_root ?? "/";
var params = ""{
    "processId": null,
    "clientInfo": { "name": "iide", "version": "0.1.0" },
    "rootUri": "%s",
    "workspaceFolders": [
        { "uri": "%s", "name": "project" }
    ],
    "capabilities": {
        "textDocument": {
            "publishDiagnostics": { "relatedInformation": true },
            "documentSymbol": {
                "hierarchicalDocumentSymbolSupport": true,
                "symbolKind": {
                    "valueSet": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,
                }
            }
        },
        "workspace": {
            "configuration": true
        },
        "window": { "workDoneProgress": true }
    },
    "initializationOptions": {
        "python": {},
        "settings": {
            "pyright": { "analysis": { "diagnosticMode": "workspace" } },
            "basedpyright": { "analysis": { "diagnosticMode": "workspace" } }
        }
    }
}""".printf (root_uri, root_uri);
var parser = new Json.Parser ();
try {
    parser.load_from_data (params);
} catch (GLib.Error e) {
    return new Json.Node (Json.NodeType.NULL);
}
return parser.get_root ();
}

public override Json.Node? server_response_result (Json.Object response) {
    string method = response.get_string_member ("method");
    if (method != "workspace/configuration") {
        return new Json.Node (Json.NodeType.NULL);
    }
}

```

```

var items = response.get_object_member ("params").get_array_member ("items");
var results = new Json.Array ();

foreach (var item in items.get_elements ()) {
    var section = item.get_object ().get_string_member ("section");
    var settings = new Json.Object ();

    if (section == "basedpyright.analysis" || section == "python.analysis") {
        //          'analysis',
        settings.set_string_member ("diagnosticMode", "workspace");
    } else if (section == "basedpyright" || section == "pyright" || section == "pyth
        //          ,          'analysis'
        var analysis_obj = new Json.Object ();
        analysis_obj.set_string_member ("diagnosticMode", "workspace");
        settings.set_object_member ("analysis", analysis_obj);
    } else {
        //          ,
        settings = new Json.Object ();
    }

    results.add_object_element (settings);
}

var results_node = new Json.Node (Json.NodeType.ARRAY);
results_node.set_array (results);
return results_node;
}
}

```

File: src/Services/LSP/LspDiagnosticsMark.vala

```

public class Iide.LspDiagnosticsMark : GtkSource.Mark {
    public int severity { get; construct set; }
    public string diagnostic_message { get; construct set; }

    public LspDiagnosticsMark(string lsp_category, int lsp_severity, string lsp_message) {
        Object(name: null,
            category: lsp_category,
            severity: lsp_severity,
            diagnostic_message: lsp_message);
    }

    private static string category_from_severity(int severity) {
        switch (severity) {
            case 1:

```

```

        return "lsp_error";
    case 2:
        return "lsp_warning";
    case 3:
    case 4:
        return "lsp_info";
    }
    return "lsp_error";
}

public LspDiagnosticsMark.from_lsp_diagnostic(LspDiagnostic diagnostics) {
    Object(name: null,
        category: category_from_severity(diagnostics.severity),
        severity: diagnostics.severity,
        diagnostic_message: diagnostics.message);
}

public static void set_mark_attributes(GtkSource.View text_view) {
    var error_mark_attrs = new GtkSource.MarkAttributes();
    var err_bg = Gdk.RGBA();
    err_bg.parse("#e01b2430");
    error_mark_attrs.set_background(err_bg);
    text_view.set_mark_attributes("lsp_error", error_mark_attrs, 100);

    var warning_mark_attrs = new GtkSource.MarkAttributes();
    var warn_bg = Gdk.RGBA();
    warn_bg.parse("#f5c21130");
    warning_mark_attrs.set_background(warn_bg);
    text_view.set_mark_attributes("lsp_warning", warning_mark_attrs, 90);

    var info_mark_attrs = new GtkSource.MarkAttributes();
    text_view.set_mark_attributes("lsp_info", info_mark_attrs, 80);
}

public static void clear_mark_attributes(GtkSource.View text_view) {
    var buffer = (GtkSource.Buffer) text_view.buffer;
    Gtk.TextIter start, end;
    buffer.get_start_iter(out start);
    buffer.get_end_iter(out end);

    buffer.remove_source_marks(start, end, "lsp_error");
    buffer.remove_source_marks(start, end, "lsp_warning");
    buffer.remove_source_marks(start, end, "lsp_info");
}

public string ? get_icon_name() {

```

```

        switch (severity) {
        case 1:
            return "dialog-error";
        case 2:
            return "dialog-warning";
        case 3:
        case 4:
            return "dialog-information";
        }
        return null;
    }
}

```

File: src/Services/TreeSitter/cpp/CppHighlighter.vala

```

[CCode (cname = "tree_sitter_cpp")]
extern unowned TreeSitter.Language ? get_lang_cpp ();

public class Ide.CppHighlighter : BaseTreeSitterHighlighter {
    public CppHighlighter (SourceView view) {
        base (view);
    }

    protected override unowned TreeSitter.Language get_ts_language () {
        return get_lang_cpp ();
    }

    protected override string query_source () {
        return ""
        ; Functions

        (call_expression
         function: (qualified_identifier
                   name: (identifier) @function))

        (template_function
         name: (identifier) @function)

        (template_method
         name: (field_identifier) @function)

        (template_function
         name: (identifier) @function)

        (function_declarator

```

```

    declarator: (qualified_identifier
                name: (identifier) @function))

(function_declarator
  declarator: (field_identifier) @function)

; Types

(namespace_identifier) @type
(#match? @type "^[A-Z]")

(auto) @type

; Constants

(this) @variable.builtin
(null "nullptr" @constant)

; Modules

(module_name
  (identifier) @module)

; Keywords

[
  "catch"
  "class"
  "co_await"
  "co_return"
  "co_yield"
  "constexpr"
  "constinit"
  "constexpr"
  "delete"
  "explicit"
  "final"
  "friend"
  "mutable"
  "namespace"
  "noexcept"
  "new"
  "override"
  "private"
  "protected"
  "public"
  "template"

```

```

        "throw"
        "try"
        "typename"
        "using"
        "concept"
        "requires"
        "virtual"
        "import"
        "export"
        "module"
    ] @keyword

    ; Strings

    (raw_string_literal) @string
    """;
}

protected override bool is_container_node (string node_type) {
    return node_type in new string[] {
        "class_specifier", "struct_specifier", "function_definition", "namespace_
    };
}
}
}

```

File: src/Services/TreeSitter/rust/RustHighlighter.vala

```

[CCode (cname = "tree_sitter_rust")]
extern unowned TreeSitter.Language ? get_language_rust ();

public class Iide.RustHighlighter : BaseTreeSitterHighlighter {
    public RustHighlighter (SourceView view) {
        base (view);
    }

    protected override unowned TreeSitter.Language get_ts_language () {
        return get_language_rust ();
    }

    protected override string query_source () {
        return ""
        ; Identifiers

        (type_identifier) @type
        (primitive_type) @type.builtin
    }
}

```

```

(field_identifier) @property

; Identifier conventions

; Assume all-caps names are constants
((identifier) @constant
 (#match? @constant "[A-Z][A-Z\\d_]+$"))

; Assume uppercase names are enum constructors
((identifier) @constructor
 (#match? @constructor "[A-Z]"))

; Assume that uppercase names in paths are types
((scoped_identifier
  path: (identifier) @type)
 (#match? @type "[A-Z]"))
((scoped_identifier
  path: (scoped_identifier
    name: (identifier) @type))
 (#match? @type "[A-Z]"))
((scoped_type_identifier
  path: (identifier) @type)
 (#match? @type "[A-Z]"))
((scoped_type_identifier
  path: (scoped_identifier
    name: (identifier) @type))
 (#match? @type "[A-Z]"))

; Assume all qualified names in struct patterns are enum constructors. (They're
; either that, or struct names; highlighting both as constructors seems to be
; the less glaring choice of error, visually.)
(struct_pattern
  type: (scoped_type_identifier
    name: (type_identifier) @constructor))

; Function calls

(call_expression
  function: (identifier) @function)
(call_expression
  function: (field_expression
    field: (field_identifier) @function.method))
(call_expression
  function: (scoped_identifier
    "::"
    name: (identifier) @function))

```



```

(generic_function
  function: (identifier) @function)
(generic_function
  function: (scoped_identifier
    name: (identifier) @function))
(generic_function
  function: (field_expression
    field: (field_identifier) @function.method))

(macro_invocation
  macro: (identifier) @function.macro
  "!" @function.macro)

; Function definitions

(function_item (identifier) @function)
(function_signature_item (identifier) @function)

(line_comment) @comment
(block_comment) @comment

(line_comment (doc_comment)) @comment.documentation
(block_comment (doc_comment)) @comment.documentation

"(" @punctuation.bracket
")" @punctuation.bracket
"[" @punctuation.bracket
"]" @punctuation.bracket
"{" @punctuation.bracket
"}" @punctuation.bracket

(type_arguments
  "<" @punctuation.bracket
  ">" @punctuation.bracket)
(type_parameters
  "<" @punctuation.bracket
  ">" @punctuation.bracket)

"::" @punctuation.delimiter
":" @punctuation.delimiter
"." @punctuation.delimiter
"," @punctuation.delimiter
";" @punctuation.delimiter

(parameter (identifier) @variable.parameter)

```

```

(lifetime (identifier) @label)

"as" @keyword
"async" @keyword
"await" @keyword
"break" @keyword
"const" @keyword
"continue" @keyword
"default" @keyword
"dyn" @keyword
"else" @keyword
"enum" @keyword
"extern" @keyword
"fn" @keyword
"for" @keyword
"gen" @keyword
"if" @keyword
"impl" @keyword
"in" @keyword
"let" @keyword
"loop" @keyword
"macro_rules!" @keyword
"match" @keyword
"mod" @keyword
"move" @keyword
"pub" @keyword
"raw" @keyword
"ref" @keyword
"return" @keyword
"static" @keyword
"struct" @keyword
"trait" @keyword
"type" @keyword
"union" @keyword
"unsafe" @keyword
"use" @keyword
"where" @keyword
"while" @keyword
"yield" @keyword
(crate) @keyword
(mutable_specifier) @keyword
(use_list (self) @keyword)
(scoped_use_list (self) @keyword)
(scoped_identifier (self) @keyword)
(super) @keyword

```

```

        (self) @variable.builtin

        (char_literal) @string
        (string_literal) @string
        (raw_string_literal) @string

        (boolean_literal) @constant.builtin
        (integer_literal) @constant.builtin
        (float_literal) @constant.builtin

        (escape_sequence) @escape

        (attribute_item) @attribute
        (inner_attribute_item) @attribute

        "*" @operator
        "&" @operator
        "'" @operator
        "\"";
    }

    protected override bool is_container_node (string node_type) {
        return node_type in new string[] {
            "function_item", "impl_item", "struct_item", "enum_item", "trait_item", "
        };
    }
}

```

File: src/Services/TreeSitter/vala/ValaHighlighter.vala

```

[CCode (cname = "tree_sitter_vala")]
extern unowned TreeSitter.Language ? get_lang_vala ();

public class Ide.ValaHighlighter : BaseTreeSitterHighlighter {
    public ValaHighlighter (SourceView view) {
        base (view);
    }

    protected override unowned TreeSitter.Language get_ts_language () {
        return get_lang_vala ();
    }

    protected override string query_source () {
        return ""
    }
}

```

```

; highlights.scm

(comment) @comment
(type) @type
(unqualified_type) @type
(attribute) @attribute
(method_declaration (symbol (symbol) @type (identifier) @function.method))
(method_declaration (symbol (identifier) @function.method))
(local_function_declaration (identifier) @function)
(destructor_declaration (identifier) @function)
(creation_method_declaration (symbol (symbol) @type (identifier) @constructor))
(creation_method_declaration (symbol (identifier) @constructor))
(enum_declaration (symbol) @type)
(enum_value (identifier) @constant)
(errordomain_declaration (symbol) @type)
(errorcode (identifier) @constant)
(constant_declaration (identifier) @constant)
(method_call_expression (member_access_expression (identifier) @function))
(lambda_expression (identifier) @variable.parameter)
(parameter (identifier) @variable.parameter)
(property_declaration (symbol (identifier) @property))
[
  (this_access)
  (base_access)
  (value_access)
] @variable.builtin
(boolean) @constant.builtin
(character) @constant
(integer) @number
(null) @constant.builtin
(real) @number
(regex) @constant
(string) @string
[
  (escape_sequence)
  (string_formatter)
] @string.special
(template_string) @string
(template_string_expression) @string.special
(verbatim_string) @string
[
  "var"
  "void"
] @type.builtin

[

```

"abstract"
"and"
"as"
"async"
"break"
"case"
"catch"
"class"
"const"
"construct"
"continue"
"default"
"delegate"
"delete"
"do"
"dynamic"
"else"
"enum"
"errordomain"
"extern"
"finally"
"for"
"foreach"
"get"
"if"
"in"
"inline"
"interface"
"internal"
"is"
"lock"
"namespace"
"new"
"not"
"or"
"out"
"override"
"owned"
"partial"
"private"
"protected"
"public"
"ref"
"return"
"set"
"signal"

```

"sizeof"
"static"
"struct"
"switch"
"throw"
"throws"
"try"
"typeof"
"unowned"
"using"
"virtual"
"weak"
"while"
"with"
"yield"
] @keyword

```

```

[
"="
"=="
"+"
"+="
"_"
"-="
"| "
"| ="
"&"
"&="
"^"
"^="
"/"
"/="
"*"
"*="
%"
"%="
"<<"
"<<="
">>"
">>="
"."
"? ."
"->"
"!"
"~"
"??"

```

```

        "?"
        ":"
        "<"
        "<="
        ">"
        ">="
        "||"
        "&&"
        "=>"
    ] @operator

    [
        ","
        ";"
    ] @punctuation.delimiter

    [
        "("
        ")"
        "{"
        "}"
        "["
        "]"
    ] @punctuation.bracket
    """;
}

protected override bool is_container_node (string node_type) {
    return node_type in new string[] { "class_definition", "function_definition" };
}
}

```

File: src/Services/TreeSitter/IndentBlock.vala

```

/*
*/

public struct Iide.IndentBlock {
    public int start_line;
    public int end_line;
    public int indent_level;
}

```

File: src/Services/NavigationHistoryService.vala

```
using Gee;

public class Iide.NavigationPoint : Object {
    public File file { get; construct; }
    public int line { get; construct; }
    public int column { get; construct; }

    public NavigationPoint (File file, int line, int column) {
        Object (file: file, line: line, column: column);
    }
}

public class Iide.NavigationHistoryService : Object {
    private static NavigationHistoryService? _instance = null;

    // libgee LinkedList - Deque
    private Deque<NavigationPoint> back_stack = new LinkedList<NavigationPoint> ();
    private Deque<NavigationPoint> forward_stack = new LinkedList<NavigationPoint> ();
    private const int MAX_HISTORY = 50;
    private bool is_navigate = true;

    public static NavigationHistoryService get_instance () {
        if (_instance == null) {
            _instance = new NavigationHistoryService ();
        }
        return _instance;
    }

    private NavigationHistoryService () {}

    public void start_navigation () {
        is_navigate = false;
    }

    /**
     * .
     * (Go to Definition) .
     */
    public void push_point (File file, int line, int column) {
        if (is_navigate)
            return;

        var point = new NavigationPoint (file, line, column);
```



```

//          : push = offer_head
back_stack.offer_head (point);

//          "   "
forward_stack.clear ();

//
if (back_stack.size > MAX_HISTORY) {
    back_stack.poll_tail ();
}
}

public void navigate_back () {
    if (back_stack.size < 2)//          (   +   )
        return;

    // 1.
    var current = back_stack.poll_head ();
    forward_stack.offer_head (current);

    // 2.          ,          "   "
    var target = back_stack.peek_head ();

    is_navigate = true;

    // 3.          (   )
    if (!DocumentManager.get_instance ().navigate_to (target)) {
        //          ,          .
        //          .
        back_stack.poll_head ();
        navigate_back ();
    }

    is_navigate = false;
}

public void navigate_forward () {
    if (forward_stack.is_empty)
        return;

    var point = forward_stack.poll_head ();

    is_navigate = true;

    if (DocumentManager.get_instance ().navigate_to (point)) {

```

```

        back_stack.offer_head (point);
    } else {
        navigate_forward ();
    }

    is_navigate = false;
}
}

```

File: src/Services/SourceNodeItem.vala

```

/*
*/
public struct Iide.SourceNodePosition {
    public uint32 row;
    public uint32 column;
}

public struct Iide.SourceNodeItem {
    public string name;
    public string type;
    public SourceNodePosition start_point;
    public Gee.List<SourceNodeItem?> siblings; //
    public Gee.List<SourceNodeItem?> children;
}

```

File: src/Services/StyleManager.vala

```

public class Iide.TagPair : Object {
    public Gtk.TextTag light { get; construct; }
    public Gtk.TextTag dark { get; construct; }

    public TagPair (Gtk.TextTag l, Gtk.TextTag d) {
        Object (light: l, dark: d);
    }

    public Gtk.TextTag get_by_index (int index) {
        return (index == 0) ? light : dark;
    }
}

public class Iide.StyleService : Object {
    private static StyleService? instance;
    public Gtk.TextTagTable shared_table { get; private set; }
}

```

```

//      Gee.HashMap      TagPair
private Gee.HashMap<string, TagPair> registry;

public static StyleService get_instance () {
    if (instance == null)instance = new StyleService ();
    return instance;
}

private StyleService () {
    shared_table = new Gtk.TextTagTable ();
    registry = new Gee.HashMap<string, TagPair> ();

    var folding_tag = new Gtk.TextTag ("$FOLD_HIDE");
    folding_tag.invisible = true; //
    shared_table.add (folding_tag);

    // ---      ---
    setup_tag ("function", "#1a5fb4", "#62a0ea", true);
    setup_tag ("keyword", "#a51d2d", "#ff7b72", true);
    setup_tag ("string", "#26a269", "#8ff0a4", false);
    setup_tag ("comment", "#5e5c64", "#94989b", false, true); // Italic
    setup_tag ("type", "#422d5b", "#f9c06b", true);
    setup_tag ("constant", "#986a44", "#d0b5f3", false);

    // ---      ---
    setup_tag ("variable", "#241f31", "#ffffff", false);
    setup_tag ("variable.parameter", "#3584e4", "#78aeed", false);
    setup_tag ("variable.builtin", "#1a5fb4", "#62a0ea", false, true); // self, cls

    // ---      ---
    setup_tag ("number", "#3071db", "#78aeed", false);
    setup_tag ("boolean", "#986a44", "#d0b5f3", true);

    // ---      (Rust      , Vala      ) ---
    setup_tag ("function.macro", "#63452c", "#c061cb", false);
    setup_tag ("attribute", "#63452c", "#c061cb", false);
    setup_tag ("label", "#422d5b", "#f9c06b", false); // Lifetimes Rust ('a)

    // ---      ---
    setup_tag ("operator", "#241f31", "#ffa348", false);
    setup_tag ("punctuation.bracket", "#241f31", "#d3d3d7", false);
    setup_tag ("punctuation.delimiter", "#241f31", "#d3d3d7", false);

    // ---      (#include, #define,      C/C++) ---
    setup_tag ("keyword.directive", "#63452c", "#c061cb", false);
    setup_tag ("keyword.control.import", "#63452c", "#c061cb", true);

```

```

// ---          C++ (Namespace, Qualifier) ---
setup_tag ("namespace", "#422d5b", "#f9c06b", false);
setup_tag ("type.qualifier", "#a51d2d", "#ff7b72", false); // const, static, volatile

// ---          ---
setup_tag ("function.call", "#1c71d8", "#78aeed", false);
setup_tag ("function.method", "#1a5fb4", "#62a0ea", true);

//          StyleService
setup_tag ("bracket.lv11", "#3584e4", "#78aeed", false); //
setup_tag ("bracket.lv12", "#26a269", "#8ff0a4", false); //
setup_tag ("bracket.lv13", "#e66100", "#ffa348", false); //
setup_tag ("bracket.lv14", "#e01b24", "#ff7b72", false); //
setup_tag ("bracket.lv15", "#9141ac", "#c061cb", false); //

//          (      query      @none)
setup_tag ("none", null, null, false);
}

private void setup_tag (string name, string? light_color, string? dark_color, bool bold,
//
var tag_light = new Gtk.TextTag (name + ":light");
tag_light.foreground = light_color;
if (bold)tag_light.weight = Pango.Weight.BOLD;
if (italic)tag_light.style = Pango.Style.ITALIC;
shared_table.add (tag_light);

//
var tag_dark = new Gtk.TextTag (name + ":dark");
tag_dark.foreground = dark_color;
if (bold)tag_dark.weight = Pango.Weight.BOLD;
if (italic)tag_dark.style = Pango.Style.ITALIC;
shared_table.add (tag_dark);

//
var pair = new TagPair (tag_light, tag_dark);
registry.set (name, pair);
}

public Gtk.TextTag? get_tag (string name, int theme_index) {
var pair = registry.get (name);
if (pair != null) {
return pair.get_by_index (theme_index);
}
return null;
}

```

```

    }
}

```

File: src/Widgets/Find/Engines/FzfSearchEngine.vala

```

public class Ide.FzfSearchEngine : SearchEngine, Object {

    private Ide.ProjectManager project_manager;

    private const int MAX_RESULTS = 100;

    public FzfSearchEngine () {
        Object ();
        project_manager = ProjectManager.get_instance ();
    }

    public string search_entry_placeholder () {
        return _("Enter file name (min 3 chars)...");
    }

    public string search_progress_message () {
        return _("Searching files...");
    }

    public string search_kind () {
        return "files";
    }

    public string search_title () {
        return _("Files");
    }

    public string search_icon_name () {
        return "document-open-symbolic";
    }

    public async Gee.List<SearchResult> perform_search (string query, GLib.Cancellable cancellable,
        var cache = yield ensure_file_cache ();

        return perform_search_sync (query, cache);
    }

    public async Gee.List<Ide.FileEntry> ensure_file_cache () {
        if (project_manager.cache_valid) {
            return project_manager.get_file_cache ();
        }
    }
}

```

```

    }

    // 1. ID
    ulong handler_id = 0;

    // 2. , callback
    handler_id = project_manager.file_cache_updated.connect (() => {
        // ( )
        SignalHandler.disconnect (project_manager, handler_id);

        // async
        ensure_file_cache.callback ();
    });

    // 3.
    yield;

    return project_manager.get_file_cache ();
}

private Gee.List<SearchResult> perform_search_sync (string current_query, Gee.List<Ide
    var all_results = new Gee.ArrayList<SearchResult> ();
    if (cache == null) {
        return all_results;
    }

    var query = current_query.down ();

    {
        foreach (var f in cache) {
            var matches = new Gee.ArrayList<MatchRange> ();
            int score = fuzzy_match_with_positions (f.name.down (), query, matches);

            if (score > 20) {
                all_results.add (new SearchResult (f.path,
                                                    f.relative_path,
                                                    -1,
                                                    f.name,
                                                    matches,
                                                    null,
                                                    score));
                if (all_results.size >= MAX_RESULTS)break;
            }
        }
    }
}

```

```

        return all_results;
    }

private int fuzzy_match_with_positions (string text, string query, Gee.List<MatchRange>
    if (text.length == 0 || query.length == 0) {
        return 0;
    }

    var text_lower = text.down ();
    var query_lower = query.down ();

    if (text_lower.contains (query_lower)) {
        var pos = text_lower.index_of (query_lower);
        matches.add (new MatchRange (pos, pos + (int) query_lower.length));
        if (pos == 0) {
            return 1000 + (1000 - text.length);
        }
        return 500 + (1000 - pos);
    }

    int score = 0;
    int consecutive = 0;
    int last_match = -1;
    bool prev_was_sep = true;
    bool matched_all = true;
    var match_positions = new Gee.ArrayList<int> ();

    for (int qi = 0; qi < query_lower.length; qi++) {
        bool found = false;
        for (int idx = last_match + 1; idx < text_lower.length; idx++) {
            if (text_lower[idx] == query_lower[qi]) {
                found = true;
                match_positions.add (idx);
                last_match = idx;
                consecutive++;

                if (idx == 0 || prev_was_sep) {
                    score += 150;
                } else if (consecutive > 1) {
                    score += consecutive * 10;
                } else {
                    score += 15;
                }
                break;
            } else {
                score -= 1;
            }
        }
    }

```

```

        }
    }

    if (!found) {
        matched_all = false;
        break;
    }

    unichar c = last_match >= 0 && last_match < (int) text.length ? text[last_match] : 0;
    prev_was_sep = !c.isalnum () && c != '_';
}

if (!matched_all) {
    return 0;
}

int i = 0;
while (i < match_positions.size) {
    int start = match_positions[i];
    int end = start + 1;

    while (i + 1 < match_positions.size && match_positions[i + 1] == match_positions[i] + 1) {
        end = match_positions[i + 1] + 1;
        i++;
    }

    matches.add (new MatchRange (start, end));
    i++;
}

return score;
}
}

```

File: src/Widgets/Find/Engines/TextSearchEngine.vala

```

public class Ide.TextSearchEngine : SearchEngine, Object {

    private Ide.ProjectManager project_manager;

    private Gee.List<SearchResult> search_cache = new Gee.ArrayList<SearchResult> ();
    private string cache_query = "";
    private ThreadPool<SearchTask> thread_pool;

    private const int MAX_RESULTS = 100;
}

```



```

private class SearchTask {
    public Gee.List<Ide.FileEntry> files;
    public string query;
    public Gee.List<SearchResult> results;
    public Cancellable cancellable;

    public SearchTask (Gee.List<Ide.FileEntry> files, string query, Cancellable cancellable) {
        this.files = files;
        this.query = query;
        this.results = new Gee.ArrayList<SearchResult> ();
        this.results = new Gee.ArrayList<SearchResult> ();
    }
}

private void search_files_in_task (SearchTask task) {
    //

    foreach (var file_entry in task.files) {
        if (task.cancellable.is_cancelled ())
            return;

        try {
            var file = GLib.File.new_for_path (file_entry.path);
            var dis = new DataInputStream (file.read ());
            string line;
            int line_num = 0;
            int matches_in_this_file = 0;

            while ((line = dis.read_line ()) != null) {
                if (task.cancellable.is_cancelled ())
                    return;

                var matches = new Gee.ArrayList<MatchRange> ();
                int score = fuzzy_match_with_positions (line, task.query, matches);

                if (score > 20) {
                    // -
                    if (matches_in_this_file >= 50) {
                        break;
                    }
                    matches_in_this_file++;

                    task.results.add (new SearchResult (
                        file_entry.path,
                        file_entry.relative_path,

```

```

        line_num,
        line,
        matches,
        null,
        score

        ));
    }
    line_num++;
}
dis.close ();
} catch (Error e) {
}
}
}

public TextSearchEngine () {
    Object ();
    project_manager = ProjectManager.get_instance ();

    // 1.
    try {
        thread_pool = new ThreadPool<SearchTask>.with_owned_data ((task) => { this.search
            (int) GLib.get_num_processors (),
            false);
    } catch (ThreadError e) {
        critical ("                : %s", e.message);
    }
}

public string search_entry_placeholder () {
    return _("Enter text (min 3 chars)...");
}

public string search_progress_message () {
    return _("Searching text in files...");
}

public string search_kind () {
    return "text";
}

public string search_title () {
    return _("Text");
}

public string search_icon_name () {

```

```

        return "edit-find-symbolic";
    }

    public async Gee.List<SearchResult> perform_search (string query, GLib.Cancellable cancellable) {
        var cache = yield ensure_file_cache ();

        return yield inner_perform_search (query, cache, cancellable);
    }

    public async Gee.List<Iide.FileEntry> ensure_file_cache () {
        if (project_manager.cache_valid) {
            return project_manager.get_text_file_cache ();
        }

        // 1. ID
        ulong handler_id = 0;

        // 2. , callback
        handler_id = project_manager.file_cache_updated.connect (() => {
            // ( )
            SignalHandler.disconnect (project_manager, handler_id);

            // async
            ensure_file_cache.callback ();
        });

        // 3.
        yield;

        return project_manager.get_text_file_cache ();
    }

    private async Gee.List<SearchResult> inner_perform_search (string current_query,
                                                                Gee.List<Iide.FileEntry>? cache,
                                                                GLib.Cancellable search_cancellable) {
        var all_results = new Gee.ArrayList<SearchResult> ();
        if (cache == null) {
            return all_results;
        }

        string new_query = current_query.strip ();

        // : ?
        // : ,
        bool can_use_cache = search_cache.size > 0 &&
            cache_query.length > 0 &&

```

```

        new_query.has_prefix (cache_query);

    if (can_use_cache) {
        //      (      )
        return filter_cache_to_ui (new_query);
    } else {
        //      (      +      )

        //
        cache_query = "";
        search_cache = new Gee.ArrayList<SearchResult> ();

        //
        return yield perform_full_disk_search_async (new_query, search_cancellable);
    }
}

private Gee.List<SearchResult> filter_cache_to_ui (string query) {
    var filtered_results = new Gee.ArrayList<SearchResult> ();

    foreach (var res in search_cache) {
        var matches = new Gee.ArrayList<MatchRange> ();
        //      score      (      )
        int score = fuzzy_match_with_positions (res.line_content, query, matches);

        //  UI
        if (score > 50) {
            filtered_results.add (new SearchResult (
                res.file_path, res.relative_path,
                res.line_number, res.line_content,
                matches, null, score
            ));
        }
    }

    //      -200
    filtered_results.sort ((a, b) => b.score - a.score);
    return filtered_results;
}

private int fuzzy_match_with_positions (string text, string query, Gee.List<MatchRange>
    if (text.length == 0 || query.length == 0) {
        return 0;
    }

    var text_lower = text.down ();

```

```

var query_lower = query.down ();

if (text_lower.contains (query_lower)) {
    var pos = text_lower.index_of (query_lower);
    matches.add (new MatchRange (pos, pos + (int) query_lower.length));
    if (pos == 0) {
        return 1000 + (1000 - text.length);
    }
    return 500 + (1000 - pos);
}

int score = 0;
int consecutive = 0;
int last_match = -1;
bool prev_was_sep = true;
bool matched_all = true;
var match_positions = new Gee.ArrayList<int> ();

for (int qi = 0; qi < query_lower.length; qi++) {
    bool found = false;
    for (int idx = last_match + 1; idx < text_lower.length; idx++) {
        if (text_lower[idx] == query_lower[qi]) {
            found = true;
            match_positions.add (idx);
            last_match = idx;
            consecutive++;

            if (idx == 0 || prev_was_sep) {
                score += 150;
            } else if (consecutive > 1) {
                score += consecutive * 10;
            } else {
                score += 15;
            }
            break;
        } else {
            score -= 1;
        }
    }

    if (!found) {
        matched_all = false;
        break;
    }

    unichar c = last_match >= 0 && last_match < (int) text.length ? text[last_match]

```

```

        prev_was_sep = !c.isalnum () && c != '_';
    }

    if (!matched_all) {
        return 0;
    }

    int i = 0;
    while (i < match_positions.size) {
        int start = match_positions[i];
        int end = start + 1;

        while (i + 1 < match_positions.size && match_positions[i + 1] == match_positions[i] + 1) {
            i++;
        }

        matches.add (new MatchRange (start, end));
        i++;
    }

    return score;
}

private async Gee.List<SearchResult> perform_full_disk_search_async (string current_query) {
    var text_files = project_manager.get_text_file_cache ();
    if (text_files == null || text_files.size == 0) {
        return new Gee.ArrayList<SearchResult> ();
    }

    int num_threads = (int) GLib.get_num_processors ();
    int files_per_thread = (text_files.size + num_threads - 1) / num_threads;
    var tasks = new Gee.ArrayList<SearchTask> ();

    // 1.
    for (int t = 0; t < num_threads; t++) {
        int start = t * files_per_thread;
        int end = int.min (start + files_per_thread, text_files.size);
        if (start >= text_files.size) break;

        var task = new SearchTask (text_files.slice (start, end), current_query, current_query);
        tasks.add (task);

        try {
            thread_pool.add (task);
        } catch (ThreadError e) { /* handle error */
    }

```

```

    }
}

// 2.      thread.join() -
//          ,          UI
while (thread_pool.get_num_threads () > 0 || thread_pool.unprocessed () > 0) {
    if (current_run_cancellable.is_cancelled ())
        return new Gee.ArrayList<SearchResult> ();

    //          50
    Timeout.add (50, perform_full_disk_search_async.callback);
    yield;
}

//          ,          -
if (current_run_cancellable.is_cancelled ())
    return new Gee.ArrayList<SearchResult> ();

var search_cache_tmp = new Gee.ArrayList<SearchResult> ();
foreach (var task in tasks) {
    search_cache_tmp.add_all (task.results);
}

// 2.          (>50)
var all_task_results = new Gee.ArrayList<SearchResult> ();
foreach (var res in search_cache_tmp) {
    if (res.score > 50)all_task_results.add (res);
}

LoggerService.get_instance ().debug ("SEARCH TEXT", "results count=" + all_task_resu
var all_results = new Gee.ArrayList<SearchResult> ();
if (all_task_results.size > 4000) {
    var top_results = new Gee.TreeSet<SearchResult> ((a, b) => {
        //          score (    ).    score    ,
        int res = b.score - a.score;
        if (res == 0) {
            int path_res = a.file_path.ascii_casecmp (b.file_path);
            if (path_res == 0) {
                return a.line_number - b.line_number;
            }
        }
        return res;
    });
}

foreach (var task in tasks) {
    if (current_run_cancellable.is_cancelled ())

```

```

        return new Gee.ArrayList<SearchResult> ();

        foreach (var match in task.results) {
            top_results.add (match);

            // - « » ( )
            if (top_results.size > MAX_RESULTS) {
                top_results.remove (top_results.last ());
            }
        }

        // all_results -200
        all_results = new Gee.ArrayList<SearchResult> ();
        all_results.add_all (top_results);
    } else {
        all_task_results.sort ((a, b) => b.score - a.score);

        all_results = new Gee.ArrayList<SearchResult> ();
        for (int i = 0; i < all_task_results.size && i < MAX_RESULTS; i++) {
            all_results.add (all_task_results[i]);
        }
    }

    // 1. (>20)
    search_cache = search_cache_tmp;
    cache_query = current_query;

    return all_results;
}
}

```

File: src/Widgets/Find/SearchEngine.vala

```

public interface Iide.SearchEngine : GLib.Object {
    public abstract string search_entry_placeholder ();
    public abstract string search_progress_message ();
    public abstract string search_kind ();
    public abstract string search_title ();
    public abstract string search_icon_name ();

    public abstract async Gee.List<SearchResult> perform_search (string query, GLib.Cancellable? cancellable);
}

```


File: src/Widgets/Find/SearchPage.vala

```
public class Ide.SearchPage : Gtk.Box {
    private SearchEngine search_engine;

    private Gtk.SearchEntry search_entry;
    private SearchResultsView results_view;
    private Gtk.Stack status_stack;
    private Gtk.Spinner spinner;

    private GLib.Cancellable? search_cancellable = null;
    private uint debounce_id = 0;

    public signal void close_requested ();

    public SearchPage (SearchEngine search_engine) {
        Object (
            orientation : Gtk.Orientation.VERTICAL,
            spacing: 8,
            margin_top: 12,
            margin_bottom: 12,
            margin_start: 12,
            margin_end: 12
        );
        this.search_engine = search_engine;

        search_entry = new Gtk.SearchEntry () {
            placeholder_text = search_engine.search_entry_placeholder (),
            expand = true
        };
        this.append (search_entry);

        results_view = new SearchResultsView ();

        status_stack = new Gtk.Stack ();
        status_stack.transition_type = Gtk.StackTransitionType.CROSSFADE;

        var loading_box = new Gtk.Box (Gtk.Orientation.VERTICAL, 12);
        loading_box.valign = Gtk.Align.CENTER;
        loading_box.halign = Gtk.Align.CENTER;

        spinner = new Gtk.Spinner ();
        spinner.set_size_request (32, 32);

        var loading_label = new Gtk.Label (search_engine.search_progress_message ());
        loading_label.add_css_class ("dim-label");
```

```

loading_box.append (spinner);
loading_box.append (loading_label);

status_stack.add_named (loading_box, "loading");
status_stack.add_named (results_view, "results");

this.append (status_stack);

status_stack.visible_child_name = "results";

search_entry.search_changed.connect (on_search_changed);

var key_controller = new Gtk.EventControllerKey ();
key_controller.key_pressed.connect (on_key_pressed);
search_entry.add_controller (key_controller);

results_view.list_view.activate.connect (on_list_activated);
search_entry.activate.connect (on_list_activated);
}

public string search_kind () {
    return search_engine.search_kind ();
}

public string search_title () {
    return search_engine.search_title ();
}

public string search_icon_name () {
    return search_engine.search_icon_name ();
}

public void handle_activated () {
    search_entry.grab_focus ();
}

private void on_list_activated () {
    open_selected ();
}

private void open_selected (bool close_search = true) {
    if (results_view.open_selected () && close_search) {
        close_requested ();
    }
}

```

```

private bool on_key_pressed (Gtk.EventControllerKey controller, uint keyval, uint keyco
    if (keyval == Gdk.Key.Escape) {
        close_requested ();
        return true;
    } else if (keyval == Gdk.Key.Return || keyval == Gdk.Key.KP_Enter) {
        open_selected ((modifiers & Gdk.ModifierType.SHIFT_MASK) == 0);
        return true;
    } else if (keyval == Gdk.Key.Up || keyval == Gdk.Key.KP_Up) {
        results_view.select_up ();
        return true;
    } else if (keyval == Gdk.Key.Down || keyval == Gdk.Key.KP_Down) {
        results_view.select_down ();
        return true;
    }
    return false;
}

private void on_search_changed () {
    if (debounce_id > 0) {
        GLib.Source.remove (debounce_id);
        debounce_id = 0;
    }

    string query = search_entry.get_text ().strip ();
    if (query.length < 3) {
        results_view.update_results (null);
        status_stack.visible_child_name = "results";
        return;
    }

    debounce_id = GLib.Timeout.add (300, () => {
        perform_search.begin (query);
        debounce_id = 0;
        return false;
    });
}

private async void perform_search (string query) {
    if (search_cancellable != null) {
        search_cancellable.cancel ();
    }
    search_cancellable = new GLib.Cancellable ();

    //
    status_stack.visible_child_name = "loading";

```

```

        spinner.start ();

        try {
            var results = yield search_engine.perform_search (query, search_cancellable);

            update_results (results);
        } catch (GLib.IOError.CANCELLED e) {
        } catch (GLib.Error e) {
            warning ("Search Error: %s", e.message);

            //
            status_stack.visible_child_name = "results";
            spinner.stop ();
        }
    }

    private void update_results (Gee.List<SearchResult>? new_results) {
        results_view.update_results (new_results);

        //
        spinner.stop ();
        status_stack.visible_child_name = "results";
    }
}

```

File: src/Widgets/Find/SearchWindow.vala

```

using Gtk;
using Adw;

public class Iide.SearchWindow : Adw.Window {
    private ViewStack view_stack;
    private ViewSwitcher view_switcher;
    private Window parent_window;
    private Iide.DocumentManager document_manager;

    public SearchWindow (Window parent_window, Iide.DocumentManager document_manager) {
        Object (transient_for: parent_window, modal: true);
        this.parent_window = parent_window;
        this.document_manager = document_manager;
        set_default_size (600, 450);

        //
        var content = new Box (Orientation.VERTICAL, 0);
        set_content (content);
    }
}

```

```

//
var header_bar = new Adw.HeaderBar ();
view_switcher = new ViewSwitcher ();
header_bar.set_title_widget (view_switcher);
content.append (header_bar);

//
view_stack = new ViewStack ();
view_switcher.stack = view_stack;
content.append (view_stack);

setup_pages ();

view_stack.notify["visible-child"].connect_after (() => {
    SearchPage? active_page = view_stack.visible_child as SearchPage;
    if (active_page != null) {
        active_page.handle_activated ();
    }
});
}

private void setup_pages () {
    var fzf_search_page = new SearchPage (new FzfSearchEngine ());
    view_stack.add_titled_with_icon (
        fzf_search_page,
        fzf_search_page.search_kind (),
        fzf_search_page.search_title (),
        fzf_search_page.search_icon_name ());
    fzf_search_page.close_requested.connect_after (close);

    var symbols_search_page = new SearchPage (new SymbolsSearchEngine ());
    view_stack.add_titled_with_icon (
        symbols_search_page,
        symbols_search_page.search_kind (),
        symbols_search_page.search_title (),
        symbols_search_page.search_icon_name ());
    symbols_search_page.close_requested.connect_after (close);

    var text_search_page = new SearchPage (new TextSearchEngine ());
    view_stack.add_titled_with_icon (
        text_search_page,
        text_search_page.search_kind (),
        text_search_page.search_title (),
        text_search_page.search_icon_name ());
    text_search_page.close_requested.connect_after (close);
}

```

```

    }

    // ( , Ctrl+Shift+O)
    public void set_active_page (string search_kind) {
        var active_page = view_stack.get_child_by_name (search_kind) as SearchPage;
        if (active_page != null) {
            view_stack.visible_child = active_page;
            active_page.handle_activated ();
        }
    }
}

```

File: src/Widgets/Panels/LogPanel.vala

```

public class Ide.LogPanel : BasePanel {
    public LogPanel () {
        base ("Logs", SymbIconProvider.get_instance ().icon_name (IconID.APP_LOG));
        child = new LogView ();
        can_maximize = true;
    }

    public override Panel.Position initial_pos () {
        return new Panel.Position () { area = Panel.Area.BOTTOM };
    }

    public override string panel_id () {
        return "LogPanel";
    }
}

```

File: src/Widgets/Panels/ProjectPanel.vala

```

public class Ide.ProjectPanel : BasePanel {
    public FileTreeView folder_view;

    public ProjectPanel () {
        base ("Project Tree", ImageFactory.icon_name_for_id (IconID.APP_PROJECT));
        folder_view = new FileTreeView ();
        child = folder_view;
        can_maximize = true;

        //
        var project_manager = ProjectManager.get_instance ();
        project_manager.project_opened.connect ((project_root) => {
            folder_view.set_root_file (project_root);
        });
    }
}

```

```

    });

    project_manager.project_closed.connect (() => {
        folder_view.set_root_file (null);
    });

    // Handle file activation to open documents
    var document_manager = DocumentManager.get_instance ();
    folder_view.file_activated.connect ((item) => {
        if (!item.is_directory) {
            document_manager.open_document (item.file, null);
        }
    });
}

public override Panel.Position initial_pos () {
    return new Panel.Position () { area = Panel.Area.START };
}

public override string panel_id () {
    return "ProjectPanel";
}
}

```

File: src/Widgets/Panels/TerminalPanel.vala

```

public class Iide.TerminalPanel : BasePanel {
    public TerminalPanel () {
        base ("Terminal", SymbIconProvider.get_instance ().icon_name (IconID.APP_TERMINAL));
        child = new Iide.Terminal ();
        can_maximize = true;
    }

    public override Panel.Position initial_pos () {
        return new Panel.Position () { area = Panel.Area.BOTTOM };
    }

    public override string panel_id () {
        return "TerminalPanel";
    }
}

```

File: src/Widgets/TextView/DiagnosticsPopover.vala

```

public class Iide.DiagnosticsPopover : Gtk.Popover {

```

```

private Gtk.ListBox list_box;
private GtkSource.View text_view;

public DiagnosticsPopover (Gtk.Widget parent, GtkSource.View view) {
    this.set_parent (parent);
    this.text_view = view;

    //
    this.set_position (Gtk.PositionType.TOP); // -
    this.set_has_arrow (true);
    this.set_autohide (true);

    var scroll = new Gtk.ScrolledWindow () {
        max_content_height = 400,
        propagate_natural_height = true,
        width_request = 400
    };

    list_box = new Gtk.ListBox ();
    list_box.add_css_class ("boxed-list");
    list_box.set_selection_mode (Gtk.SelectionMode.NONE); // ,

    scroll.set_child (list_box);
    this.set_child (scroll);

    list_box.row_activated.connect ((row) => {
        var diag_row = row as Adw.ActionRow;
        if (diag_row != null) {
            // ( . )
            Gtk.TextIter start_line, end_line;

            // 1.
            var buffer = (GtkSource.Buffer) text_view.buffer;
            var mark = diag_row.get_data<LspDiagnosticsMark> ("mark");
            if (mark == null) {
                return;
            }
            buffer.get_iter_at_mark (out start_line, mark);

            // 2.
            start_line.set_line_offset (0);

            // 3.
            end_line = start_line;
            if (!end_line.ends_line ()) {
                end_line.forward_to_line_end ();
            }
        }
    });
}

```



```

    }

    // 4.
    // GTK select_range( , )
    buffer.select_range (start_line, end_line);

    // 5.
    text_view.scroll_to_iter (start_line, 0.1, false, 0, 0.5);
    text_view.grab_focus ();

    //
    this.popdown ();
}
});
}

public void refresh () {
    // 1.
    Gtk.Widget? child;
    while ((child = list_box.get_first_child ()) != null) {
        list_box.remove (child);
    }

    var buffer = (GtkSource.Buffer) text_view.buffer;
    bool found_any = false;

    // 2.
    int line_count = buffer.get_line_count ();
    for (int i = 0; i < line_count; i++) {
        Gtk.TextIter iter;
        buffer.get_iter_at_line (out iter, i);

        var iter_marks = iter.get_marks ();

        //
        if (iter_marks != null) {
            foreach (var m in iter_marks) {
                if (m is LspDiagnosticsMark) {
                    add_diagnostic_row ((LspDiagnosticsMark) m);
                    found_any = true;
                }
            }
        }
    }

    // 3.
    " "

```

```

    if (!found_any) {
        var status_page = new Adw.StatusPage () {
            title = "No issues found",
            icon_name = "emblem-ok-symbolic",
            description = "LSP servers haven't reported any problems."
        };
        status_page.add_css_class ("compact"); // libadwaita 1.2+
        list_box.append (status_page);
    }
}

private void add_diagnostic_row (LspDiagnosticsMark mark) {
    Gtk.TextIter iter;
    var buffer = (GtkSource.Buffer) text_view.buffer;
    buffer.get_iter_at_mark (out iter, mark);
    int line = iter.get_line () + 1;

    var row = new Adw.ActionRow () {
        title = mark.diagnostic_message,
        subtitle = @"Line $line",
        activatable = true, //
        selectable = false
    };
    row.set_data ("mark", mark);

    // ( )
    switch (mark.severity) {
        case 1:
            row.add_prefix (SymbIconProvider.get_instance ().image (IconID.COD_ERROR));
            break;
        case 2: case 3: case 4:
            row.add_prefix (SymbIconProvider.get_instance ().image (IconID.COD_WARNING));
            break;
    }

    // -
    row.activated.connect (() => {
        Gtk.TextIter start_line, end_line;

        // 1.
        buffer.get_iter_at_mark (out start_line, mark);

        // 2.
        start_line.set_line_offset (0);

        // 3.

```

```

        end_line = start_line;
        if (!end_line.ends_line ()) {
            end_line.forward_to_line_end ();
        }

        // 4.
        // GTK select_range(    ,    )
        buffer.select_range (start_line, end_line);

        // 5.
        text_view.scroll_to_iter (start_line, 0.1, false, 0, 0.5);
        text_view.grab_focus ();

        //
        this.popdown ();
    });

    list_box.append (row);
}
}

```

File: src/Widgets/TextView/FontZoomer.vala

```

using Gtk;
using GtkSource;
using GLib;

public class Iide.FontZoomer : Object {
    private View src_view;
    private int zoom_level;
    private Iide.SettingsService settings;
    public signal void zoom_changed(int level);

    public FontZoomer(View src_view) {
        this.src_view = src_view;
        this.settings = Iide.SettingsService.get_instance();

        if (!this.src_view.has_css_class("text-view")) {
            this.src_view.add_css_class("text-view");
        }

        this.zoom_level = settings.editor_font_size;
        if (this.zoom_level < FontSizeHelper.MIN_ZOOM_LEVEL || this.zoom_level > FontSizeHelper.MAX_ZOOM_LEVEL)
            this.zoom_level = FontSizeHelper.DEFAULT_ZOOM_LEVEL;
    }
}

```

```

        this.src_view.add_css_class("zoom-" + zoom_level.to_string());

        var scroll_controller = new EventControllerScroll(EventControllerScrollFlags.VERTICAL);
        scroll_controller.scroll.connect((dx, dy) => {
            var event = scroll_controller.get_current_event();
            if (event == null) return false;

            var modifiers = event.get_modifier_state();
            if ((modifiers & Gdk.ModifierType.CONTROL_MASK) != 0) {
                if (dy < 0) {
                    zoom_in();
                } else if (dy > 0) {
                    zoom_out();
                }
                return true;
            }
            return false;
        });

        this.src_view.add_controller(scroll_controller);
    }

    public void zoom_in() {
        if (zoom_level < FontSizeHelper.MAX_ZOOM_LEVEL) {
            src_view.remove_css_class("zoom-" + zoom_level.to_string());
            zoom_level++;
            src_view.add_css_class("zoom-" + zoom_level.to_string());
            settings.editor_font_size = zoom_level;
            zoom_changed(zoom_level);
        }
    }

    public void zoom_out() {
        if (zoom_level > FontSizeHelper.MIN_ZOOM_LEVEL) {
            src_view.remove_css_class("zoom-" + zoom_level.to_string());
            zoom_level--;
            src_view.add_css_class("zoom-" + zoom_level.to_string());
            settings.editor_font_size = zoom_level;
            zoom_changed(zoom_level);
        }
    }

    public void zoom_reset() {
        src_view.remove_css_class("zoom-" + zoom_level.to_string());
        zoom_level = FontSizeHelper.DEFAULT_ZOOM_LEVEL;
        src_view.add_css_class("zoom-" + zoom_level.to_string());
    }

```

```

        settings.editor_font_size = zoom_level;
        zoom_changed(zoom_level);
    }

    public void set_zoom_level(int level) {
        if (level < FontSizeHelper.MIN_ZOOM_LEVEL || level > FontSizeHelper.MAX_ZOOM_LEVEL)
            return;
    }
    if (zoom_level == level) {
        return;
    }
    src_view.remove_css_class("zoom-" + zoom_level.to_string());
    zoom_level = level;
    src_view.add_css_class("zoom-" + zoom_level.to_string());
    zoom_changed(zoom_level);
}

public int get_zoom_level() {
    return zoom_level;
}
}

```

File: src/Widgets/TextView/LineNumbersGutter.vala

```

using Gtk;
using GtkSource;
using Cairo;

namespace Iide {

    public class LineNumbersGutter : GtkSource.GutterRenderer {
        private Pango.Layout? pango_layout = null;
        private int current_width = 38; //

        public LineNumbersGutter () {
            Object ();
            // (IDE)
            this.set_alignment_mode (GtkSource.GutterRendererAlignmentMode.CELL);
            this.set_xalign (1.0f);
        }

        // ( )
        public override void measure (Gtk.Orientation orientation, int for_size, out int min, out int max) {
            minimum_baseline = natural_baseline = -1;
            if (orientation == Gtk.Orientation.HORIZONTAL) {

```

```

        minimum = natural = this.current_width;
    } else {
        minimum = natural = 0;
    }
}

public override void snapshot_line (Gtk.Snapshot snapshot, GtkSource.GutterLines line,
// 1. (0, UI +1)
Gtk.TextIter current_line_iter;
lines.get_iter_at_line (out current_line_iter, line);
int real_line_number = current_line_iter.get_line () + 1;

// 2. Y-
int cell_y, cell_height;
lines.get_line_yrange (line, GtkSource.GutterRendererAlignmentMode.CELL, out cell_y, out cell_height);

// - 0.
// , .
if (cell_height <= 0) {
    return;
}

var view = this.get_view ();
if (view == null) return;

// 3.
this.pango_layout = view.create_pango_layout ("");
// var font_desc = view.get_font_desc ();
// if (font_desc != null) {
//     this.pango_layout.set_font_description (font_desc);
// } else {
//     this.pango_layout.set_font_description (Pango.FontDescription.from_string (""));
// }

//
string num_str = real_line_number.to_string ();
this.pango_layout.set_text (num_str, -1);

// (100, 1000 ..)
Pango.Rectangle ink_rect, logical_rect;
this.pango_layout.get_extents (out ink_rect, out logical_rect);
int text_width_px = logical_rect.width / Pango.SCALE;
int text_height_px = logical_rect.height / Pango.SCALE;

// +
int calculated_width = text_width_px + 12;

```

```

        if (this.current_width != calculated_width && calculated_width > 38) {
            this.current_width = calculated_width;
            view.queue_resize (); //
        }

        // 4. CAIRO
        var bounds = Graphene.Rect ();
        bounds.init (0.0f, (float) cell_y, (float) this.current_width, (float) cell_height);

        var cr = snapshot.append_cairo (bounds);

        // : (Alpha 60%)
        cr.set_source_rgba (0.5, 0.5, 0.5, 0.6);

        // 6
        double draw_x = (double) this.current_width - text_width_px - 6.0;
        double draw_y = (double) cell_y + ((cell_height - text_height_px) / 2.0);

        cr.move_to (draw_x, draw_y);
        Pango.cairo_show_layout (cr, this.pango_layout);
    }
}
}

```

File: src/Widgets/ToolViews/LogView.vala

```

/*
 * logview.vala
 *
 * Copyright 2026 kai
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program. If not, see <https://www.gnu.org/licenses/>.
 *
 * SPDX-License-Identifier: GPL-3.0-or-later

```

```

*/

public class Iide.LogView : Gtk.Box {
    private Gtk.TextView text_view;
    private Gtk.TextBuffer buffer;
    private Gtk.ScrolledWindow scrolled_window;
    private Gtk.Button clear_button;
    private Gtk.ToggleButton wrap_button;
    private Gtk.DropDown level_filter;
    private Gtk.SearchEntry search_entry;
    private Iide.LoggerService logger;

    private Gtk.TextTag tag_debug;
    private Gtk.TextTag tag_info;
    private Gtk.TextTag tag_warning;
    private Gtk.TextTag tag_error;
    private Gtk.TextTag tag_critical;
    private Gtk.TextTag tag_bold;
    private Gtk.TextTag tag_dim;

    private GLib.Mutex buffer_lock;
    private bool auto_scroll = true;
    private const int MAX_VISIBLE_LINES = 5000;
    private int line_count = 0;

    public LogView () {
        Object (orientation: Gtk.Orientation.VERTICAL, spacing: 0);
    }

    construct {
        logger = Iide.LoggerService.get_instance ();

        var icon_provider = SymbIconProvider.get_instance ();

        var toolbar = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 6);
        toolbar.add_css_class ("toolbar");
        toolbar.margin_start = 6;
        toolbar.margin_end = 6;
        toolbar.margin_top = 4;
        toolbar.margin_bottom = 4;

        clear_button = new Gtk.Button () {
            icon_name = icon_provider.icon_name (IconID.LOG_ERASE),
            tooltip_text = _("Clear logs")
        };
        clear_button.clicked.connect (on_clear_clicked);
    }
}

```



```

wrap_button = new Gtk.ToggleButton () {
    icon_name = icon_provider.icon_name (IconID.TXT_WRAP),
    tooltip_text = _("Word wrap")
};
wrap_button.active = true;
wrap_button.toggled.connect (on_wrap_toggled);

var level_label = new Gtk.Label (_("Level:")) {
    margin_start = 12
};

var level_list = new Gtk.StringList ({
    _("All"),
    "DEBUG",
    "INFO",
    "WARN",
    "ERROR",
    "CRITICAL"
});
level_filter = new Gtk.DropDown (level_list, null);
level_filter.selected = 0;
level_filter.notify["selected"].connect (on_level_changed);

search_entry = new Gtk.SearchEntry () {
    placeholder_text = _("Search logs..."),
    expand = true
};
search_entry.search_changed.connect (on_search_changed);

toolbar.append (clear_button);
toolbar.append (wrap_button);
toolbar.append (new Gtk.Separator (Gtk.Orientation.VERTICAL));
toolbar.append (level_label);
toolbar.append (level_filter);
toolbar.append (search_entry);

buffer = new Gtk.TextBuffer (null);
tag_debug = buffer.create_tag (null, foreground: "#888888");
tag_info = buffer.create_tag (null, foreground: "#4a9eff", weight: Pango.Weight.BOLD);
tag_warning = buffer.create_tag (null, foreground: "#f5a623", weight: Pango.Weight.BOLD);
tag_error = buffer.create_tag (null, foreground: "#e74c3c", weight: Pango.Weight.BOLD);
tag_critical = buffer.create_tag (null, foreground: "#ff4757", weight: Pango.Weight.BOLD);
tag_bold = buffer.create_tag (null, weight: Pango.Weight.BOLD);
tag_dim = buffer.create_tag (null, foreground: "#666666", style: Pango.Style.ITALIC);

```

```

text_view = new Gtk.TextView () {
    buffer = buffer,
    editable = false,
    monospace = true,
    wrap_mode = Gtk.WrapMode.WORD_CHAR,
    hscroll_policy = Gtk.ScrollablePolicy.NATURAL,
    vscroll_policy = Gtk.ScrollablePolicy.NATURAL
};
text_view.add_css_class ("log-view");

scrolled_window = new Gtk.ScrolledWindow () {
    child = text_view,
    hexpand = true,
    vexpand = true
};

append (toolbar);
append (scrolled_window);

logger.log_added.connect (on_log_added);
logger.log_cleared.connect (on_log_cleared);

foreach (var entry in logger.get_entries ()) {
    append_entry (entry);
}

}

private void on_log_added (Iide.LogEntry entry) {
    append_entry (entry);
}

private void append_entry (Iide.LogEntry entry) {
    buffer_lock.lock ();
    try {
        var timestamp = entry.get_timestamp_string ();
        var level_str = entry.level.to_string ();
        var domain = entry.domain;
        var message = entry.message;

        Gtk.TextTag tag;
        switch (entry.level) {
            case Iide.LogLevel.DEBUG:
                tag = tag_debug;
                break;
            case Iide.LogLevel.INFO:
                tag = tag_info;

```

```

        break;
    case Iide.LogLevel.WARNING:
        tag = tag_warning;
        break;
    case Iide.LogLevel.ERROR:
        tag = tag_error;
        break;
    case Iide.LogLevel.CRITICAL:
        tag = tag_critical;
        break;
    default:
        tag = tag_dim;
        break;
}

var ts_text = "[%s] ".printf (timestamp);
var level_text = "[%s] ".printf (level_str);
var domain_text = "[%s] ".printf (domain);

int offset = buffer.get_char_count ();

Gtk.TextIter iter;
buffer.get_end_iter (out iter);
buffer.insert (ref iter, ts_text, ts_text.length);
buffer.get_end_iter (out iter);
buffer.insert (ref iter, level_text, level_text.length);
buffer.get_end_iter (out iter);
buffer.insert (ref iter, domain_text, domain_text.length);
buffer.get_end_iter (out iter);
buffer.insert (ref iter, message, message.length);

if (entry.details != null) {
    buffer.get_end_iter (out iter);
    buffer.insert (ref iter, "\n ", 3);
    buffer.get_end_iter (out iter);
    buffer.insert (ref iter, entry.details, entry.details.length);
}

buffer.get_end_iter (out iter);
buffer.insert (ref iter, "\n", 1);

Gtk.TextIter start, end;
buffer.get_iter_at_offset (out start, offset);
buffer.get_iter_at_offset (out end, offset + (int) ts_text.length);
buffer.apply_tag (tag_dim, start, end);

```

```

        buffer.get_iter_at_offset (out start, offset + (int) ts_text.length);
        buffer.get_iter_at_offset (out end, offset + (int) ts_text.length + (int) level_text.length);
        buffer.apply_tag (tag, start, end);

        buffer.get_iter_at_offset (out start, offset + (int) ts_text.length + (int) level_text.length);
        buffer.get_iter_at_offset (out end, offset + (int) ts_text.length + (int) level_text.length + (int) level_text.length);
        buffer.apply_tag (tag_bold, start, end);

        if (entry.details != null) {
            int detail_start = offset + (int) ts_text.length + (int) level_text.length + (int) level_text.length;
            buffer.get_iter_at_offset (out start, detail_start);
            buffer.get_iter_at_offset (out end, detail_start + 3);
            buffer.apply_tag (tag_dim, start, end);
        }

        line_count++;
        if (auto_scroll) {
            scroll_to_end ();
        }

        trim_buffer ();
    } finally {
        buffer_lock.unlock ();
    }
}

private void scroll_to_end () {
    Gtk.TextIter end;
    buffer.get_end_iter (out end);
    text_view.scroll_to_iter (end, 0.0, true, 0.0, 1.0);
}

private void trim_buffer () {
    if (line_count > MAX_VISIBLE_LINES) {
        buffer_lock.lock ();
        try {
            Gtk.TextIter start;
            buffer.get_iter_at_line (out start, line_count - MAX_VISIBLE_LINES);
            Gtk.TextIter end = start;
            buffer.get_iter_at_line (out end, line_count - MAX_VISIBLE_LINES + 100);
            buffer.delete (ref start, ref end);
            line_count -= 100;
        } finally {
            buffer_lock.unlock ();
        }
    }
}

```

```

}

private void on_clear_clicked () {
    logger.clear ();
}

private void on_log_cleared () {
    buffer.set_text ("");
    line_count = 0;
}

private void on_wrap_toggled () {
    text_view.wrap_mode = wrap_button.active ? Gtk.WrapMode.WORD_CHAR : Gtk.WrapMode.NONE;
}

private void on_level_changed () {
    rebuild_log_view ();
}

private void on_search_changed () {
    rebuild_log_view ();
}

private bool matches_search (Iide.LogEntry entry, string search_text) {
    if (search_text == "") {
        return true;
    }
    var lower_search = search_text.down ();
    return entry.message.down ().contains (lower_search) ||
        entry.domain.down ().contains (lower_search) ||
        (entry.details != null && entry.details.down ().contains (lower_search));
}

private void rebuild_log_view () {
    buffer_lock.lock ();
    try {
        buffer.set_text ("");
        line_count = 0;

        var filter_level = (int) level_filter.selected;
        var search_text = search_entry.text;

        foreach (var entry in logger.get_entries ()) {
            if (filter_level > 0) {
                var entry_level = (int) entry.level + 1;
                if (entry_level != filter_level) {

```

```

        continue;
    }
}
if (!matches_search (entry, search_text)) {
    continue;
}
append_entry_unlocked (entry);
}
} finally {
    buffer_lock.unlock ();
}
}

private void append_entry_unlocked (Iide.LogEntry entry) {
    Gtk.TextIter end;
    buffer.get_end_iter (out end);

    var timestamp = entry.get_timestamp_string ();
    var level_str = entry.level.to_string ();
    var domain = entry.domain;
    var message = entry.message;

    Gtk.TextTag tag;
    switch (entry.level) {
    case Iide.LogLevel.DEBUG:
        tag = tag_debug;
        break;
    case Iide.LogLevel.INFO:
        tag = tag_info;
        break;
    case Iide.LogLevel.WARNING:
        tag = tag_warning;
        break;
    case Iide.LogLevel.ERROR:
        tag = tag_error;
        break;
    case Iide.LogLevel.CRITICAL:
        tag = tag_critical;
        break;
    default:
        tag = tag_dim;
        break;
    }

    var ts_text = "[%s] ".printf (timestamp);
    var level_text = "[%s] ".printf (level_str);

```

```

var domain_text = "[%s] ".printf (domain);

var ts_start = end;
buffer.insert (ref end, ts_text, ts_text.length);
var ts_end = end;
buffer.apply_tag (tag_dim, ts_start, ts_end);

var level_start = end;
buffer.insert (ref end, level_text, level_text.length);
var level_end = end;
buffer.apply_tag (tag, level_start, level_end);

var domain_start = end;
buffer.insert (ref end, domain_text, domain_text.length);
var domain_end = end;
buffer.apply_tag (tag_bold, domain_start, domain_end);

buffer.insert (ref end, message, message.length);

if (entry.details != null) {
    var detail_start = end;
    buffer.insert (ref end, "\n ", 3);
    var detail_end = end;
    buffer.apply_tag (tag_dim, detail_start, detail_end);

    buffer.insert (ref end, entry.details, entry.details.length);
}

buffer.insert (ref end, "\n", 1);

line_count++;
}
}

```

File: symbols/meson.build

```

gnome = import('gnome')
python = import('python').find_installation('python3')

icons_json = files('icons.json')
font_file = files('fonts/SYMBOLIC.ttf')
script = files('export_glyph.py')

# SVG ( build/symbols/gen_icons)
gen_dir = meson.current_build_dir() / 'gen_icons'

```

```

symbols_gen = custom_target(
    'symbols-gen',
    input: [icons_json, font_file],
    # Meson
    output: ['symbols.gresource.xml', 'gen_icons'],
    command: [
        python, script,
        '@INPUT0@', # json
        '@INPUT1@', # ttf
        gen_dir, # SVG
        '@OUTPUT0@' # XML (Meson build/symbols/symbols.gresource.xml)
    ],
    build_by_default: true
)

symbols_res = gnome.compile_resources(
    'symbols-resources',
    symbols_gen[0], # output (XML)
    source_dir: gen_dir,
    dependencies: symbols_gen
)

```

File: Doxyfile

```

# Doxyfile 1.16.1

# This file describes the settings to be used by the documentation system
# Doxygen (www.doxygen.org) for a project.
#
# All text after a double hash (##) is considered a comment and is placed in
# front of the TAG it is preceding.
#
# All text after a single hash (#) is considered a comment and will be ignored.
# The format is:
# TAG = value [value, ...]
# For lists, items can also be appended using:
# TAG += value [value, ...]
# Values that contain spaces should be placed between quotes (\" \").
#
# Note:
#
# Use Doxygen to compare the used configuration file with the template
# configuration file:
# doxygen -x [configFile]

```



```

# Use Doxygen to compare the used configuration file with the template
# configuration file without replacing the environment variables or CMake type
# replacement variables:
# doxygen -x_noenv [configFile]

#-----
# Project related configuration options
#-----

# This tag specifies the encoding used for all characters in the configuration
# file that follow. The default is UTF-8 which is also the encoding used for all
# text before the first occurrence of this tag. Doxygen uses libiconv (or the
# iconv built into libc) for the transcoding. See
# https://www.gnu.org/software/libiconv/ for the list of possible encodings.
# The default value is: UTF-8.

DOXYFILE_ENCODING      = UTF-8

# The PROJECT_NAME tag is a single word (or a sequence of words surrounded by
# double-quotes, unless you are using Doxywizard) that should identify the
# project for which the documentation is generated. This name is used in the
# title of most generated pages and in a few other places.
# The default value is: My Project.

PROJECT_NAME           = "IIDE"

# The PROJECT_NUMBER tag can be used to enter a project or revision number. This
# could be handy for archiving the generated documentation or if some version
# control system is used.

PROJECT_NUMBER         =

# Using the PROJECT_BRIEF tag one can provide an optional one line description
# for a project that appears at the top of each page and should give viewers a
# quick idea about the purpose of the project. Keep the description short.

PROJECT_BRIEF          =

# With the PROJECT_LOGO tag one can specify a logo or an icon that is included
# in the documentation. The maximum height of the logo should not exceed 55
# pixels and the maximum width should not exceed 200 pixels. Doxygen will copy
# the logo to the output directory.

PROJECT_LOGO           =

# With the PROJECT_ICON tag one can specify an icon that is included in the tabs

```

```

# when the HTML document is shown. Doxygen will copy the logo to the output
# directory.

PROJECT_ICON          =

# The OUTPUT_DIRECTORY tag is used to specify the (relative or absolute) path
# into which the generated documentation will be written. If a relative path is
# entered, it will be relative to the location where Doxygen was started. If
# left blank the current directory will be used.

OUTPUT_DIRECTORY      = build/docs

# If the CREATE_SUBDIRS tag is set to YES then Doxygen will create up to 4096
# sub-directories (in 2 levels) under the output directory of each output format
# and will distribute the generated files over these directories. Enabling this
# option can be useful when feeding Doxygen a huge amount of source files, where
# putting all generated files in the same directory would otherwise cause
# performance problems for the file system. Adapt CREATE_SUBDIRS_LEVEL to
# control the number of sub-directories.
# The default value is: NO.

CREATE_SUBDIRS        = NO

# Controls the number of sub-directories that will be created when
# CREATE_SUBDIRS tag is set to YES. Level 0 represents 16 directories, and every
# level increment doubles the number of directories, resulting in 4096
# directories at level 8 which is the default and also the maximum value. The
# sub-directories are organized in 2 levels, the first level always has a fixed
# number of 16 directories.
# Minimum value: 0, maximum value: 8, default value: 8.
# This tag requires that the tag CREATE_SUBDIRS is set to YES.

CREATE_SUBDIRS_LEVEL  = 8

# If the ALLOW_UNICODE_NAMES tag is set to YES, Doxygen will allow non-ASCII
# characters to appear in the names of generated files. If set to NO, non-ASCII
# characters will be escaped, for example _xE3_x81_x84 will be used for Unicode
# U+3044.
# The default value is: NO.

ALLOW_UNICODE_NAMES   = NO

# The OUTPUT_LANGUAGE tag is used to specify the language in which all
# documentation generated by Doxygen is written. Doxygen will use this
# information to generate all constant output in the proper language.
# Possible values are: Afrikaans, Arabic, Armenian, Brazilian, Bulgarian,

```

```
# Catalan, Chinese, Chinese-Traditional, Croatian, Czech, Danish, Dutch, English
# (United States), Esperanto, Farsi (Persian), Finnish, French, German, Greek,
# Hindi, Hungarian, Indonesian, Italian, Japanese, Japanese-en (Japanese with
# English messages), Korean, Korean-en (Korean with English messages), Latvian,
# Lithuanian, Macedonian, Norwegian, Persian (Farsi), Polish, Portuguese,
# Romanian, Russian, Serbian, Serbian-Cyrillic, Slovak, Slovene, Spanish,
# Swedish, Turkish, Ukrainian and Vietnamese.
# The default value is: English.
```

```
OUTPUT_LANGUAGE          = English
```

```
# If the BRIEF_MEMBER_DESC tag is set to YES, Doxygen will include brief member
# descriptions after the members that are listed in the file and class
# documentation (similar to Javadoc). Set to NO to disable this.
# The default value is: YES.
```

```
BRIEF_MEMBER_DESC       = YES
```

```
# If the REPEAT_BRIEF tag is set to YES, Doxygen will prepend the brief
# description of a member or function before the detailed description
#
# Note: If both HIDE_UNDOC_MEMBERS and BRIEF_MEMBER_DESC are set to NO, the
# brief descriptions will be completely suppressed.
# The default value is: YES.
```

```
REPEAT_BRIEF            = YES
```

```
# This tag implements a quasi-intelligent brief description abbreviator that is
# used to form the text in various listings. Each string in this list, if found
# as the leading text of the brief description, will be stripped from the text
# and the result, after processing the whole list, is used as the annotated
# text. Otherwise, the brief description is used as-is. If left blank, the
# following values are used ($name is automatically replaced with the name of
# the entity):The $name class, The $name widget, The $name file, is, provides,
# specifies, contains, represents, a, an and the.
```

```
ABBREVIATE_BRIEF        = "The $name class" \
                           "The $name widget" \
                           "The $name file" \
                           is \
                           provides \
                           specifies \
                           contains \
                           represents \
                           a \
                           an \
```

the

```
# If the ALWAYS_DETAILED_SEC and REPEAT_BRIEF tags are both set to YES then
# Doxygen will generate a detailed section even if there is only a brief
# description.
# The default value is: NO.
```

```
ALWAYS_DETAILED_SEC      = NO
```

```
# If the INLINE_INHERITED_MEMB tag is set to YES, Doxygen will show all
# inherited members of a class in the documentation of that class as if those
# members were ordinary class members. Constructors, destructors and assignment
# operators of the base classes will not be shown.
# The default value is: NO.
```

```
INLINE_INHERITED_MEMB    = NO
```

```
# If the FULL_PATH_NAMES tag is set to YES, Doxygen will prepend the full path
# before file name in the file list and in the header files. If set to NO the
# shortest path that makes the file name unique will be used
# The default value is: YES.
```

```
FULL_PATH_NAMES          = YES
```

```
# The STRIP_FROM_PATH tag can be used to strip a user-defined part of the path.
# Stripping is only done if one of the specified strings matches the left-hand
# part of the path. The tag can be used to show relative paths in the file list.
# If left blank the directory from which Doxygen is run is used as the path to
# strip.
#
# Note that you can specify absolute paths here, but also relative paths, which
# will be relative from the directory where Doxygen is started.
# This tag requires that the tag FULL_PATH_NAMES is set to YES.
```

```
STRIP_FROM_PATH          =
```

```
# The STRIP_FROM_INC_PATH tag can be used to strip a user-defined part of the
# path mentioned in the documentation of a class, which tells the reader which
# header file to include in order to use a class. If left blank only the name of
# the header file containing the class definition is used. Otherwise one should
# specify the list of include paths that are normally passed to the compiler
# using the -I flag.
```

```
STRIP_FROM_INC_PATH      =
```

```
# If the SHORT_NAMES tag is set to YES, Doxygen will generate much shorter (but
```

```
# less readable) file names. This can be useful if your file system doesn't
# support long names like on DOS, Mac, or CD-ROM.
# The default value is: NO.
```

```
SHORT_NAMES          = NO
```

```
# If the JAVADOC_AUTOBRIEF tag is set to YES then Doxygen will interpret the
# first line (until the first dot, question mark or exclamation mark) of a
# Javadoc-style comment as the brief description. If set to NO, the Javadoc-
# style will behave just like regular Qt-style comments (thus requiring an
# explicit @brief command for a brief description.)
# The default value is: NO.
```

```
JAVADOC_AUTOBRIEF    = NO
```

```
# If the JAVADOC_BANNER tag is set to YES then Doxygen will interpret a line
# such as
# /*****
# as being the beginning of a Javadoc-style comment "banner". If set to NO, the
# Javadoc-style will behave just like regular comments and it will not be
# interpreted by Doxygen.
# The default value is: NO.
```

```
JAVADOC_BANNER       = NO
```

```
# If the QT_AUTOBRIEF tag is set to YES then Doxygen will interpret the first
# line (until the first dot, question mark or exclamation mark) of a Qt-style
# comment as the brief description. If set to NO, the Qt-style will behave just
# like regular Qt-style comments (thus requiring an explicit \brief command for
# a brief description.)
# The default value is: NO.
```

```
QT_AUTOBRIEF         = NO
```

```
# The MULTILINE_CPP_IS_BRIEF tag can be set to YES to make Doxygen treat a
# multi-line C++ special comment block (i.e. a block of /*! or ///  
# a brief description. This used to be the default behavior. The new default is
# to treat a multi-line C++ comment block as a detailed description. Set this
# tag to YES if you prefer the old behavior instead.
#
# Note that setting this tag to YES also means that rational rose comments are
# not recognized any more.
# The default value is: NO.
```

```
MULTILINE_CPP_IS_BRIEF = NO
```

```

# By default Python docstrings are displayed as preformatted text and Doxygen's
# special commands cannot be used. By setting PYTHON_DOCSTRING to NO the
# Doxygen's special commands can be used and the contents of the docstring
# documentation blocks is shown as Doxygen documentation.
# The default value is: YES.

PYTHON_DOCSTRING          = YES

# If the INHERIT_DOCS tag is set to YES then an undocumented member inherits the
# documentation from any documented member that it re-implements.
# The default value is: YES.

INHERIT_DOCS              = YES

# If the SEPARATE_MEMBER_PAGES tag is set to YES then Doxygen will produce a new
# page for each member. If set to NO, the documentation of a member will be part
# of the file/class/namespace that contains it.
# The default value is: NO.

SEPARATE_MEMBER_PAGES     = NO

# The TAB_SIZE tag can be used to set the number of spaces in a tab. Doxygen
# uses this value to replace tabs by spaces in code fragments.
# Minimum value: 1, maximum value: 16, default value: 4.

TAB_SIZE                  = 4

# This tag can be used to specify a number of aliases that act as commands in
# the documentation. An alias has the form:
# name=value
# For example adding
# "sideeffect=@par Side Effects:^^"
# will allow you to put the command \sideeffect (or @sideeffect) in the
# documentation, which will result in a user-defined paragraph with heading
# "Side Effects:". Note that you cannot put \n's in the value part of an alias
# to insert newlines (in the resulting output). You can put ^^ in the value part
# of an alias to insert a newline as if a physical newline was in the original
# file. When you need a literal { or } or , in the value part of an alias you
# have to escape them by means of a backslash (\), this can lead to conflicts
# with the commands \{ and \} for these it is advised to use the version @{ and
# @} or use a double escape (\\{ and \\})

ALIASES                   =

# Set the OPTIMIZE_OUTPUT_FOR_C tag to YES if your project consists of C sources
# only. Doxygen will then generate output that is more tailored for C. For

```

```
# instance, some of the names that are used will be different. The list of all
# members will be omitted, etc.
# The default value is: NO.
```

```
OPTIMIZE_OUTPUT_FOR_C = NO
```

```
# Set the OPTIMIZE_OUTPUT_JAVA tag to YES if your project consists of Java or
# Python sources only. Doxygen will then generate output that is more tailored
# for that language. For instance, namespaces will be presented as packages,
# qualified scopes will look different, etc.
# The default value is: NO.
```

```
OPTIMIZE_OUTPUT_JAVA = NO
```

```
# Set the OPTIMIZE_FOR_FORTRAN tag to YES if your project consists of Fortran
# sources. Doxygen will then generate output that is tailored for Fortran.
# The default value is: NO.
```

```
OPTIMIZE_FOR_FORTRAN = NO
```

```
# Set the OPTIMIZE_OUTPUT_VHDL tag to YES if your project consists of VHDL
# sources. Doxygen will then generate output that is tailored for VHDL.
# The default value is: NO.
```

```
OPTIMIZE_OUTPUT_VHDL = NO
```

```
# Set the OPTIMIZE_OUTPUT_SLICE tag to YES if your project consists of Slice
# sources only. Doxygen will then generate output that is more tailored for that
# language. For instance, namespaces will be presented as modules, types will be
# separated into more groups, etc.
# The default value is: NO.
```

```
OPTIMIZE_OUTPUT_SLICE = NO
```

```
# Doxygen selects the parser to use depending on the extension of the files it
# parses. With this tag you can assign which parser to use for a given
# extension. Doxygen has a built-in mapping, but you can override or extend it
# using this tag. The format is ext=language, where ext is a file extension, and
# language is one of the parsers supported by Doxygen: IDL, Java, JavaScript,
# Csharp (C#), C, C++, Lex, D, PHP, md (Markdown), Objective-C, Python, Slice,
# VHDL, Fortran (fixed format Fortran: FortranFixed, free formatted Fortran:
# FortranFree, unknown formatted Fortran: Fortran. In the later case the parser
# tries to guess whether the code is fixed or free formatted code, this is the
# default for Fortran type files). For instance to make Doxygen treat .inc files
# as Fortran files (default is PHP), and .f files as C (default is Fortran),
# use: inc=Fortran f=C.
```

```

#
# Note: For files without extension you can use no_extension as a placeholder.
#
# Note that for custom extensions you also need to set FILE_PATTERNS otherwise
# the files are not read by Doxygen. When specifying no_extension you should add
# * to the FILE_PATTERNS.
#
# Note see also the list of default file extension mappings.

EXTENSION_MAPPING      = vala=CSharp

# If the MARKDOWN_SUPPORT tag is enabled then Doxygen pre-processes all comments
# according to the Markdown format, which allows for more readable
# documentation. See https://daringfireball.net/projects/markdown/ for details.
# The output of markdown processing is further processed by Doxygen, so you can
# mix Doxygen, HTML, and XML commands with Markdown formatting. Disable only in
# case of backward compatibilities issues.
# The default value is: YES.

MARKDOWN_SUPPORT       = YES

# If the MARKDOWN_STRICT tag is enabled then Doxygen treats text in comments as
# Markdown formatted also in cases where Doxygen's native markup format
# conflicts with that of Markdown. This is only relevant in cases where
# backticks are used. Doxygen's native markup style allows a single quote to end
# a text fragment started with a backtick and then treat it as a piece of quoted
# text, whereas in Markdown such text fragment is treated as verbatim and only
# ends when a second matching backtick is found. Also, Doxygen's native markup
# format requires double quotes to be escaped when they appear in a backtick
# section, whereas this is not needed for Markdown.
# The default value is: YES.
# This tag requires that the tag MARKDOWN_SUPPORT is set to YES.

MARKDOWN_STRICT        = YES

# When the TOC_INCLUDE_HEADINGS tag is set to a non-zero value, all headings up
# to that level are automatically included in the table of contents, even if
# they do not have an id attribute.
# Note: This feature currently applies only to Markdown headings.
# Minimum value: 0, maximum value: 99, default value: 6.
# This tag requires that the tag MARKDOWN_SUPPORT is set to YES.

TOC_INCLUDE_HEADINGS   = 6

# The MARKDOWN_ID_STYLE tag can be used to specify the algorithm used to
# generate identifiers for the Markdown headings. Note: Every identifier is

```



```

# unique.
# Possible values are: DOXYGEN use a fixed 'autotoc_md' string followed by a
# sequence number starting at 0 and GITHUB use the lower case version of title
# with any whitespace replaced by '-' and punctuation characters removed.
# The default value is: DOXYGEN.
# This tag requires that the tag MARKDOWN_SUPPORT is set to YES.

MARKDOWN_ID_STYLE      = DOXYGEN

# When enabled Doxygen tries to link words that correspond to documented
# classes, or namespaces to their corresponding documentation. Such a link can
# be prevented in individual cases by putting a % sign in front of the word or
# globally by setting AUTOLINK_SUPPORT to NO. Words listed in the
# AUTOLINK_IGNORE_WORDS tag are excluded from automatic linking.
# The default value is: YES.

AUTOLINK_SUPPORT       = YES

# This tag specifies a list of words that, when matching the start of a word in
# the documentation, will suppress auto links generation, if it is enabled via
# AUTOLINK_SUPPORT. This list does not affect links explicitly created using #
# or the \link or \ref commands.
# This tag requires that the tag AUTOLINK_SUPPORT is set to YES.

AUTOLINK_IGNORE_WORDS  =

# If you use STL classes (i.e. std::string, std::vector, etc.) but do not want
# to include (a tag file for) the STL sources as input, then you should set this
# tag to YES in order to let Doxygen match functions declarations and
# definitions whose arguments contain STL classes (e.g. func(std::string);
# versus func(std::string) {}). This also makes the inheritance and
# collaboration diagrams that involve STL classes more complete and accurate.
# The default value is: NO.

BUILTIN_STL_SUPPORT    = NO

# If you use Microsoft's C++/CLI language, you should set this option to YES to
# enable parsing support.
# The default value is: NO.

CPP_CLI_SUPPORT        = NO

# Set the SIP_SUPPORT tag to YES if your project consists of sip (see:
# https://www.riverbankcomputing.com/software) sources only. Doxygen will parse
# them like normal C++ but will assume all classes use public instead of private
# inheritance when no explicit protection keyword is present.

```

The default value is: NO.

SIP_SUPPORT = NO

For Microsoft's IDL there are propget and propput attributes to indicate
getter and setter methods for a property. Setting this option to YES will make
Doxygen to replace the get and set methods by a property in the documentation.
This will only work if the methods are indeed getting or setting a simple
type. If this is not the case, or you want to show the methods anyway, you
should set this option to NO.
The default value is: YES.

IDL_PROPERTY_SUPPORT = YES

If member grouping is used in the documentation and the DISTRIBUTE_GROUP_DOC
tag is set to YES then Doxygen will reuse the documentation of the first
member in the group (if any) for the other members of the group. By default
all members of a group must be documented explicitly.
The default value is: NO.

DISTRIBUTE_GROUP_DOC = NO

If one adds a struct or class to a group and this option is enabled, then also
any nested class or struct is added to the same group. By default this option
is disabled and one has to add nested compounds explicitly via \ingroup.
The default value is: NO.

GROUP_NESTED_COMPOUNDS = NO

Set the SUBGROUPING tag to YES to allow class member groups of the same type
(for instance a group of public functions) to be put as a subgroup of that
type (e.g. under the Public Functions section). Set it to NO to prevent
subgrouping. Alternatively, this can be done per class using the
\nosubgrouping command.
The default value is: YES.

SUBGROUPING = YES

When the INLINE_GROUPED_CLASSES tag is set to YES, classes, structs and unions
are shown inside the group in which they are included (e.g. using \ingroup)
instead of on a separate page (for HTML and Man pages) or section (for LaTeX
and RTF).

Note that this feature does not work in combination with
SEPARATE_MEMBER_PAGES.
The default value is: NO.

INLINE_GROUPED_CLASSES = NO

When the INLINE_SIMPLE_STRUCTS tag is set to YES, structs, classes, and unions
with only public data fields or simple typedef fields will be shown inline in
the documentation of the scope in which they are defined (i.e. file,
namespace, or group documentation), provided this scope is documented. If set
to NO, structs, classes, and unions are shown on a separate page (for HTML and
Man pages) or section (for LaTeX and RTF).
The default value is: NO.

INLINE_SIMPLE_STRUCTS = NO

When TYPEDEF_HIDES_STRUCT tag is enabled, a typedef of a struct, union, or
enum is documented as struct, union, or enum with the name of the typedef. So
typedef struct TypeS {} TypeT, will appear in the documentation as a struct
with name TypeT. When disabled the typedef will appear as a member of a file,
namespace, or class. And the struct will be named TypeS. This can typically be
useful for C code in case the coding convention dictates that all compound
types are typedef'ed and only the typedef is referenced, never the tag name.
The default value is: NO.

TYPEDEF_HIDES_STRUCT = NO

The size of the symbol lookup cache can be set using LOOKUP_CACHE_SIZE. This
cache is used to resolve symbols given their name and scope. Since this can be
an expensive process and often the same symbol appears multiple times in the
code, Doxygen keeps a cache of pre-resolved symbols. If the cache is too small
Doxygen will become slower. If the cache is too large, memory is wasted. The
cache size is given by this formula: $2^{(16+LOOKUP_CACHE_SIZE)}$. The valid range
is 0..9, the default is 0, corresponding to a cache size of $2^{16}=65536$
symbols. At the end of a run Doxygen will report the cache usage and suggest
the optimal cache size from a speed point of view.
Minimum value: 0, maximum value: 9, default value: 0.

LOOKUP_CACHE_SIZE = 0

The NUM_PROC_THREADS specifies the number of threads Doxygen is allowed to use
during processing. When set to 0 Doxygen will based this on the number of
cores available in the system. You can set it explicitly to a value larger
than 0 to get more control over the balance between CPU load and processing
speed. At this moment only the input processing can be done using multiple
threads. Since this is still an experimental feature the default is set to 1,
which effectively disables parallel processing. Please report any issues you
encounter. Generating dot graphs in parallel is controlled by the
DOT_NUM_THREADS setting.

```

# Minimum value: 0, maximum value: 512, default value: 1.

NUM_PROC_THREADS          = 1

# If the TIMESTAMP tag is set different from NO then each generated page will
# contain the date or date and time when the page was generated. Setting this to
# NO can help when comparing the output of multiple runs.
# Possible values are: YES, NO, DATETIME and DATE.
# The default value is: NO.

TIMESTAMP                  = NO

#-----
# Build related configuration options
#-----

# If the EXTRACT_ALL tag is set to YES, Doxygen will assume all entities in
# documentation are documented, even if no documentation was available. Private
# class members and static file members will be hidden unless the
# EXTRACT_PRIVATE respectively EXTRACT_STATIC tags are set to YES.
# Note: This will also disable the warnings about undocumented members that are
# normally produced when WARNINGS is set to YES.
# The default value is: NO.

EXTRACT_ALL                = YES

# If the EXTRACT_PRIVATE tag is set to YES, all private members of a class will
# be included in the documentation.
# The default value is: NO.

EXTRACT_PRIVATE            = YES

# If the EXTRACT_PRIV_VIRTUAL tag is set to YES, documented private virtual
# methods of a class will be included in the documentation.
# The default value is: NO.

EXTRACT_PRIV_VIRTUAL       = YES

# If the EXTRACT_PACKAGE tag is set to YES, all members with package or internal
# scope will be included in the documentation.
# The default value is: NO.

EXTRACT_PACKAGE            = NO

# If the EXTRACT_STATIC tag is set to YES, all static members of a file will be
# included in the documentation.

```

```

# The default value is: NO.

EXTRACT_STATIC          = YES

# If the EXTRACT_LOCAL_CLASSES tag is set to YES, classes (and structs) defined
# locally in source files will be included in the documentation. If set to NO,
# only classes defined in header files are included. Does not have any effect
# for Java sources.
# The default value is: YES.

EXTRACT_LOCAL_CLASSES   = YES

# This flag is only useful for Objective-C code. If set to YES, local methods,
# which are defined in the implementation section but not in the interface are
# included in the documentation. If set to NO, only methods in the interface are
# included.
# The default value is: NO.

EXTRACT_LOCAL_METHODS   = NO

# If this flag is set to YES, the members of anonymous namespaces will be
# extracted and appear in the documentation as a namespace called
# 'anonymous_namespace{file}', where file will be replaced with the base name of
# the file that contains the anonymous namespace. By default anonymous namespace
# are hidden.
# The default value is: NO.

EXTRACT_ANON_NSPPACES   = NO

# If this flag is set to YES, the name of an unnamed parameter in a declaration
# will be determined by the corresponding definition. By default unnamed
# parameters remain unnamed in the output.
# The default value is: YES.

RESOLVE_UNNAMED_PARAMS  = YES

# If the HIDE_UNDOC_MEMBERS tag is set to YES, Doxygen will hide all
# undocumented members inside documented classes or files. If set to NO these
# members will be included in the various overviews, but no documentation
# section is generated. This option has no effect if EXTRACT_ALL is enabled.
# The default value is: NO.

HIDE_UNDOC_MEMBERS      = NO

# If the HIDE_UNDOC_CLASSES tag is set to YES, Doxygen will hide all
# undocumented classes that are normally visible in the class hierarchy. If set

```

```
# to NO, these classes will be included in the various overviews. This option
# will also hide undocumented C++ concepts if enabled. This option has no effect
# if EXTRACT_ALL is enabled.
# The default value is: NO.
```

```
HIDE_UNDOC_CLASSES      = NO
```

```
# If the HIDE_UNDOC_NAMESPACES tag is set to YES, Doxygen will hide all
# undocumented namespaces that are normally visible in the namespace hierarchy.
# If set to NO, these namespaces will be included in the various overviews. This
# option has no effect if EXTRACT_ALL is enabled.
# The default value is: YES.
```

```
HIDE_UNDOC_NAMESPACES  = YES
```

```
# If the HIDE_FRIEND_COMPOUNDS tag is set to YES, Doxygen will hide all friend
# declarations. If set to NO, these declarations will be included in the
# documentation.
# The default value is: NO.
```

```
HIDE_FRIEND_COMPOUNDS  = NO
```

```
# If the HIDE_IN_BODY_DOCS tag is set to YES, Doxygen will hide any
# documentation blocks found inside the body of a function. If set to NO, these
# blocks will be appended to the function's detailed documentation block.
# The default value is: NO.
```

```
HIDE_IN_BODY_DOCS      = NO
```

```
# The INTERNAL_DOCS tag determines if documentation that is typed after a
# \internal command is included. If the tag is set to NO then the documentation
# will be excluded. Set it to YES to include the internal documentation.
# The default value is: NO.
```

```
INTERNAL_DOCS           = NO
```

```
# With the correct setting of option CASE_SENSE_NAMES Doxygen will better be
# able to match the capabilities of the underlying filesystem. In case the
# filesystem is case sensitive (i.e. it supports files in the same directory
# whose names only differ in casing), the option must be set to YES to properly
# deal with such files in case they appear in the input. For filesystems that
# are not case sensitive the option should be set to NO to properly deal with
# output files written for symbols that only differ in casing, such as for two
# classes, one named CLASS and the other named Class, and to also support
# references to files without having to specify the exact matching casing. On
# Windows (including Cygwin) and macOS, users should typically set this option
```

```

# to NO, whereas on Linux or other Unix flavors it should typically be set to
# YES.
# Possible values are: SYSTEM, NO and YES.
# The default value is: SYSTEM.

CASE_SENSE_NAMES          = SYSTEM

# If the HIDE_SCOPE_NAMES tag is set to NO then Doxygen will show members with
# their full class and namespace scopes in the documentation. If set to YES, the
# scope will be hidden.
# The default value is: NO.

HIDE_SCOPE_NAMES          = NO

# If the HIDE_COMPOUND_REFERENCE tag is set to NO (default) then Doxygen will
# append additional text to a page's title, such as Class Reference. If set to
# YES the compound reference will be hidden.
# The default value is: NO.

HIDE_COMPOUND_REFERENCE= NO

# If the SHOW_HEADERFILE tag is set to YES then the documentation for a class
# will show which file needs to be included to use the class.
# The default value is: YES.

SHOW_HEADERFILE           = YES

# If the SHOW_INCLUDE_FILES tag is set to YES then Doxygen will put a list of
# the files that are included by a file in the documentation of that file.
# The default value is: YES.

SHOW_INCLUDE_FILES        = YES

# If the SHOW_GROUPED_MEMB_INC tag is set to YES then Doxygen will add for each
# grouped member an include statement to the documentation, telling the reader
# which file to include in order to use the member.
# The default value is: NO.

SHOW_GROUPED_MEMB_INC     = NO

# If the FORCE_LOCAL_INCLUDES tag is set to YES then Doxygen will list include
# files with double quotes in the documentation rather than with sharp brackets.
# The default value is: NO.

FORCE_LOCAL_INCLUDES      = NO

```

If the `INLINE_INFO` tag is set to YES then a tag `[inline]` is inserted in the
documentation for inline members.
The default value is: YES.

`INLINE_INFO` = YES

If the `SORT_MEMBER_DOCS` tag is set to YES then Doxygen will sort the
(detailed) documentation of file and class members alphabetically by member
name. If set to NO, the members will appear in declaration order.
The default value is: YES.

`SORT_MEMBER_DOCS` = YES

If the `SORT_BRIEF_DOCS` tag is set to YES then Doxygen will sort the brief
descriptions of file, namespace and class members alphabetically by member
name. If set to NO, the members will appear in declaration order. Note that
this will also influence the order of the classes in the class list.
The default value is: NO.

`SORT_BRIEF_DOCS` = NO

If the `SORT_MEMBERS_CTORS_1ST` tag is set to YES then Doxygen will sort the
(brief and detailed) documentation of class members so that constructors and
destructors are listed first. If set to NO the constructors will appear in the
respective orders defined by `SORT_BRIEF_DOCS` and `SORT_MEMBER_DOCS`.
Note: If `SORT_BRIEF_DOCS` is set to NO this option is ignored for sorting brief
member documentation.
Note: If `SORT_MEMBER_DOCS` is set to NO this option is ignored for sorting
detailed member documentation.
The default value is: NO.

`SORT_MEMBERS_CTORS_1ST` = NO

If the `SORT_GROUP_NAMES` tag is set to YES then Doxygen will sort the hierarchy
of group names into alphabetical order. If set to NO the group names will
appear in their defined order.
The default value is: NO.

`SORT_GROUP_NAMES` = NO

If the `SORT_BY_SCOPE_NAME` tag is set to YES, the class list will be sorted by
fully-qualified names, including namespaces. If set to NO, the class list will
be sorted only by class name, not including the namespace part.
Note: This option is not very useful if `HIDE_SCOPE_NAMES` is set to YES.
Note: This option applies only to the class list, not to the alphabetical
list.

The default value is: NO.

`SORT_BY_SCOPE_NAME` = NO

If the `STRICT_PROTO_MATCHING` option is enabled and Doxygen fails to do proper
type resolution of all parameters of a function it will reject a match between
the prototype and the implementation of a member function even if there is
only one candidate or it is obvious which candidate to choose by doing a
simple string match. By disabling `STRICT_PROTO_MATCHING` Doxygen will still
accept a match between prototype and implementation in such cases.
The default value is: NO.

`STRICT_PROTO_MATCHING` = NO

The `GENERATE_TODOLIST` tag can be used to enable (YES) or disable (NO) the todo
list. This list is created by putting `\todo` commands in the documentation.
The default value is: YES.

`GENERATE_TODOLIST` = YES

The `GENERATE_TESTLIST` tag can be used to enable (YES) or disable (NO) the test
list. This list is created by putting `\test` commands in the documentation.
The default value is: YES.

`GENERATE_TESTLIST` = YES

The `GENERATE_BUGLIST` tag can be used to enable (YES) or disable (NO) the bug
list. This list is created by putting `\bug` commands in the documentation.
The default value is: YES.

`GENERATE_BUGLIST` = YES

The `GENERATE_DEPRECATEDLIST` tag can be used to enable (YES) or disable (NO)
the deprecated list. This list is created by putting `\deprecated` commands in
the documentation.
The default value is: YES.

`GENERATE_DEPRECATEDLIST`= YES

The `GENERATE_REQUIREMENTS` tag can be used to enable (YES) or disable (NO) the
requirements page. When enabled, this page is automatically created when at
least one comment block with a `\requirement` command appears in the input.
The default value is: YES.

`GENERATE_REQUIREMENTS` = YES

```

# The REQ_TRACEABILITY_INFO tag controls if traceability information is shown on
# the requirements page (only relevant when using \requirement comment blocks).
# The setting NO will disable the traceability information altogether. The
# setting UNSATISFIED_ONLY will show a list of requirements that are missing a
# satisfies relation (through the command: \satisfies). Similarly the setting
# UNVERIFIED_ONLY will show a list of requirements that are missing a verifies
# relation (through the command: \verifies). Setting the tag to YES (the
# default) will show both lists if applicable.
# Possible values are: YES, NO, UNSATISFIED_ONLY and UNVERIFIED_ONLY.
# The default value is: YES.
# This tag requires that the tag GENERATE_REQUIREMENTS is set to YES.

```

```

REQ_TRACEABILITY_INFO = YES

```

```

# The ENABLED_SECTIONS tag can be used to enable conditional documentation
# sections, marked by \if <section_label> ... \endif and \cond <section_label>
# ... \endcond blocks.

```

```

ENABLED_SECTIONS =

```

```

# The MAX_INITIALIZER_LINES tag determines the maximum number of lines that the
# initial value of a variable or macro / define can have for it to appear in the
# documentation. If the initializer consists of more lines than specified here
# it will be hidden. Use a value of 0 to hide initializers completely. The
# appearance of the value of individual variables and macros / defines can be
# controlled using \showinitializer or \hideinitializer command in the
# documentation regardless of this setting.
# Minimum value: 0, maximum value: 10000, default value: 30.

```

```

MAX_INITIALIZER_LINES = 30

```

```

# Set the SHOW_USED_FILES tag to NO to disable the list of files generated at
# the bottom of the documentation of classes and structs. If set to YES, the
# list will mention the files that were used to generate the documentation.
# The default value is: YES.

```

```

SHOW_USED_FILES = YES

```

```

# Set the SHOW_FILES tag to NO to disable the generation of the Files page. This
# will remove the Files entry from the Quick Index and from the Folder Tree View
# (if specified).
# The default value is: YES.

```

```

SHOW_FILES = YES

```

```

# Set the SHOW_NAMESPACES tag to NO to disable the generation of the Namespaces

```

```
# page. This will remove the Namespaces entry from the Quick Index and from the
# Folder Tree View (if specified).
# The default value is: YES.
```

```
SHOW_NAMESPACES      = YES
```

```
# The FILE_VERSION_FILTER tag can be used to specify a program or script that
# Doxygen should invoke to get the current version for each file (typically from
# the version control system). Doxygen will invoke the program by executing (via
# popen()) the command command input-file, where command is the value of the
# FILE_VERSION_FILTER tag, and input-file is the name of an input file provided
# by Doxygen. Whatever the program writes to standard output is used as the file
# version. For an example see the documentation.
```

```
FILE_VERSION_FILTER   =
```

```
# The LAYOUT_FILE tag can be used to specify a layout file which will be parsed
# by Doxygen. The layout file controls the global structure of the generated
# output files in an output format independent way. To create the layout file
# that represents Doxygen's defaults, run Doxygen with the -l option. You can
# optionally specify a file name after the option, if omitted DoxygenLayout.xml
# will be used as the name of the layout file. See also section "Changing the
# layout of pages" for information.
#
# Note that if you run Doxygen from a directory containing a file called
# DoxygenLayout.xml, Doxygen will parse it automatically even if the LAYOUT_FILE
# tag is left empty.
```

```
LAYOUT_FILE           =
```

```
# The CITE_BIB_FILES tag can be used to specify one or more bib files containing
# the reference definitions. This must be a list of .bib files. The .bib
# extension is automatically appended if omitted. This requires the bibtex tool
# to be installed. See also https://en.wikipedia.org/wiki/BibTeX for more info.
# For LaTeX the style of the bibliography can be controlled using
# LATEX_BIB_STYLE. To use this feature you need bibtex and perl available in the
# search path. See also \cite for info how to create references.
```

```
CITE_BIB_FILES        =
```

```
# The EXTERNAL_TOOL_PATH tag can be used to extend the search path (PATH
# environment variable) so that external tools such as latex and gs can be
# found.
# Note: Directories specified with EXTERNAL_TOOL_PATH are added in front of the
# path already specified by the PATH variable, and are added in the order
# specified.
```

```

# Note: This option is particularly useful for macOS version 14 (Sonoma) and
# higher, when running Doxygen from Doxywizard, because in this case any user-
# defined changes to the PATH are ignored. A typical example on macOS is to set
# EXTERNAL_TOOL_PATH = /Library/TeX/texbin /usr/local/bin
# together with the standard path, the full search path used by doxygen when
# launching external tools will then become
# PATH=/Library/TeX/texbin:/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin

EXTERNAL_TOOL_PATH      =

#-----
# Configuration options related to warning and progress messages
#-----

# The QUIET tag can be used to turn on/off the messages that are generated to
# standard output by Doxygen. If QUIET is set to YES this implies that the
# messages are off.
# The default value is: NO.

QUIET                   = NO

# The WARNINGS tag can be used to turn on/off the warning messages that are
# generated to standard error (stderr) by Doxygen. If WARNINGS is set to YES
# this implies that the warnings are on.
#
# Tip: Turn warnings on while writing the documentation.
# The default value is: YES.

WARNINGS                = YES

# If the WARN_IF_UNDOCUMENTED tag is set to YES then Doxygen will generate
# warnings for undocumented members. If EXTRACT_ALL is set to YES then this flag
# will automatically be disabled.
# The default value is: YES.

WARN_IF_UNDOCUMENTED    = YES

# If the WARN_IF_DOC_ERROR tag is set to YES, Doxygen will generate warnings for
# potential errors in the documentation, such as documenting some parameters in
# a documented function twice, or documenting parameters that don't exist or
# using markup commands wrongly.
# The default value is: YES.

WARN_IF_DOC_ERROR       = YES

# If WARN_IF_INCOMPLETE_DOC is set to YES, Doxygen will warn about incomplete

```

```

# function parameter documentation. If set to NO, Doxygen will accept that some
# parameters have no documentation without warning.
# The default value is: YES.

WARN_IF_INCOMPLETE_DOC = YES

# This WARN_NO_PARAMDOC option can be enabled to get warnings for functions that
# are documented, but have no documentation for their parameters or return
# value. If set to NO, Doxygen will only warn about wrong parameter
# documentation, but not about the absence of documentation. If EXTRACT_ALL is
# set to YES then this flag will automatically be disabled. See also
# WARN_IF_INCOMPLETE_DOC
# The default value is: NO.

WARN_NO_PARAMDOC          = NO

# If WARN_IF_UNDOC_ENUM_VAL option is set to YES, Doxygen will warn about
# undocumented enumeration values. If set to NO, Doxygen will accept
# undocumented enumeration values. If EXTRACT_ALL is set to YES then this flag
# will automatically be disabled.
# The default value is: NO.

WARN_IF_UNDOC_ENUM_VAL = NO

# If WARN_LAYOUT_FILE option is set to YES, Doxygen will warn about issues found
# while parsing the user defined layout file, such as missing or wrong elements.
# See also LAYOUT_FILE for details. If set to NO, problems with the layout file
# will be suppressed.
# The default value is: YES.

WARN_LAYOUT_FILE          = YES

# If the WARN_AS_ERROR tag is set to YES then Doxygen will immediately stop when
# a warning is encountered. If the WARN_AS_ERROR tag is set to FAIL_ON_WARNINGS
# then Doxygen will continue running as if WARN_AS_ERROR tag is set to NO, but
# at the end of the Doxygen process Doxygen will return with a non-zero status.
# If the WARN_AS_ERROR tag is set to FAIL_ON_WARNINGS_PRINT then Doxygen behaves
# like FAIL_ON_WARNINGS but in case no WARN_LOGFILE is defined Doxygen will not
# write the warning messages in between other messages but write them at the end
# of a run, in case a WARN_LOGFILE is defined the warning messages will be
# besides being in the defined file also be shown at the end of a run, unless
# the WARN_LOGFILE is defined as - i.e. standard output (stdout) in that case
# the behavior will remain as with the setting FAIL_ON_WARNINGS.
# Possible values are: NO, YES, FAIL_ON_WARNINGS and FAIL_ON_WARNINGS_PRINT.
# The default value is: NO.

```

```

WARN_AS_ERROR          = NO

# The WARN_FORMAT tag determines the format of the warning messages that Doxygen
# can produce. The string should contain the $file, $line, and $text tags, which
# will be replaced by the file and line number from which the warning originated
# and the warning text. Optionally the format may contain $version, which will
# be replaced by the version of the file (if it could be obtained via
# FILE_VERSION_FILTER)
# See also: WARN_LINE_FORMAT
# The default value is: $file:$line: $text.

WARN_FORMAT            = "$file:$line: $text"

# In the $text part of the WARN_FORMAT command it is possible that a reference
# to a more specific place is given. To make it easier to jump to this place
# (outside of Doxygen) the user can define a custom "cut" / "paste" string.
# Example:
# WARN_LINE_FORMAT = "'vi $file +$line'"
# See also: WARN_FORMAT
# The default value is: at line $line of file $file.

WARN_LINE_FORMAT       = "at line $line of file $file"

# The WARN_LOGFILE tag can be used to specify a file to which warning and error
# messages should be written. If left blank the output is written to standard
# error (stderr). In case the file specified cannot be opened for writing the
# warning and error messages are written to standard error. When as file - is
# specified the warning and error messages are written to standard output
# (stdout).

WARN_LOGFILE           =

#-----
# Configuration options related to the input files
#-----

# The INPUT tag is used to specify the files and/or directories that contain
# documented source files. You may enter file names like myfile.cpp or
# directories like /usr/src/myproject. Separate the files or directories with
# spaces. See also FILE_PATTERNS and EXTENSION_MAPPING
# Note: If this tag is empty the current directory is searched.

INPUT                  = src

# This tag can be used to specify the character encoding of the source files
# that Doxygen parses. Internally Doxygen uses the UTF-8 encoding. Doxygen uses

```

```
# libiconv (or the iconv built into libc) for the transcoding. See the libiconv
# documentation (see:
# https://www.gnu.org/software/libiconv/) for the list of possible encodings.
# See also: INPUT_FILE_ENCODING
# The default value is: UTF-8.
```

```
INPUT_ENCODING          = UTF-8
```

```
# This tag can be used to specify the character encoding of the source files
# that Doxygen parses. The INPUT_FILE_ENCODING tag can be used to specify
# character encoding on a per file pattern basis. Doxygen will compare the file
# name with each pattern and apply the encoding instead of the default
# INPUT_ENCODING if there is a match. The character encodings are a list of the
# form: pattern=encoding (like *.php=ISO-8859-1).
# See also: INPUT_ENCODING for further information on supported encodings.
```

```
INPUT_FILE_ENCODING     =
```

```
# If the value of the INPUT tag contains directories, you can use the
# FILE_PATTERNS tag to specify one or more wildcard patterns (like *.cpp and
# *.h) to filter out the source-files in the directories.
#
# Note that for custom extensions or not directly supported extensions you also
# need to set EXTENSION_MAPPING for the extension otherwise the files are not
# read by Doxygen.
#
# Note the list of default checked file patterns might differ from the list of
# default file extension mappings.
#
# If left blank the following patterns are tested:*.c, *.cc, *.cxx, *.cxxm,
# *.cpp, *.cppm, *.ccm, *.c++, *.c++m, *.java, *.ii, *.ixx, *.ipp, *.i++, *.inl,
# *.idl, *.ddl, *.odl, *.h, *.hh, *.hxx, *.hpp, *.h++, *.l, *.cs, *.d, *.php,
# *.php4, *.php5, *.phtml, *.inc, *.m, *.markdown, *.md, *.mm, *.dox (to be
# provided as Doxygen C comment), *.py, *.pyw, *.f90, *.f95, *.f03, *.f08,
# *.f18, *.f, *.for, *.vhd, *.vhdl, *.ucf, *.qsf and *.ice.
```

```
FILE_PATTERNS           = *.vala *.vapi
                        *.cc \
                        *.cxx \
                        *.cxxm \
                        *.cpp \
                        *.cppm \
                        *.ccm \
                        *.c++ \
                        *.c++m \
                        *.java \
```

```

*.ii \
*.ixx \
*.ipp \
*.i++ \
*.inl \
*.idl \
*.ddl \
*.odl \
*.h \
*.hh \
*.hxx \
*.hpp \
*.h++ \
*.l \
*.cs \
*.d \
*.php \
*.php4 \
*.php5 \
*.phtml \
*.inc \
*.m \
*.markdown \
*.md \
*.mm \
*.dox \
*.py \
*.pyw \
*.f90 \
*.f95 \
*.f03 \
*.f08 \
*.f18 \
*.f \
*.for \
*.vhd \
*.vhdl \
*.ucf \
*.qsf \
*.ice

```

```

# The RECURSIVE tag can be used to specify whether or not subdirectories should
# be searched for input files as well.
# The default value is: NO.

```

```

RECURSIVE          = YES

```



```
# The EXCLUDE tag can be used to specify files and/or directories that should be
# excluded from the INPUT source files. This way you can easily exclude a
# subdirectory from a directory tree whose root is specified with the INPUT tag.
#
# Note that relative paths are relative to the directory from which Doxygen is
# run.
```

```
EXCLUDE =
```

```
# The EXCLUDE_SYMLINKS tag can be used to select whether or not files or
# directories that are symbolic links (a Unix file system feature) are excluded
# from the input.
# The default value is: NO.
```

```
EXCLUDE_SYMLINKS = NO
```

```
# If the value of the INPUT tag contains directories, you can use the
# EXCLUDE_PATTERNS tag to specify one or more wildcard patterns to exclude
# certain files from those directories.
#
# Note that the wildcards are matched against the file with absolute path, so to
# exclude all test directories for example use the pattern */test/*
```

```
EXCLUDE_PATTERNS =
```

```
# The EXCLUDE_SYMBOLS tag can be used to specify one or more symbol names
# (namespaces, classes, functions, etc.) that should be excluded from the
# output. The symbol name can be a fully qualified name, a word, or if the
# wildcard * is used, a substring. Examples: ANamespace, AClass,
# ANamespace::AClass, ANamespace::*Test
```

```
EXCLUDE_SYMBOLS =
```

```
# The EXAMPLE_PATH tag can be used to specify one or more files or directories
# that contain example code fragments that are included (see the \include
# command).
```

```
EXAMPLE_PATH =
```

```
# If the value of the EXAMPLE_PATH tag contains directories, you can use the
# EXAMPLE_PATTERNS tag to specify one or more wildcard pattern (like *.cpp and
# *.h) to filter out the source-files in the directories. If left blank all
# files are included.
```

```
EXAMPLE_PATTERNS = *
```

```
# If the EXAMPLE_RECURSIVE tag is set to YES then subdirectories will be
# searched for input files to be used with the \include or \dontinclude commands
# irrespective of the value of the RECURSIVE tag.
# The default value is: NO.
```

```
EXAMPLE_RECURSIVE      = NO
```

```
# The IMAGE_PATH tag can be used to specify one or more files or directories
# that contain images that are to be included in the documentation (see the
# \image command).
```

```
IMAGE_PATH             =
```

```
# The INPUT_FILTER tag can be used to specify a program that Doxygen should
# invoke to filter for each input file. Doxygen will invoke the filter program
# by executing (via popen()) the command:
#
# <filter> <input-file>
#
# where <filter> is the value of the INPUT_FILTER tag, and <input-file> is the
# name of an input file. Doxygen will then use the output that the filter
# program writes to standard output. If FILTER_PATTERNS is specified, this tag
# will be ignored.
#
# Note that the filter must not add or remove lines; it is applied before the
# code is scanned, but not when the output code is generated. If lines are added
# or removed, the anchors will not be placed correctly.
#
# Note that Doxygen will use the data processed and written to standard output
# for further processing, therefore nothing else, like debug statements or used
# commands (so in case of a Windows batch file always use @echo OFF), should be
# written to standard output.
#
# Note that for custom extensions or not directly supported extensions you also
# need to set EXTENSION_MAPPING for the extension otherwise the files are not
# properly processed by Doxygen.
```

```
INPUT_FILTER           =
```

```
# The FILTER_PATTERNS tag can be used to specify filters on a per file pattern
# basis. Doxygen will compare the file name with each pattern and apply the
# filter if there is a match. The filters are a list of the form: pattern=filter
# (like *.cpp=my_cpp_filter). See INPUT_FILTER for further information on how
# filters are used. If the FILTER_PATTERNS tag is empty or if none of the
# patterns match the file name, INPUT_FILTER is applied.
```

```

#
# Note that for custom extensions or not directly supported extensions you also
# need to set EXTENSION_MAPPING for the extension otherwise the files are not
# properly processed by Doxygen.

FILTER_PATTERNS          =

# If the FILTER_SOURCE_FILES tag is set to YES, the input filter (if set using
# INPUT_FILTER) will also be used to filter the input files that are used for
# producing the source files to browse (i.e. when SOURCE_BROWSER is set to YES).
# The default value is: NO.

FILTER_SOURCE_FILES      = NO

# The FILTER_SOURCE_PATTERNS tag can be used to specify source filters per file
# pattern. A pattern will override the setting for FILTER_PATTERN (if any) and
# it is also possible to disable source filtering for a specific pattern using
# *.ext= (so without naming a filter).
# This tag requires that the tag FILTER_SOURCE_FILES is set to YES.

FILTER_SOURCE_PATTERNS =

# If the USE_MDFILE_AS_MAINPAGE tag refers to the name of a markdown file that
# is part of the input, its contents will be placed on the main page
# (index.html). This can be useful if you have a project on for instance GitHub
# and want to reuse the introduction page also for the Doxygen output.

USE_MDFILE_AS_MAINPAGE =

# If the IMPLICIT_DIR_DOCS tag is set to YES, any README.md file found in sub-
# directories of the project's root, is used as the documentation for that sub-
# directory, except when the README.md starts with a \dir, \page or \mainpage
# command. If set to NO, the README.md file needs to start with an explicit \dir
# command in order to be used as directory documentation.
# The default value is: YES.

IMPLICIT_DIR_DOCS        = YES

# The Fortran standard specifies that for fixed formatted Fortran code all
# characters from position 72 are to be considered as comment. A common
# extension is to allow longer lines before the automatic comment starts. The
# setting FORTRAN_COMMENT_AFTER will also make it possible that longer lines can
# be processed before the automatic comment starts.
# Minimum value: 7, maximum value: 10000, default value: 72.

FORTRAN_COMMENT_AFTER    = 72

```

```

#-----
# Configuration options related to source browsing
#-----

# If the SOURCE_BROWSER tag is set to YES then a list of source files will be
# generated. Documented entities will be cross-referenced with these sources.
#
# Note: To get rid of all source code in the generated output, make sure that
# also VERBATIM_HEADERS is set to NO.
# The default value is: NO.

SOURCE_BROWSER          = NO

# Setting the INLINE_SOURCES tag to YES will include the body of functions,
# multi-line macros, enums or list initialized variables directly into the
# documentation.
# The default value is: NO.

INLINE_SOURCES          = NO

# Setting the STRIP_CODE_COMMENTS tag to YES will instruct Doxygen to hide any
# special comment blocks from generated source code fragments. Normal C, C++ and
# Fortran comments will always remain visible.
# The default value is: YES.

STRIP_CODE_COMMENTS     = YES

# If the REFERENCED_BY_RELATION tag is set to YES then for each documented
# entity all documented functions referencing it will be listed.
# The default value is: NO.

REFERENCED_BY_RELATION = NO

# If the REFERENCES_RELATION tag is set to YES then for each documented function
# all documented entities called/used by that function will be listed.
# The default value is: NO.

REFERENCES_RELATION     = NO

# If the REFERENCES_LINK_SOURCE tag is set to YES and SOURCE_BROWSER tag is set
# to YES then the hyperlinks from functions in REFERENCES_RELATION and
# REFERENCED_BY_RELATION lists will link to the source code. Otherwise they will
# link to the documentation.
# The default value is: YES.

```

REFERENCES_LINK_SOURCE = YES

If SOURCE_TOOLTIPS is enabled (the default) then hovering a hyperlink in the
source code will show a tooltip with additional information such as prototype,
brief description and links to the definition and documentation. Since this
will make the HTML file larger and loading of large files a bit slower, you
can opt to disable this feature.
The default value is: YES.
This tag requires that the tag SOURCE_BROWSER is set to YES.

SOURCE_TOOLTIPS = YES

If the USE_HTAGS tag is set to YES then the references to source code will
point to the HTML generated by the htags(1) tool instead of Doxygen built-in
source browser. The htags tool is part of GNU's global source tagging system
(see <https://www.gnu.org/software/global/global.html>). You will need version
4.8.6 or higher.

#

To use it do the following:

- # - Install the latest version of global
- # - Enable SOURCE_BROWSER and USE_HTAGS in the configuration file
- # - Make sure the INPUT points to the root of the source tree
- # - Run doxygen as normal

#

Doxygen will invoke htags (and that will in turn invoke gtags), so these
tools must be available from the command line (i.e. in the search path).

#

The result: instead of the source browser generated by Doxygen, the links to
source code will now point to the output of htags.

The default value is: NO.

This tag requires that the tag SOURCE_BROWSER is set to YES.

USE_HTAGS = NO

If the VERBATIM_HEADERS tag is set the YES then Doxygen will generate a
verbatim copy of the header file for each class for which an include is
specified. Set to NO to disable this.

See also: Section \class.

The default value is: YES.

VERBATIM_HEADERS = YES

If the CLANG_ASSISTED_PARSING tag is set to YES then Doxygen will use the
clang parser (see:
<http://clang.llvm.org/>) for more accurate parsing at the cost of reduced
performance. This can be particularly helpful with template rich C++ code for

```

# which Doxygen's built-in parser lacks the necessary type information.
# Note: The availability of this option depends on whether or not Doxygen was
# generated with the -Duse_libclang=ON option for CMake.
# The default value is: NO.

CLANG_ASSISTED_PARSING = NO

# If the CLANG_ASSISTED_PARSING tag is set to YES and the CLANG_ADD_INC_PATHS
# tag is set to YES then Doxygen will add the directory of each input to the
# include path.
# The default value is: YES.
# This tag requires that the tag CLANG_ASSISTED_PARSING is set to YES.

CLANG_ADD_INC_PATHS      = YES

# If clang assisted parsing is enabled you can provide the compiler with command
# line options that you would normally use when invoking the compiler. Note that
# the include paths will already be set by Doxygen for the files and directories
# specified with INPUT and INCLUDE_PATH.
# This tag requires that the tag CLANG_ASSISTED_PARSING is set to YES.

CLANG_OPTIONS            =

# If clang assisted parsing is enabled you can provide the clang parser with the
# path to the directory containing a file called compile_commands.json. This
# file is the compilation database (see:
# http://clang.llvm.org/docs/HowToSetupToolingForLLVM.html) containing the
# options used when the source files were built. This is equivalent to
# specifying the -p option to a clang tool, such as clang-check. These options
# will then be passed to the parser. Any options specified with CLANG_OPTIONS
# will be added as well.
# Note: The availability of this option depends on whether or not Doxygen was
# generated with the -Duse_libclang=ON option for CMake.

CLANG_DATABASE_PATH      =

#-----
# Configuration options related to the alphabetical class index
#-----

# If the ALPHABETICAL_INDEX tag is set to YES, an alphabetical index of all
# compounds will be generated. Enable this if the project contains a lot of
# classes, structs, unions or interfaces.
# The default value is: YES.

ALPHABETICAL_INDEX       = YES

```

```
# The IGNORE_PREFIX tag can be used to specify a prefix (or a list of prefixes)
# that should be ignored while generating the index headers. The IGNORE_PREFIX
# tag works for classes, function and member names. The entity will be placed in
# the alphabetical list under the first letter of the entity name that remains
# after removing the prefix.
# This tag requires that the tag ALPHABETICAL_INDEX is set to YES.
```

```
IGNORE_PREFIX          =
```

```
#-----
# Configuration options related to the HTML output
#-----
```

```
# If the GENERATE_HTML tag is set to YES, Doxygen will generate HTML output
# The default value is: YES.
```

```
GENERATE_HTML          = YES
```

```
# The HTML_OUTPUT tag is used to specify where the HTML docs will be put. If a
# relative path is entered the value of OUTPUT_DIRECTORY will be put in front of
# it.
# The default directory is: html.
# This tag requires that the tag GENERATE_HTML is set to YES.
```

```
HTML_OUTPUT            = html
```

```
# The HTML_FILE_EXTENSION tag can be used to specify the file extension for each
# generated HTML page (for example: .htm, .php, .asp).
# The default value is: .html.
# This tag requires that the tag GENERATE_HTML is set to YES.
```

```
HTML_FILE_EXTENSION    = .html
```

```
# The HTML_HEADER tag can be used to specify a user-defined HTML header file for
# each generated HTML page. If the tag is left blank Doxygen will generate a
# standard header.
```

```
#
```

```
# To get valid HTML the header file that includes any scripts and style sheets
# that Doxygen needs, which is dependent on the configuration options used (e.g.
# the setting GENERATE_TREEVIEW). It is highly recommended to start with a
# default header using
```

```
# doxygen -w html new_header.html new_footer.html new_stylesheet.css
```

```
# YourConfigFile
```

```
# and then modify the file new_header.html. See also section "Doxygen usage"
```

```
# for information on how to generate the default header that Doxygen normally
```

uses.
Note: The header is subject to change so you typically have to regenerate the
default header when upgrading to a newer version of Doxygen. For a description
of the possible markers and block names see the documentation.
This tag requires that the tag GENERATE_HTML is set to YES.

HTML_HEADER =

The HTML_FOOTER tag can be used to specify a user-defined HTML footer for each
generated HTML page. If the tag is left blank Doxygen will generate a standard
footer. See HTML_HEADER for more information on how to generate a default
footer and what special commands can be used inside the footer. See also
section "Doxygen usage" for information on how to generate the default footer
that Doxygen normally uses.
This tag requires that the tag GENERATE_HTML is set to YES.

HTML_FOOTER =

The HTML_STYLESHEET tag can be used to specify a user-defined cascading style
sheet that is used by each HTML page. It can be used to fine-tune the look of
the HTML output. If left blank Doxygen will generate a default style sheet.
See also section "Doxygen usage" for information on how to generate the style
sheet that Doxygen normally uses.
Note: It is recommended to use HTML_EXTRA_STYLESHEET instead of this tag, as
it is more robust and this tag (HTML_STYLESHEET) will in the future become
obsolete.
This tag requires that the tag GENERATE_HTML is set to YES.

HTML_STYLESHEET =

The HTML_EXTRA_STYLESHEET tag can be used to specify additional user-defined
cascading style sheets that are included after the standard style sheets
created by Doxygen. Using this option one can overrule certain style aspects.
This is preferred over using HTML_STYLESHEET since it does not replace the
standard style sheet and is therefore more robust against future updates.
Doxygen will copy the style sheet files to the output directory.
Note: The order of the extra style sheet files is of importance (e.g. the last
style sheet in the list overrules the setting of the previous ones in the
list).
Note: Since the styling of scrollbars can currently not be overruled in
Webkit/Chromium, the styling will be left out of the default doxygen.css if
one or more extra stylesheets have been specified. So if scrollbar
customization is desired it has to be added explicitly. For an example see the
documentation.
This tag requires that the tag GENERATE_HTML is set to YES.

HTML_EXTRA_STYLESHEET =

The HTML_EXTRA_FILES tag can be used to specify one or more extra images or
other source files which should be copied to the HTML output directory. Note
that these files will be copied to the base HTML output directory. Use the
\$relpath^ marker in the HTML_HEADER and/or HTML_FOOTER files to load these
files. In the HTML_STYLESHEET file, use the file name only. Also note that the
files will be copied as-is; there are no commands or markers available.
This tag requires that the tag GENERATE_HTML is set to YES.

HTML_EXTRA_FILES =

The HTML_COLORSTYLE tag can be used to specify if the generated HTML output
should be rendered with a dark or light theme.
Possible values are: LIGHT always generates light mode output, DARK always
generates dark mode output, AUTO_LIGHT automatically sets the mode according
to the user preference, uses light mode if no preference is set (the default),
AUTO_DARK automatically sets the mode according to the user preference, uses
dark mode if no preference is set and TOGGLE allows a user to switch between
light and dark mode via a button.
The default value is: AUTO_LIGHT.
This tag requires that the tag GENERATE_HTML is set to YES.

HTML_COLORSTYLE = AUTO_LIGHT

The HTML_COLORSTYLE_HUE tag controls the color of the HTML output. Doxygen
will adjust the colors in the style sheet and background images according to
this color. Hue is specified as an angle on a color-wheel, see
<https://en.wikipedia.org/wiki/Hue> for more information. For instance the value
0 represents red, 60 is yellow, 120 is green, 180 is cyan, 240 is blue, 300
purple, and 360 is red again.
Minimum value: 0, maximum value: 359, default value: 220.
This tag requires that the tag GENERATE_HTML is set to YES.

HTML_COLORSTYLE_HUE = 220

The HTML_COLORSTYLE_SAT tag controls the purity (or saturation) of the colors
in the HTML output. For a value of 0 the output will use gray-scales only. A
value of 255 will produce the most vivid colors.
Minimum value: 0, maximum value: 255, default value: 100.
This tag requires that the tag GENERATE_HTML is set to YES.

HTML_COLORSTYLE_SAT = 100

The HTML_COLORSTYLE_GAMMA tag controls the gamma correction applied to the
luminance component of the colors in the HTML output. Values below 100

gradually make the output lighter, whereas values above 100 make the output
darker. The value divided by 100 is the actual gamma applied, so 80 represents
a gamma of 0.8, The value 220 represents a gamma of 2.2, and 100 does not
change the gamma.
Minimum value: 40, maximum value: 240, default value: 80.
This tag requires that the tag GENERATE_HTML is set to YES.

HTML_COLORSTYLE_GAMMA = 80

If the HTML_DYNAMIC_MENUS tag is set to YES then the generated HTML
documentation will contain a main index with vertical navigation menus that
are dynamically created via JavaScript. If disabled, the navigation index will
consists of multiple levels of tabs that are statically embedded in every HTML
page. Disable this option to support browsers that do not have JavaScript,
like the Qt help browser.
The default value is: YES.
This tag requires that the tag GENERATE_HTML is set to YES.

HTML_DYNAMIC_MENUS = YES

If the HTML_DYNAMIC_SECTIONS tag is set to YES then the generated HTML
documentation will contain sections that can be hidden and shown after the
page has loaded.
The default value is: NO.
This tag requires that the tag GENERATE_HTML is set to YES.

HTML_DYNAMIC_SECTIONS = NO

If the HTML_CODE_FOLDING tag is set to YES then classes and functions can be
dynamically folded and expanded in the generated HTML source code.
The default value is: YES.
This tag requires that the tag GENERATE_HTML is set to YES.

HTML_CODE_FOLDING = YES

If the HTML_COPY_CLIPBOARD tag is set to YES then Doxygen will show an icon in
the top right corner of code and text fragments that allows the user to copy
its content to the clipboard. Note this only works if supported by the browser
and the web page is served via a secure context (see:
<https://www.w3.org/TR/secure-contexts/>), i.e. using the https: or file:
protocol.
The default value is: YES.
This tag requires that the tag GENERATE_HTML is set to YES.

HTML_COPY_CLIPBOARD = YES

```
# Doxygen stores a couple of settings persistently in the browser (via e.g.
# cookies). By default these settings apply to all HTML pages generated by
# Doxygen across all projects. The HTML_PROJECT_COOKIE tag can be used to store
# the settings under a project specific key, such that the user preferences will
# be stored separately.
# This tag requires that the tag GENERATE_HTML is set to YES.
```

```
HTML_PROJECT_COOKIE      =
```

```
# With HTML_INDEX_NUM_ENTRIES one can control the preferred number of entries
# shown in the various tree structured indices initially; the user can expand
# and collapse entries dynamically later on. Doxygen will expand the tree to
# such a level that at most the specified number of entries are visible (unless
# a fully collapsed tree already exceeds this amount). So setting the number of
# entries 1 will produce a full collapsed tree by default. 0 is a special value
# representing an infinite number of entries and will result in a full expanded
# tree by default.
# Minimum value: 0, maximum value: 9999, default value: 100.
# This tag requires that the tag GENERATE_HTML is set to YES.
```

```
HTML_INDEX_NUM_ENTRIES = 100
```

```
# If the GENERATE_DOCSET tag is set to YES, additional index files will be
# generated that can be used as input for Apple's Xcode 3 integrated development
# environment (see:
# https://developer.apple.com/xcode/), introduced with OSX 10.5 (Leopard). To
# create a documentation set, Doxygen will generate a Makefile in the HTML
# output directory. Running make will produce the docset in that directory and
# running make install will install the docset in
# ~/Library/Developer/Shared/Documentation/DocSets so that Xcode will find it at
# startup. See https://developer.apple.com/library/archive/featuredarticles/Doxy
# genXcode/\_index.html for more information.
# The default value is: NO.
# This tag requires that the tag GENERATE_HTML is set to YES.
```

```
GENERATE_DOCSET           = NO
```

```
# This tag determines the name of the docset feed. A documentation feed provides
# an umbrella under which multiple documentation sets from a single provider
# (such as a company or product suite) can be grouped.
# The default value is: Doxygen generated docs.
# This tag requires that the tag GENERATE_DOCSET is set to YES.
```

```
DOCSET_FEEDNAME           = "Doxygen generated docs"
```

```
# This tag determines the URL of the docset feed. A documentation feed provides
```

```
# an umbrella under which multiple documentation sets from a single provider
# (such as a company or product suite) can be grouped.
# This tag requires that the tag GENERATE_DOCSET is set to YES.
```

```
DOCSET_FEEDURL      =
```

```
# This tag specifies a string that should uniquely identify the documentation
# set bundle. This should be a reverse domain-name style string, e.g.
# com.mycompany.MyDocSet. Doxygen will append .docset to the name.
# The default value is: org.doxygen.Project.
# This tag requires that the tag GENERATE_DOCSET is set to YES.
```

```
DOCSET_BUNDLE_ID    = org.doxygen.Project
```

```
# The DOCSET_PUBLISHER_ID tag specifies a string that should uniquely identify
# the documentation publisher. This should be a reverse domain-name style
# string, e.g. com.mycompany.MyDocSet.documentation.
# The default value is: org.doxygen.Publisher.
# This tag requires that the tag GENERATE_DOCSET is set to YES.
```

```
DOCSET_PUBLISHER_ID = org.doxygen.Publisher
```

```
# The DOCSET_PUBLISHER_NAME tag identifies the documentation publisher.
# The default value is: Publisher.
# This tag requires that the tag GENERATE_DOCSET is set to YES.
```

```
DOCSET_PUBLISHER_NAME = Publisher
```

```
# If the GENERATE_HTMLHELP tag is set to YES then Doxygen generates three
# additional HTML index files: index.hhp, index.hhc, and index.hhk. The
# index.hhp is a project file that can be read by Microsoft's HTML Help Workshop
# on Windows. In the beginning of 2021 Microsoft took the original page, with
# a.o. the download links, offline (the HTML help workshop was already many
# years in maintenance mode). You can download the HTML help workshop from the
# web archives at Installation executable (see:
# http://web.archive.org/web/20160201063255/http://download.microsoft.com/download/ad/0/A/9/0A939EF6-E31C-430F-A3DF-DFAE7960D564/htmlhelp.exe).
#
```

```
# The HTML Help Workshop contains a compiler that can convert all HTML output
# generated by Doxygen into a single compiled HTML file (.chm). Compiled HTML
# files are now used as the Windows 98 help format, and will replace the old
# Windows help format (.hlp) on all Windows platforms in the future. Compressed
# HTML files also contain an index, a table of contents, and you can search for
# words in the documentation. The HTML workshop also contains a viewer for
# compressed HTML files.
# The default value is: NO.
```

```

# This tag requires that the tag GENERATE_HTML is set to YES.

GENERATE_HTMLHELP      = NO

# The CHM_FILE tag can be used to specify the file name of the resulting .chm
# file. You can add a path in front of the file if the result should not be
# written to the html output directory.
# This tag requires that the tag GENERATE_HTMLHELP is set to YES.

CHM_FILE               =

# The HHC_LOCATION tag can be used to specify the location (absolute path
# including file name) of the HTML help compiler (hhc.exe). If non-empty,
# Doxygen will try to run the HTML help compiler on the generated index.hhp.
# The file has to be specified with full path.
# This tag requires that the tag GENERATE_HTMLHELP is set to YES.

HHC_LOCATION           =

# The GENERATE_CHI flag controls if a separate .chi index file is generated
# (YES) or that it should be included in the main .chm file (NO).
# The default value is: NO.
# This tag requires that the tag GENERATE_HTMLHELP is set to YES.

GENERATE_CHI           = NO

# The CHM_INDEX_ENCODING is used to encode HtmlHelp index (hhk), content (hhc)
# and project file content.
# This tag requires that the tag GENERATE_HTMLHELP is set to YES.

CHM_INDEX_ENCODING     =

# The BINARY_TOC flag controls whether a binary table of contents is generated
# (YES) or a normal table of contents (NO) in the .chm file. Furthermore it
# enables the Previous and Next buttons.
# The default value is: NO.
# This tag requires that the tag GENERATE_HTMLHELP is set to YES.

BINARY_TOC             = NO

# The TOC_EXPAND flag can be set to YES to add extra items for group members to
# the table of contents of the HTML help documentation and to the tree view.
# The default value is: NO.
# This tag requires that the tag GENERATE_HTMLHELP is set to YES.

TOC_EXPAND             = NO

```

The SITEMAP_URL tag is used to specify the full URL of the place where the
generated documentation will be placed on the server by the user during the
deployment of the documentation. The generated sitemap is called sitemap.xml
and placed on the directory specified by HTML_OUTPUT. In case no SITEMAP_URL
is specified no sitemap is generated. For information about the sitemap
protocol see <https://www.sitemaps.org>
This tag requires that the tag GENERATE_HTML is set to YES.

SITEMAP_URL =

If the GENERATE_QHP tag is set to YES and both QHP_NAMESPACE and
QHP_VIRTUAL_FOLDER are set, an additional index file will be generated that
can be used as input for Qt's qhelpgenerator to generate a Qt Compressed Help
(.qch) of the generated HTML documentation.
The default value is: NO.
This tag requires that the tag GENERATE_HTML is set to YES.

GENERATE_QHP = NO

If the QHG_LOCATION tag is specified, the QCH_FILE tag can be used to specify
the file name of the resulting .qch file. The path specified is relative to
the HTML output folder.
This tag requires that the tag GENERATE_QHP is set to YES.

QCH_FILE =

The QHP_NAMESPACE tag specifies the namespace to use when generating Qt Help
Project output. For more information please see Qt Help Project / Namespace
(see:
<https://doc.qt.io/archives/qt-4.8/qthelpproject.html#namespace>).
The default value is: org.doxygen.Project.
This tag requires that the tag GENERATE_QHP is set to YES.

QHP_NAMESPACE = org.doxygen.Project

The QHP_VIRTUAL_FOLDER tag specifies the namespace to use when generating Qt
Help Project output. For more information please see Qt Help Project / Virtual
Folders (see:
<https://doc.qt.io/archives/qt-4.8/qthelpproject.html#virtual-folders>).
The default value is: doc.
This tag requires that the tag GENERATE_QHP is set to YES.

QHP_VIRTUAL_FOLDER = doc

If the QHP_CUST_FILTER_NAME tag is set, it specifies the name of a custom

```

# filter to add. For more information please see Qt Help Project / Custom
# Filters (see:
# https://doc.qt.io/archives/qt-4.8/qthelpproject.html#custom-filters).
# This tag requires that the tag GENERATE_QHP is set to YES.

QHP_CUST_FILTER_NAME    =

# The QHP_CUST_FILTER_ATTRS tag specifies the list of the attributes of the
# custom filter to add. For more information please see Qt Help Project / Custom
# Filters (see:
# https://doc.qt.io/archives/qt-4.8/qthelpproject.html#custom-filters).
# This tag requires that the tag GENERATE_QHP is set to YES.

QHP_CUST_FILTER_ATTRS  =

# The QHP_SECT_FILTER_ATTRS tag specifies the list of the attributes this
# project's filter section matches. Qt Help Project / Filter Attributes (see:
# https://doc.qt.io/archives/qt-4.8/qthelpproject.html#filter-attributes).
# This tag requires that the tag GENERATE_QHP is set to YES.

QHP_SECT_FILTER_ATTRS  =

# The QHG_LOCATION tag can be used to specify the location (absolute path
# including file name) of Qt's qhelpgenerator. If non-empty Doxygen will try to
# run qhelpgenerator on the generated .qhp file.
# This tag requires that the tag GENERATE_QHP is set to YES.

QHG_LOCATION           =

# If the GENERATE_ECLIPSEHELP tag is set to YES, additional index files will be
# generated, together with the HTML files, they form an Eclipse help plugin. To
# install this plugin and make it available under the help contents menu in
# Eclipse, the contents of the directory containing the HTML and XML files needs
# to be copied into the plugins directory of eclipse. The name of the directory
# within the plugins directory should be the same as the ECLIPSE_DOC_ID value.
# After copying Eclipse needs to be restarted before the help appears.
# The default value is: NO.
# This tag requires that the tag GENERATE_HTML is set to YES.

GENERATE_ECLIPSEHELP    = NO

# A unique identifier for the Eclipse help plugin. When installing the plugin
# the directory name containing the HTML and XML files should also have this
# name. Each documentation set should have its own identifier.
# The default value is: org.doxygen.Project.
# This tag requires that the tag GENERATE_ECLIPSEHELP is set to YES.

```

ECLIPSE_DOC_ID = org.doxygen.Project

If you want full control over the layout of the generated HTML pages it might
be necessary to disable the index and replace it with your own. The
DISABLE_INDEX tag can be used to turn on/off the condensed index (tabs) at top
of each HTML page. A value of NO enables the index and the value YES disables
it. Since the tabs in the index contain the same information as the navigation
tree, you can set this option to YES if you also set GENERATE_TREEVIEW to YES.
The default value is: NO.
This tag requires that the tag GENERATE_HTML is set to YES.

DISABLE_INDEX = NO

The GENERATE_TREEVIEW tag is used to specify whether a tree-like index
structure should be generated to display hierarchical information. If the tag
value is set to YES, a side panel will be generated containing a tree-like
index structure (just like the one that is generated for HTML Help). For this
to work a browser that supports JavaScript, DHTML, CSS and frames is required
(i.e. any modern browser). Windows users are probably better off using the
HTML help feature. Via custom style sheets (see HTML_EXTRA_STYLESHEET) one can
further fine tune the look of the index (see "Fine-tuning the output"). As an
example, the default style sheet generated by Doxygen has an example that
shows how to put an image at the root of the tree instead of the PROJECT_NAME.
Since the tree basically has more details information than the tab index, you
could consider setting DISABLE_INDEX to YES when enabling this option.
The default value is: YES.
This tag requires that the tag GENERATE_HTML is set to YES.

GENERATE_TREEVIEW = YES

When GENERATE_TREEVIEW is set to YES, the PAGE_OUTLINE_PANEL option determines
if an additional navigation panel is shown at the right hand side of the
screen, displaying an outline of the contents of the main page, similar to
e.g. <https://developer.android.com/reference> If GENERATE_TREEVIEW is set to
NO, this option has no effect.
The default value is: YES.
This tag requires that the tag GENERATE_HTML is set to YES.

PAGE_OUTLINE_PANEL = YES

When GENERATE_TREEVIEW is set to YES, the FULL_SIDEBAR option determines if
the side bar is limited to only the treeview area (value NO) or if it should
extend to the full height of the window (value YES). Setting this to YES gives
a layout similar to e.g. <https://docs.readthedocs.io> with more room for
contents, but less room for the project logo, title, and description. If


```

# GENERATE_TREEVIEW is set to NO, this option has no effect.
# The default value is: NO.
# This tag requires that the tag GENERATE_HTML is set to YES.

FULL_SIDEBAR          = NO

# The ENUM_VALUES_PER_LINE tag can be used to set the number of enum values that
# Doxygen will group on one line in the generated HTML documentation.
#
# Note that a value of 0 will completely suppress the enum values from appearing
# in the overview section.
# Minimum value: 0, maximum value: 20, default value: 4.
# This tag requires that the tag GENERATE_HTML is set to YES.

ENUM_VALUES_PER_LINE  = 4

# When the SHOW_ENUM_VALUES tag is set doxygen will show the specified
# enumeration values besides the enumeration mnemonics.
# The default value is: NO.

SHOW_ENUM_VALUES      = NO

# If the treeview is enabled (see GENERATE_TREEVIEW) then this tag can be used
# to set the initial width (in pixels) of the frame in which the tree is shown.
# Minimum value: 0, maximum value: 1500, default value: 250.
# This tag requires that the tag GENERATE_HTML is set to YES.

TREEVIEW_WIDTH        = 250

# If the EXT_LINKS_IN_WINDOW option is set to YES, Doxygen will open links to
# external symbols imported via tag files in a separate window.
# The default value is: NO.
# This tag requires that the tag GENERATE_HTML is set to YES.

EXT_LINKS_IN_WINDOW   = NO

# If the OBFUSCATE_EMAILS tag is set to YES, Doxygen will obfuscate email
# addresses.
# The default value is: YES.
# This tag requires that the tag GENERATE_HTML is set to YES.

OBFUSCATE_EMAILS      = YES

# If the HTML_FORMULA_FORMAT option is set to svg, Doxygen will use the pdf2svg
# tool (see https://github.com/dawbarton/pdf2svg) or inkscape (see
# https://inkscape.org) to generate formulas as SVG images instead of PNGs for

```

```
# the HTML output. These images will generally look nicer at scaled resolutions.
# Possible values are: png (the default) and svg (looks nicer but requires the
# pdf2svg or inkscape tool).
# The default value is: png.
# This tag requires that the tag GENERATE_HTML is set to YES.
```

```
HTML_FORMULA_FORMAT      = png
```

```
# Use this tag to change the font size of LaTeX formulas included as images in
# the HTML documentation. When you change the font size after a successful
# Doxygen run you need to manually remove any form_*.png images from the HTML
# output directory to force them to be regenerated.
# Minimum value: 8, maximum value: 50, default value: 10.
# This tag requires that the tag GENERATE_HTML is set to YES.
```

```
FORMULA_FONTSIZE         = 10
```

```
# The FORMULA_MACROFILE can contain LaTeX \newcommand and \renewcommand commands
# to create new LaTeX commands to be used in formulas as building blocks. See
# the section "Including formulas" for details.
```

```
FORMULA_MACROFILE        =
```

```
# Enable the USE_MATHJAX option to render LaTeX formulas using MathJax (see
# https://www.mathjax.org) which uses client side JavaScript for the rendering
# instead of using pre-rendered bitmaps. Use this if you do not have LaTeX
# installed or if you want to formulas look prettier in the HTML output. When
# enabled you may also need to install MathJax separately and configure the path
# to it using the MATHJAX_RELPATH option.
# The default value is: NO.
# This tag requires that the tag GENERATE_HTML is set to YES.
```

```
USE_MATHJAX              = NO
```

```
# With MATHJAX_VERSION it is possible to specify the MathJax version to be used.
# Note that the different versions of MathJax have different requirements with
# regards to the different settings, so it is possible that also other MathJax
# settings have to be changed when switching between the different MathJax
# versions.
# Possible values are: MathJax_2, MathJax_3 and MathJax_4.
# The default value is: MathJax_2.
# This tag requires that the tag USE_MATHJAX is set to YES.
```

```
MATHJAX_VERSION          = MathJax_2
```

```
# When MathJax is enabled you can set the default output format to be used for
```

```
# the MathJax output. For more details about the output format see MathJax
# version 2 (see:
# https://docs.mathjax.org/en/v2.7/output.html), MathJax version 3 (see:
# https://docs.mathjax.org/en/v3.2/output/index.html) and MathJax version 4
# (see:
# https://docs.mathjax.org/en/v4.0/output/index.htm).
# Possible values are: HTML-CSS (which is slower, but has the best
# compatibility. This is the name for Mathjax version 2, for MathJax version 3
# this will be translated into chtml), NativeMML (i.e. MathML. Only supported
# for MathJax 2. For MathJax version 3 chtml will be used instead.), chtml (This
# is the name for Mathjax version 3, for MathJax version 2 this will be
# translated into HTML-CSS) and SVG.
# The default value is: HTML-CSS.
# This tag requires that the tag USE_MATHJAX is set to YES.
```

```
MATHJAX_FORMAT = HTML-CSS
```

```
# When MathJax is enabled you need to specify the location relative to the HTML
# output directory using the MATHJAX_RELPATH option. For Mathjax version 2 the
# destination directory should contain the MathJax.js script. For instance, if
# the mathjax directory is located at the same level as the HTML output
# directory, then MATHJAX_RELPATH should be ../mathjax.s For Mathjax versions 3
# and 4 the destination directory should contain the tex-<format>.js script
# (where <format> is either chtml or svg). The default value points to the
# MathJax Content Delivery Network so you can quickly see the result without
# installing MathJax. However, it is strongly recommended to install a local
# copy of MathJax from https://www.mathjax.org before deployment. The default
# value is:
# - in case of MathJax version 2: https://cdn.jsdelivr.net/npm/mathjax@2
# - in case of MathJax version 3: https://cdn.jsdelivr.net/npm/mathjax@3
# - in case of MathJax version 4: https://cdn.jsdelivr.net/npm/mathjax@4
# This tag requires that the tag USE_MATHJAX is set to YES.
```

```
MATHJAX_RELPATH =
```

```
# The MATHJAX_EXTENSIONS tag can be used to specify one or more MathJax
# extension names that should be enabled during MathJax rendering. For example
# for MathJax version 2 (see https://docs.mathjax.org/en/v2.7/tex.html):
# MATHJAX_EXTENSIONS = TeX/AMSmath TeX/AMSsymbols
# For example for MathJax version 3 (see
# https://docs.mathjax.org/en/v3.2/input/tex/extensions/):
# MATHJAX_EXTENSIONS = ams
# For example for MathJax version 4 (see
# https://docs.mathjax.org/en/v4.0/input/tex/extensions/):
# MATHJAX_EXTENSIONS = units
# Note that for Mathjax version 4 quite a few extensions are already
```

```
# automatically loaded. To disable a package in Mathjax version 4 one can use
# the package name prepended with a minus sign (- like MATHJAX_EXTENSIONS +=
# -textmacros)
# This tag requires that the tag USE_MATHJAX is set to YES.
```

```
MATHJAX_EXTENSIONS      =
```

```
# The MATHJAX_CODEFILE tag can be used to specify a file with JavaScript pieces
# of code that will be used on startup of the MathJax code. See the Mathjax site
# for more details:
# - MathJax version 2 (see:
# https://docs.mathjax.org/en/v2.7/)
# - MathJax version 3 (see:
# https://docs.mathjax.org/en/v3.2/)
# - MathJax version 4 (see:
# https://docs.mathjax.org/en/v4.0/) For an example see the documentation.
# This tag requires that the tag USE_MATHJAX is set to YES.
```

```
MATHJAX_CODEFILE      =
```

```
# When the SEARCHENGINE tag is enabled Doxygen will generate a search box for
# the HTML output. The underlying search engine uses JavaScript and DHTML and
# should work on any modern browser. Note that when using HTML help
# (GENERATE_HTMLHELP), Qt help (GENERATE_QHP), or docsets (GENERATE_DOCSET)
# there is already a search function so this one should typically be disabled.
# For large projects the JavaScript based search engine can be slow, then
# enabling SERVER_BASED_SEARCH may provide a better solution. It is possible to
# search using the keyboard; to jump to the search box use <access key> + S
# (what the <access key> is depends on the OS and browser, but it is typically
# <CTRL>, <ALT>/<option>, or both). Inside the search box use the <cursor down
# key> to jump into the search results window, the results can be navigated
# using the <cursor keys>. Press <Enter> to select an item or <escape> to cancel
# the search. The filter options can be selected when the cursor is inside the
# search box by pressing <Shift>+<cursor down>. Also here use the <cursor keys>
# to select a filter and <Enter> or <escape> to activate or cancel the filter
# option.
# The default value is: YES.
# This tag requires that the tag GENERATE_HTML is set to YES.
```

```
SEARCHENGINE           = YES
```

```
# When the SERVER_BASED_SEARCH tag is enabled the search engine will be
# implemented using a web server instead of a web client using JavaScript. There
# are two flavors of web server based searching depending on the EXTERNAL_SEARCH
# setting. When disabled, Doxygen will generate a PHP script for searching and
# an index file used by the script. When EXTERNAL_SEARCH is enabled the indexing
```

```

# and searching needs to be provided by external tools. See the section
# "External Indexing and Searching" for details.
# The default value is: NO.
# This tag requires that the tag SEARCHENGINE is set to YES.

SERVER_BASED_SEARCH      = NO

# When EXTERNAL_SEARCH tag is enabled Doxygen will no longer generate the PHP
# script for searching. Instead the search results are written to an XML file
# which needs to be processed by an external indexer. Doxygen will invoke an
# external search engine pointed to by the SEARCHENGINE_URL option to obtain the
# search results.
#
# Doxygen ships with an example indexer (doxyindexer) and search engine
# (doxysearch.cgi) which are based on the open source search engine library
# Xapian (see:
# https://xapian.org/).
#
# See the section "External Indexing and Searching" for details.
# The default value is: NO.
# This tag requires that the tag SEARCHENGINE is set to YES.

EXTERNAL_SEARCH          = NO

# The SEARCHENGINE_URL should point to a search engine hosted by a web server
# which will return the search results when EXTERNAL_SEARCH is enabled.
#
# Doxygen ships with an example indexer (doxyindexer) and search engine
# (doxysearch.cgi) which are based on the open source search engine library
# Xapian (see:
# https://xapian.org/). See the section "External Indexing and Searching" for
# details.
# This tag requires that the tag SEARCHENGINE is set to YES.

SEARCHENGINE_URL         =

# When SERVER_BASED_SEARCH and EXTERNAL_SEARCH are both enabled the unindexed
# search data is written to a file for indexing by an external tool. With the
# SEARCHDATA_FILE tag the name of this file can be specified.
# The default file is: searchdata.xml.
# This tag requires that the tag SEARCHENGINE is set to YES.

SEARCHDATA_FILE          = searchdata.xml

# When SERVER_BASED_SEARCH and EXTERNAL_SEARCH are both enabled the
# EXTERNAL_SEARCH_ID tag can be used as an identifier for the project. This is

```

```

# useful in combination with EXTRA_SEARCH_MAPPINGS to search through multiple
# projects and redirect the results back to the right project.
# This tag requires that the tag SEARCHENGINE is set to YES.

EXTERNAL_SEARCH_ID      =

# The EXTRA_SEARCH_MAPPINGS tag can be used to enable searching through Doxygen
# projects other than the one defined by this configuration file, but that are
# all added to the same external search index. Each project needs to have a
# unique id set via EXTERNAL_SEARCH_ID. The search mapping then maps the id of
# to a relative location where the documentation can be found. The format is:
# EXTRA_SEARCH_MAPPINGS = tagname1=loc1 tagname2=loc2 ...
# This tag requires that the tag SEARCHENGINE is set to YES.

EXTRA_SEARCH_MAPPINGS   =

#-----
# Configuration options related to the LaTeX output
#-----

# If the GENERATE_LATEX tag is set to YES, Doxygen will generate LaTeX output.
# The default value is: YES.

GENERATE_LATEX           = YES

# The LATEX_OUTPUT tag is used to specify where the LaTeX docs will be put. If a
# relative path is entered the value of OUTPUT_DIRECTORY will be put in front of
# it.
# The default directory is: latex.
# This tag requires that the tag GENERATE_LATEX is set to YES.

LATEX_OUTPUT             = latex

# The LATEX_CMD_NAME tag can be used to specify the LaTeX command name to be
# invoked.
#
# Note that when not enabling USE_PDFLATEX the default is latex when enabling
# USE_PDFLATEX the default is pdflatex and when in the later case latex is
# chosen this is overwritten by pdflatex. For specific output languages the
# default can have been set differently, this depends on the implementation of
# the output language.
# This tag requires that the tag GENERATE_LATEX is set to YES.

LATEX_CMD_NAME           =

# The MAKEINDEX_CMD_NAME tag can be used to specify the command name to generate

```

```

# index for LaTeX.
# Note: This tag is used in the Makefile / make.bat.
# See also: LATEX_MAKEINDEX_CMD for the part in the generated output file
# (.tex).
# The default file is: makeindex.
# This tag requires that the tag GENERATE_LATEX is set to YES.

MAKEINDEX_CMD_NAME      = makeindex

# The LATEX_MAKEINDEX_CMD tag can be used to specify the command name to
# generate index for LaTeX. In case there is no backslash (\) as first character
# it will be automatically added in the LaTeX code.
# Note: This tag is used in the generated output file (.tex).
# See also: MAKEINDEX_CMD_NAME for the part in the Makefile / make.bat.
# The default value is: makeindex.
# This tag requires that the tag GENERATE_LATEX is set to YES.

LATEX_MAKEINDEX_CMD     = makeindex

# If the COMPACT_LATEX tag is set to YES, Doxygen generates more compact LaTeX
# documents. This may be useful for small projects and may help to save some
# trees in general.
# The default value is: NO.
# This tag requires that the tag GENERATE_LATEX is set to YES.

COMPACT_LATEX           = NO

# The PAPER_TYPE tag can be used to set the paper type that is used by the
# printer.
# Possible values are: a4 (210 x 297 mm), letter (8.5 x 11 inches), legal (8.5 x
# 14 inches) and executive (7.25 x 10.5 inches).
# The default value is: a4.
# This tag requires that the tag GENERATE_LATEX is set to YES.

PAPER_TYPE              = a4

# The EXTRA_PACKAGES tag can be used to specify one or more LaTeX package names
# that should be included in the LaTeX output. The package can be specified just
# by its name or with the correct syntax as to be used with the LaTeX
# \usepackage command. To get the times font for instance you can specify :
# EXTRA_PACKAGES=times or EXTRA_PACKAGES={times}
# To use the option intlimits with the amsmath package you can specify:
# EXTRA_PACKAGES=[intlimits]{amsmath}
# If left blank no extra packages will be included.
# This tag requires that the tag GENERATE_LATEX is set to YES.

```

EXTRA_PACKAGES

=

```
# The LATEX_HEADER tag can be used to specify a user-defined LaTeX header for
# the generated LaTeX document. The header should contain everything until the
# first chapter. If it is left blank Doxygen will generate a standard header. It
# is highly recommended to start with a default header using
# doxygen -w latex new_header.tex new_footer.tex new_stylesheet.sty
# and then modify the file new_header.tex. See also section "Doxygen usage" for
# information on how to generate the default header that Doxygen normally uses.
#
# Note: Only use a user-defined header if you know what you are doing!
# Note: The header is subject to change so you typically have to regenerate the
# default header when upgrading to a newer version of Doxygen. The following
# commands have a special meaning inside the header (and footer): For a
# description of the possible markers and block names see the documentation.
# This tag requires that the tag GENERATE_LATEX is set to YES.
```

LATEX_HEADER

=

```
# The LATEX_FOOTER tag can be used to specify a user-defined LaTeX footer for
# the generated LaTeX document. The footer should contain everything after the
# last chapter. If it is left blank Doxygen will generate a standard footer. See
# LATEX_HEADER for more information on how to generate a default footer and what
# special commands can be used inside the footer. See also section "Doxygen
# usage" for information on how to generate the default footer that Doxygen
# normally uses. Note: Only use a user-defined footer if you know what you are
# doing!
# This tag requires that the tag GENERATE_LATEX is set to YES.
```

LATEX_FOOTER

=

```
# The LATEX_EXTRA_STYLESHEET tag can be used to specify additional user-defined
# LaTeX style sheets that are included after the standard style sheets created
# by Doxygen. Using this option one can overrule certain style aspects. Doxygen
# will copy the style sheet files to the output directory.
# Note: The order of the extra style sheet files is of importance (e.g. the last
# style sheet in the list overrules the setting of the previous ones in the
# list).
# This tag requires that the tag GENERATE_LATEX is set to YES.
```

LATEX_EXTRA_STYLESHEET =

```
# The LATEX_EXTRA_FILES tag can be used to specify one or more extra images or
# other source files which should be copied to the LATEX_OUTPUT output
# directory. Note that the files will be copied as-is; there are no commands or
# markers available.
```


This tag requires that the tag GENERATE_LATEX is set to YES.

LATEX_EXTRA_FILES =

If the PDF_HYPERLINKS tag is set to YES, the LaTeX that is generated is
prepared for conversion to PDF (using ps2pdf or pdflatex). The PDF file will
contain links (just like the HTML output) instead of page references. This
makes the output suitable for online browsing using a PDF viewer.
The default value is: YES.
This tag requires that the tag GENERATE_LATEX is set to YES.

PDF_HYPERLINKS = YES

If the USE_PDFLATEX tag is set to YES, Doxygen will use the engine as
specified with LATEX_CMD_NAME to generate the PDF file directly from the LaTeX
files. Set this option to YES, to get a higher quality PDF documentation.

See also section LATEX_CMD_NAME for selecting the engine.
The default value is: YES.
This tag requires that the tag GENERATE_LATEX is set to YES.

USE_PDFLATEX = YES

The LATEX_BATCHMODE tag signals the behavior of LaTeX in case of an error.
Possible values are: NO same as ERROR_STOP, YES same as BATCH, BATCH In batch
mode nothing is printed on the terminal, errors are scrolled as if <return> is
hit at every error; missing files that TeX tries to input or request from
keyboard input (\read on a not open input stream) cause the job to abort,
NON_STOP In nonstop mode the diagnostic message will appear on the terminal,
but there is no possibility of user interaction just like in batch mode,
SCROLL In scroll mode, TeX will stop only for missing files to input or if
keyboard input is necessary and ERROR_STOP In errorstop mode, TeX will stop at
each error, asking for user intervention.
The default value is: NO.
This tag requires that the tag GENERATE_LATEX is set to YES.

LATEX_BATCHMODE = NO

If the LATEX_HIDE_INDICES tag is set to YES then Doxygen will not include the
index chapters (such as File Index, Compound Index, etc.) in the output.
The default value is: NO.
This tag requires that the tag GENERATE_LATEX is set to YES.

LATEX_HIDE_INDICES = NO

The LATEX_BIB_STYLE tag can be used to specify the style to use for the

```

# bibliography, e.g. plainnat, or ieetr. See
# https://en.wikipedia.org/wiki/BibTeX and \cite for more info.
# The default value is: plainnat.
# This tag requires that the tag GENERATE_LATEX is set to YES.

LATEX_BIB_STYLE          = plainnat

# The LATEX_EMOJI_DIRECTORY tag is used to specify the (relative or absolute)
# path from which the emoji images will be read. If a relative path is entered,
# it will be relative to the LATEX_OUTPUT directory. If left blank the
# LATEX_OUTPUT directory will be used.
# This tag requires that the tag GENERATE_LATEX is set to YES.

LATEX_EMOJI_DIRECTORY    =

#-----
# Configuration options related to the RTF output
#-----

# If the GENERATE_RTF tag is set to YES, Doxygen will generate RTF output. The
# RTF output is optimized for Word 97 and may not look too pretty with other RTF
# readers/editors.
# The default value is: NO.

GENERATE_RTF              = NO

# The RTF_OUTPUT tag is used to specify where the RTF docs will be put. If a
# relative path is entered the value of OUTPUT_DIRECTORY will be put in front of
# it.
# The default directory is: rtf.
# This tag requires that the tag GENERATE_RTF is set to YES.

RTF_OUTPUT                = rtf

# If the COMPACT_RTF tag is set to YES, Doxygen generates more compact RTF
# documents. This may be useful for small projects and may help to save some
# trees in general.
# The default value is: NO.
# This tag requires that the tag GENERATE_RTF is set to YES.

COMPACT_RTF               = NO

# If the RTF_HYPERLINKS tag is set to YES, the RTF that is generated will
# contain hyperlink fields. The RTF file will contain links (just like the HTML
# output) instead of page references. This makes the output suitable for online
# browsing using Word or some other Word compatible readers that support those

```

```

# fields.
#
# Note: WordPad (write) and others do not support links.
# The default value is: NO.
# This tag requires that the tag GENERATE_RTF is set to YES.

RTF_HYPERLINKS          = NO

# Load stylesheet definitions from file. Syntax is similar to Doxygen's
# configuration file, i.e. a series of assignments. You only have to provide
# replacements, missing definitions are set to their default value.
#
# See also section "Doxygen usage" for information on how to generate the
# default style sheet that Doxygen normally uses.
# This tag requires that the tag GENERATE_RTF is set to YES.

RTF_STYLESHEET_FILE     =

# Set optional variables used in the generation of an RTF document. Syntax is
# similar to Doxygen's configuration file. A template extensions file can be
# generated using doxygen -e rtf extensionFile.
# This tag requires that the tag GENERATE_RTF is set to YES.

RTF_EXTENSIONS_FILE     =

# The RTF_EXTRA_FILES tag can be used to specify one or more extra images or
# other source files which should be copied to the RTF_OUTPUT output directory.
# Note that the files will be copied as-is; there are no commands or markers
# available.
# This tag requires that the tag GENERATE_RTF is set to YES.

RTF_EXTRA_FILES         =

#-----
# Configuration options related to the man page output
#-----

# If the GENERATE_MAN tag is set to YES, Doxygen will generate man pages for
# classes and files.
# The default value is: NO.

GENERATE_MAN            = NO

# The MAN_OUTPUT tag is used to specify where the man pages will be put. If a
# relative path is entered the value of OUTPUT_DIRECTORY will be put in front of
# it. A directory man3 will be created inside the directory specified by

```

```

# MAN_OUTPUT.
# The default directory is: man.
# This tag requires that the tag GENERATE_MAN is set to YES.

MAN_OUTPUT                = man

# The MAN_EXTENSION tag determines the extension that is added to the generated
# man pages. In case the manual section does not start with a number, the number
# 3 is prepended. The dot (.) at the beginning of the MAN_EXTENSION tag is
# optional.
# The default value is: .3.
# This tag requires that the tag GENERATE_MAN is set to YES.

MAN_EXTENSION              = .3

# The MAN_SUBDIR tag determines the name of the directory created within
# MAN_OUTPUT in which the man pages are placed. If defaults to man followed by
# MAN_EXTENSION with the initial . removed.
# This tag requires that the tag GENERATE_MAN is set to YES.

MAN_SUBDIR                 =

# If the MAN_LINKS tag is set to YES and Doxygen generates man output, then it
# will generate one additional man file for each entity documented in the real
# man page(s). These additional files only source the real man page, but without
# them the man command would be unable to find the correct page.
# The default value is: NO.
# This tag requires that the tag GENERATE_MAN is set to YES.

MAN_LINKS                  = NO

#-----
# Configuration options related to the XML output
#-----

# If the GENERATE_XML tag is set to YES, Doxygen will generate an XML file that
# captures the structure of the code including all documentation.
# The default value is: NO.

GENERATE_XML                = NO

# The XML_OUTPUT tag is used to specify where the XML pages will be put. If a
# relative path is entered the value of OUTPUT_DIRECTORY will be put in front of
# it.
# The default directory is: xml.
# This tag requires that the tag GENERATE_XML is set to YES.

```

XML_OUTPUT = xml

If the XML_PROGRAMLISTING tag is set to YES, Doxygen will dump the program
listings (including syntax highlighting and cross-referencing information) to
the XML output. Note that enabling this will significantly increase the size
of the XML output.
The default value is: YES.
This tag requires that the tag GENERATE_XML is set to YES.

XML_PROGRAMLISTING = YES

If the XML_NS_MEMB_FILE_SCOPE tag is set to YES, Doxygen will include
namespace members in file scope as well, matching the HTML output.
The default value is: NO.
This tag requires that the tag GENERATE_XML is set to YES.

XML_NS_MEMB_FILE_SCOPE = NO

#-----
Configuration options related to the DOCBOOK output
#-----

If the GENERATE_DOCBOOK tag is set to YES, Doxygen will generate Docbook files
that can be used to generate PDF.
The default value is: NO.

GENERATE_DOCBOOK = NO

The DOCBOOK_OUTPUT tag is used to specify where the Docbook pages will be put.
If a relative path is entered the value of OUTPUT_DIRECTORY will be put in
front of it.
The default directory is: docbook.
This tag requires that the tag GENERATE_DOCBOOK is set to YES.

DOCBOOK_OUTPUT = docbook

#-----
Configuration options for the AutoGen Definitions output
#-----

If the GENERATE_AUTOGEN_DEF tag is set to YES, Doxygen will generate an
AutoGen Definitions (see <https://autogen.sourceforge.net/>) file that captures
the structure of the code including all documentation. Note that this feature
is still experimental and incomplete at the moment.
The default value is: NO.

GENERATE_AUTOGEN_DEF = NO

#-----
Configuration options related to Sqlite3 output
#-----

If the GENERATE_SQLITE3 tag is set to YES Doxygen will generate a Sqlite3
database with symbols found by Doxygen stored in tables.
The default value is: NO.

GENERATE_SQLITE3 = NO

The SQLITE3_OUTPUT tag is used to specify where the Sqlite3 database will be
put. If a relative path is entered the value of OUTPUT_DIRECTORY will be put
in front of it.
The default directory is: sqlite3.
This tag requires that the tag GENERATE_SQLITE3 is set to YES.

SQLITE3_OUTPUT = sqlite3

The SQLITE3_RECREATE_DB tag is set to YES, the existing doxygen_sqlite3.db
database file will be recreated with each Doxygen run. If set to NO, Doxygen
will warn if a database file is already found and not modify it.
The default value is: YES.
This tag requires that the tag GENERATE_SQLITE3 is set to YES.

SQLITE3_RECREATE_DB = YES

#-----
Configuration options related to the Perl module output
#-----

If the GENERATE_PERLMOD tag is set to YES, Doxygen will generate a Perl module
file that captures the structure of the code including all documentation.

Note that this feature is still experimental and incomplete at the moment.
The default value is: NO.

GENERATE_PERLMOD = NO

If the PERLMOD_LATEX tag is set to YES, Doxygen will generate the necessary
Makefile rules, Perl scripts and LaTeX code to be able to generate PDF and DVI
output from the Perl module output.
The default value is: NO.
This tag requires that the tag GENERATE_PERLMOD is set to YES.

PERLMOD_LATEX = NO

If the PERLMOD_PRETTY tag is set to YES, the Perl module output will be nicely
formatted so it can be parsed by a human reader. This is useful if you want to
understand what is going on. On the other hand, if this tag is set to NO, the
size of the Perl module output will be much smaller and Perl will parse it
just the same.
The default value is: YES.
This tag requires that the tag GENERATE_PERLMOD is set to YES.

PERLMOD_PRETTY = YES

The names of the make variables in the generated doxyrules.make file are
prefixed with the string contained in PERLMOD_MAKEVAR_PREFIX. This is useful
so different doxyrules.make files included by the same Makefile don't
overwrite each other's variables.
This tag requires that the tag GENERATE_PERLMOD is set to YES.

PERLMOD_MAKEVAR_PREFIX =

#-----
Configuration options related to the preprocessor
#-----

If the ENABLE_PREPROCESSING tag is set to YES, Doxygen will evaluate all
C-preprocessor directives found in the sources and include files.
The default value is: YES.

ENABLE_PREPROCESSING = YES

If the MACRO_EXPANSION tag is set to YES, Doxygen will expand all macro names
in the source code. If set to NO, only conditional compilation will be
performed. Macro expansion can be done in a controlled way by setting
EXPAND_ONLY_PREDEF to YES.
The default value is: NO.
This tag requires that the tag ENABLE_PREPROCESSING is set to YES.

MACRO_EXPANSION = NO

If the EXPAND_ONLY_PREDEF and MACRO_EXPANSION tags are both set to YES then
the macro expansion is limited to the macros specified with the PREDEFINED and
EXPAND_AS_DEFINED tags.
The default value is: NO.
This tag requires that the tag ENABLE_PREPROCESSING is set to YES.

EXPAND_ONLY_PREDEF = NO

If the SEARCH_INCLUDES tag is set to YES, the include files in the
INCLUDE_PATH will be searched if a #include is found.
The default value is: YES.
This tag requires that the tag ENABLE_PREPROCESSING is set to YES.

SEARCH_INCLUDES = YES

The INCLUDE_PATH tag can be used to specify one or more directories that
contain include files that are not input files but should be processed by the
preprocessor. Note that the INCLUDE_PATH is not recursive, so the setting of
RECURSIVE has no effect here.
This tag requires that the tag SEARCH_INCLUDES is set to YES.

INCLUDE_PATH =

You can use the INCLUDE_FILE_PATTERNS tag to specify one or more wildcard
patterns (like *.h and *.hpp) to filter out the header-files in the
directories. If left blank, the patterns specified with FILE_PATTERNS will be
used.
This tag requires that the tag ENABLE_PREPROCESSING is set to YES.

INCLUDE_FILE_PATTERNS =

The PREDEFINED tag can be used to specify one or more macro names that are
defined before the preprocessor is started (similar to the -D option of e.g.
gcc). The argument of the tag is a list of macros of the form: name or
name=definition (no spaces). If the definition and the "=" are omitted, "=1"
is assumed. To prevent a macro definition from being undefined via #undef or
recursively expanded use the := operator instead of the = operator.
This tag requires that the tag ENABLE_PREPROCESSING is set to YES.

PREDEFINED =

If the MACRO_EXPANSION and EXPAND_ONLY_PREDEF tags are set to YES then this
tag can be used to specify a list of macro names that should be expanded. The
macro definition that is found in the sources will be used. Use the PREDEFINED
tag if you want to use a different macro definition that overrules the
definition found in the source code.
This tag requires that the tag ENABLE_PREPROCESSING is set to YES.

EXPAND_AS_DEFINED =

If the SKIP_FUNCTION_MACROS tag is set to YES then Doxygen's preprocessor will
remove all references to function-like macros that are alone on a line, have


```

# an all uppercase name, and do not end with a semicolon. Such function macros
# are typically used for boiler-plate code, and will confuse the parser if not
# removed.
# The default value is: YES.
# This tag requires that the tag ENABLE_PREPROCESSING is set to YES.

SKIP_FUNCTION_MACROS    = YES

#-----
# Configuration options related to external references
#-----

# The TAGFILES tag can be used to specify one or more tag files. For each tag
# file the location of the external documentation should be added. The format of
# a tag file without this location is as follows:
# TAGFILES = file1 file2 ...
# Adding location for the tag files is done as follows:
# TAGFILES = file1=loc1 "file2 = loc2" ...
# where loc1 and loc2 can be relative or absolute paths or URLs. See the
# section "Linking to external documentation" for more information about the use
# of tag files.
# Note: Each tag file must have a unique name (where the name does NOT include
# the path). If a tag file is not located in the directory in which Doxygen is
# run, you must also specify the path to the tagfile here.

TAGFILES                =

# When a file name is specified after GENERATE_TAGFILE, Doxygen will create a
# tag file that is based on the input files it reads. See section "Linking to
# external documentation" for more information about the usage of tag files.

GENERATE_TAGFILE        =

# If the ALLEXTERNALS tag is set to YES, all external classes and namespaces
# will be listed in the class and namespace index. If set to NO, only the
# inherited external classes will be listed.
# The default value is: NO.

ALLEXTERNALS           = NO

# If the EXTERNAL_GROUPS tag is set to YES, all external groups will be listed
# in the topic index. If set to NO, only the current project's groups will be
# listed.
# The default value is: YES.

EXTERNAL_GROUPS        = YES

```

```

# If the EXTERNAL_PAGES tag is set to YES, all external pages will be listed in
# the related pages index. If set to NO, only the current project's pages will
# be listed.
# The default value is: YES.

EXTERNAL_PAGES          = YES

#-----
# Configuration options related to diagram generator tools
#-----

# If set to YES the inheritance and collaboration graphs will hide inheritance
# and usage relations if the target is undocumented or is not a class.
# The default value is: YES.

HIDE_UNDOC_RELATIONS    = YES

# If you set the HAVE_DOT tag to YES then Doxygen will assume the dot tool is
# available from the path. This tool is part of Graphviz (see:
# https://www.graphviz.org/), a graph visualization toolkit from AT&T and Lucent
# Bell Labs. The other options in this section have no effect if this option is
# set to NO
# The default value is: NO.

HAVE_DOT                = YES

# The DOT_NUM_THREADS specifies the number of dot invocations Doxygen is allowed
# to run in parallel. When set to 0 Doxygen will base this on the number of
# processors available in the system. You can set it explicitly to a value
# larger than 0 to get control over the balance between CPU load and processing
# speed.
# Minimum value: 0, maximum value: 512, default value: 0.
# This tag requires that the tag HAVE_DOT is set to YES.

DOT_NUM_THREADS         = 0

# DOT_COMMON_ATTR is common attributes for nodes, edges and labels of
# subgraphs. When you want a differently looking font in the dot files that
# Doxygen generates you can specify fontname, fontcolor and fontsize attributes.
# For details please see <a href=https://graphviz.org/doc/info/attrs.html>Node,
# Edge and Graph Attributes specification</a> You need to make sure dot is able
# to find the font, which can be done by putting it in a standard location or by
# setting the DOTFONTPATH environment variable or by setting DOT_FONTPATH to the
# directory containing the font. Default graphviz fontsize is 14.
# The default value is: fontname=Helvetica,fontsize=10.

```

```

# This tag requires that the tag HAVE_DOT is set to YES.

DOT_COMMON_ATTR          = "fontname=Helvetica,fontsize=10"

# DOT_EDGE_ATTR is concatenated with DOT_COMMON_ATTR. For elegant style you can
# add 'arrowhead=open, arrowtail=open, arrowsize=0.5'. <a
# href=https://graphviz.org/doc/info/arrows.html>Complete documentation about
# arrows shapes.</a>
# The default value is: labelfontname=Helvetica,labelfontsize=10.
# This tag requires that the tag HAVE_DOT is set to YES.

DOT_EDGE_ATTR            = "labelfontname=Helvetica,labelfontsize=10"

# DOT_NODE_ATTR is concatenated with DOT_COMMON_ATTR. For view without boxes
# around nodes set 'shape=plain' or 'shape=plaintext' <a
# href=https://www.graphviz.org/doc/info/shapes.html>Shapes specification</a>
# The default value is: shape=box,height=0.2,width=0.4.
# This tag requires that the tag HAVE_DOT is set to YES.

DOT_NODE_ATTR            = "shape=box,height=0.2,width=0.4"

# You can set the path where dot can find font specified with fontname in
# DOT_COMMON_ATTR and others dot attributes.
# This tag requires that the tag HAVE_DOT is set to YES.

DOT_FONTPATH             =

# If the CLASS_GRAPH tag is set to YES or GRAPH or BUILTIN then Doxygen will
# generate a graph for each documented class showing the direct and indirect
# inheritance relations. In case the CLASS_GRAPH tag is set to YES or GRAPH and
# HAVE_DOT is enabled as well, then dot will be used to draw the graph. In case
# the CLASS_GRAPH tag is set to YES and HAVE_DOT is disabled or if the
# CLASS_GRAPH tag is set to BUILTIN, then the built-in generator will be used.
# If the CLASS_GRAPH tag is set to TEXT the direct and indirect inheritance
# relations will be shown as texts / links. Explicit enabling an inheritance
# graph or choosing a different representation for an inheritance graph of a
# specific class, can be accomplished by means of the command \inheritancegraph.
# Disabling an inheritance graph can be accomplished by means of the command
# \hideinheritancegraph.
# Possible values are: NO, YES, TEXT, GRAPH and BUILTIN.
# The default value is: YES.

CLASS_GRAPH              = YES

# If the COLLABORATION_GRAPH tag is set to YES then Doxygen will generate a
# graph for each documented class showing the direct and indirect implementation

```

```
# dependencies (inheritance, containment, and class references variables) of the
# class with other documented classes. Explicit enabling a collaboration graph,
# when COLLABORATION_GRAPH is set to NO, can be accomplished by means of the
# command \collaborationgraph. Disabling a collaboration graph can be
# accomplished by means of the command \hidecollaborationgraph.
# The default value is: YES.
# This tag requires that the tag HAVE_DOT is set to YES.
```

```
COLLABORATION_GRAPH    = YES
```

```
# If the GROUP_GRAPHS tag is set to YES then Doxygen will generate a graph for
# groups, showing the direct groups dependencies. Explicit enabling a group
# dependency graph, when GROUP_GRAPHS is set to NO, can be accomplished by means
# of the command \groupgraph. Disabling a directory graph can be accomplished by
# means of the command \hidegroupgraph. See also the chapter Grouping in the
# manual.
# The default value is: YES.
# This tag requires that the tag HAVE_DOT is set to YES.
```

```
GROUP_GRAPHS           = YES
```

```
# If the UML_LOOK tag is set to YES, Doxygen will generate inheritance and
# collaboration diagrams in a style similar to the OMG's Unified Modeling
# Language.
# The default value is: NO.
# This tag requires that the tag HAVE_DOT is set to YES.
```

```
UML_LOOK               = YES
```

```
# If the UML_LOOK tag is enabled, the fields and methods are shown inside the
# class node. If there are many fields or methods and many nodes the graph may
# become too big to be useful. The UML_LIMIT_NUM_FIELDS threshold limits the
# number of items for each type to make the size more manageable. Set this to 0
# for no limit. Note that the threshold may be exceeded by 50% before the limit
# is enforced. So when you set the threshold to 10, up to 15 fields may appear,
# but if the number exceeds 15, the total amount of fields shown is limited to
# 10.
# Minimum value: 0, maximum value: 100, default value: 10.
# This tag requires that the tag UML_LOOK is set to YES.
```

```
UML_LIMIT_NUM_FIELDS   = 10
```

```
# If the UML_LOOK tag is enabled, field labels are shown along the edge between
# two class nodes. If there are many fields and many nodes the graph may become
# too cluttered. The UML_MAX_EDGE_LABELS threshold limits the number of items to
# make the size more manageable. Set this to 0 for no limit.
```

Minimum value: 0, maximum value: 100, default value: 10.
This tag requires that the tag UML_LOOK is set to YES.

UML_MAX_EDGE_LABELS = 10

If the DOT_UML_DETAILS tag is set to NO, Doxygen will show attributes and
methods without types and arguments in the UML graphs. If the DOT_UML_DETAILS
tag is set to YES, Doxygen will add type and arguments for attributes and
methods in the UML graphs. If the DOT_UML_DETAILS tag is set to NONE, Doxygen
will not generate fields with class member information in the UML graphs. The
class diagrams will look similar to the default class diagrams but using UML
notation for the relationships.
Possible values are: NO, YES and NONE.
The default value is: NO.
This tag requires that the tag UML_LOOK is set to YES.

DOT_UML_DETAILS = NO

The DOT_WRAP_THRESHOLD tag can be used to set the maximum number of characters
to display on a single line. If the actual line length exceeds this threshold
significantly it will be wrapped across multiple lines. Some heuristics are
applied to avoid ugly line breaks.
Minimum value: 0, maximum value: 1000, default value: 17.
This tag requires that the tag HAVE_DOT is set to YES.

DOT_WRAP_THRESHOLD = 17

If the TEMPLATE_RELATIONS tag is set to YES then the inheritance and
collaboration graphs will show the relations between templates and their
instances.
The default value is: NO.
This tag requires that the tag HAVE_DOT is set to YES.

TEMPLATE_RELATIONS = NO

If the INCLUDE_GRAPH, ENABLE_PREPROCESSING and SEARCH_INCLUDES tags are set to
YES then Doxygen will generate a graph for each documented file showing the
direct and indirect include dependencies of the file with other documented
files. Explicit enabling an include graph, when INCLUDE_GRAPH is set to NO,
can be accomplished by means of the command \includegraph. Disabling an
include graph can be accomplished by means of the command \hideincludegraph.
The default value is: YES.
This tag requires that the tag HAVE_DOT is set to YES.

INCLUDE_GRAPH = YES

```

# If the INCLUDED_BY_GRAPH, ENABLE_PREPROCESSING and SEARCH_INCLUDES tags are
# set to YES then Doxygen will generate a graph for each documented file showing
# the direct and indirect include dependencies of the file with other documented
# files. Explicit enabling an included by graph, when INCLUDED_BY_GRAPH is set
# to NO, can be accomplished by means of the command \includedbygraph. Disabling
# an included by graph can be accomplished by means of the command
# \hideincludedbygraph.
# The default value is: YES.
# This tag requires that the tag HAVE_DOT is set to YES.

```

```
INCLUDED_BY_GRAPH      = YES
```

```

# If the CALL_GRAPH tag is set to YES then Doxygen will generate a call
# dependency graph for every global function or class method.
#
# Note that enabling this option will significantly increase the time of a run.
# So in most cases it will be better to enable call graphs for selected
# functions only using the \callgraph command. Disabling a call graph can be
# accomplished by means of the command \hidecallgraph.
# The default value is: NO.
# This tag requires that the tag HAVE_DOT is set to YES.

```

```
CALL_GRAPH             = NO
```

```

# If the CALLER_GRAPH tag is set to YES then Doxygen will generate a caller
# dependency graph for every global function or class method.
#
# Note that enabling this option will significantly increase the time of a run.
# So in most cases it will be better to enable caller graphs for selected
# functions only using the \callergraph command. Disabling a caller graph can be
# accomplished by means of the command \hidecallergraph.
# The default value is: NO.
# This tag requires that the tag HAVE_DOT is set to YES.

```

```
CALLER_GRAPH           = NO
```

```

# If the GRAPHICAL_HIERARCHY tag is set to YES then Doxygen will graphical
# hierarchy of all classes instead of a textual one.
# The default value is: YES.
# This tag requires that the tag HAVE_DOT is set to YES.

```

```
GRAPHICAL_HIERARCHY    = YES
```

```

# If the DIRECTORY_GRAPH tag is set to YES then Doxygen will show the
# dependencies a directory has on other directories in a graphical way. The
# dependency relations are determined by the #include relations between the

```

```
# files in the directories. Explicit enabling a directory graph, when
# DIRECTORY_GRAPH is set to NO, can be accomplished by means of the command
# \directorygraph. Disabling a directory graph can be accomplished by means of
# the command \hidedirectorygraph.
# The default value is: YES.
# This tag requires that the tag HAVE_DOT is set to YES.
```

```
DIRECTORY_GRAPH          = YES
```

```
# The DIR_GRAPH_MAX_DEPTH tag can be used to limit the maximum number of levels
# of child directories generated in directory dependency graphs by dot.
# Minimum value: 1, maximum value: 25, default value: 1.
# This tag requires that the tag DIRECTORY_GRAPH is set to YES.
```

```
DIR_GRAPH_MAX_DEPTH      = 1
```

```
# The DOT_IMAGE_FORMAT tag can be used to set the image format of the images
# generated by dot. For an explanation of the image formats see the section
# output formats in the documentation of the dot tool (Graphviz (see:
# https://www.graphviz.org/)).
```

```
#
# Note the formats svg:cairo and svg:cairo:cairo cannot be used in combination
# with INTERACTIVE_SVG (the INTERACTIVE_SVG will be set to NO).
# Possible values are: png, jpg, gif, svg, png:gd, png:gd:gd, png:cairo,
# png:cairo:gd, png:cairo:cairo, png:cairo:gdiplus, png:gdiplus,
# png:gdiplus:gdiplus, svg:cairo, svg:cairo:cairo, svg:svg, svg:svg:core,
# gif:cairo, gif:cairo:gd, gif:cairo:gdiplus, gif:gdiplus, gif:gdiplus:gdiplus,
# gif:gd, gif:gd:gd, jpg:cairo, jpg:cairo:gd, jpg:cairo:gdiplus, jpg:gd,
# jpg:gd:gd, jpg:gdiplus and jpg:gdiplus:gdiplus.
# The default value is: png.
# This tag requires that the tag HAVE_DOT is set to YES.
```

```
DOT_IMAGE_FORMAT         = png
```

```
# If DOT_IMAGE_FORMAT is set to svg or svg:svg or svg:svg:core, then this option
# can be set to YES to enable generation of interactive SVG images that allow
# zooming and panning.
```

```
#
# Note that this requires a modern browser other than Internet Explorer. Tested
# and working are Firefox, Chrome, Safari, and Opera.
```

```
#
# Note This option will be automatically disabled when DOT_IMAGE_FORMAT is set
# to svg:cairo or svg:cairo:cairo.
# The default value is: NO.
# This tag requires that the tag HAVE_DOT is set to YES.
```

INTERACTIVE_SVG = YES

The DOT_PATH tag can be used to specify the path where the dot tool can be
found. If left blank, it is assumed the dot tool can be found in the path.
This tag requires that the tag HAVE_DOT is set to YES.

DOT_PATH =

The DOTFILE_DIRS tag can be used to specify one or more directories that
contain dot files that are included in the documentation (see the \dotfile
command).
This tag requires that the tag HAVE_DOT is set to YES.

DOTFILE_DIRS =

You can include diagrams made with dia in Doxygen documentation. Doxygen will
then run dia to produce the diagram and insert it in the documentation. The
DIA_PATH tag allows you to specify the directory where the dia binary resides.
If left empty dia is assumed to be found in the default search path.

DIA_PATH =

The DIAFILE_DIRS tag can be used to specify one or more directories that
contain dia files that are included in the documentation (see the \diafile
command).

DIAFILE_DIRS =

When using PlantUML, the PLANTUML_JAR_PATH tag should be used to specify the
path where java can find the plantuml.jar file or to the filename of jar file
to be used. If left blank, it is assumed PlantUML is not used or called during
a preprocessing step. Doxygen will generate a warning when it encounters a
\startuml command in this case and will not generate output for the diagram.

PLANTUML_JAR_PATH =

When using PlantUML, the PLANTUML_CFG_FILE tag can be used to specify a
configuration file for PlantUML.

PLANTUML_CFG_FILE =

When using PlantUML, the specified paths are searched for files specified by
the !include statement in a PlantUML block.

PLANTUML_INCLUDE_PATH =


```
# The PLANTUMLFILE_DIRS tag can be used to specify one or more directories that
# contain PlantUml files that are included in the documentation (see the
# \plantumlfile command).
```

```
PLANTUMLFILE_DIRS      =
```

```
# The DOT_GRAPH_MAX_NODES tag can be used to set the maximum number of nodes
# that will be shown in the graph. If the number of nodes in a graph becomes
# larger than this value, Doxygen will truncate the graph, which is visualized
# by representing a node as a red box. Note that if the number of direct
# children of the root node in a graph is already larger than
# DOT_GRAPH_MAX_NODES then the graph will not be shown at all. Also note that
# the size of a graph can be further restricted by MAX_DOT_GRAPH_DEPTH.
# Minimum value: 0, maximum value: 10000, default value: 50.
# This tag requires that the tag HAVE_DOT is set to YES.
```

```
DOT_GRAPH_MAX_NODES    = 50
```

```
# The MAX_DOT_GRAPH_DEPTH tag can be used to set the maximum depth of the graphs
# generated by dot. A depth value of 3 means that only nodes reachable from the
# root by following a path via at most 3 edges will be shown. Nodes that lay
# further from the root node will be omitted. Note that setting this option to 1
# or 2 may greatly reduce the computation time needed for large code bases. Also
# note that the size of a graph can be further restricted by
# DOT_GRAPH_MAX_NODES. Using a depth of 0 means no depth restriction.
# Minimum value: 0, maximum value: 1000, default value: 0.
# This tag requires that the tag HAVE_DOT is set to YES.
```

```
MAX_DOT_GRAPH_DEPTH    = 0
```

```
# Set the DOT_MULTI_TARGETS tag to YES to allow dot to generate multiple output
# files in one run (i.e. multiple -o and -T options on the command line). This
# makes dot run faster, but since only newer versions of dot (>1.8.10) support
# this, this feature is disabled by default.
# The default value is: NO.
# This tag requires that the tag HAVE_DOT is set to YES.
```

```
DOT_MULTI_TARGETS      = NO
```

```
# If the GENERATE_LEGEND tag is set to YES Doxygen will generate a legend page
# explaining the meaning of the various boxes and arrows in the dot generated
# graphs.
# Note: This tag requires that UML_LOOK isn't set, i.e. the Doxygen internal
# graphical representation for inheritance and collaboration diagrams is used.
# The default value is: YES.
# This tag requires that the tag HAVE_DOT is set to YES.
```

GENERATE_LEGEND = YES

If the DOT_CLEANUP tag is set to YES, Doxygen will remove the intermediate
files that are used to generate the various graphs.

#

Note: This setting is not only used for dot files but also for msc temporary
files.

The default value is: YES.

DOT_CLEANUP = YES

You can define message sequence charts within Doxygen comments using the \msc
command. If the MSCGEN_TOOL tag is left empty (the default), then Doxygen will
use a built-in version of mscgen tool to produce the charts. Alternatively,
the MSCGEN_TOOL tag can also specify the name an external tool. For instance,
specifying prog as the value, Doxygen will call the tool as prog -T
<outfile_format> -o <outputfile> <inputfile>. The external tool should support
output file formats "png", "eps", "svg", and "ismap".

MSCGEN_TOOL =

The MSCFILE_DIRS tag can be used to specify one or more directories that
contain msc files that are included in the documentation (see the \mscfile
command).

MSCFILE_DIRS =

File: src/Services/Actions/AppActionsManager.vala

```
/*  
*/
```

```
public class Iide.AppActionsManager: Object {  
    private static AppActionsManager? _instance;  
    private GLib.Settings settings;  
    private Gee.HashMap<string, Iide.AppAction> actions;  
    private Gee.HashMap<string, string> shortcuts;  
  
    public static AppActionsManager get_instance () {  
        if (_instance == null) {  
            _instance = new AppActionsManager ();  
        }  
        return _instance;  
    }  
}
```

```

private AppActionsManager () {
    settings = SettingsService.get_instance ().settings;
    actions = new Gee.HashMap<string, Iide.AppAction> ();
    shortcuts = new Gee.HashMap<string, string> ();
    load_shortcuts_from_gsettings();
}

public Gee.Collection<Iide.AppAction> get_all_actions () {
    return actions.values;
}

public Iide.AppAction get_action(string action_id) {
    return actions.get(action_id);
}

private void load_shortcuts_from_gsettings () {
    var json_str = settings.get_string ("shortcuts");
    if (json_str == null || json_str == "") {
        save_shortcuts_to_gsettings ();
        return;
    }

    try {
        var parser = new Json.Parser ();
        parser.load_from_data (json_str);
        var root = parser.get_root ();
        if (root.get_node_type () == Json.NodeType.OBJECT) {
            var obj = root.get_object ();
            obj.foreach_member ((obj, name, node) => {
                shortcuts.set (name, node.get_string ());
            });
        }
    } catch (Error e) {
        warning ("Failed to parse shortcuts: %s", e.message);
    }
}

private void save_shortcuts_to_gsettings () {
    var root = new Json.Node (Json.NodeType.OBJECT);
    var obj = new Json.Object ();
    foreach (var entry in shortcuts.entries) {
        obj.set_string_member (entry.key, entry.value);
    }
    root.set_object (obj);
}

```

```

        var generator = new Json.Generator ();
        generator.set_root (root);
        var json_str = generator.to_data (null);
        settings.set_string ("shortcuts", json_str);
    }

private bool get_boolean_setting_safe (string key_name, bool fallback_value) {
    if (settings.settings_schema != null && settings.settings_schema.has_key (key_name)) {
        return settings.get_boolean (key_name);
    }
    return fallback_value;
}

public void register_action (Gtk.Application app, Iide.AppAction action) {
    actions.set (action.id, action);
    if (shortcuts.has_key (action.id)) {
        action.set_shortcut (shortcuts.get (action.id));
    }

    if (action.is_toggle) {
        bool saved_state = get_boolean_setting_safe (action.id, true);
        action.update_state (saved_state);

        var toggle_action = new SimpleAction.stateful (
            action.id,
            null,
            new Variant.boolean (action.state)
        );
        toggle_action.activate.connect (() => {
            action.execute ();
        });
        action.state_changed.connect ((new_state) => {
            toggle_action.set_state (new Variant.boolean (new_state));
            settings.set_boolean (action.id, new_state);
        });
        app.add_action (toggle_action);
    } else {
        var simple_action = new SimpleAction (action.id, null);
        simple_action.activate.connect (() => {
            action.execute ();
        });
        app.add_action (simple_action);
    }

    if (action.shortcut != null && action.shortcut != "") {
        app.set_accels_for_action ("app." + action.id, { action.shortcut });
    }
}

```

```

    }

    action.shortcut_changed.connect ((new_shortcut) => {
        if (new_shortcut != null && new_shortcut != "") {
            app.set_accels_for_action ("app." + action.id, { action.shortcut });
        } else {
            app.set_accels_for_action ("app." + action.id, {});
        }
        if (new_shortcut == null && shortcuts.has_key (action.id)) {
            shortcuts.unset (action.id);
        } else {
            shortcuts.set(action.id, new_shortcut);
        }
        save_shortcuts_to_gsettings();
    });
}

public void unregister_action (string action_id) {
    actions.unset (action_id);
    shortcuts.unset (action_id);
}
}

```

File: src/Services/LSP/DiagnosticsService.vala

```

public class Ide.DiagnosticsService : Object {
    private static DiagnosticsService? instance;

    //      : [ClientID] -> [FileURI] -> [      ]
    private Gee.HashMap<int, Gee.HashMap<string, Gee.List<LspDiagnostic>>> server_map;
    private uint update_timeout_id = 0;

    public signal void diagnostics_updated (int client_id, string uri);
    public signal void total_count_changed (int total_errors, int total_warnings);

    public static DiagnosticsService get_instance () {
        if (instance == null)instance = new DiagnosticsService ();
        return instance;
    }

    private DiagnosticsService () {
        server_map = new Gee.HashMap<int, Gee.HashMap<string, Gee.List<LspDiagnostic>>> ();
    }

    /**

```

```

*
*           Map,           Entry (           ).
*/
public Gee.Map<int, Gee.HashMap<string, Gee.List<LspDiagnostic>>> get_server_map () {
    return server_map;
}

/**
*           ID.
*           "Clangd",       "14056232".
*/
public string get_server_name (int client_id) {
    //           LspService,
    var client = IdeLspService.get_instance ().get_client_by_hash (client_id);
    if (client != null) {
        return client.name (); //           ,           LspClient           name
    }
    return @"Unknown Server ($client_id)";
}

/**
*           LSP-           .
*/
public Gee.List<LspDiagnostic>? get_diagnostics_for_file (int client_id, string uri) {
    if (server_map.has_key (client_id)) {
        var client_files = server_map.get (client_id);
        if (client_files.has_key (uri)) {
            //           ( Vala Gee.List -           )
            return client_files.get (uri);
        }
    }
    return null;
}

public void update_diagnostics (int client_id, string uri, Gee.List<LspDiagnostic> list) {
    if (!server_map.has_key (client_id)) {
        server_map.set (client_id, new Gee.HashMap<string, Gee.List<LspDiagnostic>> ());
    }

    var client_files = server_map.get (client_id);

    if (list.size == 0) {
        client_files.unset (uri);
    } else {
        client_files.set (uri, list);
    }
}

```

```

        diagnostics_updated (client_id, uri);

        //          emit_totals():
        if (update_timeout_id == 0) {
            update_timeout_id = Timeout.add (300, () => {
                emit_totals ();
                update_timeout_id = 0;
                return Source.REMOVE;
            });
        }
    }

    //
    public void remove_client (int client_id) {
        if (server_map.unset (client_id)) {
            emit_totals ();
        }
    }

    private void emit_totals () {
        int e = 0; int w = 0;
        foreach (var client_files in server_map.values) {
            foreach (var list in client_files.values) {
                foreach (var d in list) {
                    if (d.severity == 1)e++;
                    else w++;
                }
            }
        }
        total_count_changed (e, w);
    }
}

```

File: src/Services/TreeSitter/python/PythonIndenter.vala

```

/*
*/
public class Iide.PythonIndenter: GtkSource.Indenter, Object {
    private BaseTreeSitterHighlighter ts_highlighter;
    protected TreeSitter.Query query;
    public bool is_valid = true;
    private const string source = ""
        ;
        ,
        (class_definition) @indent.begin
        (function_definition) @indent.begin

```

```

        (if_statement) @indent.begin
        (elif_clause) @indent.begin
        (else_clause) @indent.begin
        (for_statement) @indent.begin
        (while_statement) @indent.begin
        (with_statement) @indent.begin
        (match_statement) @indent.begin
        (case_clause) @indent.begin
        (try_statement) @indent.begin
        (except_clause) @indent.begin
        (finally_clause) @indent.begin
        (parenthesized_expression) @indent.begin
        (list) @indent.begin
        (dictionary) @indent.begin
        (argument_list) @indent.begin
        (parameters) @indent.begin
    """;

public PythonIndenter (BaseTreeSitterHighlighter ts_highlighter, TreeSitter.Language language,
    Object ();
    this.ts_highlighter = ts_highlighter;

    uint32 error_offset;
    TreeSitter.QueryError error_type;
    this.query = new TreeSitter.Query (lang, source, (uint32) source.length, out error_offset, out error_type);

    if (error_type != TreeSitter.QueryError.None) {
        LoggerService.get_instance ().error ("TS", "TreeSitter Query Error at %u: %s".printf (error_offset, error_type));
        this.is_valid = false;
    }
}

//
private string get_line_indent_text (Gtk.TextIter line_start) {
    Gtk.TextIter line_end = line_start;
    line_end.forward_to_line_end ();
    string text = line_start.get_buffer ().get_text (line_start, line_end, false);

    string indent = "";
    //
    for (int i = 0; i < text.length; i++) {
        unichar c = text.get_char (i);
        if (c.isspace ()) {
            indent += c.to_string ();
        } else {
            break;
        }
    }
}

```



```

    }
}
return indent;
}

public void indent (GtkSource.View view, ref Gtk.TextIter iter) {
    if (!is_valid)
        return;

    if (ts_highlighter.get_tree () == null)
        return;

    ts_highlighter.flush_changes();

    // 1.      (
    Gtk.TextIter anchor = iter;
    bool found_anchor = false;
    while (anchor.backward_line ()) {
        Gtk.TextIter line_end = anchor;
        line_end.forward_to_line_end ();
        if (view.buffer.get_text (anchor, line_end, false).strip ().length > 0) {
            found_anchor = true;
            break;
        }
    }
    if (!found_anchor)
        return;

    // 2.
    var base_indent = get_line_indent_text (anchor);

    // 3.      Tree-Sitter
    Gtk.TextIter anchor_end = anchor;
    anchor_end.forward_to_line_end ();
    uint32 end_byte = get_byte_offset_safe (anchor_end);

    if (end_byte > 0) {
        var cursor = new TreeSitter.QueryCursor ();
        //      (
        cursor.set_byte_range (end_byte - 1, end_byte);
        cursor.exec (this.query, ts_highlighter.get_tree ().root_node ());

        TreeSitter.QueryMatch match;
        while (cursor.next_match (out match)) {
            for (int i = 0; i < match.capture_count; i++) {
                unowned TreeSitter.Node node = match.captures[i].node;
            }
        }
    }
}

```

```

        //      1:
        if (node.start_point ().row == (uint32) anchor.get_line ()) {

            //      2 ( ):
            //
            if (node.end_byte () <= end_byte) {
                continue;
            }

            base_indent += "      ";
            break;
        }
    }
}

// 4.
view.buffer.begin_user_action ();

view.buffer.insert (ref iter, base_indent, base_indent.length);

// Remove space symbols from current position...
Gtk.TextIter line_start = iter;
Gtk.TextIter line_end = line_start;
while (line_end.get_char ().isspace () && !line_end.ends_line ()) {
    line_end.forward_char ();
}

if (!line_start.equal (line_end)) {
    view.buffer.delete (ref line_start, ref line_end);
}

view.buffer.end_user_action ();
}

public bool is_trigger (GtkSource.View view, Gtk.TextIter location, Gdk.ModifierType sta
    return (keyval == Gdk.Key.Return || keyval == Gdk.Key.KP_Enter);
}
}

```

File: src/Services/PendingChange.vala

```

/*
*/

```

```

public class Ide.PendingChange : GLib.Object {
    public int start_offset;    //
    public int end_offset;     //
    public string text;        //          ( delete_range -      )

    //
    public int start_line;
    public int start_utf16_char;
    public int start_utf8_char;
    public int end_line;
    public int end_utf16_char;
    public int end_utf8_char;

    public PendingChange (string t, Gtk.TextIter s_iter, Gtk.TextIter? e_iter = null) {
        this.text = t;
        this.start_offset = s_iter.get_offset ();
        this.end_offset = e_iter != null? e_iter.get_offset () : this.start_offset;

        //
        this.calculate_position (s_iter, out this.start_line, out this.start_utf16_char, out
        if (e_iter != null) {
            this.calculate_position (e_iter, out this.end_line, out this.end_utf16_char, out
        } else {
            this.end_line = this.start_line;
            this.end_utf16_char = this.start_utf16_char;
            this.end_utf8_char = this.start_utf8_char;
        }
    }

    private void calculate_position (Gtk.TextIter iter, out int line_number, out int utf16_c
        line_number = iter.get_line ();

        //
        Gtk.TextIter line_start = iter;
        line_start.set_line_offset (0);

        //
        string line_text = line_start.get_text (iter);

        //      UTF-16 code units
        int utf16_count = 0;
        int utf8_count = 0;
        int i = 0;
        unichar c;
        while (line_text.get_next_char (ref i, out c)) {

```

```

        utf8_count++;
        if (c <= 0xFFFF) {
            utf16_count++;
        } else {
            utf16_count += 2;
        }
    }
    utf16_char = utf16_count;
    utf8_char = utf8_count;
}
}

```

File: src/Widgets/TextView/GutterMarkRenderer.vala

```

using Gtk;
using GtkSource;
using GLib;

public class Iide.GutterMarkRenderer : GutterRenderer {
    private int current_icon_size = 16;

    private Gtk.IconPaintable error_paintable;
    private Gtk.IconPaintable warning_paintable;
    private Gdk.RGBA[] error_colors = {
        Gdk.RGBA() {red = 1.0f, green = 0.0f, blue = 0.0f, alpha = 0.86f}
    };
    private Gdk.RGBA[] warning_colors = {
        Gdk.RGBA() {red = 0.7f, green = 0.7f, blue = 0.0f, alpha = 0.86f}
    };

    public GutterMarkRenderer() {
        Object();
        init_paintable_icons ();
    }

    private void init_paintable_icons() {
        var icon_provider = SymbIconProvider.get_instance ();
        var display = Gdk.Display.get_default ();
        var theme = Gtk.IconTheme.get_for_display (display);

        var error_icon_name = icon_provider.icon_name (IconID.COD_ERROR);
        var error_gicon = new GLib.ThemedIcon (error_icon_name);
        error_paintable = theme.lookup_by_gicon (error_gicon, current_icon_size, 1, Gtk.Text

        var warning_icon_name = icon_provider.icon_name (IconID.COD_WARNING);
    }
}

```

```

        var warning_gicon = new GLib.ThemedIcon (warning_icon_name);
        warning_paintable = theme.lookup_by_gicon (warning_gicon, current_icon_size, 1, Gtk.
    }

    public void set_icons_size (int size) {
        if (current_icon_size == size) {
            return;
        }
        current_icon_size = size;
        queue_resize ();

        var gutter = (Gutter) get_parent ();
        if (gutter != null) {
            gutter.queue_allocate ();
        }
    }

    public override void measure (Gtk.Orientation orientation, int for_size, out int minimum
        minimum_baseline = natural_baseline = -1;
        if (orientation == Gtk.Orientation.HORIZONTAL) {
            minimum = natural = current_icon_size;
        } else {
            minimum = natural = 0;
        }
    }

    public override void snapshot_line (Gtk.Snapshot snapshot, GutterLines lines, uint line)
        var buffer = get_buffer ();
        if (buffer == null) {
            return;
        }

        Gtk.TextIter iter;
        lines.get_iter_at_line (out iter, line);
        var marks = iter.get_marks ();

        bool has_error = false;
        bool has_warning = false;
        foreach (var text_mark in marks) {
            var lsp_mark = text_mark as LspDiagnosticsMark;
            if (lsp_mark != null) {
                if (lsp_mark.severity == 1) {
                    has_error = true;
                } else {
                    has_warning = true;
                }
            }
        }
    }

```

```

    }
}

if (!has_error && !has_warning) {
    return;
}

int y, height;
lines.get_line_yrange (line, GutterRendererAlignmentMode.CELL, out y, out height);

var snapshot_size = (float) current_icon_size;
var x = (float) xpad;
var y_pos = (float) y + ((float) height - snapshot_size) / 2.0f;

var point = Graphene.Point () { x = x, y = y_pos };
snapshot.translate (point);
if (has_warning)
    warning_paintable.snapshot_symbolic (snapshot, snapshot_size, snapshot_size, warning);
if (has_error)
    error_paintable.snapshot_symbolic (snapshot, snapshot_size, snapshot_size, error);
snapshot.translate (Graphene.Point () { x = -x, y = -y_pos });
}
}

```

File: src/iide.gresource.xml

```

<?xml version="1.0" encoding="UTF-8" ?>
<gresources>
  <gresource prefix="/org/github/kai66673/iide">
    <file preprocess="xml-stripblanks">window.ui</file>
    <file preprocess="xml-stripblanks">shortcuts-dialog.ui</file>
    <file>fonts/SymbolsNerdFont-Regular.ttf</file>
    <file>style.css</file>
  </gresource>
  <gresource prefix="/org/github/kai66673/iide/icons">
    <file
      compressed="true"
      alias="scalable/mimetypes/text-x-python-symbolic.svg"
    >icons/mimetypes/text-x-python-symbolic.svg</file>
    <file
      compressed="true"
      alias="scalable/mimetypes/text-x-vala-symbolic.svg"
    >icons/mimetypes/text-x-vala-symbolic.svg</file>
    <file
      compressed="true"

```

```

        alias="scalable/mimetypes/text-x-meson-symbolic.svg"
    >icons/mimetypes/text-x-meson-symbolic.svg</file>
<file
    compressed="true"
    alias="scalable/mimetypes/text-markdown-symbolic.svg"
    >icons/mimetypes/text-markdown-symbolic.svg</file>
<file
    compressed="true"
    alias="scalable/mimetypes/text-x-javascript-symbolic.svg"
    >icons/mimetypes/text-x-javascript-symbolic.svg</file>
<file
    compressed="true"
    alias="scalable/mimetypes/text-xml-symbolic.svg"
    >icons/mimetypes/text-xml-symbolic.svg</file>
</gresource>
</gresources>

```

File: vapi/libtreesitter.vapi

```

[CCode (cheader_filename = "tree_sitter/api.h")]
namespace TreeSitter {

    [CCode (cname = "TREE_SITTER_LANGUAGE_VERSION")]
    public const int LANGUAGE_VERSION;

    [CCode (cname = "TREE_SITTER_MIN_COMPATIBLE_LANGUAGE_VERSION")]
    public const int MIN_COMPATIBLE_LANGUAGE_VERSION;

    [CCode (cname = "TSInputEncoding", cprefix = "TSInputEncoding", has_type_id = false)]
    public enum InputEncoding {
        UTF8,
        UTF16
    }

    [CCode (cname = "TSSymbolType", cprefix = "TSSymbolType", has_type_id = false)]
    public enum SymbolType {
        Regular,
        Anonymous,
        Auxiliary
    }

    [CCode (cname = "TSLogType", cprefix = "TSLogType", has_type_id = false)]
    public enum LogType {
        Parse,
        Lex
    }

```

```

}

[SimpleType]
[CCode (cname = "TSSymbol", has_type_id = false)]
public struct Symbol : uint16 {
}

[CCode (cname = "TSLanguage")]
[Compact]
public class Language {
    uint32 version;
    uint32 symbol_count;
    uint32 alias_count;
    uint32 token_count;
    uint32 external_token_count;
    // const char **symbol_names;
    // const TSSymbolMetadata *symbol_metadata;
    // const uint16 *parse_table;
    // const TSParseActionEntry *parse_actions;
    // const TSLexMode *lex_modes;
    // const TSSymbol *alias_sequences;
    uint16 max_alias_sequence_length;
    // bool (*lex_fn)(TSLexer *, TSSStateId);
    // bool (*keyword_lex_fn)(TSLexer *, TSSStateId);
    // Symbol keyword_capture_token;
/*
struct {
    const bool *states;
    const TSSymbol *symbol_map;
    void (*create)();
    void (*destroy)(void *);
    bool (*scan)(void *, TSLexer *, const bool *symbol_whitelist);
    unsigned (*serialize)(void *, char *);
    void (*deserialize)(void *, const char *, unsigned);
} external_scanner;
*/
    [CCode (cname = "ts_language_symbol_count")]
    public uint32 get_symbol_count ();

    [CCode (cname = "ts_language_symbol_name")]
    public unowned string get_symbol_name (Symbol symbol);

    [CCode (cname = "ts_language_symbol_for_name")]
    public Symbol symbol_for_name (string name);

    [CCode (cname = "ts_language_symbol_type")]

```



```

        public SymbolType get_symbol_type (Symbol symbol);

        [CCode (cname = "ts_language_abi_version")]
        public uint32 get_version ();
    }

    [CCode (cname = "TSParser", free_function = "ts_parser_delete")]
    [Compact]
    public class Parser {
        [CCode (cname = "ts_parser_new")]
        public Parser ();

        [CCode (cname = "ts_parser_language")]
        public unowned Language get_language ();

        [CCode (cname = "ts_parser_set_language")]
        public bool set_language (Language language);

        [CCode (cname = "ts_parser_parse_string", array_length_type = "uint32_t")]
        public Tree parse_string (Tree? tree, uint8[] source);

        [CCode (cname = "ts_parser_parse")]
        public Tree ? parse (Tree? old_tree, Input input);
    }

    [CCode (cname = "TSTree", free_function = "ts_tree_delete")]
    [Compact]
    public class Tree {
        [CCode (cname = "ts_tree_root_node")]
        public Node root_node ();

        [CCode (cname = "ts_tree_edit")]
        public void edit (InputEdit edit);

        [CCode (cname = "ts_tree_get_changed_ranges")]
        public Range[] get_changed_ranges (Tree new_tree);
    }

    [SimpleType]
    [CCode (cname = "TSPoint", has_type_id = false)]
    public struct Point {
        uint32 row;
        uint32 column;
    }

    [CCode (cname = "TSRange", has_type_id = false)]

```

```

public struct Range {
    Point start_point;
    Point end_point;
    uint32 start_byte;
    uint32 end_byte;
}

[CCode (cname = "TSInputReadFunc", has_target = false)]
public delegate string ? InputReadFunc (void* payload, uint32 byte_index, Point position);

[CCode (cname = "TSInput", has_type_id = false)]
[SimpleType] //
public struct Input {
    public void* payload;
    public InputReadFunc read;
    public InputEncoding encoding;
}

[CCode (cname = "TSLogger", has_type_id = false)]
public struct Logger {
    void* payload;
    // void (*log)(void *payload, TSLogType, const char *);
}

[CCode (cname = "TSInputEdit", has_type_id = false)]
public struct InputEdit {
    uint32 start_byte;
    uint32 old_end_byte;
    uint32 new_end_byte;
    Point start_point;
    Point old_end_point;
    Point new_end_point;
}

[SimpleType]
[CCode (cname = "TSNode", has_type_id = false)]
public struct Node {

    [CCode (cname = "ts_node_type")]
    public unowned string type ();

    [CCode (cname = "ts_node_string")]
    public string to_str ();

    [CCode (cname = "ts_node_start_point")]
    public Point start_point ();
}

```

```

[CCode (cname = "ts_node_end_point")]
public Point end_point ();

[CCode (cname = "ts_node_start_byte")]
public uint32 start_byte ();

[CCode (cname = "ts_node_end_byte")]
public uint32 end_byte ();

[CCode (cname = "ts_node_child_count")]
public uint32 child_count ();

[CCode (cname = "ts_node_child")]
public unowned Node child (uint32 index);

[CCode (cname = "ts_node_descendant_for_byte_range")]
public Node descendant_for_byte_range (uint32 start, uint32 end);

[CCode (cname = "ts_node_named_descendant_for_byte_range")]
public Node named_descendant_for_byte_range (uint32 start, uint32 end);

[CCode (cname = "ts_node_parent")]
public Node parent ();

[CCode (cname = "ts_node_is_null")]
public bool is_null ();

[CCode (cname = "ts_node_child_by_field_name")]
public Node child_by_field_name (string field_name, uint32 name_len = 0);

[CCode (cname = "ts_node_named_child_count")]
public uint32 named_child_count ();

[CCode (cname = "ts_node_named_child")]
public TreeSitter.Node named_child (uint32 child_index);
}

[CCode (cname = "TSTreeCursor", free_function = "ts_tree_cursor_delete", has_type_id = 1)
[Compact]
public class TreeCursor {
    [CCode (cname = "ts_tree_cursor_goto_next_sibling")]
    public bool goto_next_sibling ();

    [CCode (cname = "ts_tree_cursor_goto_first_child")]
    public bool goto_first_child ();
}

```

```

    [CCode (cname = "ts_tree_cursor_current_node")]
    public Node current_node ();

    [CCode (cname = "ts_tree_cursor_goto_parent")]
    public bool goto_parent ();
}

[CCode (cname = "TSQueryError", cprefix = "TSQueryError", has_type_id = false)]
public enum QueryError {
    None,
    Syntax,
    NodeType,
    Field,
    Capture,
    Structure,
    Language
}

[CCode (cname = "TSQuery", free_function = "ts_query_delete")]
[Compact]
public class Query {
    [CCode (cname = "ts_query_new")]
    public Query (Language language, string source, uint32 source_len, out uint32 error) {
        ...
    }

    [CCode (cname = "ts_query_capture_count")]
    public uint32 capture_count ();

    [CCode (cname = "ts_query_capture_name_for_id", array_length = false)]
    public unowned string capture_name_for_id (uint32 id, out uint32 length);

    [CCode (cname = "ts_query_string_count")]
    public uint32 string_count ();
}

[CCode (cname = "TSQueryMatch", has_type_id = false, destroy_function = "")]
public struct QueryMatch {
    public uint32 id;
    public uint16 pattern_index;
    public uint16 capture_count;
    [CCode (array_length_cname = "capture_count")]
    public QueryCapture[] captures;
}

[CCode (cname = "TSQueryCapture", has_type_id = false)]
public struct QueryCapture {
    ...
}

```

```

        public Node node;
        public uint32 index;
    }

    [CCode (cname = "TSQueryCursor", free_function = "ts_query_cursor_delete")]
    [Compact]
    public class QueryCursor {
        [CCode (cname = "ts_query_cursor_new")]
        public QueryCursor ();

        [CCode (cname = "ts_query_cursor_exec")]
        public void exec (Query query, Node node);

        [CCode (cname = "ts_query_cursor_next_match")]
        public bool next_match (out QueryMatch match);

        [CCode (cname = "ts_query_cursor_set_byte_range")]
        public void set_byte_range (uint32 start, uint32 end);
    }
}

/*

TSLogger ts_parser_logger(const TSParser *);
void ts_parser_set_logger(TSParser *, TSLogger);
void ts_parser_print_dot_graphs(TSParser *, FILE *);
void ts_parser_halt_on_error(TSParser *, bool);
TSTree *ts_parser_parse(TSParser *, const TSTree *, TSInput);
bool ts_parser_enabled(const TSParser *);
void ts_parser_set_enabled(TSParser *, bool);
size_t ts_parser_operation_limit(const TSParser *);
void ts_parser_set_operation_limit(TSParser *, size_t);
void ts_parser_reset(TSParser *);
void ts_parser_set_included_ranges(TSParser *, const TSTree *, uint32_t);
const TSTree *ts_parser_included_ranges(const TSParser *, uint32_t *);

TSTree *ts_tree_copy(const TSTree *);
void ts_tree_edit(TSTree *, const TSInputEdit *);
TSTree *ts_tree_get_changed_ranges(const TSTree *, const TSTree *, uint32_t *);
void ts_tree_print_dot_graph(const TSTree *, FILE *);
const TSTree *ts_tree_language(const TSTree *);

TSSymbol ts_node_symbol(TSNode);
char *ts_node_string(TSNode);
bool ts_node_eq(TSNode, TSNode);
bool ts_node_is_null(TSNode);

```

```

bool ts_node_is_named(TSNode);
bool ts_node_is_missing(TSNode);
bool ts_node_has_changes(TSNode);
bool ts_node_has_error(TSNode);
TSNode ts_node_parent(TSNode);
TSNode ts_node_child(TSNode, uint32_t);
TSNode ts_node_named_child(TSNode, uint32_t);
uint32_t ts_node_child_count(TSNode);
uint32_t ts_node_named_child_count(TSNode);
TSNode ts_node_next_sibling(TSNode);
TSNode ts_node_next_named_sibling(TSNode);
TSNode ts_node_prev_sibling(TSNode);
TSNode ts_node_prev_named_sibling(TSNode);
TSNode ts_node_first_child_for_byte(TSNode, uint32_t);
TSNode ts_node_first_named_child_for_byte(TSNode, uint32_t);
TSNode ts_node_descendant_for_byte_range(TSNode, uint32_t, uint32_t);
TSNode ts_node_named_descendant_for_byte_range(TSNode, uint32_t, uint32_t);
TSNode ts_node_descendant_for_point_range(TSNode, TSPoint, TSPoint);
TSNode ts_node_named_descendant_for_point_range(TSNode, TSPoint, TSPoint);
void ts_node_edit(TSNode *, const TSInputEdit *);

void ts_tree_cursor_delete(TSTreeCursor *);
void ts_tree_cursor_reset(TSTreeCursor *, TSNode);
int64_t ts_tree_cursor_goto_first_child_for_byte(TSTreeCursor *, uint32_t);

*/

```

File: .gitignore

File: meson.build

```

project(
  'iide',
  ['c', 'vala'],
  version: '0.1.0',
  meson_version: '>= 1.0.0',
  default_options: [
    'warning_level=2',
    'werror=false',
  ],
)

add_project_arguments('-DGETTEXT_PACKAGE="' + meson.project_name() + '"', language: 'c')

```

```

tree_sitter_sources = [
    'vendor/tree-sitter/lib/src/lib.c',
    'parsers/ts_helper.c',
    'parsers/bash/src/parser.c',
    'parsers/bash/src/scanner.c',
    'parsers/json/src/parser.c',
    'parsers/cpp/src/parser.c',
    'parsers/cpp/src/scanner.c',
    'parsers/go/src/parser.c',
    'parsers/html/src/parser.c',
    'parsers/html/src/scanner.c',
    'parsers/javascript/src/parser.c',
    'parsers/javascript/src/scanner.c',
    'parsers/python/src/parser.c',
    'parsers/python/src/scanner.c',
    'parsers/ruby/src/parser.c',
    'parsers/ruby/src/scanner.c',
    'parsers/rust/src/parser.c',
    'parsers/rust/src/scanner.c',
    'parsers/yaml/src/parser.c',
    'parsers/yaml/src/scanner.c',
    'parsers/vala/src/parser.c',
]

tree_sitter_lib = static_library(
    'tree-sitter-runtime',
    tree_sitter_sources,
    include_directories: [
        include_directories('vendor/tree-sitter/lib/src'),
        include_directories('vendor/tree-sitter/lib/include'),
        include_directories('parsers/vala/src'),
    ],
)

tree_sitter_vapi = meson.get_compiler('vala').find_library('libtreesitter', dirs: join_paths

tree_sitter_xml_sources = [
    'parsers/xml/xml/src/parser.c',
    'parsers/xml/xml/src/scanner.c',
]

tree_sitter_xml = static_library(
    'tree-sitter-xml',
    tree_sitter_xml_sources,
    include_directories: [

```

```

        include_directories('parsers/xml/xml/src'),
    ],
)

tree_sitter_c_sources = [
    'parsers/c/src/parser.c',
]

tree_sitter_c = static_library(
    'tree-sitter-c',
    tree_sitter_c_sources,
    include_directories: [
        include_directories('parsers/c/src'),
    ],
)

tree_sitter_typescript_sources = [
    'parsers/typescript/typescript/src/parser.c',
    'parsers/typescript/typescript/src/scanner.c',
]

tree_sitter_typescript = static_library(
    'tree-sitter-typescript',
    tree_sitter_typescript_sources,
    include_directories: [
        include_directories('parsers/typescript/typescript/src'),
    ],
)

tree_sitter_php_sources = [
    'parsers/php/php/src/parser.c',
    'parsers/php/php/src/scanner.c',
]

tree_sitter_php = static_library(
    'tree-sitter-php',
    tree_sitter_php_sources,
    include_directories: [
        include_directories('parsers/php/php/src'),
    ],
)

tree_sitter_dep = declare_dependency(
    link_with: [
        tree_sitter_lib,
        tree_sitter_xml,
    ],
)

```



```

        tree_sitter_c,
        tree_sitter_typescript,
        tree_sitter_php,
    ],
    include_directories: include_directories(join_paths('vendor', 'tree-sitter', 'lib', 'in
dependencies: tree_sitter_vapi,
)

i18n = import('i18n')
gnome = import('gnome')
valac = meson.get_compiler('vala')

srcdir = meson.project_source_root() / 'src'

config_h = configuration_data()
config_h.set_quoted('PACKAGE_VERSION', meson.project_version())
config_h.set_quoted('GETTEXT_PACKAGE', 'iide')
config_h.set_quoted('LOCALEDIR', get_option('prefix') / get_option('localedir'))
configure_file(output: 'config.h', configuration: config_h)

config_dep = valac.find_library('config', dirs: srcdir)
config_inc = include_directories('.')

subdir('data')
subdir('symbols')
subdir('src')
subdir('po')

gnome.post_install(
    glib_compile_schemas: true,
    gtk_update_icon_cache: true,
    update_desktop_database: true,
)

local_settings_conf = configuration_data()
local_settings_conf.set('PREFIX', get_option('prefix'))
local_settings_sh = configure_file(
    input: 'scripts/install-local-settings.sh.in',
    output: 'install-local-settings.sh',
    configuration: local_settings_conf,
)
meson.add_install_script(local_settings_sh)

doxygen = find_program('doxygen', required: false)
gen_docs_script = find_program('scripts/gen-docs.sh')
if doxygen.found()

```

```

        run_target('docs',
            command: [gen_docs_script]
        )
    endif

```

File: src/Services/LSP/LspDocumentClient.vala

```

/*
*/

public class Ide.LspDocumentClient: GLib.Object {
    private SourceView source_view;

    private Gee.ArrayList<PendingChange> pending_queue = new Gee.ArrayList<PendingChange> ();
    private int document_version = 0;
    private uint debounce_id = 0;
    private uint debounce_full_id = 0;
    private int lsp_sync_kind = 0; // 0 - , 1 - incremental, 2 - full sync

    private ulong change_handler_id = 0;

    public LspDocumentClient(SourceView source_view) {
        Object();
        this.source_view = source_view;
    }

    public void add_change (PendingChange nc) {
        if (!pending_queue.is_empty) {
            // TODO: merge changes...
        }

        this.document_version++;
        pending_queue.add (nc);
        reset_timer ();
    }

    private void reset_timer () {
        if (debounce_id > 0)
            Source.remove (debounce_id);
        debounce_id = Timeout.add (400, () => {
            flush_changes ();
            return Source.REMOVE;
        });
    }
}

```

```

public void flush_changes () {
    if (lsp_sync_kind != 1)
        return;
    if (pending_queue.is_empty)
        return;

    var changes = pending_queue;
    pending_queue = new Gee.ArrayList<PendingChange> ();
    if (debounce_id > 0) {
        Source.remove (debounce_id);
        debounce_id = 0;
    }

    //
    var lsp_service = IdeLspService.get_instance ();
    lsp_service.send_did_change.begin (this.source_view.uri, this.document_version, char

}

private async void flush_changes_async () {
    if (pending_queue.is_empty)return;

    var changes = pending_queue;
    pending_queue = new Gee.ArrayList<PendingChange> ();
    if (debounce_id > 0) {
        Source.remove (debounce_id);
        debounce_id = 0;
    }

    //
    var lsp_service = IdeLspService.get_instance ();
    yield lsp_service.send_did_change (this.source_view.uri, this.document_version, char

}

private async void flush_changes_full_async () {
    this.document_version++;

    //
    ( , Main Loop )
    string current_text = this.source_view.buffer.text;

    var client = IdeLspService.get_instance ().get_client_for_uri (this.source_view.uri);
    if (client != null) {
        try {
            //
            yield client.text_document_did_change (this.source_view.uri, this.document_v

            debug ("LSP: Full sync sent for %s (v%d)", this.source_view.uri, this.docume

```

```

        } catch (Error e) {
            warning ("LSP: Full sync error: %s", e.message);
        }
    }
}

public async void sync_changes_async () {
    switch (lsp_sync_kind) {
        case 1:
            yield flush_changes_async ();

            break;
        case 2:
            yield flush_changes_full_async ();

            break;
    }
}

public void bind_lsp_client (LspClient? client) {
    if (client != null)
        setup_lsp_sync (client);
    else
        setup_no_lsp_sync ();
}

//
private void setup_lsp_sync (LspClient client) {
    this.apply_sync_strategy (client);
}

private void setup_no_lsp_sync () {
    lsp_sync_kind = 0;
    if (change_handler_id > 0)
        this.source_view.disconnect (change_handler_id);

    // ( )
    this.pending_queue.clear ();
}

private void apply_sync_strategy (LspClient client) {
    // (Full Sync)
    if (client.capabilities.sync_kind != TextDocumentSyncKind.INCREMENTAL) {
        // 'before'
        if (change_handler_id > 0)
            this.source_view.disconnect (change_handler_id);
    }
}

```

```

//          (          Full Sync)
this.pending_queue.clear ();

//          Full Sync (          )
this.source_view.buffer.changed.connect_after (() => {
    this.start_debounce_full_timer ();
});

//          (          )
client.text_document_did_change.begin (
    this.source_view.uri, this.document_version, this.source_view.buffer.text);
lsp_sync_kind = 2;
} else {
    lsp_sync_kind = 1;
    reset_timer ();
}
}

private void start_debounce_full_timer () {
    // 1.          ,
    if (debounce_full_id > 0) {
        Source.remove (debounce_full_id);
    }

    // 2.          (          , 500          )
    debounce_full_id = Timeout.add (500, () => {
        this.flush_changes_full_async.begin (); //
        debounce_full_id = 0;
        return Source.REMOVE;
    });
}
}
}

```

File: src/Services/DocumentManager.vala

```

/*
 * documentmanager.vala
 *
 * Copyright 2026 kai
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or

```

```

* (at your option) any later version.
*
* This program is distributed in the hope that it will be useful,
* but WITHOUT ANY WARRANTY; without even the implied warranty of
* MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
* GNU General Public License for more details.
*
* You should have received a copy of the GNU General Public License
* along with this program. If not, see <https://www.gnu.org/licenses/>.
*
* SPDX-License-Identifier: GPL-3.0-or-later
*/

```

```

using Gee;
using GLib;
using Gtk;
using Panel;

```

```

public class Ide.DocumentManager : GLib.Object {
    public Window window;
    private IdeLspService lsp_service;

    private string? current_workspace_root;

    private LoggerService logger = LoggerService.get_instance ();
    private static DocumentManager? _instance;

    public static DocumentManager get_instance () {
        return _instance;
    }

    public Gee.HashMap<string, TextView> _documents;
    public Gee.HashMap<string, TextView> documents {
        get {
            _documents = new Gee.HashMap<string, TextView> ();
            window.grid.foreach_frame ((frame) => {
                var pages = frame.get_pages ();

                for (uint i = 0; i < pages.get_n_items (); i++) {
                    var item = pages.get_item (i) as Adw.TabPage;
                    if (item == null) {
                        continue;
                    }
                    var child = item.get_child ();
                    if (child == null) {
                        continue;
                    }
                }
            });
        }
    }
}

```

```

        }
        var text_view = child as TextView;
        if (text_view != null) {
            _documents.set (text_view.uri, text_view);
        }
    }
});
return _documents;
}
}

public DocumentManager (Window window) {
    this.window = window;
    DocumentManager._instance = this;
    // documents = new Gee.HashMap<string, TextView> ();
    lsp_service = IdeLspService.get_instance ();

    lsp_service.diagnostics_updated.connect ((uri, diagnostics) => {
        var doc = documents.get (uri);
        if (doc != null) {
            var lsp_diagnostics = new Gee.ArrayList<LspDiagnostic> ();
            foreach (var diag in diagnostics) {
                var d = new LspDiagnostic ();
                d.severity = diag.severity;
                d.message = diag.message;
                d.start_line = diag.start_line;
                d.start_column = diag.start_column;
                d.end_line = diag.end_line;
                d.end_column = diag.end_column;
                lsp_diagnostics.add (d);
            }

            Idle.add (() => {
                doc.update_diagnostics (lsp_diagnostics);
                return false;
            });
        }
    });
}

public void set_workspace_root (string? root) {
    current_workspace_root = root;
}

public signal void document_opened (TextView document);

```

```

public Panel.Widget? open_document (GLib.File file, Panel.Position ? pos) {
    return open_document_with_selection (file, -1, -1, -1, pos);
}

public Panel.Widget? open_document_with_selection (GLib.File file, int line, int start_col, int end_col,
string uri = file.get_uri ();

var docs = documents;
if (docs.has_key (uri)) {
    var widget = docs.get (uri);
    widget.raise ();
    widget.view_grab_focus ();
    if (line >= 0) {
        widget.select_and_scroll (line, start_col, end_col, false);
    }
    return widget;
} else {
    var shared_table = Iide.StyleService.get_instance ().shared_table;
    var buffer = new GtkSource.Buffer (shared_table);
    var source_file = new GtkSource.File ();
    source_file.location = file;
    var file_loader = new GtkSource.FileLoader (buffer, source_file);
    Iide.TextView? panel_widget = null;
    file_loader.load_async.begin (Priority.DEFAULT, null, null, (obj, res) => {
        try {
            file_loader.load_async.end (res);
            panel_widget = new Iide.TextView (file, buffer, window);

            panel_widget.buffer_saved.connect (() => {
                string content = ((GtkSource.Buffer) panel_widget.text_view.buffer).get_text ();
                lsp_service.change_document.begin (uri, content);
            });

            if (pos == null) {
                window.grid.add (panel_widget);
            } else {
                window.add_widget (panel_widget, pos);
            }
            panel_widget.raise ();
            panel_widget.view_grab_focus ();
            logger.debug ("Doc", "Document panel widget created: " + uri);

            if (line >= 0) {
                // TODO: with timeout...
                panel_widget.select_and_scroll (line, start_col, end_col, true);
            }
        }
    });
}

```



```

        string content = buffer.text;
        string? lang_id = lsp_service.get_language_id_for_file (file);
        if (lang_id != null) {
            lsp_service.open_document.begin (uri, lang_id, content, current_work
        }
    } catch (Error e) {
        logger.error ("Doc", "Error Opening File", "Failed to read file %s: %s".
    }
});
return panel_widget;
}

public Panel.Widget? get_document_for_file (GLib.File file) {
    string uri = file.get_uri ();
    return documents.get (uri);
}

public bool is_file_open (GLib.File file) {
    return documents.has_key (file.get_uri ());
}

public Gee.ArrayList<string> get_open_document_uris () {
    var uris = new Gee.ArrayList<string> ();
    foreach (var uri in documents.keys) {
        uris.add (uri);
    }
    return uris;
}

public void open_document_by_uri (string uri) {
    var file = GLib.File.new_for_uri (uri);
    if (file.query_exists (null)) {
        open_document (file, null);
    }
}

public bool navigate_to (NavigationPoint nav_point) {
    var docs = documents;
    var uri = nav_point.file.get_uri ();
    if (docs.has_key (uri)) {
        var widget = docs.get (uri);
        widget.raise ();
        widget.view_grab_focus ();
        widget.select_and_scroll (nav_point.line, nav_point.column, nav_point.column, fa

```

```

        return true;
    }
    return false;
}
}

```

File: src/Services/ProjectManager.vala

```

using Gtk;
using GLib;
using Gee;

public class Iide.FileEntry : Object {
    public string path { get; construct; }
    public string name { get; construct; }
    public bool is_text_file { get; construct; }
    public string relative_path { get; construct; }
    public string display_name { get; construct; }

    public FileEntry (string path, string name, bool is_text_file, string relative_path) {
        Object (
            path: path,
            name: name,
            is_text_file: is_text_file,
            relative_path: relative_path,
            display_name: "%s → %s".printf (name, relative_path)
        );
    }
}

public class Iide.LanguageConfig : GLib.Object {
    public string language_id { get; construct set; }
    public string[] server_command { get; construct set; }
    public string[] file_patterns { get; construct set; }

    public LanguageConfig (string language_id, string[] server_command, string[] file_patterns) {
        Object (
            language_id: language_id,
            server_command: server_command,
            file_patterns: file_patterns
        );
    }
}

public class Iide.ProjectManager : Object {

```

```

private static ProjectManager? _instance;
private GLib.File? current_project_root;
private string? current_project_name;
private Iide.SettingsService settings;
private Gee.HashMap<string, LanguageConfig> language_configs;

private Gee.List<Iide.FileEntry> file_cache;
private Gee.List<Iide.FileEntry> text_file_cache;
public bool cache_valid = false;
private bool cache_loading = false;
private FileMonitor? directory_monitor;

private const string[] EXCLUDED_DIRS = {
    "node_modules", "target", "build", "__pycache__",
    ".git", ".svn", ".hg", "vendor", ".cargo",
    ".cache", ".local", ".config"
};

public signal void project_opened (GLib.File project_root);
public signal void project_closed ();
public signal void file_cache_updated ();
public signal void file_cache_invalidated ();

public static unowned ProjectManager get_instance () {
    if (_instance == null) {
        _instance = new ProjectManager ();
    }
    return _instance;
}

public ProjectManager () {
    current_project_root = null;
    current_project_name = null;
    settings = Iide.SettingsService.get_instance ();
    language_configs = new Gee.HashMap<string, LanguageConfig> ();
    file_cache = new Gee.ArrayList<Iide.FileEntry> ();
    text_file_cache = new Gee.ArrayList<Iide.FileEntry> ();
    init_default_language_configs ();
}

private void init_default_language_configs () {
    language_configs.set ("c", new LanguageConfig ("c", { "clangd" }, { "*.c", "*.h" }));
    language_configs.set ("cpp", new LanguageConfig ("cpp", { "clangd" }, { "*.cpp", "*.h" }));
    language_configs.set ("python", new LanguageConfig ("python", { "basedpyright-langs" }, { "*.py" }));
    language_configs.set ("rust", new LanguageConfig ("rust", { "rust-analyzer" }, { "*.rs" }));
    language_configs.set ("go", new LanguageConfig ("go", { "gopls" }, { "*.go" }));
}

```

```

        language_configs.set ("typescript", new LanguageConfig ("typescript", { "typescript":
        language_configs.set ("javascript", new LanguageConfig ("javascript", { "typescript":
        language_configs.set ("json", new LanguageConfig ("json", { "vscode-json-languageser
        language_configs.set ("html", new LanguageConfig ("html", { "vscode-html-language-se
        language_configs.set ("css", new LanguageConfig ("css", { "vscode-css-language-serve
    }

    private string? last_loaded_config_path;

    public Gee.Collection<LanguageConfig> get_language_configs () {
        return language_configs.values;
    }

    public LanguageConfig ? get_language_config (string language_id) {
        return language_configs.get (language_id);
    }

    public void load_lsp_config (GLib.File project_root) {
        string? config_path = project_root.get_path ();
        if (config_path == null) {
            config_path = project_root.get_uri ();
        }

        if (config_path != null && config_path == last_loaded_config_path) {
            return;
        }

        init_default_language_configs ();

        var config_dir = project_root.get_child (".iide");
        var lsp_file = config_dir.get_child ("lsp.json");

        if (!lsp_file.query_exists (null)) {
            message ("No .iide/lsp.json found at %s", lsp_file.get_path ());
            last_loaded_config_path = config_path;
            return;
        }

        try {
            var parser = new Json.Parser ();
            parser.load_from_file (lsp_file.get_path ());
            var root = parser.get_root ();
            var obj = (Json.Object) root;

            if (obj.has_member ("languages")) {
                var languages = obj.get_array_member ("languages");

```

```

        int lang_count = (int) languages.get_length ();
        for (int j = 0; j < lang_count; j++) {
            var node = languages.get_element (j);
            var lang_obj = node.get_object ();
            if (lang_obj == null) continue;

            string? lang_id = null;
            string[] ? server_cmd = null;
            string[] ? patterns = null;

            if (lang_obj.has_member ("id")) {
                lang_id = lang_obj.get_string_member ("id");
            }
            if (lang_obj.has_member ("server")) {
                var server_arr = lang_obj.get_array_member ("server");
                int server_count = (int) server_arr.get_length ();
                server_cmd = new string[server_count];
                for (int i = 0; i < server_count; i++) {
                    server_cmd[i] = server_arr.get_string_element (i);
                }
            }
            if (lang_obj.has_member ("patterns")) {
                var patterns_arr = lang_obj.get_array_member ("patterns");
                int patterns_count = (int) patterns_arr.get_length ();
                patterns = new string[patterns_count];
                for (int i = 0; i < patterns_count; i++) {
                    patterns[i] = patterns_arr.get_string_element (i);
                }
            }

            if (lang_id != null) {
                merge_language_config (lang_id, server_cmd, patterns);
            }
        }
    }

    message ("Loaded LSP config from %s", lsp_file.get_path ());
    last_loaded_config_path = config_path;
} catch (Error e) {
    warning ("Failed to load LSP config: %s", e.message);
}
}

private void merge_language_config (string lang_id, string[]? server_cmd, string[]? patterns) {
    var config = language_configs.get (lang_id);
    if (config != null) {

```

```

        if (server_cmd != null && server_cmd.length > 0) {
            config.server_command = server_cmd;
        }
        if (patterns != null && patterns.length > 0) {
            config.file_patterns = patterns;
        }
        message ("Merged LSP config for '%s'", lang_id);
        return;
    }

    if (server_cmd != null && patterns != null) {
        language_configs.set (lang_id, new LanguageConfig (lang_id, server_cmd, patterns));
        message ("Added new LSP config for '%s'", lang_id);
    }
}

public void open_project_file (GLib.File project_root) {
    if (!project_root.query_exists (null)) {
        stderr.printf ("Project directory does not exist: %s\n", project_root.get_path ());
        return;
    }

    if (current_project_root != null) {
        close_project ();
    }

    init_default_language_configs ();

    current_project_root = project_root;
    current_project_name = project_root.get_basename ();

    settings.current_project_path = project_root.get_path ();
    settings.add_recent_project (project_root.get_path ());
    settings.last_open_directory = project_root.get_parent ().get_path ();

    load_lsp_config (project_root);

    rebuild_file_cache_async.begin ();

    setup_directory_monitor (project_root);

    project_opened (project_root);
}

private void setup_directory_monitor (GLib.File dir) {
    if (directory_monitor != null) {

```

```

        directory_monitor.cancel ();
    }

    try {
        directory_monitor = dir.monitor_directory (FileMonitorFlags.NONE, null);
        directory_monitor.changed.connect ((src, dst, event) => {
            if (event == FileMonitorEvent.CHANGES_DONE_HINT) {
                invalidate_file_cache ();
            }
        });
    } catch (Error e) {
        warning ("Failed to setup directory monitor: %s", e.message);
    }
}

public void close_project () {
    if (directory_monitor != null) {
        directory_monitor.cancel ();
        directory_monitor = null;
    }

    if (current_project_root != null) {
        current_project_root = null;
        current_project_name = null;
        cache_valid = false;
        file_cache.clear ();
        text_file_cache.clear ();
        settings.current_project_path = "";
        project_closed ();
    }
}

public async void rebuild_file_cache_async () {
    if (current_project_root == null) {
        return;
    }

    if (cache_loading) {
        return;
    }

    cache_loading = true;
    file_cache.clear ();
    text_file_cache.clear ();

    try {

```

```

        yield scan_directory_async (current_project_root, current_project_root.get_path

        file_cache.sort ((a, b) => a.name.collate (b.name));
        text_file_cache.sort ((a, b) => a.name.collate (b.name));
        cache_valid = true;
        file_cache_updated ();
    } catch (Error e) {
        warning ("Error rebuilding file cache: %s", e.message);
    }

    cache_loading = false;
}

private bool is_text_file (GLib.FileInfo file_info) {
    return ContentType.is_a (file_info.get_content_type (), "text/plain");
}

private async void scan_directory_async (GLib.File dir, string base_path) throws Error {
    var enumerator = yield dir.enumerate_children_async ("standard::name,standard::type",
        FileQueryInfoFlags.NONE,
        Priority.DEFAULT,
        null);

    while (true) {
        var files = yield enumerator.next_files_async (100, Priority.DEFAULT, null);

        if (files == null || files.length () == 0) {
            break;
        }

        foreach (var info in files) {
            var name = info.get_name ();
            if (name.has_prefix (".")) {
                continue;
            }

            var file_type = info.get_file_type ();
            if (file_type == FileType.DIRECTORY) {
                bool excluded = false;
                foreach (var excluded_name in EXCLUDED_DIRS) {
                    if (name == excluded_name) {
                        excluded = true;
                        break;
                    }
                }
            }
            if (!excluded) {

```



```

        var child = dir.get_child (name);
        yield scan_directory_async (child, base_path);
    }
} else if (file_type == FileType.REGULAR) {
    var path = dir.get_child (name).get_path ();
    if (path != null) {
        var relative = path.substring (base_path.length + 1);
        bool is_text = is_text_file (info);
        file_cache.add (new Iide.FileEntry (
                                                    path,
                                                    name,
                                                    is_text,
                                                    relative));

        if (is_text) {
            text_file_cache.add (new Iide.FileEntry (
                                                        path,
                                                        name,
                                                        true,
                                                        relative));
        }
    }
}
}
}

public Gee.List<Iide.FileEntry>? get_file_cache () {
    if (!cache_valid) {
        return null;
    }
    return file_cache;
}

public Gee.List<Iide.FileEntry>? get_text_file_cache () {
    if (!cache_valid) {
        return null;
    }
    return text_file_cache;
}

public async void ensure_file_cache_async () {
    if (cache_valid || cache_loading) {
        return;
    }
    yield rebuild_file_cache_async ();
}

```

```

public void invalidate_file_cache () {
    cache_valid = false;
    file_cache_invalidated ();
}

public string ? get_workspace_root_path () {
    if (current_project_root != null) {
        return current_project_root.get_path ();
    }
    return null;
}

public void open_project_by_path (string path) {
    if (path != null && path != "") {
        var file = GLib.File.new_for_path (path);
        if (file.query_exists (null)) {
            open_project_file (file);
        }
    }
}

public GLib.File? get_current_project_root () {
    return current_project_root;
}

public string ? get_current_project_name () {
    return current_project_name;
}

public bool has_open_project () {
    return current_project_root != null;
}

public async void open_project_dialog (Window parent_window) {
    var dialog = new FileDialog () {
        title = _("Open Project"),
        modal = true
    };

    var last_dir = settings.last_open_directory;
    if (last_dir != null && last_dir != "") {
        dialog.initial_folder = GLib.File.new_for_path (last_dir);
    }

    try {

```

```

        var file = yield dialog.select_folder (parent_window, null);

        if (file != null) {
            open_project_file (file);
        }
    } catch {
        // User dismissed dialog or other error - silently ignore
    }
}
}

```

File: src/Services/SettingsService.vala

```

/*
 * settingservice.vala
 *
 * Copyright 2026 kai
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program. If not, see <https://www.gnu.org/licenses/>.
 *
 * SPDX-License-Identifier: GPL-3.0-or-later
 */

public enum Iide.ColorScheme {
    SYSTEM,
    LIGHT,
    DARK;

    public static ColorScheme from_string (string value) {
        switch (value) {
            case "light":
                return LIGHT;
            case "dark":
                return DARK;
        }
    }
}

```

```

        default:
            return SYSTEM;
    }
}

public string to_string () {
    switch (this) {
        case LIGHT:
            return "light";
        case DARK:
            return "dark";
        default:
            return "system";
    }
}

public Adw.ColorScheme to_adw_color_scheme () {
    switch (this) {
        case LIGHT:
            return Adw.ColorScheme.FORCE_LIGHT;
        case DARK:
            return Adw.ColorScheme.FORCE_DARK;
        default:
            return Adw.ColorScheme.DEFAULT;
    }
}

}

public class Ide.FontSizeHelper : Object {
    public const int DEFAULT_ZOOM_LEVEL = 6;
    public const int MIN_ZOOM_LEVEL = 1;
    public const int MAX_ZOOM_LEVEL = 15;
    private const int[] FONT_SIZES = { 4, 6, 8, 10, 12, 14, 16, 18, 20, 24, 28, 32, 36, 40,

    public static int get_size_for_zoom_level (int zoom_level) {
        if (zoom_level >= MIN_ZOOM_LEVEL && zoom_level <= MAX_ZOOM_LEVEL) {
            return FONT_SIZES[zoom_level - 1];
        }
        return FONT_SIZES[DEFAULT_ZOOM_LEVEL - 1];
    }

    public static int[] get_available_sizes () {
        return FONT_SIZES;
    }
}
}

```

```

public class Iide.SettingsService : Object {
    private static SettingsService? _instance;
    private Settings _settings;

    public Settings settings { get { return _settings; } }

    public signal void editor_setting_changed (string key);

    public static SettingsService get_instance () {
        if (_instance == null) {
            _instance = new SettingsService ();
        }
        return _instance;
    }

    private SettingsService () {
        _settings = new Settings ("org.github.kai66673.iide");
    }

    public ColorScheme color_scheme {
        get {
            return ColorScheme.from_string (settings.get_string ("color-scheme"));
        }
        set {
            settings.set_string ("color-scheme", value.to_string ());
            editor_setting_changed ("color-scheme");
        }
    }

    public int editor_font_size {
        get {
            return (int) settings.get_double ("editor-font-size");
        }
        set {
            settings.set_double ("editor-font-size", (double) value);
            editor_setting_changed ("editor-font-size");
        }
    }

    public bool show_minimap {
        get {
            return settings.get_boolean ("show-minimap");
        }
        set {
            settings.set_boolean ("show-minimap", value);
            editor_setting_changed ("show-minimap");
        }
    }
}

```

```

    }
}

public bool show_line_numbers {
    get {
        return settings.get_boolean ("show-line-numbers");
    }
    set {
        settings.set_boolean ("show-line-numbers", value);
        editor_setting_changed ("show-line-numbers");
    }
}

public bool show_diagnostics_marks {
    get {
        return settings.get_boolean ("show-diagnostics-marks");
    }
    set {
        settings.set_boolean ("show-diagnostics-marks", value);
        editor_setting_changed ("show-diagnostics-marks");
    }
}

public bool show_folding_gutter {
    get {
        return settings.get_boolean ("show-folding-gutter");
    }
    set {
        settings.set_boolean ("show-folding-gutter", value);
        editor_setting_changed ("show-folding-gutter");
    }
}

public bool highlight_current_line {
    get {
        return settings.get_boolean ("highlight-current-line");
    }
    set {
        settings.set_boolean ("highlight-current-line", value);
        editor_setting_changed ("highlight-current-line");
    }
}

public bool auto_indent {
    get {

```

```

        return settings.get_boolean ("auto-indent");
    }
    set {
        settings.set_boolean ("auto-indent", value);
        editor_setting_changed ("auto-indent");
    }
}

public int panel_start_width {
    get {
        return (int) settings.get_double ("panel-start-width");
    }
    set {
        settings.set_double ("panel-start-width", (double) value);
    }
}

public int panel_end_width {
    get {
        return (int) settings.get_double ("panel-end-width");
    }
    set {
        settings.set_double ("panel-end-width", (double) value);
    }
}

public int panel_bottom_height {
    get {
        return (int) settings.get_double ("panel-bottom-height");
    }
    set {
        settings.set_double ("panel-bottom-height", (double) value);
    }
}

public int panel_bottom_width {
    get {
        return (int) settings.get_double ("panel-bottom-width");
    }
    set {
        settings.set_double ("panel-bottom-width", (double) value);
    }
}

public bool reveal_start_panel {
    get {

```

```

        return settings.get_boolean ("reveal-start-panel");
    }
    set {
        settings.set_boolean ("reveal-start-panel", value);
    }
}

public bool reveal_end_panel {
    get {
        return settings.get_boolean ("reveal-end-panel");
    }
    set {
        settings.set_boolean ("reveal-end-panel", value);
    }
}

public bool reveal_bottom_panel {
    get {
        return settings.get_boolean ("reveal-bottom-panel");
    }
    set {
        settings.set_boolean ("reveal-bottom-panel", value);
    }
}

public string[] recent_projects {
    owned get {
        return settings.get_strv ("recent-projects");
    }
}

public int max_recent_projects {
    get {
        return (int) settings.get_double ("max-recent-projects");
    }
    set {
        settings.set_double ("max-recent-projects", (double) value);
    }
}

public string last_open_directory {
    owned get {
        return settings.get_string ("last-open-directory");
    }
    set {
        settings.set_string ("last-open-directory", value);
    }
}

```



```

    }
}

public string current_project_path {
    owned get {
        return settings.get_string ("current-project-path");
    }
    set {
        settings.set_string ("current-project-path", value);
    }
}

public Gee.ArrayList<string> open_documents {
    owned get {
        var arr = settings.get_strv ("open-documents");
        var list = new Gee.ArrayList<string> ();
        foreach (var s in arr) {
            list.add (s);
        }
        return (owned) list;
    }
    set {
        var arr = new string[value.size];
        int i = 0;
        foreach (var s in value) {
            arr[i++] = s;
        }
        settings.set_strv ("open-documents", arr);
    }
}

public string panel_layout {
    owned get {
        return settings.get_string ("panel-layout");
    }
    set {
        settings.set_string ("panel-layout", value);
    }
}

public string grid_layout {
    owned get {
        return settings.get_string ("grid-layout");
    }
    set {
        settings.set_string ("grid-layout", value);
    }
}

```

```

        Settings.sync ();
    }
}

public int window_width {
    get {
        return (int) settings.get_double ("window-width");
    }
    set {
        settings.set_double ("window-width", (double) value);
    }
}

public int window_height {
    get {
        return (int) settings.get_double ("window-height");
    }
    set {
        settings.set_double ("window-height", (double) value);
    }
}

public bool window_maximized {
    get {
        return settings.get_boolean ("window-maximized");
    }
    set {
        settings.set_boolean ("window-maximized", value);
    }
}

public void add_recent_project (string path) {
    var projects = new List<string> ();
    projects.prepend (path);

    foreach (var p in recent_projects) {
        if (p != path) {
            bool found = false;
            foreach (var existing in projects) {
                if (existing == path) {
                    found = true;
                    break;
                }
            }
            if (!found) {
                projects.append (p);
            }
        }
    }
}

```

```

        }
    }

    var arr = new string[projects.length ()];
    int i = 0;
    foreach (var p in projects) {
        arr[i++] = p;
    }
    settings.set_strv ("recent-projects", arr);
}

public void remove_recent_project (string path) {
    var projects = new List<string> ();
    foreach (var p in recent_projects) {
        if (p != path) {
            projects.append (p);
        }
    }

    var arr = new string[projects.length ()];
    int i = 0;
    foreach (var p in projects) {
        arr[i++] = p;
    }
    settings.set_strv ("recent-projects", arr);
}

public void clear_recent_projects () {
    settings.set_strv ("recent-projects", {});
}
}

```

File: src/Services/Utils.vala

```

using GLib;

[CCode(cheader_filename = "unistd.h")]
extern int getpid();

namespace Iide {
    public string mime_type_for_file(File file) {
        string mime_type = "text-x";
        try {
            var info = file.query_info("standard:*", FileQueryInfoFlags.NONE, null);

```

```

        mime_type = ContentType.get_mime_type(info.get_attribute_as_string(FileAttribute
    } catch (Error e) {
    }
    return mime_type;
}

public int application_pid() {
    return getpid();
}

public void copy_resource_to_file(string resource_path, string local_path) {
    // 1.      File      (      resource:/// )
    var resource_file = File.new_for_uri(resource_path);

    // 2.      File
    var local_file = File.new_for_path(local_path);

    try {
        // 3.      .      OVERWRITE      ,      .
        resource_file.copy(local_file, FileCopyFlags.OVERWRITE, null, null);
        print("      : %s\n", local_path);
    } catch (Error e) {
        stderr.printf("      : %s\n", e.message);
    }
}

public string ? escape_pango(string? text) {
    if (text == null)
        return null;
    return text
        .replace("&", "&")
        .replace("<", "<")
        .replace(">", ">");
}

private uint32 get_byte_offset_safe (Gtk.TextIter iter) {
    int target_line = iter.get_line ();
    int current_line_bytes = iter.get_line_index (); //      (0(1))

    //
    if (target_line == 0) {
        return (uint32) current_line_bytes;
    }

    uint32 total_bytes = 0;
    Gtk.TextIter line_runner;

```

```

        iter.get_buffer ().get_start_iter (out line_runner);

        //
        //      get_bytes_in_line()  TextIter      B-      GTK,
        //      (      \n)      !
        for (int i = 0; i < target_line; i++) {
            total_bytes += (uint32) line_runner.get_bytes_in_line ();
            if (!line_runner.forward_line ()) break;
        }

        //
        total_bytes += (uint32) current_line_bytes;

        return total_bytes;
    }
}

```

File: src/Widgets/Panels/DiagnosticsPanel.vala

```

using Gee;
using Adw;

public class Ide.DiagnosticsRow : Adw.ActionRow {
    private LspDiagnostic diagnostic;
    private string uri;

    public DiagnosticsRow (string uri, LspDiagnostic diagnostic) {
        Object (
            title: diagnostic.message,
            subtitle: @"Line $(diagnostic.start_line + 1)",
            activatable: true
        );
        this.diagnostic = diagnostic;
        this.uri = uri;

        var icon_provider = SymbIconProvider.get_instance ();
        switch (diagnostic.severity) {
            case 1:
                add_prefix (icon_provider.image (IconID.COD_ERROR));
                break;
            case 2: case 3: case 4:
                add_prefix (icon_provider.image (IconID.COD_WARNING));
                break;
        }
    }
}

```

```

}

public class Iide.FileRow : Adw.ExpanderRow {
    //      Box      ListBox
    private Gtk.ListBox content_list;
    private string uri;

    public FileRow (string uri) {
        Object (title: Path.get_basename (uri.replace ("file://", "")));
        this.uri = uri;

        //      ListBox      activated
        content_list = new Gtk.ListBox ();
        // content_list.add_css_class ("boxed-list"); //
        content_list.set_selection_mode (Gtk.SelectionMode.NONE);

        //      ListBox
        add_row (content_list);
    }

    public void update_rows (Gee.List<LspDiagnostic>? diags) {
        //      ListBox
        Gtk.Widget? child;
        while ((child = content_list.get_first_child ()) != null) {
            content_list.remove (child);
        }

        subtitle = @"$(diags.size) issues";
        if (diags != null) {
            foreach (var diag in diags) {
                var row = new DiagnosticsRow (uri, diag);

                //      :
                row.activated.connect (() => {
                    //
                    var file = GLib.File.new_for_uri (uri);
                    DocumentManager.get_instance ().open_document_with_selection (file, diag);
                });

                content_list.append (row);
            }
        }
    }
}

public class Iide.DiagnosticsPanel : BasePanel {

```

```

private Gtk.ScrolledWindow scrolled;
private Gtk.ListBox main_list;
private Adw.StatusPage empty_page;

//                               : [URI] -> ExpanderRow
private HashMap<string, FileRow> file_rows = new HashMap<string, FileRow> ();
//                               : [ClientID] -> Label
private HashMap<int, Gtk.Widget> server_headers = new HashMap<int, Gtk.Widget> ();

public DiagnosticsPanel () {
    base ("Diagnostics", SymbIIconProvider.get_instance ().icon_name (IconID.APP_ISSUES));
    can_maximize = true;

    main_list = new Gtk.ListBox ();
    main_list.set_selection_mode (Gtk.SelectionMode.NONE);

    scrolled = new Gtk.ScrolledWindow () {
        vexpand = true,
        child = main_list
    };

    empty_page = new Adw.StatusPage () {
        title = "No issues found",
        icon_name = "emblem-ok-symbolic",
        vexpand = true
    };

    this.set_child (empty_page);

    var service = DiagnosticsService.get_instance ();
    //
    service.diagnostics_updated.connect (on_file_diagnostics_updated);
    service.total_count_changed.connect (on_total_changed);
}

public override Panel.Position initial_pos () {
    return new Panel.Position () { area = Panel.Area.BOTTOM };
}

public override string panel_id () {
    return "DiagnosticsPanel";
}

private void on_total_changed (int e, int w) {
    //           "           "
    if (e == 0 && w == 0) {

```

```

        if (this.get_child () != empty_page)this.set_child (empty_page);
    } else {
        if (this.get_child () != scrolled)this.set_child (scrolled);
    }
}

//
private void on_file_diagnostics_updated (int client_id, string uri) {
    var service = DiagnosticsService.get_instance ();
    var diags = service.get_diagnostics_for_file (client_id, uri);

    // 1.
    if (diags == null || diags.size == 0) {
        if (file_rows.has_key (uri)) {
            var row = file_rows.get (uri);
            main_list.remove (row);
            file_rows.unset (uri);
        }
        return;
    }

    // 2.
    ensure_server_header (client_id);

    // 3.
    FileRow file_row;
    if (file_rows.has_key (uri)) {
        file_row = file_rows.get (uri);
    } else {
        file_row = new FileRow (uri);
        main_list.append (file_row);
        file_rows.set (uri, file_row);
    }

    file_row.update_rows (diags);
}

private void ensure_server_header (int client_id) {
    if (!server_headers.has_key (client_id)) {
        var service = DiagnosticsService.get_instance ();
        var header = new Gtk.Label (service.get_server_name (client_id)) {
            xalign = 0, margin_start = 12, margin_top = 12, margin_bottom = 6
        };
        header.add_css_class ("heading");
        main_list.append (header);
        server_headers.set (client_id, header);
    }
}

```



```

    }
}
}

```

File: src/main.vala

```

/* main.vala
 *
 * Copyright 2026 kai
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program. If not, see <https://www.gnu.org/licenses/>.
 *
 * SPDX-License-Identifier: GPL-3.0-or-later
 */

int main (string[] args) {
    GLib.Environment.set_variable ("GSK_RENDERER", "ngl", true);

    Intl.bindtextdomain (Config.GETTEXT_PACKAGE, Config.LOCALEDIR);
    Intl.bind_textdomain_codeset (Config.GETTEXT_PACKAGE, "UTF-8");
    Intl.textdomain (Config.GETTEXT_PACKAGE);

    // GtkSource.init ();
    Adw.init ();

    var app = new Iide.Application ();
    return app.run (args);
}

```

File: src/style.css

```

textview {
    ----
    .log-view text {

```

```

-----
.text-view.zoom-1 {
-----
.lsp_error_line {
-----
.lsp_warning_line {
-----
.lsp_info_line {
-----
.text-view.zoom-2 {
-----
.text-view.zoom-3 {
-----
.text-view.zoom-4 {
-----
.text-view.zoom-5 {
-----
.text-view.zoom-6 {
-----
.text-view.zoom-7 {
-----
.text-view.zoom-8 {
-----
.text-view.zoom-9 {
-----
.text-view.zoom-10 {
-----
.text-view.zoom-11 {
-----
.text-view.zoom-12 {
-----
.text-view.zoom-13 {
-----
.text-view.zoom-14 {
-----
.text-view.zoom-15 {
-----
.textview-map {
-----
.breadcrumbs-btn {
-----
.small-menu-button {
-----
.small-menu-button>button {
-----
/*      :      contents      */

```

```

.small-menu-button>button>contents {
----
/*          - */
----
/*          , */
.small-menu-button image {
----
/* 3.          (          StyleManager) */
image.def-icon {
----
/* - */
----
/* 2.          .dark.
   Libadwaita          .dark  AduApplicationWindow,
   StyleManager          . */
.dark image.def-icon {
----
/* - */
----
/* */
/* - */
.nerd-icon {
----
/* */
----
/*          IDE */
.icon-orange {
----
/* , */
.icon-purple {
----
/* , */
.icon-blue {
----
/* , */
.icon-gray {
----
/* , */
----
.editor-status-bar .diagnostic-box {
----
/* */
----
/* */
----
/* , */

```

```

----
.icon-cyan {
----
.icon-green {
----
.icon-yellow {
----
.icon-tan {
----
/*          Completion (      ) */
.completion-row .nerd-icon {
----
.editor-status-bar .diagnostic-box.hover {
----
/*          */
----
/*          */
----
/*          ,          */
.editor-status-bar .diagnostic-box.hover image {
----
button.error label {
----
color: @error_fg_color;
----
button.error image {

```

File: symbols/export_glyph.py

```

#!/usr/bin/env -S uv run --script
# /// script
# dependencies = [
#     "pygobject",
#     "pycairo",
# ]
# ///
----
def export_glyph(symbol, font_desc, output_dir, icon_name)
----
filename = os.path.join(output_dir, f"{icon_name}-symbolic.svg")
surface = cairo.SVGSurface(filename, 256, 256)
cr = cairo.Context(surface)
layout = PangoCairo.create_layout(cr)
----
scale = (256 * 0.84) / max(w, h)

```

```

----
content = f.read()
----
generated_warning = [
----
def generate_provider_source_file(icon_names)
----
src_path = (
----
# Ide.IconID
----
enums = [f"    {iid[0]} = {i}" for i, iid in enumerate(icon_names)]
----
# Ide.SymbIconProvider
----
def main()
----
#           (Meson                               )
----
#           (                               Arch/TypeLib)
font_dir = os.path.dirname(os.path.abspath(ttf_path))
----
config = json.load(f)
----
prefix = config.get("prefix", "def-")
font_family = config.get("font_name", "Sans")
styles_data = config.get("styles", {})
icons_data = config.get("icons", {})
----
#           :           ,           SVG
actions_dir = os.path.join(out_base, "scalable", "actions")
----
icon_names = []
----
styles = {style_name: colors for style_name, colors in styles_data.items()}
icon_indexes = []
----
#           JSON
----
#           JSON
style_name = ""
----
char_raw = data.get("char")
style_name = data.get("style")
----
char_raw = data

```

```

----
full_name = f"{prefix}-{name}"
----
name.replace("-", "_").upper(), # IconID
f"{name}", # (def-)icon-name(-symbolic)
style_name, # css-class-name
----
#      HEX-      (0x...      U+...)
----
char_parsed = chr(
----
char_parsed = str(char_raw)
----
# 1.      SVG
#      (      , 200),      Pango
----
#      CSS
cwd = os.path.dirname(__file__)
css_file_name = "symbols.css"
----
#      XML      GLib
----
#      ,      Vala
----
# Generate SymbIconProvider.vala

```

File: src/Services/LSP/LspService.vala

```

using GLib;
using Gee;

public class Ide.IdeLspService : GLib.Object {
    private static IdeLspService? _instance;
    private Gee.HashMap<string, LspClient> clients;
    private Gee.HashMap<string, string> uri_to_client_key;
    private Gee.HashMap<string, int> document_versions;
    private Gee.HashMap<string, bool> client_starting;
    private Gee.ArrayList<PendingOpen> pending_opens;

    private LoggerService logger = LoggerService.get_instance ();

    public signal void diagnostics_updated (string uri, ArrayList<LspDiagnostic> diagnostics);

    public class PendingOpen {
        public string uri;
    }
}

```

```

        public string language_id;
        public string content;
        public string? workspace_root;
        public SourceView view;

        public PendingOpen (string uri, string language_id, string content, string? workspace_root) {
            this.uri = uri;
            this.language_id = language_id;
            this.content = content;
            this.workspace_root = workspace_root;
            this.view = view;
        }
    }

    // [ClientHash] -> [Token] -> LspTaskInfo
    private Gee.HashMap<int, Gee.HashMap<string, LspTaskInfo?>> progress_map =
        new Gee.HashMap<int, Gee.HashMap<string, LspTaskInfo?>> ();

    public signal void tasks_changed (Gee.List<LspTaskInfo?> active_tasks);

    construct {
        clients = new Gee.HashMap<string, LspClient> ();
        uri_to_client_key = new Gee.HashMap<string, string> ();
        document_versions = new Gee.HashMap<string, int> ();
        client_starting = new Gee.HashMap<string, bool> ();
        pending_opens = new Gee.ArrayList<PendingOpen> ();
    }

    public static unowned IdeLspService get_instance () {
        if (_instance == null) {
            _instance = new IdeLspService ();
        }
        return _instance;
    }

    public string ? get_language_id_for_file (GLib.File file) {
        string filename = file.get_basename () ?? "";

        switch (filename) {
            case "CMakeLists.txt" :
                return "cmake";
            case ".gitignore":
                return "git-config";
            case "meson.build":
                return "meson";
            case "PKGBUILD":

```

```

        return "bash";
default:
    break;
}

string path = file.get_path () ?? "";
int dot_pos = path.last_index_of (".");
if (dot_pos >= 0 && dot_pos < path.length - 1) {
    string ext = path[dot_pos + 1 : path.length].down ();
    switch (ext) {
        case "py":
            return "python";
        case "c":
        case "h":
            return "c";
        case "cpp":
        case "cc":
        case "cxx":
        case "hpp":
        case "hxx":
            return "cpp";
        case "vala":
        case "vapi":
            return "vala";
        case "rs":
            return "rust";
        case "go":
            return "go";
        case "js":
        case "ts":
            return "javascript";
        case "json":
            return "json";
        case "xml":
            return "xml";
        case "html":
        case "htm":
            return "html";
        case "css":
            return "css";
        case "md":
        case "markdown":
            return "markdown";
        case "sh":
        case "bash":
        case "zsh":
    }
}

```



```

        return "bash";
    case "yaml":
    case "yml":
        return "yaml";
    }
}

return null;
}

public LspClient ? get_client_by_hash (int client_id) {
    foreach (var client in clients.values) {
        if (client.get_hash () == client_id) {
            return client;
        }
    }
    return null;
}

public LspClient[] get_clients () {
    return clients.values.to_array ();
}

public void register_client (LspClient client) {
    int id = client.get_hash ();

    client.progress_updated.connect ((token, msg, perc, active) => {
        if (!progress_map.has_key (id))
            progress_map.set (id, new Gee.HashMap<string, LspTaskInfo?> ());

        var client_tasks = progress_map.get (id);

        if (active) {
            var info = LspTaskInfo () {
                server_name = client.name (),
                message = msg,
                percentage = perc
            };
            client_tasks.set (token, info);
        } else {
            client_tasks.unset (token);
        }

        emit_tasks_changed ();
    });
}

```

```

private void emit_tasks_changed () {
    var all_tasks = new Gee.ArrayList<LspTaskInfo?> ();
    foreach (var client_map in progress_map.values) {
        foreach (var task in client_map.values) {
            all_tasks.add (task);
        }
    }
    tasks_changed (all_tasks);
}

public async void open_document (string uri, string language_id, string content, string server_key) {
    var server_key = LspRegistry.get_lsp_id (language_id);
    if (server_key == null) {
        debug ("IdeLspService: No LSP server configured for language: %s", language_id);
        Idle.add (() => {
            view.bind_lsp_client (null);
            return Source.REMOVE;
        });
        return;
    }

    debug ("IdeLspService: Opening document %s (lang=%s)", uri, language_id);

    if (clients.has_key (server_key)) {
        var client = clients.get (server_key);
        uri_to_client_key.set (uri, server_key);
        document_versions.set (uri, 1);
        try {
            yield client.text_document_did_open (uri, language_id, 1, content);
        } catch (Error e) {
            logger.error ("LSP", "Failed to open document %s: %s".printf (uri, e.message));
        }

        Idle.add (() => {
            view.bind_lsp_client (client);
            return Source.REMOVE;
        });

        return;
    }

    if (client_starting.get (server_key) == true) {
        pending_opens.add (new PendingOpen (uri, language_id, content, workspace_root, v));
        return;
    }
}

```

```

    client_starting.set (server_key, true);

    var config = LspRegistry.get_config (server_key);
    if (config == null) {
        client_starting.set (server_key, false);
        return;
    }

    var client = new LspClient (config);
    client.diagnostics_received.connect ((uri, diagnostics) => {
        diagnostics_updated (uri, diagnostics);
    });

    bool started = yield client.start_server_async (workspace_root);

    logger.info ("LSP", "Started server for %s: %b (%s)".printf (server_key, started, workspace_root));

    if (started) {
        clients.set (server_key, client);
        uri_to_client_key.set (uri, server_key);
        document_versions.set (uri, 1);
        try {
            yield client.text_document_did_open (uri, language_id, 1, content);
        } catch (Error e) {
            logger.error ("LSP", "Failed to open document %s: %s".printf (uri, e.message));
        }

        Idle.add (() => {
            view.bind_lsp_client (client);
            return Source.REMOVE;
        });

        yield process_pending_opens ();
    } else {
        // TODO: restart logic...
        Idle.add (() => {
            view.bind_lsp_client (null);
            return Source.REMOVE;
        });

        warning ("IdeLspService: Failed to start LSP server for %s", server_key);
    }

    client_starting.set (server_key, false);
}

```

```

private async void process_pending_opens () {
    var opens_to_process = new ArrayList<PendingOpen> ();
    foreach (var open in pending_opens) {
        opens_to_process.add (open);
    }
    pending_opens.clear ();

    foreach (var open in opens_to_process) {
        yield open_document (open.uri, open.language_id, open.content, open.workspace_ro
    }
}

public async void change_document (string uri, string content, int? change_start = null,
    var server_key = uri_to_client_key.get (uri);

    logger.debug ("LSP", "Changed doc: " + uri + " / server_key: " + server_key);
    if (server_key == null || !clients.has_key (server_key)) {
        return;
    }

    var client = clients.get (server_key);
    var version = document_versions.get (uri);
    document_versions.set (uri, version + 1);

    try {
        yield client.text_document_did_change (uri, version + 1, content);
    } catch (Error e) {
        logger.error ("LSP", "Failed to change document (FULL sync) %s: %s".printf (uri,
    }
}

public async void send_did_change (string uri, int version, Gee.ArrayList<PendingChange>
    var server_key = uri_to_client_key.get (uri);
    if (server_key == null || !clients.has_key (server_key)) {
        return;
    }

    var client = clients.get (server_key);
    try {
        yield client.send_did_change (uri, version, changes);
    } catch (Error e) {
        logger.error ("LSP", "Failed to change document (INCREMENTAL sync) %s: %s".printf
    }
}

```

```

public async void close_document (string uri) {
    var server_key = uri_to_client_key.get (uri);
    if (server_key == null || !clients.has_key (server_key)) {
        return;
    }

    var client = clients.get (server_key);
    try {
        yield client.text_document_did_close (uri);
    } catch (Error e) {
        logger.error ("LSP", "Failed to close document %s: %s".printf (uri, e.message));
    }

    uri_to_client_key.unset (uri);
    document_versions.unset (uri);
}

public LspClient ? get_client_for_uri (string uri) {
    var server_key = uri_to_client_key.get (uri);
    if (server_key == null) {
        return null;
    }
    return clients.get (server_key);
}

public async LspCompletionResult ? request_completion (string uri,
                                                        int line,
                                                        int character,
                                                        string? trigger_character = null,
                                                        CompletionTriggerKind trigger_kind) {

    var client = get_client_for_uri (uri);
    if (client == null) {
        return null;
    }
    try {
        return yield client.request_completion (uri, line, character, trigger_character, trigger_kind);
    } catch (Error e) {
        logger.error ("LSP", "Failed to request completion for %s: %s".printf (uri, e.message));
        return null;
    }
}

public async string ? request_hover (string uri, int line, int character) {
    var client = get_client_for_uri (uri);
    if (client == null) {
        return null;
    }
}

```

```

    }
    try {
        return yield client.request_hover (uri, line, character);
    } catch (Error e) {
        logger.error ("LSP", "Failed to request hover for %s: %s".printf (uri, e.message));
        return null;
    }
}

public async Gee.ArrayList<LspLocation>? goto_definition (string uri, int line, int character) {
    var client = get_client_for_uri (uri);
    if (client == null)
        return null;
    try {
        return yield client.request_definition (uri, line, character);
    } catch (Error e) {
        logger.error ("LSP", "Failed to request definition for %s: %s".printf (uri, e.message));
        return null;
    }
}

public async Gee.List<DocumentLspSymbol>? document_symbols(string uri) {
    var client = get_client_for_uri (uri);
    if (client == null)
        return null;
    try {
        return yield client.document_symbols (uri);
    } catch (Error e) {
        logger.error ("LSP", "Failed to request document symbols for %s: %s".printf (uri, e.message));
        return null;
    }
}
}

```

File: src/Widgets/Find/Engines/SymbolsSearchEngine.vala

```

public class Ide.SymbolsSearchEngine : SearchEngine, Object {

    public string search_entry_placeholder () {
        return _("Enter symbol name (min 3 chars)...");
    }

    public string search_progress_message () {
        return _("Searching symbols...");
    }
}

```

```

public string search_kind () {
    return "symbols";
}

public string search_title () {
    return _("Symbols");
}

public string search_icon_name () {
    return "emblem-system-symbolic";
}

public async Gee.List<SearchResult> perform_search (string query, GLib.Cancellable cancellable) {
    var clients = IdeLspService.get_instance ().get_clients ();
    var results = new Gee.ArrayList<WorkspaceLspSymbol> ();
    foreach (var client in clients) {
        results.add_all (yield client.workspace_symbols (query, cancellable));
    }
    return to_search_results (results);
}

private Gee.List<SearchResult> to_search_results (Gee.List<WorkspaceLspSymbol> results) {
    var items = new Gee.ArrayList<SearchResult> ();
    foreach (var sym in results) {
        var file_path = sym.uri.replace ("file://", "");
        items.add (new SearchResult (
            file_path,
            file_path,
            sym.start_line,
            sym.name,
            null,
            ImageFactory.create_for_symbol (sym.kind)));
    }
    return items;
}
}

```

File: src/Widgets/Find/SearchResult.vala

```

public class Ide.MatchRange : Object {
    public int start { get; construct; }
    public int end { get; construct; }

    public MatchRange (int start, int end) {

```

```

        Object (start: start, end: end);
    }
}

public class Iide.SearchResult : Object {
    public string file_path { get; construct; }
    public string relative_path { get; construct; }
    public int line_number { get; construct; }
    public string line_content { get; construct; }
    public Gee.List<MatchRange>? matches { get; construct; }
    public Gtk.Widget? icon_widget { get; construct; }
    public int score { get; construct; }

    public SearchResult (string file_path,
        string relative_path,
        int line_number,
        string line_content,
        Gee.List<MatchRange>? matches = null,
        Gtk.Widget? icon_widget = null,
        int score = 0) {
        Object (
            file_path : file_path,
            relative_path : relative_path,
            line_number : line_number,
            line_content : line_content,
            matches: matches,
            icon_widget: icon_widget,
            score: score
        );
    }
}

```

File: src/Widgets/TextView/BreadcrumbFileNavigator.vala

```

public class Iide.BreadcrumbFileNavigator : Gtk.Box {
    private GLib.File root_file;
    private GLib.File current_folder;

    public Gtk.SearchEntry search_entry;
    private Gtk.Button back_button;
    private Gtk.ListBox list_box;
    private Gtk.ScrolledWindow scrolled;
    private Gtk.Stack stack;
}

```



```

private void open_file (GLib.File file) {
    DocumentManager.get_instance ().open_document (file, null);
}

public signal void close_requested ();

public BreadcrumbFileNavigator (GLib.File file) {
    Object (orientation: Gtk.Orientation.VERTICAL, spacing: 6);
    this.root_file = file;
    this.current_folder = file;
    this.set_size_request (300, -1);
    this.margin_top = this.margin_bottom = this.margin_start = this.margin_end = 6;

    // --- Header ---
    var header = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 4);

    back_button = new Gtk.Button.from_icon_name ("go-previous-symbolic");
    back_button.add_css_class ("flat");
    back_button.clicked.connect (() => {
        var parent = current_folder.get_parent ();
        if (parent != null)
            load_directory.begin (parent, (obj, res) => {
                load_directory.end (res);
                this.current_folder = parent;
                this.search_entry.text = "";
                this.refresh_state ();
            });
    });

    search_entry = new Gtk.SearchEntry ();
    search_entry.hexpand = true;
    search_entry.focusable = true; //

    header.append (back_button);
    header.append (search_entry);
    this.append (header);

    // --- Stack ---
    stack = new Gtk.Stack ();
    stack.transition_type = Gtk.StackTransitionType.CROSSFADE;

    var spinner = new Gtk.Spinner ();
    spinner.start ();
    stack.add_named (spinner, "loading");

    list_box = new Gtk.ListBox ();

```

```

list_box.add_css_class ("navigation-sidebar");
list_box.row_activated.connect ((row) => { open_or_select_row (row); });

list_box.set_filter_func ((row) => {
    var text = search_entry.get_text ().down ();
    if (text == "")return true;
    var name = row.get_data<string> ("file-name").down ();
    return name.contains (text);
});

scrolled = new Gtk.ScrolledWindow ();
scrolled.propagate_natural_height = true;
scrolled.set_min_content_height (0);
scrolled.set_max_content_height (400);
scrolled.set_child (list_box);
stack.add_named (scrolled, "list");

this.append (stack);

search_entry.search_changed.connect (() => {
    list_box.invalidate_filter ();
    select_first_visible_row ();
});

//
load_directory.begin (this.current_folder, (obj, res) => {
    load_directory.end (res);
    this.refresh_state ();
});

var key_controller = new Gtk.EventControllerKey ();
key_controller.key_pressed.connect (on_key_pressed);
search_entry.add_controller (key_controller);
list_box.set_selection_mode (Gtk.SelectionMode.SINGLE);

search_entry.activate.connect (() => {
    open_or_select_row (list_box.get_selected_row ());
});
}

private void select_first_visible_row () {
    var row = list_box.get_first_child ();
    while (row != null) {
        var list_row = row as Gtk.ListBoxRow;
        if (list_row.get_child_visible ()) {
            list_box.select_row (list_row);
        }
    }
}

```

```

        break;
    }
    row = row.get_next_sibling ();
}

private void refresh_state () {
    this.search_entry.grab_focus ();
    select_first_visible_row ();
}

private void open_or_select_row (Gtk.ListBoxRow? row) {
    if (row == null)
        return;

    var name = row.get_data<string> ("file-name");
    var type = row.get_data<GLib.FileType> ("file-type");
    var selected = current_folder.get_child (name);

    if (type == GLib.FileType.DIRECTORY) {
        //
        load_directory.begin (selected, (obj, res) => {
            load_directory.end (res);
            this.current_folder = selected;
            this.search_entry.text = "";
            this.refresh_state ();
        });
    } else {
        open_file (selected);
    }
}

private bool on_key_pressed (Gtk.EventControllerKey controller, uint keyval, uint keycode) {
    if (keyval == Gdk.Key.Escape) {
        close_requested ();
        return true;
    } else if (keyval == Gdk.Key.Up || keyval == Gdk.Key.KP_Up) {
        move_selection_up ();
        return true;
    } else if (keyval == Gdk.Key.Down || keyval == Gdk.Key.KP_Down) {
        move_selection_down ();
        return true;
    }
    return false;
}

```

```

private void move_selection_down () {
    var selected_row = list_box.get_selected_row ();
    if (selected_row == null) {
        select_first_visible_row ();
        return;
    }

    var next_row_widget = selected_row.get_next_sibling ();
    while (next_row_widget != null) {
        var list_row = next_row_widget as Gtk.ListBoxRow;
        if (list_row.get_child_visible ()) {
            list_box.select_row (list_row);
            return;
        }
        next_row_widget = next_row_widget.get_next_sibling ();
    }

    if (!selected_row.get_child_visible ())
        select_first_visible_row ();
}

private void move_selection_up () {
    var selected_row = list_box.get_selected_row ();
    if (selected_row == null) {
        select_first_visible_row ();
        return;
    }

    var prev_row_widget = selected_row.get_prev_sibling ();
    while (prev_row_widget != null) {
        var list_row = prev_row_widget as Gtk.ListBoxRow;
        if (list_row.get_child_visible ()) {
            list_box.select_row (list_row);
            return;
        }
        prev_row_widget = prev_row_widget.get_prev_sibling ();
    }

    if (!selected_row.get_child_visible ())
        select_first_visible_row ();
}

private async void load_directory (GLib.File folder) {
    stack.visible_child_name = "loading";
    list_box.remove_all ();
    back_button.sensitive = (folder.get_parent () != null);
}

```

```

search_entry.placeholder_text = folder.get_basename ();

try {
    var enumerator = yield folder.enumerate_children_async ("standard::name,standard::icon-name",
        GLib.FileQueryInfoFlags.NONE, GLib.Priority.DEFAULT, null);

    while (true) {
        var files = yield enumerator.next_files_async (50, GLib.Priority.DEFAULT, null);

        if (files == null)break;
        foreach (var info in files) {
            list_box.append (create_row (info));
        }
    }
    stack.visible_child_name = "list";
} catch (Error e) {
    stack.add_named (new Gtk.Label (e.message), "error");
    stack.visible_child_name = "error";
}

}

private Gtk.ListBoxRow create_row (GLib.FileInfo info) {
    var row = new Gtk.ListBoxRow ();
    var box = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 8);
    box.margin_start = box.margin_end = 8;
    box.margin_top = box.margin_bottom = 4;

    bool is_directory = info.get_file_type () == FileType.DIRECTORY;

    var icon = is_directory ? ImageFactory.folder_image () : ImageFactory.create_for_file ();
    var label = new Gtk.Label (info.get_name ());
    label.ellipsize = Pango.EllipsizeMode.END;

    box.append (icon);
    box.append (label);

    // , VSCode
    if (is_directory) {
        var arrow = new Gtk.Image.from_icon_name ("go-next-symbolic");
        arrow.halign = Gtk.Align.END;
        arrow.expand = true;
        arrow.opacity = 0.5;
        box.append (arrow);
    }

    row.set_child (box);
}

```

```

        row.set_data ("file-name", info.get_name ());
        row.set_data ("file-type", info.get_file_type ());
        return row;
    }
}

```

File: src/Widgets/ToolViews/ProjectView.vala

```

using Gtk;

public class Ide.FileTreeView : Box {
    private ColumnView column_view;
    private SingleSelection selection;
    private GLib.File? _root_directory = null;
    private CustomSorter sorter;

    public signal void file_activated (FileItem item);

    //          null
    public GLib.File? root_directory {
        get { return _root_directory; }
        set {
            _root_directory = value;

            if (_root_directory == null) {
                //          null,          View
                column_view.model = null;
                selection = null;
                return;
            }

            //
            var root_store = create_file_model (_root_directory);
            var sorted_root = new SortListModel (root_store, sorter);

            //          ,
            var tree_model = new TreeListModel (sorted_root, false, false, (item) => {
                var file_item = item as FileItem;
                if (file_item != null && file_item.is_directory) {
                    var child_store = create_file_model (file_item.file);
                    return new SortListModel (child_store, sorter);
                }
                return null;
            });
        }
    }
}

```

```

        selection = new SingleSelection (tree_model);
        column_view.model = selection;
    }
}

public FileTreeView (GLib.File? root_dir = null) {
    Object (orientation : Orientation.VERTICAL);

    //      View
    column_view = new ColumnView (null);
    //
    column_view.get_first_child ().set_visible (false);

    //
    if (root_dir != null) {
        root_directory = root_dir;
    }

    setup_view ();
}

public void set_root_file (GLib.File? root_dir) {
    root_directory = root_dir;
}

private void setup_view () {
    var column = new ColumnViewColumn (null, create_factory ());
    column.expand = true;
    sorter = new CustomSorter ((a, b) => {
        var fi1 = a as FileItem;
        var fi2 = b as FileItem;
        if (fi1 == null || fi2 == null) return 0;

        //      :
        if (fi1.is_directory != fi2.is_directory) {
            return fi1.is_directory ? -1 : 1;
        }
        //
        return strcmp (fi1.name.down (), fi2.name.down ());
    });
    column.sorter = sorter;
    column_view.append_column (column);

    var scroll = new ScrolledWindow ();
    scroll.vexpand = true;
    scroll.set_child (column_view);
}

```

```

this.append (scroll);

column_view.activate.connect ((pos) => {
    var item = selection.get_item (pos) as TreeListRow;
    if (item != null) {
        var file_item = item.get_item () as FileItem;
        if (file_item != null) {
            file_activated (file_item);
        }
    }
});
}

private GLib.ListStore create_file_model (GLib.File dir) {
    var store = new GLib.ListStore (typeof (FileItem));
    try {
        //
        var enumerator = dir.enumerate_children ("standard::display-name,standard::type"
        GLib.FileInfo info;
        while ((info = enumerator.next_file (null)) != null) {
            store.append (new FileItem (dir.get_child (info.get_name ()), info));
        }
    } catch (Error e) {
        //
        debug ("Cannot read directory: %s", e.message);
    }
    return store;
}

private ListItemFactory create_factory () {
    var factory = new SignalListItemFactory ();

    factory.setup.connect ((list_item) => {
        var item = (Gtk.ListItem) list_item;
        var expander = new TreeExpander ();
        var box = new Box (Orientation.HORIZONTAL, 6);
        var icon_box = new Box (Orientation.HORIZONTAL, 2);
        icon_box.set_size_request (20, 20);
        var label = new Label ("");

        box.append (icon_box);
        box.append (label);
        expander.set_child (box);
        item.set_child (expander);
    });
}

```



```

factory.bind.connect ((list_item) => {
    var item = (Gtk.ListItem) list_item;
    var tree_row = item.get_item () as TreeListRow;
    if (tree_row == null) return;

    var file_item = tree_row.get_item () as FileItem;
    var expander = item.get_child () as TreeExpander;
    var box = expander.get_child () as Box;
    var icon_box = box.get_first_child () as Box;
    var label = icon_box.get_next_sibling () as Label;

    expander.list_row = tree_row;
    if (file_item != null) {
        label.label = file_item.name;
        if (file_item.is_directory) {
            icon_box.append (ImageFactory.folder_image ());
        } else {
            icon_box.append (ImageFactory.create_for_file (file_item.file));
        }

        if (!file_item.is_directory) {
            var click = new Gtk.GestureClick ();
            click.released.connect (() => {
                file_activated (file_item);
            });
            box.add_controller (click);
        }
    }
});

return factory;
}

//
public class Iide.FileItem : Object {
    public GLib.File file { get; construct; }
    public string name { get; construct; }
    public bool is_directory { get; construct; }

    public FileItem (GLib.File file, GLib.FileInfo info) {
        Object (
            file : file,
            name : info.get_display_name (),
            is_directory: info.get_file_type () == GLib.FileType.DIRECTORY
        );
    }
}

```

```

    }
}

```

File: src/Widgets/PreferencesDialog.vala

```

/*
 * preferencesdialog.vala
 *
 * Copyright 2026 kai
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program. If not, see <https://www.gnu.org/licenses/>.
 *
 * SPDX-License-Identifier: GPL-3.0-or-later
 */

public class Iide.PreferencesDialog : Adw.PreferencesWindow {
    private Iide.SettingsService settings;
    private Adw.StyleManager style_manager;

    private Adw.ComboRow color_scheme_row;
    private Adw.SwitchRow show_minimap_row;
    private Adw.SwitchRow show_line_numbers_row;
    private Adw.SwitchRow show_marks_gutter_row;
    private Adw.SwitchRow show_folding_gutter_row;
    private Adw.SwitchRow highlight_current_line_row;
    private Adw.SwitchRow auto_indent_row;

    public PreferencesDialog () {
        Object (modal: true, destroy_with_parent: true);
        settings = Iide.SettingsService.get_instance ();
        style_manager = Adw.StyleManager.get_default ();

        title = _("Preferences");
        set_default_size (500, 450);
    }
}

```

```

        build_ui ();
    }

private void build_ui () {
    var action_manager = Ide.AppActionsManager.get_instance ();

    var appearance_page = new Adw.PreferencesPage () {
        title = _("Appearance"),
        icon_name = "preferences-desktop-appearance-symbolic"
    };

    var appearance_group = new Adw.PreferencesGroup () {
        title = _("Appearance")
    };

    color_scheme_row = new Adw.ComboRow () {
        title = _("Color Scheme"),
        model = new Gtk.StringList ({ "System", "Light", "Dark" })
    };
    var scheme_index = settings.color_scheme;
    color_scheme_row.selected = (uint) scheme_index;
    color_scheme_row.notify["selected"].connect (() => {
        var scheme = (ColorScheme) color_scheme_row.selected;
        settings.color_scheme = scheme;
        style_manager.color_scheme = scheme.to_adw_color_scheme ();
    });
    appearance_group.add (color_scheme_row);
    appearance_page.add (appearance_group);

    var editor_page = new Adw.PreferencesPage () {
        title = _("Editor"),
        icon_name = "accessories-text-editor-symbolic"
    };

    var editor_group = new Adw.PreferencesGroup () {
        title = _("Editor")
    };

    show_minimap_row = new Adw.SwitchRow () {
        title = _("Show Minimap"),
        subtitle = _("Display a minimap showing the document overview")
    };
    show_minimap_row.active = settings.show_minimap;
    show_minimap_row.notify["active"].connect (() => {
        settings.show_minimap = show_minimap_row.active;
    });
}

```

```

        action_manager.get_action ("show-minimap").update_state (show_minimap_row.active);
    });
    editor_group.add (show_minimap_row);

    show_line_numbers_row = new Adw.SwitchRow () {
        title = _("Show Line Numbers"),
        subtitle = _("Display line numbers on the editor gutter")
    };
    show_line_numbers_row.active = settings.show_line_numbers;
    show_line_numbers_row.notify["active"].connect (() => {
        settings.show_line_numbers = show_line_numbers_row.active;
        action_manager.get_action ("show-line-numbers").update_state (show_line_numbers_row.active);
    });
    editor_group.add (show_line_numbers_row);

    show_marks_gutter_row = new Adw.SwitchRow () {
        title = _("Show Diagnostics Marks"),
        subtitle = _("Display diagnostics marks on the editor gutter"),
    };
    show_marks_gutter_row.active = settings.show_diagnostics_marks;
    show_marks_gutter_row.notify["active"].connect (() => {
        settings.show_diagnostics_marks = show_marks_gutter_row.active;
        action_manager.get_action ("show-diagnostics-marks").update_state (show_marks_gutter_row.active);
    });
    editor_group.add (show_marks_gutter_row);

    show_folding_gutter_row = new Adw.SwitchRow () {
        title = _("Show Folding Gutter"),
        subtitle = _("Display folding gutter on the editor gutter"),
    };
    show_folding_gutter_row.active = settings.show_folding_gutter;
    show_folding_gutter_row.notify["active"].connect (() => {
        settings.show_folding_gutter = show_folding_gutter_row.active;
        action_manager.get_action ("show-folding-gutter").update_state (show_folding_gutter_row.active);
    });
    editor_group.add (show_folding_gutter_row);

    highlight_current_line_row = new Adw.SwitchRow () {
        title = _("Highlight Current Line"),
        subtitle = _("Highlight the line where the cursor is positioned")
    };
    highlight_current_line_row.active = settings.highlight_current_line;
    highlight_current_line_row.notify["active"].connect (() => {
        settings.highlight_current_line = highlight_current_line_row.active;
    });
    editor_group.add (highlight_current_line_row);

```

```

auto_indent_row = new Adw.SwitchRow () {
    title = _("Auto Indent"),
    subtitle = _("Automatically indent new lines based on the previous line")
};
auto_indent_row.active = settings.auto_indent;
auto_indent_row.notify["active"].connect (() => {
    settings.auto_indent = auto_indent_row.active;
});
editor_group.add (auto_indent_row);

var sizes = FontSizeHelper.get_available_sizes ();
var size_strings = new string[sizes.length];
for (int i = 0; i < sizes.length; i++) {
    size_strings[i] = "%d px".printf (sizes[i]);
}
var font_size_model = new Gtk.StringList (size_strings);
var current_level = settings.editor_font_size;
if (current_level < FontSizeHelper.MIN_ZOOM_LEVEL || current_level > FontSizeHelper.DEFAULT_ZOOM_LEVEL;
    current_level = FontSizeHelper.DEFAULT_ZOOM_LEVEL;
}

var font_size_row = new Adw.ComboRow () {
    title = _("Font Size"),
    model = font_size_model,
    selected = (uint) (current_level - 1)
};
font_size_row.notify["selected"].connect (() => {
    settings.editor_font_size = (int) font_size_row.selected + 1;
});
editor_group.add (font_size_row);
editor_page.add (editor_group);

var projects_page = new Adw.PreferencesPage () {
    title = _("Projects"),
    icon_name = "folder-open-symbolic"
};

var projects_group = new Adw.PreferencesGroup () {
    title = _("Projects")
};

var recent_projects_row = new Adw.ActionRow () {
    title = _("Recent Projects")
};
var recent_projects = settings.recent_projects;

```

```

if (recent_projects.length > 0) {
    var subtitle = string.joinv ("\n", recent_projects);
    recent_projects_row.subtitle = subtitle;
} else {
    recent_projects_row.subtitle = _("No recent projects");
}
var clear_button = new Gtk.Button () {
    icon_name = "edit-clear-symbolic",
    tooltip_text = _("Clear Recent Projects")
};
clear_button.clicked.connect (() => {
    settings.clear_recent_projects ();
    recent_projects_row.subtitle = _("No recent projects");
});
recent_projects_row.add_suffix (clear_button);
recent_projects_row.activatable_widget = clear_button;
projects_group.add (recent_projects_row);
projects_page.add (projects_group);

var shortcuts_page = new Adw.PreferencesPage () {
    title = _("Shortcuts"),
    icon_name = "input-keyboard-symbolic"
};

var shortcuts_group = new Adw.PreferencesGroup () {
    title = _("Keyboard Shortcuts")
};

var actions = action_manager.get_all_actions ();

var list_box = new Gtk.ListBox () {
    selection_mode = Gtk.SelectionMode.NONE,
    show_separators = true
};

var sorted_actions = new Gee.ArrayList<Ide.AppAction> ();
foreach (var action in actions) {
    sorted_actions.add (action);
}
sorted_actions.sort ((a, b) => {
    var cat_cmp = (a.category ?? "").collate (b.category ?? "");
    if (cat_cmp != 0) return cat_cmp;
    return a.name.collate (b.name);
});

string? current_category = null;

```

```

foreach (var action in sorted_actions) {
    if (action.category != current_category) {
        current_category = action.category;
        var header_row = new Gtk.ListBoxRow () {
            selectable = false,
            activatable = false
        };
        var header_label = new Gtk.Label (action.category ?? _("Other")) {
            halign = Gtk.Align.START,
            margin_top = 8,
            margin_bottom = 4
        };
        header_label.add_css_class ("title");
        header_label.add_css_class ("dim-label");
        header_row.child = header_label;
        list_box.append (header_row);
    }
    var row = new ShortcutRow (action);
    list_box.append (row);
}

var scrolled = new Gtk.ScrolledWindow () {
    child = list_box,
    hexpand = true,
    vexpand = true
};
scrolled.set_size_request (-1, 300);
shortcuts_group.add (scrolled);
shortcuts_page.add (shortcuts_group);

add (appearance_page);
add (editor_page);
add (projects_page);
add (shortcuts_page);
}
}

private class Ide.ShortcutRow : Gtk.ListBoxRow {
    private Ide.AppAction action;
    private Gtk.Label shortcut_label;

    public ShortcutRow (Ide.AppAction action) {
        this.action = action;

        var box = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 12) {
            margin_start = 12,

```

```

        margin_end = 12,
        margin_top = 8,
        margin_bottom = 8
    };

    var info_box = new Gtk.Box (Gtk.Orientation.VERTICAL, 2);
    var name_label = new Gtk.Label (action.name) {
        halign = Gtk.Align.START,
        expand = true
    };
    name_label.add_css_class ("title");
    var desc_label = new Gtk.Label (action.description ?? "") {
        halign = Gtk.Align.START,
        expand = true
    };
    desc_label.add_css_class ("caption");
    desc_label.add_css_class ("dim-label");
    info_box.append (name_label);
    info_box.append (desc_label);
    box.append (info_box);

    shortcut_label = new Gtk.Label (action.shortcut != null ? action.shortcut : _("None")) {
        halign = Gtk.Align.END,
        margin_start = 12
    };
    shortcut_label.add_css_class ("caption");
    shortcut_label.add_css_class ("dim-label");
    box.append (shortcut_label);

    var clear_button = new Gtk.Button () {
        icon_name = "edit-clear-symbolic",
        tooltip_text = _("Clear shortcut"),
        valign = Gtk.Align.CENTER
    };
    clear_button.clicked.connect (on_clear_clicked);
    box.append (clear_button);

    var capture_button = new Gtk.Button () {
        label = _("Change"),
        valign = Gtk.Align.CENTER
    };
    capture_button.clicked.connect (on_capture_clicked);
    box.append (capture_button);

    child = box;

```



```

        action.shortcut_changed.connect ((new_shortcut) => {
            shortcut_label.set_label (new_shortcut != null ? new_shortcut : _("None"));
        });
    }

    private void on_clear_clicked () {
        this.action.set_shortcut (null);
    }

    private void on_capture_clicked () {
        var window = this.get_ancestor (typeof (Gtk.Window)) as Gtk.Window;
        var dialog = new ShortcutCaptureWindow (action, window);
        dialog.show ();
    }
}

private class Iide.ShortcutCaptureWindow : Gtk.Window {
    private Iide.AppAction action;
    private Gtk.Label label;
    private uint keyval = 0;
    private Gdk.ModifierType modifiers;

    public ShortcutCaptureWindow (Iide.AppAction action, Gtk.Window? parent) {
        this.action = action;

        title = _("Set Shortcut for %s").printf (action.name);
        modal = true;
        if (parent != null) {
            set_transient_for (parent);
        }

        var content_box = new Gtk.Box (Gtk.Orientation.VERTICAL, 12) {
            margin_top = 20,
            margin_bottom = 20,
            margin_start = 20,
            margin_end = 20
        };

        label = new Gtk.Label (_("Press a key combination...")) {
            halign = Gtk.Align.CENTER
        };
        content_box.append (label);

        var current = action.shortcut;
        if (current != null && current != "") {
            var hint = new Gtk.Label (_("Current: %s").printf (current)) {

```

```

        halign = Gtk.Align.CENTER
    };
    hint.add_css_class ("caption");
    hint.add_css_class ("dim-label");
    content_box.append (hint);
}

var event_controller = new Gtk.EventControllerKey ();
event_controller.key_pressed.connect (on_key_pressed);
content_box.add_controller (event_controller);

var cancel_button = new Gtk.Button () {
    label = _("Cancel")
};
cancel_button.clicked.connect (() => destroy ());

var clear_button = new Gtk.Button () {
    label = _("Clear")
};
clear_button.clicked.connect (() => {
    this.action.set_shortcut (null);
    destroy ();
});

var save_button = new Gtk.Button () {
    label = _("Save")
};
save_button.add_css_class ("suggested-action");
save_button.clicked.connect (on_save);

var button_box = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 6);
button_box.append (cancel_button);
button_box.append (clear_button);
button_box.append (save_button);
button_box.halign = Gtk.Align.END;
content_box.append (button_box);

child = content_box;
set_size_request (350, 120);
}

private bool on_key_pressed (uint keyval, uint keycode, Gdk.ModifierType state) {
    var modifiers = state & (Gdk.ModifierType.SHIFT_MASK | Gdk.ModifierType.CONTROL_MASK);

    if (keyval == Gdk.Key.Escape) {
        destroy ();
    }
}

```

```

        return true;
    }

    if (keyval == Gdk.Key.BackSpace && modifiers == 0) {
        this.action.set_shortcut (null);
        destroy ();
        return true;
    }

    if (modifiers == 0 || (modifiers & Gdk.ModifierType.CONTROL_MASK) != 0 ||
        (modifiers & Gdk.ModifierType.SHIFT_MASK) != 0 ||
        (modifiers & Gdk.ModifierType.ALT_MASK) != 0 ||
        (modifiers & Gdk.ModifierType.SUPER_MASK) != 0) {

        if (modifiers != 0) {
            this.keyval = keyval;
            this.modifiers = modifiers;
            label.set_label (format_shortcut (keyval, modifiers));
        }
        return true;
    }
    return false;
}

private string format_shortcut (uint keyval, Gdk.ModifierType modifiers) {
    var accel_string = "";

    if ((modifiers & Gdk.ModifierType.CONTROL_MASK) != 0) accel_string += "<primary>";
    if ((modifiers & Gdk.ModifierType.ALT_MASK) != 0) accel_string += "<Alt>";
    if ((modifiers & Gdk.ModifierType.SHIFT_MASK) != 0) accel_string += "<Shift>";
    if ((modifiers & Gdk.ModifierType.SUPER_MASK) != 0) accel_string += "<Super>";

    var key_name = Gdk.keyval_name (keyval);
    if (key_name != null) {
        accel_string += key_name;
    }

    return accel_string;
}

private void on_save () {
    if (keyval != 0) {
        var shortcut = format_shortcut (keyval, modifiers);
        this.action.set_shortcut (shortcut);
    }
    destroy ();
}

```

```
}  
}
```

File: symbols/symbols.css

```
/*  
 * WARNING: THIS FILE IS GENERATED AUTOMATICALLY.  
 * DO NOT EDIT THIS FILE MANUALLY.  
 * ALL CHANGES WILL BE LOST UPON REGENERATION.  
 */  
.def-folder { color: #5f23b3; }  
.dark .def-folder { color: #7fcfc9; }  
----  
.def-ls-class { color: #f57900; }  
.dark .def-ls-class { color: #f57900; }  
----  
.def-ls-method { color: #9141ac; }  
.dark .def-ls-method { color: #9141ac; }  
----  
.def-ls-field { color: #3584e4; }  
.dark .def-ls-field { color: #3584e4; }  
----  
.def-ls-interface { color: #33d17a; }  
.dark .def-ls-interface { color: #33d17a; }  
----  
.def-ls-constructor { color: #3584e4; }  
.dark .def-ls-constructor { color: #3584e4; }  
----  
.def-ls-module { color: #3584e4; }  
.dark .def-ls-module { color: #3584e4; }  
----  
.def-ls-snippet { color: #33d17a; }  
.dark .def-ls-snippet { color: #33d17a; }  
----  
.def-ls-color { color: #c0bfb1; }  
.dark .def-ls-color { color: #c0bfb1; }  
----  
.def-ls-folder { color: #c0bfb1; }  
.dark .def-ls-folder { color: #c0bfb1; }  
----  
.def-ls-constant { color: #3584e4; }  
.dark .def-ls-constant { color: #3584e4; }  
----  
.def-ls-struct { color: #f57900; }  
.dark .def-ls-struct { color: #f57900; }
```

```

----
.def-ls-operator { color: #2aa1b3; }
.dark .def-ls-operator { color: #2aa1b3; }
----
.def-ls-property { color: #3584e4; }
.dark .def-ls-property { color: #3584e4; }
----
.def-ls-enum-member { color: #3584e4; }
.dark .def-ls-enum-member { color: #3584e4; }
----
.def-ls-type-parameter { color: #2aa1b3; }
.dark .def-ls-type-parameter { color: #2aa1b3; }
----
.def-ls-literal { color: #33d17a; }
.dark .def-ls-literal { color: #33d17a; }
----
.def-mt-make { color: #6d8086; }
.dark .def-mt-make { color: #6d8086; }
----
.def-mt-docker { color: #384d54; }
.dark .def-mt-docker { color: #384d54; }
----
.def-mt-json { color: #cbbcb41; }
.dark .def-mt-json { color: #cbbcb41; }
----
.def-mt-xml { color: #e37933; }
.dark .def-mt-xml { color: #e37933; }
----
.def-mt-yaml { color: #cb3e20; }
.dark .def-mt-yaml { color: #cb3e20; }
----
.def-mt-ini { color: #6d8086; }
.dark .def-mt-ini { color: #6d8086; }
----
.def-mt-markdown { color: #519aba; }
.dark .def-mt-markdown { color: #519aba; }
----
.def-mt-toml { color: #cb3e20; }
.dark .def-mt-toml { color: #cb3e20; }
----
.def-mt-vala { color: #6e44b3; }
.dark .def-mt-vala { color: #6e44b3; }
----
.def-mt-c { color: #599eff; }
.dark .def-mt-c { color: #599eff; }
----

```

```

.def-mt-h { color: #599eff; }
.dark .def-mt-h { color: #599eff; }
----

.def-mt-cpp { color: #00599c; }
.dark .def-mt-cpp { color: #00599c; }
----

.def-mt-py { color: #306998; }
.dark .def-mt-py { color: #306998; }
----

.def-mt-js { color: #f1e05a; }
.dark .def-mt-js { color: #f1e05a; }
----

.def-mt-ts { color: #2b7489; }
.dark .def-mt-ts { color: #2b7489; }
----

.def-mt-css { color: #563d7c; }
.dark .def-mt-css { color: #563d7c; }
----

.def-mt-html { color: #e34c26; }
.dark .def-mt-html { color: #e34c26; }
----

.def-mt-bash { color: #4ebd4e; }
.dark .def-mt-bash { color: #4ebd4e; }
----

.def-mt-rust { color: #dea584; }
.dark .def-mt-rust { color: #dea584; }
----

.def-mt-go { color: #00add8; }
.dark .def-mt-go { color: #00add8; }
----

.def-mt-lua { color: #000080; }
.dark .def-mt-lua { color: #000080; }
----

.def-mt-x-image { color: #a074c4; }
.dark .def-mt-x-image { color: #a074c4; }
----

.def-mt-x-audio { color: #2b7489; }
.dark .def-mt-x-audio { color: #2b7489; }
----

.def-mt-x-video { color: #2b7489; }
.dark .def-mt-x-video { color: #2b7489; }
----

.def-mt-x-text { color: #858585; }
.dark .def-mt-x-text { color: #858585; }
----

.def-mt-default { color: #858585; }

```

```

.dark .def-mt-default { color: #858585; }
----
.def-mt-x-app { color: #858585; }
.dark .def-mt-x-app { color: #858585; }
----
.def-cod-error { color: #ff3300; }
.dark .def-cod-error { color: #ff3300; }
----
.def-cod-warning { color: #cccc00; }
.dark .def-cod-warning { color: #cccc00; }
----
.def-cod-info { color: #66ccff; }
.dark .def-cod-info { color: #66ccff; }

```

File: src/Services/TreeSitter/python/PythonHighlighter.vala

```

[CCode (cname = "tree_sitter_python")]
extern unowned TreeSitter.Language ? get_lang_python ();

public class Iide.PythonHighlighter : BaseTreeSitterHighlighter {
    public PythonHighlighter (SourceView view) {
        base (view);
    }

    protected override unowned TreeSitter.Language get_ts_language () {
        return get_lang_python ();
    }

    protected override string query_source () {
        return """
            ; Identifier naming conventions

            (identifier) @variable

            ((identifier) @constructor
             (#match? @constructor "^[A-Z]"))

            ((identifier) @constant
             (#match? @constant "^[A-Z][A-Z_]*$"))

            ; Function calls

            (decorator) @function
            (decorator
             (identifier) @function)

```

```

(call
  function: (attribute attribute: (identifier) @function.method))
(call
  function: (identifier) @function)

; Builtin functions

((call
  function: (identifier) @function.builtin)
  (#match?
    @function.builtin
    "^(abs|all|any|ascii|bin|bool|breakpoint|bytearray|bytes|callable|chr|classmethod|
; Function definitions

(function_definition
  name: (identifier) @function)

(attribute attribute: (identifier) @property)
(type (identifier) @type)

; Literals

[
  (none)
  (true)
  (false)
] @constant.builtin

[
  (integer)
  (float)
] @number

(comment) @comment
(string) @string
(escape_sequence) @escape

(interpolation
  "{" @punctuation.special
  "}" @punctuation.special) @embedded

[
  "_"
  "=="

```



```

"!="
"*"
**"
***="
*="
/"
//"
//="
/="
&"
&="
%"
%= "
^"
^="
+"
->"
+= "
<"
<<"
<<="
<="
<>"
="
:="
=="
">"
">="
">>"
">>="
"| "
"|="
"~"
"@="
"and"
"in"
"is"
"not"
"or"
"is not"
"not in"
] @operators

["(" " ")" "[" "]" "{" "}"] @punctuation.bracket

[

```

```

        "as"
        "assert"
        "async"
        "await"
        "break"
        "class"
        "continue"
        "def"
        "del"
        "elif"
        "else"
        "except"
        "exec"
        "finally"
        "for"
        "from"
        "global"
        "if"
        "import"
        "lambda"
        "nonlocal"
        "pass"
        "print"
        "raise"
        "return"
        "try"
        "while"
        "with"
        "yield"
        "match"
        "case"
    ] @keyword
    """
}

protected override bool is_container_node (string node_type) {
    return node_type in new string[] {
        "function_definition", "class_definition", "module_definition"
    };
}

protected override bool is_foldable_node_type (string type) {
    //      AST Python
    return type == "function_definition" ||
        type == "class_definition" ||
        type == "if_statement" ||

```

```

        type == "for_statement" ||
        type == "while_statement" ||
        type == "try_statement" ||
        type == "with_statement" ||
        type == "match_statement" ||
        type == "case_clause";
    }

    protected override TreeSitter.Point body_start_point(TreeSitter.Node foldable_node) {
        for (uint32 j = 0; j < foldable_node.child_count (); j++) {
            var inner_child = foldable_node.child (j);
            if (inner_child.type () == ":")
                return inner_child.start_point ();
        }
        return foldable_node.start_point ();
    }

    public override GtkSource.Indenter? create_indenter() {
        return new PythonIndenter(this, get_ts_language ());
    }
}

```

File: src/Services/TreeSitter/TreeSitterDocument.vala

```

/*
*/

public class Iide.TreeSitterDocument: SourceDocument {
    public BaseTreeSitterHighlighter ts_highlighter;

    public TreeSitterDocument(SourceView source_view, BaseTreeSitterHighlighter ts_highlighter) {
        base(source_view);
        this.ts_highlighter = ts_highlighter;

        source_view.buffer.insert_text.connect ((ref location, text, len) => {
            ts_highlighter.on_insert_text (location, text, len);
        });

        source_view.buffer.delete_range.connect ((start, end) => {
            ts_highlighter.on_delete_range (start, end);
        });

        this.ts_highlighter.breadcrumbs_changed.connect((crumbs) => {
            this.breadcrumbs_changed(crumbs);
        });
    }
}

```

```

    });
    //          highlighter
    ((GtkSource.Buffer) (source_view.buffer)).highlight_syntax = false;

    //          indenter
    // source_view.auto_indent = ts_highlighter.ts_indenter == null;
    var indenter = ts_highlighter.create_indenter ();
    if (indenter != null)
        source_view.set_indenter (indenter);
    source_view.auto_indent = true;
}

protected override bool handle_key_pressed(uint keyval, uint keycode, Gdk.ModifierType m
    return ts_highlighter.handle_key_pressed (keyval, keycode, modifiers);
}

public override void expand_selection() {
    ts_highlighter.expand_selection ();
}

public override void shrink_selection() {
    ts_highlighter.shrink_selection ();
}

public override Gee.List<SourceNodeItem?> get_full_outline () {
    return ts_highlighter.get_full_outline ();
}
}

```

File: src/Services/SourceDocument.vala

```

/*
*/

public class Iide.SourceDocument: GLib.Object {
    public signal void document_changed(PendingChange new_change);
    public signal void breadcrumbs_changed (Gee.List<SourceNodeItem?> crumbs);

    protected virtual bool handle_key_pressed(uint keyval, uint keycode, Gdk.ModifierType m
    protected virtual void handle_insert_text (Gtk.TextIter iter, string text, int len_bytes
    protected virtual void handle_delete_range (Gtk.TextIter start, Gtk.TextIter end) {}

    public virtual void expand_selection() {}
    public virtual void shrink_selection() {}
    public virtual Gee.List<SourceNodeItem?> get_full_outline () {

```

```

        return new Gee.ArrayList<SourceNodeItem?> ();
    }

    public SourceDocument(SourceView source_view) {
        Object();

        source_view.buffer.insert_text.connect ((ref location, text, len) => {
            var change = new PendingChange (text, location);
            this.document_changed (change);
            handle_insert_text(location, text, len);
        });

        source_view.buffer.delete_range.connect ((start, end) => {
            var change = new PendingChange ("", start, end);
            this.document_changed (change);
            handle_delete_range (start, end);
        });

        var key_controller = new Gtk.EventControllerKey ();
        key_controller.set_propagation_phase (Gtk.PropagationPhase.CAPTURE);
        key_controller.key_pressed.connect (on_key_pressed);
        source_view.add_controller (key_controller);

        //          highlighter
        ((GtkSource.Buffer) (source_view.buffer)).highlight_syntax = true;

        //          indenter
        source_view.auto_indent = true;
    }

    private bool on_key_pressed (Gtk.EventControllerKey controller, uint keyval, uint keycode, uint modifiers) {
        return handle_key_pressed (keyval, keycode, modifiers);
    }
}

```

File: src/Widgets/TextView/LspCompletionProvider.vala

```

/*
 * LspCompletionProvider.vala
 *
 * Copyright 2026 kai
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or

```

```

* (at your option) any later version.
*
* This program is distributed in the hope that it will be useful,
* but WITHOUT ANY WARRANTY; without even the implied warranty of
* MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
* GNU General Public License for more details.
*
* You should have received a copy of the GNU General Public License
* along with this program. If not, see <https://www.gnu.org/licenses/>.
*
* SPDX-License-Identifier: GPL-3.0-or-later
*/

using GLib;
using Gee;
using Gtk;
using GtkSource;

namespace Ide {

    public class LspCompletionProposal : GLib.Object, CompletionProposal {
        public LspCompletionItem item { get; private set; }

        public LspCompletionProposal (LspCompletionItem item) {
            this.item = item;
        }

        public string get_label () {
            return item.label;
        }

        public string ? get_markup () {
            return item.label;
        }

        public string get_text () {
            return item.insert_text;
        }

        public Icon ? get_icon () {
            return null;
        }

        public string ? get_info () {
            return item.documentation;
        }
    }
}

```

```

}

public class LspCompletionProvider : GLib.Object, CompletionProvider {
    private weak SourceView source_view;
    private IdeLspService lsp_service;
    private GLib.ListStore base_store;
    private Gtk.FilterListModel filter_model;
    private string current_word = "";

    private bool filter_proposals (Object item) {
        var proposal = item as LspCompletionProposal;
        if (proposal == null) return false;

        //          filter_text    LSP,          ,          label
        string haystack = proposal.get_label ();
        string needle = this.current_word;

        if (needle == "") return true;

        //          GtkSource.Completion
        uint priority;
        return GtkSource.Completion.fuzzy_match (haystack, needle, out priority);
    }

    public LspCompletionProvider (SourceView view) {
        this.source_view = view;
        this.lsp_service = IdeLspService.get_instance ();

        // 1.
        base_store = new GLib.ListStore (typeof (LspCompletionProposal));

        // 2.          ,          filter_proposals
        var custom_filter = new Gtk.CustomFilter (filter_proposals);

        // 3.          -
        filter_model = new Gtk.FilterListModel (base_store, custom_filter);
    }

    public virtual bool is_trigger (Gtk.TextIter iter, unichar ch) {
        // 1.
        var client = IdeLspService.get_instance ().get_client_for_uri (this.source_view

        if (client == null || !client.is_initialized) {
            //          ,          (          )
            return ch == '.';
        }
    }
}

```

```

        // 2.      ,
        string s = ch.to_string ();
        if (client.capabilities.completion_triggers.contains (s)) {
            return true;
        }

        return false;
    }

    public virtual CompletionActivation get_activation (CompletionContext context) {
        //      : Ctrl+Space
        return CompletionActivation.INTERACTIVE | CompletionActivation.USER_REQUESTED;
    }

    public virtual async GLib.ListModel populate_async (CompletionContext context, GLib.
        base_store.remove_all ();
        current_word = context.get_word ();

        if (source_view == null) {
            return filter_model;
        }

        yield source_view.lsp_sync_changes_async ();

        var buffer = source_view.buffer;
        var insert_mark = buffer.get_insert ();
        TextIter iter;
        buffer.get_iter_at_mark (out iter, insert_mark);

        int line = iter.get_line ();
        int character = iter.get_line_offset ();
        string uri = source_view.uri;

        string? trigger_char = null;
        var trigger_kind = CompletionTriggerKind.INVOKED; // Invoked

        TextIter prev = iter;
        if (prev.backward_char ()) {
            unichar ch = prev.get_char ();
            string s = ch.to_string ();

            var client = IdeLspService.get_instance ().get_client_for_uri (this.source_v
            if (client != null && client.capabilities.completion_triggers.contains (s))
                trigger_char = s;
                trigger_kind = CompletionTriggerKind.TRIGGER_CHARACTER;

```



```

    }
}

var result = yield lsp_service.request_completion (uri, line, character, trigger

if (result == null || result.items.size == 0) {
    return filter_model;
}

foreach (var item in result.items) {
    var proposal = new LspCompletionProposal (item);
    base_store.append (proposal);
}

return filter_model;
}

public virtual void display (CompletionContext context, CompletionProposal proposal) {
    var p = (LspCompletionProposal) proposal;

    switch (cell.column) {
    case CompletionColumn.ICON :
        cell.set_widget (ImageFactory.create_for_completion (p.item.kind));
        break;
    case CompletionColumn.TYPED_TEXT:
        cell.text = p.get_label ();
        break;
    case CompletionColumn.DETAILS: {
        string? doc = p.item.documentation;
        if (doc != null && doc != "") {
            cell.set_markup (doc);
        }
        break;
    }
    case CompletionColumn.COMMENT: {
        string? doc = p.item.detail;
        if (doc != null && doc != "") {
            cell.set_markup (doc);
        }
        break;
    }
    default:
        break;
    }
}
}

```

```

// 2.          ( )
public virtual void activate (CompletionContext context, CompletionProposal proposal) {
    var view = context.get_view ();
    var buffer = view.get_buffer ();

    TextIter start, end;

    //          ,          GSV          "          "
    if (context.get_bounds (out start, out end)) {
        //          ,
        buffer.delete (ref start, ref end);
    } else {
        //          (          ),          ,
        buffer.get_iter_at_mark (out start, buffer.get_insert ());
        end = start;
        string? word = context.get_word ();
        if (word != null && word != "") {
            start.backward_chars (word.char_count ());
            buffer.delete (ref start, ref end);
        }
    }

    //
    var p = (LspCompletionProposal) proposal;
    buffer.insert (ref start, p.get_label (), -1);
}

//          -
public virtual void refilter (CompletionContext context, GLib.ListModel model) {
    current_word = context.get_word ();

    // 6.
    var filter = filter_model.get_filter () as Gtk.CustomFilter;
    filter.changed (Gtk.FilterChange.DIFFERENT);

    if (model.get_n_items () == 0) {
        context.get_completion ().hide ();
    }
}

public virtual string ? get_title () { return "Simple"; }
public virtual int get_priority (CompletionContext context) { return 100; }
public virtual bool is_running (CompletionContext context) { return true; }
}
}

```

File: src/Widgets/TextView/TreeSitterFoldingGutter.vala

```
/*
*/
using Gtk;
using GtkSource;
using Cairo;

namespace Iide {

    // -
    public class FoldedMarkRange : GLib.Object {
        public Gtk.TextMark start_mark;
        public Gtk.TextMark end_mark;

        public FoldedMarkRange (Gtk.TextBuffer buffer, Gtk.TextIter start, Gtk.TextIter end) {
            Object ();
            // left_gravity = true
            // /
            this.start_mark = buffer.create_mark (null, start, true);
            this.end_mark = buffer.create_mark (null, end, true);
        }

        public void free_marks (Gtk.TextBuffer buffer) {
            buffer.delete_mark (this.start_mark);
            buffer.delete_mark (this.end_mark);
        }
    }

    public class TreeSitterFoldingGutter : GtkSource.GutterRenderer {
        //
        public Gee.ArrayList<FoldedMarkRange> active_folds = new Gee.ArrayList<FoldedMarkRange> ();

        // Tree-sitter
        private Gee.List<Iide.IndentBlock?> file_blocks;

        private string folding_tag_name = "$FOLD_HIDE";
        private int current_icon_size = 16;

        // QtCreator
        private bool is_pointer_inside = false;
        private int hover_line = -1;

        public signal void fold_state_changed ();

        public TreeSitterFoldingGutter () {
```

```

Object ();
this.file_blocks = new Gee.ArrayList<Iide.IndentBlock?> ();
this.set_alignment_mode (GtkSource.GutterRendererAlignmentMode.CELL);
}

//          SourceView.vala
public void update_blocks_data (Gee.List<Iide.IndentBlock?> blocks) {
    this.file_blocks = blocks;
    this.queue_draw ();
}

//          EventController-      GTK 4
protected override void change_view (GtkSource.View? old_view) {
    base.change_view (old_view);
    var view = this.get_view ();
    if (view == null) return;

    var motion_controller = new Gtk.EventControllerMotion ();

    motion_controller.enter.connect ((x, y) => {
        this.is_pointer_inside = true;
        this.update_hover_position (x, y);
    });

    motion_controller.motion.connect ((x, y) => {
        this.update_hover_position (x, y);
    });

    motion_controller.leave.connect (() => {
        this.is_pointer_inside = false;
        this.hover_line = -1;
        this.queue_draw ();
    });

    view.add_controller (motion_controller);
}

private void update_hover_position (double x, double y) {
    var view = this.get_view ();
    Gtk.TextIter iter;
    int buffer_y;
    view.window_to_buffer_coords (Gtk.TextWindowType.TEXT, 0, (int) y, null, out buffer_y);
    view.get_iter_at_location (out iter, 0, buffer_y);

    int new_hover_line = iter.get_line ();
    if (this.hover_line != new_hover_line) {

```

```

        this.hover_line = new_hover_line;
        this.queue_draw ();
    }
}

public void set_icons_size (int size) {
    if (current_icon_size == size) {
        return;
    }
    current_icon_size = size;
    queue_resize ();

    var gutter = (Gutter) get_parent ();
    if (gutter != null) {
        gutter.queue_allocate ();
    }
}

public override void measure (Gtk.Orientation orientation, int for_size, out int minimum, out int natural) {
    minimum_baseline = natural_baseline = -1;
    if (orientation == Gtk.Orientation.HORIZONTAL) {
        minimum = natural = this.current_icon_size;
    } else {
        minimum = natural = 0;
    }
}

// QtCreator (GtkSourceView 5)
public override void snapshot_line (Gtk.Snapshot snapshot, GtkSource.GutterLines lines, out Gtk.TextIter current_line_iter, out int current_line) {
    Gtk.TextIter current_line_iter;
    lines.get_iter_at_line (out current_line_iter, line);
    int current_line = current_line_iter.get_line ();

    bool is_block_start = this.check_if_block_starts (current_line);
    bool is_inside_block = this.check_if_inside_foldable_block (current_line);

    if (!is_block_start && !is_inside_block) return;

    bool is_collapsed = this.is_line_collapsed_by_number (current_line);

    int cell_y, cell_height;
    lines.get_line_yrange (line, GtkSource.GutterRendererAlignmentMode.CELL, out cell_y, out cell_height);
    if (cell_height <= 0) return;

    var bounds = Graphene.Rect ();

```

```

        bounds.init (0.0f, (float) cell_y, (float) this.current_icon_size, (float) cell_y + cell_height);

        var cr = snapshot.append_cairo (bounds);
        cr.set_line_width (1.0);

        // Scope
        bool is_current_scope_hovered = this.check_if_line_belongs_to_hovered_block (current_line);
        if (is_current_scope_hovered) {
            cr.set_source_rgba (0.2, 0.6, 1.0, 0.9); //
        } else {
            cr.set_source_rgba (0.5, 0.5, 0.5, 0.4); //
        }

        double mid_x = this.current_icon_size / 2.0;
        double mid_y = cell_y + (cell_height / 2.0);
        double r = double.max (3.0, this.current_icon_size / 4.0);

        if (is_block_start) {
            if (is_collapsed) {
                // [+]
                this.draw_plus_icon_scaled (cr, mid_x, mid_y, r);
            } else {
                // [-]
                if (this.is_pointer_inside) {
                    this.draw_minus_icon_scaled (cr, mid_x, mid_y, r);
                }
                cr.move_to (mid_x, mid_y + r + 1.0);
                cr.line_to (mid_x, cell_y + cell_height);
                cr.stroke ();
            }
        } else if (is_inside_block && !is_collapsed) {
            cr.move_to (mid_x, cell_y);
            cr.line_to (mid_x, cell_y + cell_height);
            cr.stroke ();
        }
    }

    //
    TEXTMARK (GTK 4)
    public override void activate (Gtk.TextIter iter, Gdk.Rectangle area, uint button, int n_presses) {
        if (button != 1 || n_presses != 1) return;

        int clicked_line = iter.get_line ();
        var view = this.get_view ();
        var buffer = view.get_buffer ();

        var target_block = this.find_deepest_block_for_line (clicked_line);

```

```

if (target_block == null) return;

int start_line = target_block.start_line;
int end_line = target_block.end_line;

//
Gtk.TextIter fold_start;
buffer.get_iter_at_line (out fold_start, start_line + 1);

Gtk.TextIter fold_end;
buffer.get_iter_at_line (out fold_end, end_line + 1);
if (fold_end.is_end()) buffer.get_end_iter (out fold_end);

var invisible_tag = buffer.get_tag_table ().lookup (this.folding_tag_name);
if (invisible_tag == null) return;

//
FoldedMarkRange? existing_fold = null;
foreach (var fold in this.active_folds) {
    Gtk.TextIter mark_iter;
    buffer.get_iter_at_mark (out mark_iter, fold.start_mark);
    if (mark_iter.get_line () - 1 == start_line) {
        existing_fold = fold;
        break;
    }
}

if (existing_fold != null) {
    //
    Gtk.TextIter s, e;
    buffer.get_iter_at_mark (out s, existing_fold.start_mark);
    buffer.get_iter_at_mark (out e, existing_fold.end_mark);

    buffer.remove_tag (invisible_tag, s, e);
    existing_fold.free_marks (buffer);
    this.active_folds.remove (existing_fold);
} else {
    //
    buffer.apply_tag (invisible_tag, fold_start, fold_end);
    var new_fold = new FoldedMarkRange (buffer, fold_start, fold_end);
    this.active_folds.add (new_fold);
}

view.queue_resize ();
this.queue_draw ();
this.fold_state_changed ();

```

```

}

public override bool query_activatable (Gtk.TextIter iter, Gdk.Rectangle area) {
    int line = iter.get_line ();
    return this.find_deepest_block_for_line (line) != null;
}

// ( - REE-SITTER)

public Iide.IndentBlock? find_deepest_block_for_line (int line) {
    Iide.IndentBlock? deepest_block = null;
    int max_indent = -1;

    foreach (var block in this.file_blocks) {
        if (line >= block.start_line && line <= block.end_line) {
            if (block.indent_level > max_indent) {
                max_indent = block.indent_level;
                deepest_block = block;
            }
        }
    }
    return deepest_block;
}

private bool check_if_block_starts (int line) {
    foreach (var block in this.file_blocks) {
        if (block.start_line == line) return true;
    }
    return false;
}

private bool check_if_inside_foldable_block (int line) {
    var block = this.find_deepest_block_for_line (line);
    return block != null && line > block.start_line && line <= block.end_line;
}

private bool check_if_line_belongs_to_hovered_block (int line) {
    if (this.hover_line == -1) return false;

    var hovered_block = this.find_deepest_block_for_line (this.hover_line);
    if (hovered_block == null) return false;

    var current_block = this.find_deepest_block_for_line (line);
    if (current_block == null) return false;

    if (line >= hovered_block.start_line && line <= hovered_block.end_line) {

```



```

        if (current_block.indent_level > hovered_block.indent_level) {
            return true;
        }
        return hovered_block.start_line == current_block.start_line &&
            hovered_block.end_line == current_block.end_line;
    }
    return false;
}

public bool is_line_collapsed_by_number (int line_num) {
    var buffer = this.get_view ().get_buffer ();
    foreach (var fold in this.active_folds) {
        Gtk.TextIter mark_iter;
        buffer.get_iter_at_mark (out mark_iter, fold.start_mark);
        if (mark_iter.get_line () - 1 == line_num) {
            return true;
        }
    }
    return false;
}

//          CAIRO
private void draw_plus_icon_scaled (Cairo.Context cr, double x, double y, double r) {
    cr.rectangle (x - r, y - r, r * 2, r * 2); cr.stroke ();
    cr.move_to (x - (r - 2.0), y); cr.line_to (x + (r - 2.0), y); cr.stroke ();
    cr.move_to (x, y - (r - 2.0)); cr.line_to (x, y + (r - 2.0)); cr.stroke ();
}

private void draw_minus_icon_scaled (Cairo.Context cr, double x, double y, double r) {
    cr.rectangle (x - r, y - r, r * 2, r * 2); cr.stroke ();
    cr.move_to (x - (r - 2.0), y); cr.line_to (x + (r - 2.0), y); cr.stroke ();
}
}
}

```

File: src/Services/LSP/LspClient.vala

```

using GLib;
using Gee;

public class Iide.LspPromise : Object {
    public SourceFunc callback;
    //
    public int64 creation_time;
}

```

```

    public LspPromise (owned SourceFunc cb) {
        this.callback = (owned) cb;
        this.creation_time = get_monotonic_time ();
    }
}

public enum Iide.TextDocumentSyncKind {
    NONE = 0,
    FULL = 1,
    INCREMENTAL = 2
}

public class Iide.ServerCapabilities : Object {
    public TextDocumentSyncKind sync_kind = TextDocumentSyncKind.FULL;
    public HashSet<string> completion_triggers = new HashSet<string> ();
    public bool hover_provider = false;
    public bool definition_provider = false;

    // To features...
    public bool references_provider = false;
    public bool rename_provider = false;
}

public struct Iide.LspTaskInfo {
    public string server_name;
    public string message;
    public int percentage; // -1
}

public class Iide.LspClient : Object {
    private int next_id = 0;
    private Map<int, LspPromise> pending_requests = new HashMap<int, LspPromise> ();
    private Map<int, Json.Object?> responses = new HashMap<int, Json.Object?> ();
    private LspConfig config;

    private DiagnosticsService diagnostics_service;

    //      LSP
    private GLib.Subprocess process;

    //      (
    private GLib.DataInputStream input_stream;

    //      (
    private bool is_writing = false;
    private Deque<string> write_queue = new LinkedList<string> ();

```

```

private OutputStream output_stream;

//                                UI
public signal void diagnostics_received (string uri, Gee.ArrayList<LspDiagnostic> diags) {
    //
    public signal void log_message (int type, string message);

    //
    public ServerCapabilities capabilities { get; private set; }

    // , ,
    public signal void initialized_with_capabilities (ServerCapabilities caps);

    public bool is_initialized { get; private set; default = false; }

    public LspClient (LspConfig config) {
        this.capabilities = new ServerCapabilities ();
        this.config = config;
        this.diagnostics_service = DiagnosticsService.get_instance ();
    }

    public signal void progress_updated (string token, string message, int percentage, bool done) {
        //
    }

    public string name () { return config.command (); }

    public int get_hash () {
        //
        return (int) ((void*) this);
    }

    public async Json.Object? send_request (string method, Json.Object params, Cancellable? cancellable) {
        int id = next_id--;

        var root = new Json.Object ();
        root.set_string_member ("jsonrpc", "2.0");
        root.set_int_member ("id", id);
        root.set_string_member ("method", method);
        root.set_object_member ("params", params);

        //
        var promise = new LspPromise (send_request.callback);
        pending_requests.set (id, promise);

        // : , , " "
        ulong handler_id = 0;

```

```

        if (cancellable != null) {
            handler_id = cancellable.connect (() => {
                //
                if (pending_requests.has_key (id)) {
                    pending_requests.unset (id);
                    //
                    . Idle.add
                    UI
                    Idle.add ((owned) promise.callback);
                }
            });
        }

        try {
            //
            cancellable
            yield this.send_message_async (root, cancellable);

            //
            yield;

            //
            if (cancellable != null && cancellable.is_cancelled ()) {
                throw new IOError.CANCELLED ("
                %s (id: %d)
                ".printf (method, id));
            }

            //
            ,
            var response = responses.get (id);
            responses.unset (id);
            return response;
        } finally {
            //
            if (handler_id > 0) {
                cancellable.disconnect (handler_id);
            }
        }
    }

}

private async void send_message_async (Json.Object node, Cancellable? cancellable = null) {
    var generator = new Json.Generator ();
    var root_node = new Json.Node (Json.NodeType.OBJECT);
    root_node.set_object (node);
    generator.set_root (root_node);

    string body = generator.to_data (null);
    string message = "Content-Length: %d\r\n\r\n%s".printf ((int) body.length, body);

    write_queue.add (message);
}

```

```

    if (is_writing) return;
    is_writing = true;

    try {
        while (!write_queue.is_empty) {
            //
            if (cancellable != null && cancellable.is_cancelled ()) break;

            string current_msg = write_queue.poll_head ();
            //
            yield output_stream.write_all_async (current_msg.data, Priority.DEFAULT, cancellable);

            yield output_stream.flush_async (Priority.DEFAULT, cancellable);
        }
    } finally {
        is_writing = false;
    }
}

private async void run_read_loop () {
    try {
        while (true) {
            // 1.          Content-Length
            string? line = yield input_stream.read_line_async (Priority.DEFAULT, null);

            if (line == null) break; //

            if (line.has_prefix ("Content-Length: ")) {
                int length = int.parse (line.substring (16).strip ());

                // 2.          (\r\n\r\n)
                while (line != "" && line != null) {
                    line = yield input_stream.read_line_async (Priority.DEFAULT, null);

                    if (line != null) line = line.strip ();
                }

                // 3.          JSON
                uint8[] buffer = new uint8[length + 1];
                size_t bytes_read;
                yield input_stream.read_all_async (buffer[0 : length], Priority.DEFAULT, cancellable);

                buffer[length] = '\0'; //

                // 4.
                this.handle_payload ((string) buffer);
            }
        }
    }
}

```

```

        }
    }
} catch (Error e) {
    if (!(e is IOError.CANCELLED)) {
        debug ("LSP Read Loop Error: %s", e.message);
    }
}
}

public async void send_response_async (Json.Node id, Json.Node? result = null) throws Error {
    var root = new Json.Object ();
    root.set_string_member ("jsonrpc", "2.0");
    root.set_member ("id", id);

    //          'result'      'method'   'params'
    if (result != null) {
        root.set_member ("result", result);
    } else {
        root.set_member ("result", new Json.Node (Json.NodeType.NULL));
    }

    var generator = new Json.Generator ();
    var root_node = new Json.Node (Json.NodeType.OBJECT);
    root_node.set_object (root);
    generator.set_root (root_node);

    yield this.send_message_async (root);
}

private void handle_payload (string payload) {
    try {
        var parser = new Json.Parser ();
        parser.load_from_data (payload);
        var root = parser.get_root ().get_object ();

        if (root.has_member ("method")) {
            string method = root.get_string_member ("method");
            if (method == "window/workDoneProgress/create" && root.has_member ("id")) {
                //
                var node_id = root.get_member ("id");
                send_response_async.begin (node_id, new Json.Node (Json.NodeType.NULL));
                return;
            }
        }
        if (method == "workspace/diagnostic/refresh" && root.has_member ("id")) {
            //
            var node_id = root.get_member ("id");

```

```

        send_response_async.begin (node_id, new Json.Node (Json.NodeType.NULL));
        return;
    }
}

//          (    'id')
if (root.has_member ("id")) {
    this.handle_response (root);
}
//          (    'method',    'id')
else if (root.has_member ("method")) {
    this.handle_incoming_notification (root);
}
} catch (Error e) {
    warning ("Failed to parse LSP payload: %s", e.message);
}
}

private void handle_response (Json.Object response) {
    int id = (int) response.get_int_member ("id");

    if (pending_requests.has_key (id)) {
        var promise = pending_requests.get (id);
        pending_requests.unset (id);

        //
        responses.set (id, response);

        //
        //      Idle.add,          MainContext (UI      )
        Idle.add ((owned) promise.callback);
        return;
    }

    if (response.has_member ("method")) {
        var node_result = config.server_response_result (response);
        var node_id = response.get_member ("id");
        send_response_async.begin (node_id, node_result);
    }

    // if (response.has_member ("method")) {
    // string method = response.get_string_member ("method");
    // switch (method) {
    // case "client/registerCapability" : {
    // message ("!!!handle_response - client/registerCapability - " + json_object_to_string (response));
    // var node_id = response.get_member ("id");

```

```

        // send_response_async.begin (node_id, new Json.Node (Json.NodeType.NULL));
        // } return;
        // case "workspace/configuration" : {
        // message ("!!!handle_response - workspace/configuration - " + json_object_to_string(response));
        // handle_response_workspace_configuration (response);
        // } return;
        // }
        // }
    }

private void handle_incoming_notification (Json.Object root) {
    string method = root.get_string_member ("method");

    //
    // (params)
    Json.Object? params = null;
    if (root.has_member ("params")) {
        params = root.get_object_member ("params");
    }

    switch (method) {
    case "textDocument/publishDiagnostics" :
        if (params != null) {
            string uri = params.get_string_member ("uri");
            var json_array = params.get_array_member ("diagnostics");

            // JSON-
            // IdeLspDiagnostic
            var diag_list = this.parse_diagnostics (json_array);

            //
            // UI
            Idle.add (() => {
                //
                diagnostics_service.update_diagnostics (this.get_hash (), uri, diag_list);

                this.diagnostics_received (uri, diag_list);
                return Source.REMOVE;
            });
        }
        break;

    case "window/workDoneProgress/create" :
        message ("!!!handle_incoming_notification - window/workDoneProgress/create" + json_object_to_string(response));
        break;

    case "$/progress" :
        // message ("!!!handle_incoming_notification - $/progress" + json_object_to_string(response));
        if (params != null) {

```



```

// Token                                , Tree-sitter  LSP
string token = "";
var token_node = params.get_member ("token");
if (token_node.get_node_type () == Json.NodeType.VALUE) {
    token = token_node.get_string ();
} else {
    token = token_node.get_int ().to_string ();
}

var value = params.get_object_member ("value");
string kind = value.get_string_member ("kind");

//
bool active = (kind != "end");
if (kind == "end") {
    var sparams = new Json.Object ();
    sparams.set_string_member ("query", "");

    send_request.begin ("workspace/symbol", sparams);
}

//                                ( BasedPyright          'message')
string msg = "";
if (value.has_member ("title")) {
    msg = value.get_string_member ("title");
}
if (value.has_member ("message")) {
    string details = value.get_string_member ("message");
    msg = (msg != "") ? @"$msg: $details" : details;
}

//                                ( )
int perc = value.has_member ("percentage") ? (int) value.get_int_member ("percentage") : 0;

//                                UI
Idle.add (() => {
    this.progress_updated (token, msg, perc, active);
    return Source.REMOVE;
});

}
break;
case "window/logMessage" :
    if (params != null)handle_log_message (params);
    break;

case "window/showMessage" :

```

```

//                                     GNOME
if (params != null)debug ("LSP Show Message: %s", params.get_string_member ("message"));
break;

default:
    debug ("Unhandled LSP notification: %s", method);
    break;
}
}

private void handle_log_message (Json.Object params) {
    int type = (int) params.get_int_member ("type");
    string message = params.get_string_member ("message");

    Idle.add (() => {
        this.log_message (type, message);
        return Source.REMOVE;
    });
}

public async bool start_server_async (string? workspace_root) {
    try {
        // 1.
        string[] argv = { config.command () };
        foreach (var arg in config.args ())argv += arg;

        // 2.
        var launcher = new SubprocessLauncher (SubprocessFlags.STDOUT_PIPE | SubprocessFlags.STDERR_PIPE);
        launcher.set_cwd (workspace_root.replace ("file://", ""));
        this.process = launcher.spawnv (argv);

        // 3.
        this.output_stream = this.process.get_stdin_pipe ();
        this.input_stream = new DataInputStream (this.process.get_stdout_pipe ());

        // 4.
        (
        this.run_read_loop.begin ();

        // 5. INITIALIZE
        var init_params = config.initialize_params (workspace_root, null);
        var response = yield this.send_request ("initialize", init_params.get_object ("params"));

        if (response != null && response.has_member ("result")) {
            var result = response.get_object_member ("result");
            //
            this.parse_capabilities (result);
        }
    }
}

```

```

    }

    //      Idle.add,
    is_initialized = true;
    Idle.add (() => {
        this.initialized_with_capabilities (this.capabilities);
        IdeLspService.get_instance ().register_client (this);
        return Source.REMOVE; //
    });

    // 6.      INITIALIZED (
    yield this.send_notification_async ("initialized", new Json.Object ());

    debug ("LSP Server started and initialized for: %s", workspace_root ?? "unknown");
    return true;
} catch (Error e) {
    warning ("Failed to start LSP server: %s", e.message);
    return false;
}
}

private void parse_capabilities (Json.Object result) {
    if (!result.has_member ("capabilities"))return;
    var caps = result.get_object_member ("capabilities");

    //
    if (caps.has_member ("textDocumentSync")) {
        var sync = caps.get_member ("textDocumentSync");
        if (sync.get_node_type () == Json.NodeType.OBJECT) {
            var sync_obj = sync.get_object ();
            if (sync_obj.has_member ("change"))
                this.capabilities.sync_kind = (TextDocumentSyncKind) sync_obj.get_int_member ("change");
        } else if (sync.get_node_type () == Json.NodeType.VALUE) {
            this.capabilities.sync_kind = (TextDocumentSyncKind) sync.get_int ();
        }
    }

    //
    if (caps.has_member ("completionProvider")) {
        var comp = caps.get_object_member ("completionProvider");
        if (comp.has_member ("triggerCharacters")) {
            var triggers = comp.get_array_member ("triggerCharacters");
            this.capabilities.completion_triggers.clear ();
            foreach (var node in triggers.get_elements ()) {
                this.capabilities.completion_triggers.add (node.get_string ());
            }
        }
    }
}

```

```

    }
}

// Hover & Definition
this.capabilities.hover_provider = caps.has_member ("hoverProvider");
this.capabilities.definition_provider = caps.has_member ("definitionProvider");
}

public async void send_notification_async (string method, Json.Object params) throws Error {
    var root = new Json.Object ();
    root.set_string_member ("jsonrpc", "2.0");
    root.set_string_member ("method", method);
    root.set_object_member ("params", params);

    //
    yield this.send_message_async (root);
}

private Gee.ArrayList<LspDiagnostic> parse_diagnostics (Json.Array diagnostics_array) {
    var result = new Gee.ArrayList<LspDiagnostic> ();

    foreach (var diag_node in diagnostics_array.get_elements ()) {
        var diag_obj = diag_node.get_object ();
        var d = new LspDiagnostic ();

        //
        d.message = diag_obj.get_string_member ("message");
        if (diag_obj.has_member ("severity")) {
            d.severity = (int) diag_obj.get_int_member ("severity");
        }

        //      (Range LSP      start end      )
        var range = diag_obj.get_object_member ("range");
        var start = range.get_object_member ("start");
        var end = range.get_object_member ("end");

        d.start_line = (int) start.get_int_member ("line");
        d.start_column = (int) start.get_int_member ("character"); // character -> start_column
        d.end_line = (int) end.get_int_member ("line");
        d.end_column = (int) end.get_int_member ("character"); // character -> end_column

        result.add (d);
    }

    return result;
}

```

```

public async void text_document_did_open (string uri, string language_id, int version, s
    var params = new Json.Object ();

    var doc = new Json.Object ();
    doc.set_string_member ("uri", uri);
    doc.set_string_member ("languageId", language_id);
    doc.set_int_member ("version", version);
    doc.set_string_member ("text", content);

    params.set_object_member ("textDocument", doc);

    //          (notification),          ID
    yield this.send_notification_async ("textDocument/didOpen", params);

    debug ("LSP: Sent didOpen for %s", uri);
}

public async void text_document_did_change (string uri, int version, string content) th
    var params = new Json.Object ();

    // 1.
    var doc = new Json.Object ();
    doc.set_string_member ("uri", uri);
    doc.set_int_member ("version", version);
    params.set_object_member ("textDocument", doc);

    // 2.          (      range)
    var change = new Json.Object ();
    change.set_string_member ("text", content);

    var changes = new Json.Array ();
    changes.add_object_element (change);
    params.set_array_member ("contentChanges", changes);

    yield this.send_notification_async ("textDocument/didChange", params);
}

public async void send_did_change (string uri, int version, Gee.ArrayList<PendingChange>
    var params = new Json.Object ();

    // 1.
    var doc = new Json.Object ();
    doc.set_string_member ("uri", uri);
    doc.set_int_member ("version", version);
    params.set_object_member ("textDocument", doc);

```

```

// 2.
var json_changes = new Json.Array ();
foreach (var c in changes) {
    var obj = new Json.Object ();

    // (range)
    var range = new Json.Object ();

    var start = new Json.Object ();
    start.set_int_member ("line", c.start_line);
    start.set_int_member ("character", c.start_utf16_char);

    var end = new Json.Object ();
    end.set_int_member ("line", c.end_line);
    end.set_int_member ("character", c.end_utf16_char);

    range.set_object_member ("start", start);
    range.set_object_member ("end", end);

    obj.set_object_member ("range", range);
    obj.set_string_member ("text", c.text);

    json_changes.add_object_element (obj);
}
params.set_array_member ("contentChanges", json_changes);

yield this.send_notification_async ("textDocument/didChange", params);
}

public async void text_document_did_close (string uri) throws Error {
    var params = new Json.Object ();
    var doc = new Json.Object ();
    doc.set_string_member ("uri", uri);
    params.set_object_member ("textDocument", doc);

    yield this.send_notification_async ("textDocument/didClose", params);
}

private LspCompletionResult parse_completion_result (Json.Node node) {
    var res = new LspCompletionResult ();
    res.items = new Gee.ArrayList<LspCompletionItem> ();

    Json.Array? items_array = null;

    // 1: CompletionList { isIncomplete, items }

```

```

if (node.get_node_type () == Json.NodeType.OBJECT) {
    var obj = node.get_object ();
    if (obj.has_member ("isIncomplete")) {
        res.is_incomplete = obj.get_boolean_member ("isIncomplete");
    }
    if (obj.has_member ("items")) {
        items_array = obj.get_array_member ("items");
    }
}
//      2:                CompletionItem[]
else if (node.get_node_type () == Json.NodeType.ARRAY) {
    items_array = node.get_array ();
}

if (items_array != null) {
    foreach (var item_node in items_array.get_elements ()) {
        var item_obj = item_node.get_object ();
        var item = new LspCompletionItem ();

        item.label = item_obj.get_string_member ("label");

        //
        if (item_obj.has_member ("insertText"))
            item.insert_text = item_obj.get_string_member ("insertText");
        else
            item.insert_text = item.label;

        if (item_obj.has_member ("detail"))
            item.detail = item_obj.get_string_member ("detail");

        if (item_obj.has_member ("kind"))
            item.kind = (LspCompletionKind) item_obj.get_int_member ("kind");

        //                (                MarkupContent)
        if (item_obj.has_member ("documentation")) {
            var doc_node = item_obj.get_member ("documentation");
            if (doc_node.get_node_type () == Json.NodeType.OBJECT)
                item.documentation = doc_node.get_object ().get_string_member ("value");
            else
                item.documentation = doc_node.get_string ();
        }

        res.items.add (item);
    }
}

```

```

        return res;
    }

    public async LspCompletionResult ? request_completion (string uri, int line, int character) {
        var params = new Json.Object ();

        var doc = new Json.Object ();
        doc.set_string_member ("uri", uri);
        params.set_object_member ("textDocument", doc);

        var pos = new Json.Object ();
        pos.set_int_member ("line", line);
        pos.set_int_member ("character", character);
        params.set_object_member ("position", pos);

        var context = new Json.Object ();
        context.set_int_member ("triggerKind", (int) trigger_kind);
        if (trigger_char != null) {
            context.set_string_member ("triggerCharacter", trigger_char);
        }
        params.set_object_member ("context", context);

        var response = yield this.send_request ("textDocument/completion", params);

        if (response == null || !response.has_member ("result"))return null;

        var result_node = response.get_member ("result");
        if (result_node.get_node_type () == Json.NodeType.NULL)return null;

        return parse_completion_result (result_node);
    }

    private string ? parse_hover_result (Json.Node node) {
        if (node.get_node_type () != Json.NodeType.OBJECT)
            return null;

        var result = node.get_object ();

        //      1:          'contents'
        if (!result.has_member ("contents"))
            return null;

        var contents = result.get_member ("contents");

        //      (MarkupContent)
        if (contents.get_node_type () == Json.NodeType.OBJECT) {

```



```

        var obj = contents.get_object ();
        if (obj.has_member ("value")) {
            return obj.get_string_member ("value");
        }
    }
    //
    else {
        return contents.get_string ();
    }
}

return null;
}

public async string ? request_hover (string uri, int line, int character) throws Error {
    var params = new Json.Object ();

    var doc = new Json.Object ();
    doc.set_string_member ("uri", uri);
    params.set_object_member ("textDocument", doc);

    var pos = new Json.Object ();
    pos.set_int_member ("line", line);
    pos.set_int_member ("character", character);
    params.set_object_member ("position", pos);

    var response = yield this.send_request ("textDocument/hover", params);

    if (response == null || !response.has_member ("result")) return null;

    return parse_hover_result (response.get_member ("result"));
}

public async Gee.List<DocumentLspSymbol>? document_symbols (string uri, Cancellable ? cancellable) {
    var params = new Json.Object ();

    var doc = new Json.Object ();
    doc.set_string_member ("uri", uri);
    params.set_object_member ("textDocument", doc);

    var response = yield this.send_request ("textDocument/documentSymbol", params, cancellable);

    if (response == null || !response.has_member ("result"))
        return null;

    var node = new Json.Node(Json.NodeType.OBJECT);
    node.set_object(response);
}

```

```

        LoggerService.get_instance ().info("DS", Json.to_string (node, true));

        var result = parse_document_lsp_symbols (response.get_member ("result"));

        return result;
    }

    public async Gee.List<WorkspaceLspSymbol>? workspace_symbols (string query, Cancellable
        var params = new Json.Object ();
        params.set_string_member ("query", query);

        var response = yield this.send_request ("workspace/symbol", params, cancellable);

        if (response == null || !response.has_member ("result"))
            return null;

        return parse_workspace_lsp_symbols (response.get_member ("result"));
    }

    public Gee.List<WorkspaceLspSymbol> parse_workspace_lsp_symbols (Json.Node root_node) {
        var result = new Gee.ArrayList<WorkspaceLspSymbol> ();

        //      ,      -
        if (root_node.get_node_type () != Json.NodeType.ARRAY) return result;

        var array = root_node.get_array ();

        foreach (var element in array.get_elements ()) {
            var obj = element.get_object ();
            var symbol = new WorkspaceLspSymbol ();

            symbol.name = obj.get_string_member ("name");
            symbol.kind = (SymbolKind) obj.get_int_member ("kind");

            if (obj.has_member ("containerName")) {
                symbol.container_name = obj.get_string_member ("containerName");
            }

            //      Location (URI   Range)
            var location = obj.get_object_member ("location");
            symbol.uri = location.get_string_member ("uri");

            var range = location.get_object_member ("range");
            var start = range.get_object_member ("start");

```

```

        symbol.start_line = (int) start.get_int_member ("line");
        symbol.start_char = (int) start.get_int_member ("character");

        result.add (symbol);
    }

    return result;
}

public async Gee.ArrayList<LspLocation>? request_definition (string uri, int line, int character) {
    var params = new Json.Object ();

    var doc = new Json.Object ();
    doc.set_string_member ("uri", uri);
    params.set_object_member ("textDocument", doc);

    var pos = new Json.Object ();
    pos.set_int_member ("line", line);
    pos.set_int_member ("character", character);
    params.set_object_member ("position", pos);

    var response = yield this.send_request ("textDocument/definition", params);

    if (response == null || !response.has_member ("result"))return null;

    var result_node = response.get_member ("result");
    if (result_node.get_node_type () == Json.NodeType.NULL)return null;

    return parse_definition_result (result_node);
}

private Gee.ArrayList<LspLocation> parse_definition_result (Json.Node node) {
    var locations = new Gee.ArrayList<LspLocation> ();

    //      1:      Location {}
    if (node.get_node_type () == Json.NodeType.OBJECT) {
        var loc = parse_single_location (node.get_object ());
        if (loc != null)locations.add (loc);
    }
    //      2:      Location[]
    else if (node.get_node_type () == Json.NodeType.ARRAY) {
        var array = node.get_array ();
        foreach (var element in array.get_elements ()) {
            var loc = parse_single_location (element.get_object ());
            if (loc != null)locations.add (loc);
        }
    }
}

```

```

    }

    return locations;
}

private LspLocation ? parse_single_location (Json.Object obj) {
    var loc = new LspLocation ();
    loc.uri = obj.get_string_member ("uri");

    var range = obj.get_object_member ("range");
    var start = range.get_object_member ("start");
    var end = range.get_object_member ("end");

    loc.start_line = (int) start.get_int_member ("line");
    loc.start_column = (int) start.get_int_member ("character");
    loc.end_line = (int) end.get_int_member ("line");
    loc.end_column = (int) end.get_int_member ("character");

    return loc;
}

public Gee.List<Iide.DocumentLspSymbol> parse_document_lsp_symbols (Json.Node root_node) {
    var result = new Gee.ArrayList<Iide.DocumentLspSymbol> ();

    // ( )
    if (root_node.get_node_type () != Json.NodeType.ARRAY) return result;

    var array = root_node.get_array ();
    foreach (var element in array.get_elements ()) {
        if (element.get_node_type () != Json.NodeType.OBJECT) continue;

        var symbol = parse_single_document_lsp_symbol (element.get_object (), null);
        result.add (symbol);
    }

    return result;
}

private Iide.DocumentLspSymbol? parse_single_document_lsp_symbol (Json.Object obj, string uri) {
    // 1.
    var symbol = new Iide.DocumentLspSymbol ();
    if (symbol == null) return null;

    // 2.
    string name = "Unknown";
    if (obj.has_member ("name")) {

```

```

        name = obj.get_string_member ("name");
    }
    symbol.name = name; // notify

    if (obj.has_member ("kind")) {
        symbol.kind = (SymbolKind) obj.get_int_member ("kind");
    }

    symbol.container_name = parent_name;

    // 3. ( )
    Json.Object? range_obj = null;
    if (obj.has_member ("selectionRange")) range_obj = obj.get_object_member ("selectionRange");
    else if (obj.has_member ("range")) range_obj = obj.get_object_member ("range");

    if (range_obj != null) {
        if (range_obj.has_member ("start")) {
            var start = range_obj.get_object_member ("start");
            symbol.start_line = (int) start.get_int_member ("line");
            symbol.start_char = (int) start.get_int_member ("character");
        }
    }

    // 4. ( )
    if (obj.has_member ("children")) {
        var children_node = obj.get_member ("children");
        if (children_node != null && !children_node.is_null() && children_node.get_node_type() == Json.NodeType.OBJECT) {
            var children_array = children_node.get_array();
            foreach (var child_node in children_array.get_elements()) {
                // , get_object()
                if (child_node != null && child_node.get_node_type() == Json.NodeType.OBJECT) {
                    var child_symbol = parse_single_document_lsp_symbol (child_node.get_object());
                    if (child_symbol != null) {
                        symbol.children.add (child_symbol);
                    }
                }
            }
        }
    }

    return symbol;
}
}

```

File: src/Services/LSP/LspTypes.vala

```
public enum Iide.CompletionTriggerKind {
    /**
     * ( , Ctrl+Space)
     * , .
     */
    INVOKED = 1,

    /**
     * -
     * ( , '.', ':', '->').
     */
    TRIGGER_CHARACTER = 2,

    /**
     * ( ,
     * 'incomplete').
     */
    TRIGGER_FOR_INCOMPLETE_COMPLETIONS = 3
}

public enum Iide.LspCompletionKind {
    TEXT = 1,
    METHOD = 2,
    FUNCTION = 3,
    CONSTRUCTOR = 4,
    FIELD = 5,
    VARIABLE = 6,
    CLASS = 7,
    INTERFACE = 8,
    MODULE = 9,
    PROPERTY = 10,
    UNIT = 11,
    VALUE = 12,
    ENUM = 13,
    KEYWORD = 14,
    SNIPPET = 15,
    COLOR = 16,
    FILE = 17,
    REFERENCE = 18,
    FOLDER = 19,
    ENUM_MEMBER = 20,
    CONSTANT = 21,
    STRUCT = 22,
    EVENT = 23,
```

```

        OPERATOR = 24,
        TYPE_PARAMETER = 25;
    }

    public class Iide.LspCompletionItem : GLib.Object {
        public string label { get; set; default = ""; }
        public string? detail { get; set; }
        public string? documentation { get; set; }
        public int sort_text_priority { get; set; default = 0; }
        public int insert_text_priority { get; set; default = 0; }
        public string insert_text { get; set; default = ""; }
        public string? text_edit { get; set; }
        public int start_line { get; set; default = 0; }
        public int start_column { get; set; default = 0; }
        public int end_line { get; set; default = 0; }
        public int end_column { get; set; default = 0; }
        public LspCompletionKind kind { get; set; default = LspCompletionKind.TEXT; }
    }

    public class Iide.LspDiagnostic : GLib.Object {
        public int severity { get; set; default = 1; }
        public string message { get; set; default = ""; }
        public int start_line { get; set; default = 0; }
        public int start_column { get; set; default = 0; }
        public int end_line { get; set; default = 0; }
        public int end_column { get; set; default = 0; }

        public string to_string () {
            return "Diagnostic: (%d:%d-%d:%d) %s".printf (start_line, start_column, end_line, end_column, message);
        }
    }

    public class Iide.LspLocation : GLib.Object {
        public string uri { get; set; }
        public int start_line { get; set; }
        public int start_column { get; set; }
        public int end_line { get; set; }
        public int end_column { get; set; }
    }

    public class Iide.LspCompletionResult : GLib.Object {
        public Gee.ArrayList<LspCompletionItem> items { get; set; }
        public bool is_incomplete { get; set; default = false; }
    }

    public enum Iide.SymbolKind {

```

```

        FILE = 1, MODULE = 2, NAMESPACE = 3, PACKAGE = 4,
        CLASS = 5, METHOD = 6, PROPERTY = 7, FIELD = 8,
        CONSTRUCTOR = 9, ENUM = 10, INTERFACE = 11,
        FUNCTION = 12, VARIABLE = 13, CONSTANT = 14,
        STRING = 15, NUMBER = 16, BOOLEAN = 17,
        ARRAY = 18, OBJECT = 19, KEY = 20, NULL = 21,
        ENUM_MEMBER = 22, STRUCT = 23, EVENT = 24,
        OPERATOR = 25, TYPE_PARAMETER = 26;
    }

    public class Iide.WorkspaceLspSymbol : Object {
        public string name { get; set; }
        public SymbolKind kind { get; set; }
        public string uri { get; set; }
        public int start_line { get; set; }
        public int start_char { get; set; }
        public string? container_name { get; set; }
    }

    public class Iide.DocumentLspSymbol : Object {
        public string name { get; set; }
        public SymbolKind kind { get; set; }
        public int start_line { get; set; }
        public int start_char { get; set; }
        public string? container_name { get; set; }
        public Gee.List<DocumentLspSymbol> children { get; set; default = new Gee.ArrayList<Docu
    }

```

File: src/Services/ImageFactory.vala

```

/*
 */

public class Iide.ImageFactory {
    public static Gtk.Widget create_for_ts (string type) {
        SymbIconProvider _provider = SymbIconProvider.get_instance ();

        //      Tree-sitter
        if (type.contains ("class")) {
            return _provider.image (IconID.LS_CLASS);
        } else if (type.contains ("struct")) {
            return _provider.image (IconID.LS_STRUCT);
        } else if (type.contains ("method") || type.contains ("function")) {
            return _provider.image (IconID.LS_METHOD);
        } else if (type.contains ("field") || type.contains ("variable") || type.contains ("

```



```

        return _provider.image (IconID.LS_FIELD);
    } else if (type.contains ("enum")) {
        return _provider.image (IconID.LS_ENUM);
    } else if (type.contains ("interface")) {
        return _provider.image (IconID.LS_INTERFACE);
    }

    return _provider.image (IconID.LS_DEFAULT);
}

public static Gtk.Widget create_for_completion (Iide.LspCompletionKind kind) {
    SymbIconProvider _provider = SymbIconProvider.get_instance ();

    switch (kind) {
    case TEXT:
        return _provider.image (IconID.LS_TEXT);
    case METHOD: case FUNCTION:
        return _provider.image (IconID.LS_METHOD);
    case CONSTRUCTOR:
        return _provider.image (IconID.LS_CONSTRUCTOR);
    case FIELD: case VARIABLE:
        return _provider.image (IconID.LS_FIELD);
    case CLASS:
        return _provider.image (IconID.LS_CLASS);
    case INTERFACE:
        return _provider.image (IconID.LS_INTERFACE);
    case MODULE:
        return _provider.image (IconID.LS_MODULE);
    case PROPERTY:
        return _provider.image (IconID.LS_PROPERTY);
    case ENUM:
        return _provider.image (IconID.LS_ENUM);
    case KEYWORD:
        return _provider.image (IconID.LS_KEYWORD);
    case SNIPPET:
        return _provider.image (IconID.LS_SNIPPET);
    case COLOR:
        return _provider.image (IconID.LS_COLOR);
    case FILE:
        return _provider.image (IconID.LS_FILE);
    case FOLDER:
        return _provider.image (IconID.LS_FOLDER);
    case CONSTANT:
        return _provider.image (IconID.LS_CONSTANT);
    case STRUCT:
        return _provider.image (IconID.LS_STRUCT);
    }
}

```

```

        case OPERATOR:
            return _provider.image (IconID.LS_OPERATOR);
        default:
            return _provider.image (IconID.LS_DEFAULT);
    }
}

public static Gtk.Widget create_for_symbol (Iide.SymbolKind kind) {
    SymbIconProvider _provider = SymbIconProvider.get_instance ();

    switch (kind) {
        case FILE:
            return _provider.image (IconID.LS_FILE);
        case MODULE: case NAMESPACE: case PACKAGE:
            return _provider.image (IconID.LS_MODULE);
        case CLASS:
            return _provider.image (IconID.LS_CLASS);
        case METHOD: case FUNCTION:
            return _provider.image (IconID.LS_METHOD);
        case CONSTRUCTOR:
            return _provider.image (IconID.LS_CONSTRUCTOR);
        case PROPERTY: case FIELD:
            return _provider.image (IconID.LS_PROPERTY);
        case VARIABLE: case CONSTANT:
            return _provider.image (IconID.LS_CONSTANT);
        case STRING: case NUMBER: case BOOLEAN:
            return _provider.image (IconID.LS_LITERAL);
        case ENUM:
            return _provider.image (IconID.LS_ENUM);
        case ENUM_MEMBER:
            return _provider.image (IconID.LS_ENUM_MEMBER);
        case STRUCT:
            return _provider.image (IconID.LS_STRUCT);
        case OPERATOR:
            return _provider.image (IconID.LS_OPERATOR);
        case TYPE_PARAMETER:
            return _provider.image (IconID.LS_TYPE_PARAMETER);
        default:
            return _provider.image (IconID.LS_DEFAULT);
    }
}

public static Gtk.Image folder_image () {
    SymbIconProvider _provider = SymbIconProvider.get_instance ();
    return _provider.image (IconID.FOLDER);
}

```

```

private static string filename_extension (string basename) {
    //
    int dot_index = basename.last_index_of_char ('.');
    if (dot_index != -1) {
        return basename.substring (dot_index);
    }
    return basename;
}

private static string get_mime_type (GLib.File file) {
    try {
        //
        var info = file.query_info (FileAttribute.STANDARD_CONTENT_TYPE, FileQueryInfoFlags.NONE, null);
        var content_type = info.get_content_type ();
        return ContentType.get_mime_type (content_type);
    } catch (Error e) {
        return "application/octet-stream";
    }
}

public static Gtk.Image create_for_file (GLib.File file) {
    SymbIconProvider _provider = SymbIconProvider.get_instance ();
    return _provider.image (icon_id_for_file (file));
}

public static Gtk.Image create_for_file_info (GLib.FileInfo info) {
    SymbIconProvider _provider = SymbIconProvider.get_instance ();
    return _provider.image (icon_id_for_file_info (info));
}

public static string icon_name_for_file (GLib.File file) {
    SymbIconProvider _provider = SymbIconProvider.get_instance ();
    return _provider.icon_name (icon_id_for_file (file));
}

public static string icon_name_for_id (IconID icon_id) {
    SymbIconProvider _provider = SymbIconProvider.get_instance ();
    return _provider.icon_name (icon_id);
}

private static IconID ? icon_id_for_extension (string extension) {
    switch (extension) {
        // ---          /          ---
        case "makefile" : case ".mk": case ".build":
            return IconID.MT_MAKE;
    }
}

```

```

        case "dockerfile": case ".dockerfile":
            return IconID.MT_DOCKER;
        case ".json":
            return IconID.MT_JSON;
        case ".xml": case ".ui": case ".glade":
            return IconID.MT_XML;
        case ".yaml": case ".yml":
            return IconID.MT_YAML;
        case ".conf": case ".ini":
            return IconID.MT_INI;
        case ".md": case ".markdown":
            return IconID.MT_MARKDOWN;
        case ".toml":
            return IconID.MT_TOML;
        // --- ---
        case ".vala": case ".vapi":
            return IconID.MT_VALA;
        case ".c":
            return IconID.MT_C;
        case ".h": case ".hh": case ".hpp":
            return IconID.MT_H;
        case ".cpp": case ".cc":
            return IconID.MT_CPP;
        case ".py": case ".pyi": case ".pyw":
            return IconID.MT_PY;
        case ".js":
            return IconID.MT_JS;
        case ".ts":
            return IconID.MT_TS;
        case ".css":
            return IconID.MT_CSS;
        case ".html":
            return IconID.MT_HTML;
        case ".sh": case ".zsh": case ".bash":
            return IconID.MT_BASH;
        case ".rs":
            return IconID.MT_RUST;
        case ".go":
            return IconID.MT_GO;
        case ".lua":
            return IconID.MT_MAKE;
    }
    return null;
}

private static IconID icon_id_for_file_info (GLib.FileInfo info) {

```

```

        string extension = filename_extension (info.get_name ());
        return icon_id_for_extension (extension) ?? IconID.MT_DEFAULT;
    }

private static IconID icon_id_for_file (GLib.File file) {
    string basename = file.get_basename ().down ();
    string extension = filename_extension (basename);

    var icon_id = icon_id_for_extension (extension);
    if (icon_id != null) {
        return icon_id;
    }

    // ---      (MIME-      ) ---
    string mime = get_mime_type (file);
    if (mime.has_prefix ("text/")) {
        return IconID.MT_X_TEXT;
    } else if (mime.has_prefix ("image/")) {
        return IconID.MT_X_IMAGE;
    } else if (mime.has_prefix ("audio/")) {
        return IconID.MT_X_AUDIO;
    } else if (mime.has_prefix ("video/")) {
        return IconID.MT_X_VIDEO;
    } else if (mime.has_prefix ("application/")) {
        return IconID.MT_X_APP;
    }

    return IconID.MT_DEFAULT;
}
}

```

File: src/Widgets/Find/SearchResultsView.vala

```

public class Iide.SearchResultItem : Gtk.Box {
    private Gtk.Label name_label;
    private Gtk.Label path_label;
    private Gtk.Box icon_box;

    public SearchResultItem() {
        Object(orientation: Gtk.Orientation.HORIZONTAL, spacing: 12);
        this.margin_start = 8;
        this.margin_end = 8;
        this.margin_top = 6;
        this.margin_bottom = 6;
    }
}

```

```

        icon_box = new Gtk.Box(Gtk.Orientation.VERTICAL, 0);
        icon_box.set_size_request(20, 20);

        var text_box = new Gtk.Box(Gtk.Orientation.VERTICAL, 2);
        text_box.hexpand = true;

        name_label = new Gtk.Label(null);
        name_label.xalign = 0;
        name_label.add_css_class("title-5");
        name_label.hexpand = true;

        path_label = new Gtk.Label(null);
        path_label.add_css_class("dim-label");
        path_label.add_css_class("caption");
        path_label.xalign = 0;
        path_label.ellipsize = Pango.EllipsizeMode.START;
        path_label.hexpand = true;

        text_box.append(name_label);
        text_box.append(path_label);

        this.append(icon_box);
        this.append(text_box);
    }

private string highlight_matches(string text, Gee.List<MatchRange>? matches) {
    var escaped = escape_pango(text);

    if (matches == null || matches.size == 0) {
        return escaped;
    }

    var sb = new StringBuilder();
    int pos = 0;

    foreach (var m in matches) {
        if (m.start > pos) {
            sb.append(escaped.substring(pos, m.start - pos));
        }
        if (m.end > m.start && m.end <= (int) escaped.length) {
            sb.append("<span weight=\"bold\" background=\"#ffd700\" color=\"#000000\">");
            sb.append(escaped.substring(m.start, m.end - m.start));
            sb.append("</span>");
        }
        pos = m.end;
    }
}

```

```

        if (pos < (int) escaped.length) {
            sb.append(escaped.substring(pos));
        }

        return sb.str;
    }

    public void bind_search_result(SearchResult result) {
        var highlighted_name = highlight_matches(result.line_content, result.matches);
        var line_prefix = result.line_number == -1 ? "" : (result.line_number + 1).to_string;
        name_label.set_markup(line_prefix + highlighted_name);
        path_label.set_label(result.relative_path);
        if (result.icon_widget == null) {
            icon_box.hide();
        } else {
            icon_box.append(result.icon_widget);
        }
    }
}

//          Box -
public class Iide.SearchResultsView : Gtk.Box {
    private DocumentManager document_manager = DocumentManager.get_instance();
    public Gtk.ListView list_view; // ListView
    public Gtk.SingleSelection selection;
    private GLib.ListStore results;

    private const int MAX_RESULTS = 100;

    public SearchResultsView() {
        Object(orientation: Gtk.Orientation.VERTICAL, spacing: 0);

        results = new GLib.ListStore(typeof (SearchResult));
        selection = new Gtk.SingleSelection(results);

        var factory = new Gtk.SignalListItemFactory();
        factory.setup.connect((item) => {
            var list_item = item as Gtk.ListItem;
            list_item.child = new SearchResultItem();
        });

        factory.bind.connect((item) => {
            var list_item = item as Gtk.ListItem;
            var item_box = list_item.child as SearchResultItem;
            var result = (SearchResult) list_item.item;

```

```

        item_box.bind_search_result(result);
    });

    //      ListView
    list_view = new Gtk.ListView(selection, factory);
    list_view.hexpand = true;
    list_view.vexpand = true;
    list_view.show_separators = true;

    //      ListView      Box
    var scrolled = new Gtk.ScrolledWindow();
    scrolled.child = list_view;
    this.append(scrolled);
}

//      'this'      'list_view'
public void select_up() {
    if (selection.selected > 0) {
        selection.selected -= 1;
        list_view.scroll_to(selection.selected, Gtk.ListScrollFlags.NONE, null);
    }
}

public void select_down() {
    if (selection.selected < (int) results.n_items - 1) {
        selection.selected += 1;
        list_view.scroll_to(selection.selected, Gtk.ListScrollFlags.NONE, null);
    }
}

public void update_results(Gee.List<SearchResult>? new_results) {
    if (new_results == null || new_results.size == 0) {
        results.splice(0, results.n_items, new SearchResult[0]);
        return;
    }

    var show_count = new_results.size;
    if (show_count > MAX_RESULTS) {
        show_count = MAX_RESULTS;
    }

    var items = new SearchResult[show_count];
    for (var i = 0; i < show_count; i++) {
        items[i] = new_results[i];
    }
    update_result_list(items);
}

```



```

    }

    public void update_result_list(SearchResult[] items) {
        results.splice(0, results.n_items, items);
        if (items.length > 0) {
            selection.selected = 0;
        }
    }

    public bool open_selected() {
        var index = (int) selection.selected;
        if (index >= 0 && index < results.n_items) {
            var result = results.get_item(index) as SearchResult;
            var file = GLib.File.new_for_path(result.file_path);

            int start_col = 0;
            int end_col = 0;
            if (result.matches != null && result.matches.size > 0) {
                start_col = result.matches[0].start;
                end_col = result.matches[0].end;
            }

            document_manager.open_document_with_selection(file, result.line_number, start_col, end_col);
            return true;
        }
        return false;
    }
}

```

File: src/window.vala

```

/* window.vala
 *
 * Copyright 2026 kai
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU General Public License for more details.
 *
 */

```

```

* You should have received a copy of the GNU General Public License
* along with this program.  If not, see <https://www.gnu.org/licenses/>.
*
* SPDX-License-Identifier: GPL-3.0-or-later
*/

using Gtk;
using Adw;
using Panel;

public class Iide.Window : Panel.DocumentWorkspace {

    private Iide.DocumentManager document_manager;
    private Iide.ProjectManager project_manager;
    private Iide.SettingsService settings;

    private Gtk.Button lsp_btn;
    private Gtk.Spinner lsp_spin;
    private Gtk.Label lsp_count;
    private Gtk.Popover lsp_popover;
    private Gtk.Box lsp_list_box;

    private Gtk.Button global_diag_btn;
    private Gtk.Label global_diag_label;
    private Gtk.Image global_diag_icon;
    private DiagnosticsPanel panel_widget_diagnostics;
    private string app_error_icon_name;

    private BasePanel[] panel_widgets;

    public Window (Gtk.Application app) {
        Object (application: app);
        GtkSource.init ();
    }

    construct {
        settings = Iide.SettingsService.get_instance ();
        document_manager = new Iide.DocumentManager (this);
        project_manager = Iide.ProjectManager.get_instance ();
        document_manager.document_opened.connect ((widget) => {
            grid.add (widget);
            widget.raise ();
            widget.view_grab_focus ();
        });

        project_manager.project_opened.connect ((project_root) => {

```

```

        document_manager.set_workspace_root (project_root.get_uri ());
    });

project_manager.project_closed.connect (() => {
    document_manager.set_workspace_root (null);
});

// Header
var header = new Adw.HeaderBar ();
var menu_button = new Gtk.MenuButton ();
menu_button.icon_name = "open-menu-symbolic";

var menu = new GLib.Menu ();
menu.append (_("Open Project"), "app.open-project");
menu.append (_("Quick Open"), "app.fuzzy-finder");
menu.append (_("Search Symbol"), "app.search-symbol");
menu.append (_("Search in Files"), "app.search-in-files");
menu.append (_("Save All"), "app.save");
menu.append (_("Preferences"), "app.preferences");
menu.append (_("About"), "app.about");
menu.append (_("Quit"), "app.quit");
menu_button.set_menu_model (menu);

header.pack_end (menu_button);

var panel_layout = settings.panel_layout;
if (panel_layout != null && panel_layout != "") {
    Iide.PanelLayoutHelper.deserialize_dock (panel_layout, dock);
} else {
    dock.reveal_start = settings.reveal_start_panel;
    dock.start_width = settings.panel_start_width;
    dock.reveal_end = settings.reveal_end_panel;
    dock.end_width = settings.panel_end_width;
    dock.reveal_bottom = settings.reveal_bottom_panel;
    dock.bottom_height = settings.panel_bottom_height;
}
var start_toggle_btn = new Panel.ToggleButton (dock, Panel.Area.START);
header.pack_start (start_toggle_btn);

setup_navigation_buttons (header);

var end_toggle_btn = new Panel.ToggleButton (dock, Panel.Area.END);
header.pack_end (end_toggle_btn);

set_titlebar (header);

```

```

// Theme switcher
var style_manager = Adw.StyleManager.get_default ();
style_manager.color_scheme = settings.color_scheme.to_adw_color_scheme ();

var theme_list = new Gtk.StringList ({ "System", "Light", "Dark" });
var expr = new Gtk.PropertyExpression (typeof (Gtk.StringObject), null, "string");
var theme_dropdown = new Gtk.DropDown (theme_list, expr) {
    selected = (uint) settings.color_scheme,
    tooltip_text = _("Color Scheme"),
    show_arrow = false
};

var factory = new Gtk.SignalListItemFactory ();
factory.setup.connect ((item) => {
    var list_item = item as Gtk.ListItem;
    var box = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 6);
    var icon = new Gtk.Image ();
    var label = new Gtk.Label (null);
    label.xalign = 0;
    box.append (icon);
    box.append (label);
    list_item.set_child (box);
});
factory.bind.connect ((item) => {
    var list_item = item as Gtk.ListItem;
    var box = list_item.get_child () as Gtk.Box;
    var icon = box.get_first_child () as Gtk.Image;
    var label = icon.get_next_sibling () as Gtk.Label;
    var obj = list_item.get_item () as Gtk.StringObject;
    var text = obj.get_string ();
    label.set_label (text);

    string icon_name;
    switch (text) {
        case "System":
            icon_name = "weather-overcast-symbolic";
            break;
        case "Light":
            icon_name = "weather-clear-symbolic";
            break;
        case "Dark":
            icon_name = "weather-clear-night-symbolic";
            break;
        default:
            icon_name = "image-missing-symbolic";
            break;
    }
}

```

```

    }
    icon.icon_name = icon_name;
  });
  theme_dropdown.set_factory (factory);

  theme_dropdown.notify["selected"].connect (() => {
    var scheme = (ColorScheme) theme_dropdown.selected;
    settings.color_scheme = scheme;
    style_manager.color_scheme = scheme.to_adw_color_scheme ();

    //                                ,      CSS
    if (scheme == ColorScheme.DARK) {
      this.add_css_class ("dark");
    } else {
      this.remove_css_class ("dark");
    }
  });
  if (settings.color_scheme == ColorScheme.DARK) {
    this.add_css_class ("dark");
  }

  header.pack_end (theme_dropdown);

  // statusbar (                                layout)

  create_panels ();

  //                                layout
  restore_panels_layout ();

  //      toggle button      BOTTOM                                layout
  var bottom_toggle_btn = new Panel.ToggleButton (dock, Panel.Area.BOTTOM);
  statusbar.add_suffix (1, bottom_toggle_btn);

  setup_lsp_status ();
  setup_global_diag_widget ();

  project_manager.open_project_by_path (settings.current_project_path);
  restore_opened_documents ();

  // Handle window close
  this.close_request.connect (() => {
    save_window_settings ();
    settings.open_documents = document_manager.get_open_document_uris ();
    bool has_unsaved = false;
    foreach (var entry in document_manager.documents.entries) {

```

```

        if (entry.value is Iide.TextView) {
            var tv = (Iide.TextView) entry.value;
            if (tv.is_modified) {
                has_unsaved = true;
                break;
            }
        }
    }
    if (has_unsaved) {
        var dialog = new Adw.AlertDialog (_, "Unsaved Changes"), _("You have unsaved
        dialog.add_response ("cancel", _("Cancel"));
        dialog.add_response ("discard", _("Discard"));
        dialog.add_response ("save", _("Save"));
        dialog.set_response_appearance ("save", Adw.ResponseAppearance.SUGGESTED);
        dialog.response.connect ((response) => {
            if (response == "save") {
                foreach (var entry in document_manager.documents.entries) {
                    if (entry.value is Iide.TextView) {
                        var tv = (Iide.TextView) entry.value;
                        if (tv.is_modified) {
                            tv.save ();
                        }
                    }
                }
                settings.open_documents = document_manager.get_open_document_uris ();
                this.destroy ();
            } else if (response == "discard") {
                settings.open_documents = document_manager.get_open_document_uris ();
                this.destroy ();
            }
        });
        dialog.present (this);
        return true;
    }
    return false;
});

repair_empty_areas ();
}

private void restore_opened_documents () {
    var grid_data = settings.grid_layout;
    bool has_grid_docs = grid_data != null && grid_data != "";
    if (has_grid_docs) {
        restore_documents_from_grid_data (grid_data);
    } else {

```

```

        var open_docs = settings.open_documents;
        bool has_open_docs = open_docs.size > 0;
        if (has_open_docs) {
            foreach (var uri in open_docs) {
                document_manager.open_document_by_uri (uri);
            }
        }
    }

    Timeout.add (300, () => {
        NavigationHistoryService.get_instance ().start_navigation ();
        return Source.REMOVE;
    });
}

private void setup_navigation_buttons (Adw.HeaderBar header) {
    //
    var back_btn = new Gtk.Button.from_icon_name ("go-previous-symbolic");
    back_btn.tooltip_text = "    (Alt+Left)";
    back_btn.action_name = "app.navigation_back"; //          Action

    //
    var forward_btn = new Gtk.Button.from_icon_name ("go-next-symbolic");
    forward_btn.tooltip_text = "    (Alt+Right)";
    forward_btn.action_name = "app.navigation_forward";

    header.pack_start (back_btn);
    header.pack_start (forward_btn);
}

private void repair_empty_areas () {
    bool empty_start = true;
    bool empty_bottom = true;
    bool empty_end = true;
    dock.foreach_frame ((frame) => {
        var position = frame.get_position ();
        switch (position.area) {
            case Panel.Area.START:
                empty_start = false;
                break;
            case Panel.Area.BOTTOM:
                empty_bottom = false;
                break;
            case Panel.Area.END:
                empty_end = false;
                break;
        }
    });
}

```

```

        default:
            break;
    }
});

if (empty_start) {
    var position = new Panel.Position () {
        area = Panel.Area.START
    };
    var tmp_panel = new Panel.Widget ();
    add_widget (tmp_panel, position);
    dock.remove (tmp_panel);
}

if (empty_bottom) {
    var position = new Panel.Position () {
        area = Panel.Area.BOTTOM
    };
    var tmp_panel = new Panel.Widget ();
    add_widget (tmp_panel, position);
    dock.remove (tmp_panel);
}

if (empty_end) {
    var position = new Panel.Position () {
        area = Panel.Area.END
    };
    var tmp_panel = new Panel.Widget ();
    add_widget (tmp_panel, position);
    dock.remove (tmp_panel);
}
}

private void create_panels () {
    panel_widget_diagnostics = new DiagnosticsPanel ();

    panel_widgets = {
        new ProjectPanel (),
        new TerminalPanel (),
        new LogPanel (),
        panel_widget_diagnostics
    };
}

private void initialize_panels () {
    foreach (var panel in panel_widgets) {

```



```

        add_widget (panel, panel.initial_pos ());
    }
}

private void restore_panels_layout () {
    var dock_layout = settings.panel_layout;
    if (dock_layout == null || dock_layout == "") {
        initialize_panels ();
        return;
    }

    var widget_layouts = Iide.PaneLayoutHelper.parse_widgets (dock_layout);

    foreach (var panel_widget in panel_widgets) {
        var widget_layout = widget_layouts.get (panel_widget.panel_id ());
        var pos = widget_layout != null? widget_layout.to_pos () : panel_widget.initial_pos ();

        add_widget (panel_widget, pos);
    }
}

private void save_window_settings () {
    settings.panel_layout = Iide.PaneLayoutHelper.serialize_dock (dock);
    var grid_json = Iide.PaneLayoutHelper.serialize_grid (grid);
    settings.grid_layout = grid_json;

    bool maximized = false;
    var surface = this.get_surface ();
    if (surface != null) {
        var toplevel = surface as Gdk.Toplevel;
        if (toplevel != null) {
            var state = toplevel.get_state ();
            maximized = (state & Gdk.ToplevelState.MAXIMIZED) != 0;
        }
    }

    if (!maximized) {
        settings.window_width = (int) this.get_width ();
        settings.window_height = (int) this.get_height ();
    }
    settings.window_maximized = maximized;
}

private void restore_grid_documents (Gee.ArrayList<Iide.PaneLayoutHelper.DocumentInfo> docs) {
    uint last_col = 0;
    foreach (var doc_info in sorted_docs) {

```

```

        if (doc_info.column > last_col) {
            last_col = doc_info.column;
        }
    }

    for (uint i = 1; i <= last_col; i++) {
        grid.insert_column (i);
    }

    foreach (var doc_info in sorted_docs) {
        var file = GLib.File.new_for_uri (doc_info.uri);
        var pos = new Panel.Position ();
        pos.area = Panel.Area.CENTER;
        pos.column = doc_info.column;
        pos.row = doc_info.row;
        document_manager.open_document (file, pos);
    }
}

private void restore_documents_from_grid_data (string grid_data) {
    var docs = Iide.PanelLayoutHelper.parse_grid_documents (grid_data);
    if (docs.size == 0) {
        return;
    }

    var sorted_docs = new Gee.ArrayList<Iide.PanelLayoutHelper.DocumentInfo> ();
    foreach (var doc in docs) {
        sorted_docs.add (doc);
    }
    sorted_docs.sort ((a, b) => {
        int col_cmp = (int) (a.column - b.column);
        if (col_cmp != 0) return col_cmp;
        return (int) (a.row - b.row);
    });

    restore_grid_documents (sorted_docs);
}

public void save_modified () {
    foreach (var entry in document_manager.documents.entries) {
        var widget = entry.value;
        if (widget is Iide.TextView) {
            var tv = widget as Iide.TextView;
            if (tv.is_modified) {
                tv.save ();
            }
        }
    }
}

```

```

        }
    }
}

public void open_project_dialog () {
    project_manager.open_project_dialog.begin (this);
}

public Iide.DocumentManager get_document_manager () {
    return document_manager;
}

public SourceView ? get_active_source_view () {
    // 1.
    Panel.Widget? active_widget = this.get_grid ().get_most_recent_frame ().get_visible

    if (active_widget == null)return null;

    return (active_widget as TextView) ? .source_view;
}

private void setup_lsp_status () {
    // 1.          Statusbar
    var btn_content = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 6);
    lsp_spin = new Gtk.Spinner ();
    lsp_count = new Gtk.Label ("0");
    btn_content.append (lsp_spin);
    btn_content.append (lsp_count);

    lsp_btn = new Gtk.Button () { child = btn_content, visible = false };
    lsp_btn.add_css_class ("flat");
    this.statusbar.add_prefix (100, lsp_btn);

    // 2.          Popover
    lsp_popover = new Gtk.Popover ();
    lsp_popover.set_parent (lsp_btn);
    var scroll = new Gtk.ScrolledWindow () {
        max_content_height = 300,
        propagate_natural_height = true,
        width_request = 350 //
    };
    lsp_list_box = new Gtk.Box (Gtk.Orientation.VERTICAL, 0);
    lsp_list_box.add_css_class ("boxed-list"); //      Adwaita

    scroll.set_child (lsp_list_box);
    lsp_popover.set_child (scroll);
}

```

```

lsp_btn.clicked.connect (() => lsp_popover.popup ());

// 3.
IdeLspService.get_instance ().tasks_changed.connect (update_lsp_ui);
}

private void update_lsp_ui (Gee.List<LspTaskInfo?> tasks) {
    //
    Gtk.Widget? child;
    while ((child = lsp_list_box.get_first_child ()) != null)
        lsp_list_box.remove (child);

    if (tasks.size == 0) {
        lsp_btn.hide ();
        lsp_popover.popdown ();
        return;
    }

    lsp_btn.show ();
    lsp_spin.start ();
    lsp_count.label = tasks.size.to_string ();

    foreach (var task in tasks) {
        var row = new Adw.ActionRow () {
            title = task.server_name,
            subtitle = task.message
        };

        if (task.percentage >= 0) {
            var progress = new Gtk.ProgressBar () {
                fraction = task.percentage / 100.0,
                valign = Gtk.Align.CENTER
            };
            progress.add_css_class ("osd"); //
            row.add_suffix (progress);
        }

        lsp_list_box.append (row);
    }
}

private void setup_global_diag_widget () {
    app_error_icon_name = SymbIconProvider.get_instance ().icon_name (IconID.APP_ERROR);
    var content = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 6);
    global_diag_icon = new Gtk.Image.from_icon_name ("emblem-ok-symbolic");
}

```

```

        global_diag_label = new Gtk.Label ("OK");

        content.append (global_diag_icon);
        content.append (global_diag_label);

        global_diag_btn = new Gtk.Button ();
        global_diag_btn.set_child (content);
        global_diag_btn.add_css_class ("flat");

        //
        global_diag_btn.clicked.connect (() => {
            panel_widget_diagnostics.raise ();
        });

        this.statusbar.add_prefix (50, global_diag_btn);

        //
        DiagnosticsService.get_instance ().total_count_changed.connect (update_global_diag_s
    }

    private void update_global_diag_status (int errors, int warns) {
        if (errors == 0 && warns == 0) {
            global_diag_icon.icon_name = "emblem-ok-symbolic";
            global_diag_label.label = "OK";
            global_diag_btn.remove_css_class ("error-state"); //
        } else {
            global_diag_icon.icon_name = app_error_icon_name;
            global_diag_label.label = @"$errors / $warns";
            global_diag_btn.add_css_class ("error-state");
        }
    }
}

```

File: src/Services/SymbIconProvider.vala

```

/*
 * WARNING: THIS FILE IS GENERATED AUTOMATICALLY.
 * DO NOT EDIT THIS FILE MANUALLY.
 * ALL CHANGES WILL BE LOST UPON REGENERATION.
 */

public enum Iide.IconID {
    FOLDER = 0,
    LS_CLASS = 1,
    LS_METHOD = 2,

```

```
LS_FIELD = 3,  
LS_ENUM = 4,  
LS_INTERFACE = 5,  
LS_TEXT = 6,  
LS_CONSTRUCTOR = 7,  
LS_MODULE = 8,  
LS_KEYWORD = 9,  
LS_SNIPPET = 10,  
LS_COLOR = 11,  
LS_FILE = 12,  
LS_FOLDER = 13,  
LS_CONSTANT = 14,  
LS_STRUCT = 15,  
LS_OPERATOR = 16,  
LS_PROPERTY = 17,  
LS_ENUM_MEMBER = 18,  
LS_TYPE_PARAMETER = 19,  
LS_LITERAL = 20,  
LS_DEFAULT = 21,  
MT_MAKE = 22,  
MT_DOCKER = 23,  
MT_JSON = 24,  
MT_XML = 25,  
MT_YAML = 26,  
MT_INI = 27,  
MT_MARKDOWN = 28,  
MT_TOML = 29,  
MT_VALA = 30,  
MT_C = 31,  
MT_H = 32,  
MT_CPP = 33,  
MT_PY = 34,  
MT_JS = 35,  
MT_TS = 36,  
MT_CSS = 37,  
MT_HTML = 38,  
MT_BASH = 39,  
MT_RUST = 40,  
MT_GO = 41,  
MT_LUA = 42,  
MT_X_IMAGE = 43,  
MT_X_AUDIO = 44,  
MT_X_VIDEO = 45,  
MT_X_TEXT = 46,  
MT_X_APP = 47,  
MT_DEFAULT = 48,
```

```

APP_PROJECT = 49,
APP_LOG = 50,
APP_TERMINAL = 51,
APP_ISSUES = 52,
APP_ERROR = 53,
APP_WARNING = 54,
APP_INFO = 55,
TXT_WRAP = 56,
LOG_ERASE = 57,
COD_ERROR = 58,
COD_WARNING = 59,
COD_INFO = 60;
}

public class Ide.SymbIconProvider {
    private static SymbIconProvider? instance = null;
    public static SymbIconProvider get_instance () {
        if (instance == null) {
            instance = new SymbIconProvider ();
        }
        return instance;
    }

    private string[,] _icon_indexes;
    private string _prefix = "def-";

    private SymbIconProvider () {
        _icon_indexes = {
            { "folder", "folder" },
            { "ls-class", "ls-class" },
            { "ls-method", "ls-method" },
            { "ls-field", "ls-field" },
            { "ls-enum", "ls-class" },
            { "ls-interface", "ls-interface" },
            { "ls-text", "" },
            { "ls-constructor", "ls-constructor" },
            { "ls-module", "ls-module" },
            { "ls-keyword", "" },
            { "ls-snippet", "ls-snippet" },
            { "ls-color", "ls-color" },
            { "ls-file", "" },
            { "ls-folder", "ls-folder" },
            { "ls-constant", "" },
            { "ls-struct", "ls-struct" },
            { "ls-operator", "" },
            { "ls-property", "ls-property" },
        }
    }
}

```

```

        { "ls-enum-member", "ls-enum-member" },
        { "ls-type-parameter", "ls-type-parameter" },
        { "ls-literal", "ls-literal" },
        { "ls-default", "" },
        { "mt-make", "mt-make" },
        { "mt-docker", "mt-docker" },
        { "mt-json", "mt-json" },
        { "mt-xml", "mt-xml" },
        { "mt-yaml", "mt-xml" },
        { "mt-ini", "mt-ini" },
        { "mt-markdown", "mt-markdown" },
        { "mt-toml", "mt-toml" },
        { "mt-vala", "mt-vala" },
        { "mt-c", "mt-c" },
        { "mt-h", "mt-h" },
        { "mt-cpp", "mt-cpp" },
        { "mt-py", "mt-py" },
        { "mt-js", "mt-js" },
        { "mt-ts", "mt-ts" },
        { "mt-css", "mt-css" },
        { "mt-html", "mt-html" },
        { "mt-bash", "mt-bash" },
        { "mt-rust", "mt-rust" },
        { "mt-go", "mt-go" },
        { "mt-lua", "mt-lua" },
        { "mt-x-image", "mt-x-image" },
        { "mt-x-audio", "mt-x-audio" },
        { "mt-x-video", "mt-x-video" },
        { "mt-x-text", "mt-x-text" },
        { "mt-x-app", "mt-x-app" },
        { "mt-default", "mt-default" },
        { "app-project", "" },
        { "app-log", "" },
        { "app-terminal", "" },
        { "app-issues", "" },
        { "app-error", "" },
        { "app-warning", "" },
        { "app-info", "" },
        { "txt-wrap", "" },
        { "log-erase", "" },
        { "cod-error", "cod-error" },
        { "cod-warning", "cod-warning" },
        { "cod-info", "cod-info" },
    };
}

public Gtk.Image image (IconID icon_id, bool colorize = true, int? pixel_size = null) {

```



```

        var icon_name = _icon_indexes[(int) icon_id, 0];
        Gtk.Image img = new Gtk.Image.from_icon_name (_prefix + icon_name + "-symbolic");
        if (colorize) {
            var class_name = _icon_indexes[(int) icon_id, 1];
            if (class_name.length > 0) {
                img.add_css_class (_prefix + class_name);
            }
        }
        if (pixel_size != null) {
            img.pixel_size = pixel_size;
        }
        return img;
    }

    public string icon_name (IconID icon_id) {
        var icon_name = _icon_indexes[(int) icon_id, 0];
        return _prefix + icon_name + "-symbolic";
    }
}

```

File: src/Widgets/TextView/EditorStatusBar.vala

```

public class Iide.EditorStatusBar : Gtk.Box {
    private SourceView source_view;
    private Gtk.Label pos_label;
    private Gtk.Label mode_label;

    private Gtk.Label error_label;
    private Gtk.Label warn_label;
    private Gtk.Box diagnostic_box;

    private BreadcrumbsBar new_breadcrumbs;

    private Iide.DiagnosticsPopover diag_popover = null;

    public EditorStatusBar (SourceView source_view) {
        Object (orientation: Gtk.Orientation.HORIZONTAL, spacing: 12);
        this.source_view = source_view;
        this.add_css_class ("editor-status-bar");

        //          : Breadcrumbs
        new_breadcrumbs = new BreadcrumbsBar (source_view);
        this.append (new_breadcrumbs);
        new_breadcrumbs.update_file_path (GLib.File.new_for_uri (source_view.uri),
                                           GLib.File.new_for_path (ProjectManager.get_instanc

```

```

// spacer
var spacer_box = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 2);
spacer_box.hexpand = true;
this.append (spacer_box);

//      :
var info_box = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 12);

mode_label = new Gtk.Label ("INS");
mode_label.add_css_class ("dim-label");
mode_label.height_request = 24;

pos_label = new Gtk.Label ("1:1");

info_box.append (mode_label);
info_box.append (pos_label);
this.append (info_box);

diagnostic_box = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 8);

//      ( )
error_label = new Gtk.Label ("0");
error_label.add_css_class ("error-label"); //      CSS

//      ( )
warn_label = new Gtk.Label ("0");
warn_label.add_css_class ("warning-label");

var icon_provider = SymbIconProvider.get_instance ();

diagnostic_box.append (icon_provider.image (IconID.COD_ERROR));
diagnostic_box.append (error_label);
diagnostic_box.append (icon_provider.image (IconID.COD_WARNING));
diagnostic_box.append (warn_label);

//      -
info_box.prepend (diagnostic_box);
diagnostic_box.hide (); //      ,

init_diagnostics_interaction ();
this.diagnostic_box.add_css_class ("diagnostic-box");
}

private void init_diagnostics_interaction () {
//

```

```

var click_gesture = new Gtk.GestureClick ();

// (pressed)
click_gesture.pressed.connect ((n_press, x, y) => {
    // , Popover
    show_diagnostics_popup ();
});

//
this.diagnostic_box.add_controller (click_gesture);

// ( ) :
this.diagnostic_box.set_cursor (new Gdk.Cursor.from_name ("pointer", null));

// --- 2. (Hover) ---
var motion_controller = new Gtk.EventControllerMotion ();

//
motion_controller.enter.connect ((x, y) => {
    this.diagnostic_box.add_css_class ("hover");
    // " "
    this.diagnostic_box.set_cursor (new Gdk.Cursor.from_name ("pointer", null));
});

//
motion_controller.leave.connect (() => {
    this.diagnostic_box.remove_css_class ("hover");
    //
    this.diagnostic_box.set_cursor (null);
});

this.diagnostic_box.add_controller (motion_controller);
}

private void show_diagnostics_popup () {
    if (this.diag_popover == null) {
        // , diagnostic_box
        this.diag_popover = new Iide.DiagnosticsPopover (this.diagnostic_box, this.source);
    }

    //
    this.diag_popover.refresh ();
    this.diag_popover.popup ();
}

public void update_diagnostics (int errors, int warnings) {

```

```

        if (errors == 0 && warnings == 0) {
            diagnostic_box.hide ();
            return;
        }

        diagnostic_box.show ();
        error_label.label = errors.to_string ();
        warn_label.label = warnings.to_string ();
    }

    public void update_breadcrumbs (Gee.List<SourceNodeItem?> crumbs) {
        new_breadcrumbs.update_breadcrumbs (crumbs);
    }

    public void update_position (int line, int col, int selection_len = 0) {
        if (selection_len > 0) {
            pos_label.label = "%d:%d (%d selected)".printf (line + 1, col + 1, selection_len);
        } else {
            pos_label.label = "%d:%d".printf (line + 1, col + 1);
        }
    }

    public void update_mode (bool overwrite) {
        mode_label.label = overwrite ? "OVR" : "INS";
    }
}

```

File: src/Widgets/TextView/BreadcrumbTreeSitterNavigator.vala

```

public class Iide.BreadcrumbTreeSitterNavigator : Gtk.Box {
    private SourceView source_view;
    private Gee.List<SourceNodeItem?> siblings;
    public Gtk.SearchEntry search_entry;
    private Gtk.ListBox list_box;

    public signal void close_requested ();

    private class BreadcrumbObject : Object {
        public SourceNodeItem item;
        public BreadcrumbObject (SourceNodeItem item) {
            Object ();
            this.item = item;
        }
    }
}

```

```

public BreadcrumbTreeSitterNavigator (SourceView source_view, Gee.List<SourceNodeItem?>
    Object (orientation: Gtk.Orientation.VERTICAL, spacing: 6);
    this.source_view = source_view;
    this.siblings = siblings;
    this.set_size_request (280, -1);
    this.margin_top = 6;
    this.margin_bottom = 6;
    this.margin_start = 6;
    this.margin_end = 6;

    // ---      ---
    search_entry = new Gtk.SearchEntry ();
    search_entry.hexpand = true;
    this.append (search_entry);

    // ---      ---
    list_box = new Gtk.ListBox ();
    list_box.add_css_class ("navigation-sidebar");

    foreach (var item in siblings) {
        list_box.append (create_row (item));
    }

    //
    list_box.set_filter_func ((row) => {
        var text = search_entry.get_text ().down ();
        if (text == "")return true;

        var obj = row.get_data<BreadcrumbObject> ("item");
        var name = obj.item.name.down ();
        return name.contains (text);
    });
    search_entry.search_changed.connect (() => {
        list_box.invalidate_filter ();
        refresh_state ();
    });

    list_box.row_activated.connect (on_row_activated);
    search_entry.activate.connect (() => {
        on_row_activated (list_box.get_selected_row ());
    });

    var scroll = new Gtk.ScrolledWindow ();
    scroll.propagate_natural_height = true;
    scroll.set_max_content_height (400);
    scroll.set_child (list_box);

```

```

        this.append (scroll);

        var key_controller = new Gtk.EventControllerKey ();
        key_controller.key_pressed.connect (on_key_pressed);
        search_entry.add_controller (key_controller);
        list_box.set_selection_mode (Gtk.SelectionMode.SINGLE);

        this.refresh_state ();
    }

    private void select_first_visible_row () {
        var row = list_box.get_first_child ();
        while (row != null) {
            var list_row = row as Gtk.ListBoxRow;
            if (list_row.get_child_visible ()) {
                list_box.select_row (list_row);
                break;
            }
            row = row.get_next_sibling ();
        }
    }

    private void refresh_state () {
        this.search_entry.grab_focus ();
        select_first_visible_row ();
    }

    private bool on_key_pressed (Gtk.EventControllerKey controller, uint keyval, uint keyco
        if (keyval == Gdk.Key.Escape) {
            close_requested ();
            return true;
        } else if (keyval == Gdk.Key.Up || keyval == Gdk.Key.KP_Up) {
            move_selection_up ();
            return true;
        } else if (keyval == Gdk.Key.Down || keyval == Gdk.Key.KP_Down) {
            move_selection_down ();
            return true;
        }
        return false;
    }

    private void move_selection_down () {
        var selected_row = list_box.get_selected_row ();
        if (selected_row == null) {
            select_first_visible_row ();
            return;
        }
    }

```

```

    }

    var next_row_widget = selected_row.get_next_sibling ();
    while (next_row_widget != null) {
        var list_row = next_row_widget as Gtk.ListBoxRow;
        if (list_row.get_child_visible ()) {
            list_box.select_row (list_row);
            return;
        }
        next_row_widget = next_row_widget.get_next_sibling ();
    }

    if (!selected_row.get_child_visible ())
        select_first_visible_row ();
}

private void move_selection_up () {
    var selected_row = list_box.get_selected_row ();
    if (selected_row == null) {
        select_first_visible_row ();
        return;
    }

    var prev_row_widget = selected_row.get_prev_sibling ();
    while (prev_row_widget != null) {
        var list_row = prev_row_widget as Gtk.ListBoxRow;
        if (list_row.get_child_visible ()) {
            list_box.select_row (list_row);
            return;
        }
        prev_row_widget = prev_row_widget.get_prev_sibling ();
    }

    if (!selected_row.get_child_visible ())
        select_first_visible_row ();
}

private Gtk.ListBoxRow create_row (SourceNodeItem item) {
    var row = new Gtk.ListBoxRow ();
    var box = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 8);
    box.margin_bottom = 4;
    box.margin_top = 4;
    box.margin_start = 4;
    box.margin_end = 4;

    var icon = ImageFactory.create_for_ts (item.type);

```

```

        var label = new Gtk.Label (item.name);
        label.ellipsize = Pango.EllipsizeMode.END;

        box.append (icon);
        box.append (label);
        row.set_child (box);

        //
        row.set_data ("item", new BreadcrumbObject (item));

        return row;
    }

    private void on_row_activated (Gtk.ListBoxRow row) {
        var obj = row.get_data<BreadcrumbObject> ("item");
        source_view.goto ((int) obj.item.start_point.row,
                           (int) obj.item.start_point.column);
    }
}

```

File: symbols/icons.json

```

{
    "font_file": "fonts/SYMBOLIC.ttf",
    "font_name": "Symbols Nerd Font",
    "output_dir": "gen_icons",
    "styles": {
        "folder": [
            "#5f23b3",
            "#7fcfc9"
        ],
        "ls-class": [
            "#f57900",
            "#f57900"
        ],
        "ls-method": [
            "#9141ac",
            "#9141ac"
        ],
        "ls-field": [
            "#3584e4",
            "#3584e4"
        ],
        "ls-interface": [

```



```

        "#33d17a",
        "#33d17a"
    ],
    "ls-constructor": [
        "#3584e4",
        "#3584e4"
    ],
    "ls-module": [
        "#3584e4",
        "#3584e4"
    ],
    "ls-snippet": [
        "#33d17a",
        "#33d17a"
    ],
    "ls-color": [
        "#c0bfb1",
        "#c0bfb1"
    ],
    "ls-folder": [
        "#c0bfb1",
        "#c0bfb1"
    ],
    "ls-constant": [
        "#3584e4",
        "#3584e4"
    ],
    "ls-struct": [
        "#f57900",
        "#f57900"
    ],
    "ls-operator": [
        "#2aa1b3",
        "#2aa1b3"
    ],
    "ls-property": [
        "#3584e4",
        "#3584e4"
    ],
    "ls-enum-member": [
        "#3584e4",
        "#3584e4"
    ],
    "ls-type-parameter": [
        "#2aa1b3",
        "#2aa1b3"
    ]

```

```

],
"ls-literal": [
  "#33d17a",
  "#33d17a"
],
"mt-make": [
  "#6d8086",
  "#6d8086"
],
"mt-docker": [
  "#384d54",
  "#384d54"
],
"mt-json": [
  "#cbcb41",
  "#cbcb41"
],
"mt-xml": [
  "#e37933",
  "#e37933"
],
"mt-yaml": [
  "#cb3e20",
  "#cb3e20"
],
"mt-ini": [
  "#6d8086",
  "#6d8086"
],
"mt-markdown": [
  "#519aba",
  "#519aba"
],
"mt-toml": [
  "#cb3e20",
  "#cb3e20"
],
"mt-vala": [
  "#6e44b3",
  "#6e44b3"
],
"mt-c": [
  "#599eff",
  "#599eff"
],
"mt-h": [

```

```

        "#599eff",
        "#599eff"
    ],
    "mt-cpp": [
        "#00599c",
        "#00599c"
    ],
    "mt-py": [
        "#306998",
        "#306998"
    ],
    "mt-js": [
        "#f1e05a",
        "#f1e05a"
    ],
    "mt-ts": [
        "#2b7489",
        "#2b7489"
    ],
    "mt-css": [
        "#563d7c",
        "#563d7c"
    ],
    "mt-html": [
        "#e34c26",
        "#e34c26"
    ],
    "mt-bash": [
        "#4ebd4e",
        "#4ebd4e"
    ],
    "mt-rust": [
        "#dea584",
        "#dea584"
    ],
    "mt-go": [
        "#00add8",
        "#00add8"
    ],
    "mt-lua": [
        "#000080",
        "#000080"
    ],
    "mt-x-image": [
        "#a074c4",
        "#a074c4"
    ]

```

```

    ],
    "mt-x-audio": [
        "#2b7489",
        "#2b7489"
    ],
    "mt-x-video": [
        "#2b7489",
        "#2b7489"
    ],
    "mt-x-text": [
        "#858585",
        "#858585"
    ],
    "mt-default": [
        "#858585",
        "#858585"
    ],
    "mt-x-app": [
        "#858585",
        "#858585"
    ],
    "cod-error": [
        "#ff3300",
        "#ff3300"
    ],
    "cod-warning": [
        "#cccc00",
        "#cccc00"
    ],
    "cod-info": [
        "#66ccff",
        "#66ccff"
    ]
  ],
  "icons": {
    "folder": {
      "char": "0xeaf7",
      "style": "folder"
    },
    "ls-class": {
      "char": "0xeb5b",
      "style": "ls-class"
    },
    "ls-method": {
      "char": "0xea8c",
      "style": "ls-method"
    }
  }
}

```

```

},
"ls-field": {
  "char": "0xeb5f",
  "style": "ls-field"
},
"ls-enum": {
  "char": "0xea95",
  "style": "ls-class"
},
"ls-interface": {
  "char": "0xeb61",
  "style": "ls-interface"
},
"ls-text": {
  "char": "0xeb69"
},
"ls-constructor": {
  "char": "0xeb44",
  "style": "ls-constructor"
},
"ls-module": {
  "char": "0xeb29",
  "style": "ls-module"
},
"ls-keyword": {
  "char": "0xeb62"
},
"ls-snippet": {
  "char": "0xeb66",
  "style": "ls-snippet"
},
"ls-color": {
  "char": "0xf024b",
  "style": "ls-color"
},
"ls-file": {
  "char": "0xf0214"
},
"ls-folder": {
  "char": "0xf024b",
  "style": "ls-folder"
},
"ls-constant": {
  "char": "0xeb5d",
  "symbol": "ls-constant"
},

```

```

"ls-struct": {
  "char": "0xea91",
  "style": "ls-struct"
},
"ls-operator": {
  "char": "0xeb64",
  "style": ""
},
"ls-property": {
  "char": "0xeb65",
  "style": "ls-property"
},
"ls-enum-member": {
  "char": "0xeb5e",
  "style": "ls-enum-member"
},
"ls-type-parameter": {
  "char": "0xea92",
  "style": "ls-type-parameter"
},
"ls-literal": {
  "char": "0xea90",
  "style": "ls-literal"
},
"ls-default": {
  "char": "0xea8b"
},
"mt-make": {
  "char": "0xe673",
  "style": "mt-make"
},
"mt-docker": {
  "char": "0xf308",
  "style": "mt-docker"
},
"mt-json": {
  "char": "0xe60b",
  "style": "mt-json"
},
"mt-xml": {
  "char": "0xf05c0",
  "style": "mt-xml"
},
"mt-yaml": {
  "char": "0xe8eb",
  "style": "mt-xml"
}

```

```

},
"mt-ini": {
  "char": "0xe615",
  "style": "mt-ini"
},
"mt-markdown": {
  "char": "0xeb1d",
  "style": "mt-markdown"
},
"mt-toml": {
  "char": "0xe6b2",
  "style": "mt-toml"
},
"mt-vala": {
  "char": "0xe69e",
  "style": "mt-vala"
},
"mt-c": {
  "char": "0xe61e",
  "style": "mt-c"
},
"mt-h": {
  "char": "0xe7fe",
  "style": "mt-h"
},
"mt-cpp": {
  "char": "0xe61d",
  "style": "mt-cpp"
},
"mt-py": {
  "char": "0xe73c",
  "style": "mt-py"
},
"mt-js": {
  "char": "0xe74e",
  "style": "mt-js"
},
"mt-ts": {
  "char": "0xe628",
  "style": "mt-ts"
},
"mt-css": {
  "char": "0xe749",
  "style": "mt-css"
},
"mt-html": {

```

```

        "char": "0xe736",
        "style": "mt-html"
    },
    "mt-bash": {
        "char": "0xe736",
        "style": "mt-bash"
    },
    "mt-rust": {
        "char": "0xe7a8",
        "style": "mt-rust"
    },
    "mt-go": {
        "char": "0xe627",
        "style": "mt-go"
    },
    "mt-lua": {
        "char": "0xe620",
        "style": "mt-lua"
    },
    "mt-x-image": {
        "char": "0xf024f",
        "style": "mt-x-image"
    },
    "mt-x-audio": {
        "char": "0xf1c7",
        "style": "mt-x-audio"
    },
    "mt-x-video": {
        "char": "0xf1c8",
        "style": "mt-x-video"
    },
    "mt-x-text": {
        "char": "0xf0f6",
        "style": "mt-x-text"
    },
    "mt-x-app": {
        "char": "0xf1c3",
        "style": "mt-x-app"
    },
    "mt-default": {
        "char": "0xf016",
        "style": "mt-default"
    },
    "app-project": {
        "char": "0xf13d2"
    },

```



```

        "app-log": {
            "char": "0xf4ed"
        },
        "app-terminal": {
            "char": "0xf489"
        },
        "app-issues": {
            "char": "0xf0a04"
        },
        "app-error": {
            "char": "0xea87"
        },
        "app-warning": {
            "char": "0xea6c"
        },
        "app-info": {
            "char": "0xea74"
        },
        "txt-wrap": {
            "char": "0xf05b6"
        },
        "log-erase": {
            "char": "0xf12d"
        },
        "cod-error": {
            "char": "0xea87",
            "style": "cod-error"
        },
        "cod-warning": {
            "char": "0xea6c",
            "style": "cod-warning"
        },
        "cod-info": {
            "char": "0xea74",
            "style": "cod-info"
        }
    }
}

```

File: src/Widgets/TextView/BreadcrumbsBar.vala

```

public class Iide.BreadcrumbFileSegment : Gtk.Box {
    private GLib.File file;
    private bool is_file;
    private Gtk.MenuButton button;

```

```

private SourceView source_view;

public BreadcrumbFileSegment (SourceView source_view, GLib.File file, bool is_file) {
    Object (orientation: Gtk.Orientation.HORIZONTAL, spacing: 0);
    this.source_view = source_view;

    this.file = file;
    this.is_file = is_file;

    button = new Gtk.MenuButton ();
    button.has_frame = false;
    button.add_css_class ("flat");
    button.add_css_class ("small-menu-button");
    button.valign = Gtk.Align.CENTER;
    button.can_shrink = true;
    button.always_show_arrow = true;

    var label = new Gtk.Label (file.get_basename ());
    button.set_child (label);
    label.set_ellipsize (Pango.EllipsizeMode.END);
    label.set_width_chars (1);

    if (is_file) {
        // VSCode , Box
        var btn_box = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 4);
        var icon = new Gtk.Image.from_icon_name ("text-x-generic-symbolic");
        btn_box.append (icon);
        btn_box.append (label);
        button.set_child (btn_box);
    }

    this.append (button);

    if (!is_file) {
        setup_popover ();
    } else {
        setup_file_outline_popover ();
    }
}

private void setup_popover () {
    var popover = new Gtk.Popover ();
    button.set_popover (popover);

    button.notify["active"].connect (() => {
        if (button.active) {

```

```

        var navigator = new BreadcrumbFileNavigator (this.file);

        popover.set_child (navigator);
        navigator.search_entry.set_key_capture_widget (popover);

        navigator.close_requested.connect (() => {
            button.active = false;
            source_view.grab_focus ();
        });
    }
});
}

private void setup_file_outline_popover () {
    var popover = new Gtk.Popover ();
    button.set_popover (popover);

    button.notify["active"].connect (() => {
        if (button.active) {
            var navigator = new BreadcrumbSymbolOutlineNavigator (source_view);

            popover.set_child (navigator);
            navigator.search_entry.set_key_capture_widget (popover);
            navigator.close_requested.connect (() => {
                button.active = false;
                source_view.grab_focus ();
            });
        }
    });
}

}

public class Iide.BreadcrumbSymbolSegment : Gtk.Box {
    private SourceView source_view;
    private Gtk.MenuButton button;
    private SourceNodeItem current_item;

    public BreadcrumbSymbolSegment (SourceView source_view, SourceNodeItem item) {
        Object (orientation: Gtk.Orientation.HORIZONTAL, spacing: 0);
        this.source_view = source_view;
        this.current_item = item;

        button = new Gtk.MenuButton ();
        button.has_frame = false;
        button.add_css_class ("flat");
        button.add_css_class ("small-menu-button");
    }
}

```

```

        button.valign = Gtk.Align.CENTER;
        button.can_shrink = true;
        button.always_show_arrow = true;

        var label = new Gtk.Label (item.name);
        button.set_child (label);
        label.set_ellipsize (Pango.EllipsizeMode.END);
        label.set_width_chars (1);

        this.append (button);
        setup_popover ();
    }

    private void setup_popover () {
        var popover = new Gtk.Popover ();
        button.set_popover (popover);

        button.notify["active"].connect (() => {
            if (button.active) {
                if (this.current_item.siblings.size < 2) {
                    button.active = false;
                    this.source_view.goto ((int) this.current_item.start_point.row,
                                            (int) this.current_item.start_point.column);
                } else {
                    var navigator = new BreadcrumbTreeSitterNavigator (source_view, this.current_item);
                    popover.set_child (navigator);

                    navigator.search_entry.set_key_capture_widget (popover);
                    navigator.close_requested.connect (() => {
                        button.active = false;
                        source_view.grab_focus ();
                    });
                }
            }
        });
    }
}

public class Iide.BreadcrumbsBar : Gtk.Box {
    private Gtk.Box path_box; // : > src > main.vala
    private Gtk.Box scope_box; // LSP- : MyClass > my_method
    private SourceView source_view;

    public BreadcrumbsBar (SourceView source_view) {
        Object (orientation: Gtk.Orientation.HORIZONTAL, spacing: 0);
        this.source_view = source_view;
    }
}

```

```

        add_css_class ("breadcrumbs-bar");

        path_box = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 0);
        path_box.height_request = 24;
        path_box.hexpand = false;
        path_box.halign = Gtk.Align.START;
        scope_box = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 0);
        scope_box.height_request = 24;
        scope_box.hexpand = false;
        scope_box.halign = Gtk.Align.START;

        append (path_box);
        append (scope_box);
    }

    public void update_file_path (GLib.File file, GLib.File project_root) {
        var child = path_box.get_first_child ();
        while (child != null) {
            var next = child.get_next_sibling ();
            path_box.remove (child);
            child = next;
        }

        string relative_path = project_root.get_relative_path (file);

        var segment = new BreadcrumbFileSegment (source_view, file, true);
        path_box.append (segment);

        if (relative_path == null) {
            return;
        }

        var current_file = file.get_parent ();
        while (current_file.get_path () != project_root.get_path ()) {
            var dir_segment = new BreadcrumbFileSegment (source_view, current_file, false);
            path_box.prepend (dir_segment);
            current_file = current_file.get_parent ();
            if (current_file == null)
                break;
        }
    }

    public void update_breadcrumbs (Gee.List<SourceNodeItem?> crumbs) {
        //
        var child = scope_box.get_first_child ();
        while (child != null) {

```

```

        var next = child.get_next_sibling ();
        scope_box.remove (child);
        child = next;
    }

    foreach (var crumb in crumbs) {
        var ts_segment = new BreadcrumbSymbolSegment (source_view, crumb);
        scope_box.append (ts_segment);
    }
}

```

File: src/Widgets/TextView/BreadcrumbSymbolOutlineNavigator.vala

```

/*
*/

public class Ide.BreadcrumbSymbolOutlineNavigator : Gtk.Box {
    private SourceView source_view;
    public Gtk.SearchEntry search_entry;
    private Gtk.ListBox ts_list_box;
    private Gtk.ListBox lsp_list_box;
    private Gtk.Spinner spinner;
    private Gtk.Stack stack;
    private Gtk.ToggleButton back_button;

    public signal void close_requested ();

    private class BreadcrumbObject : Object {
        public SourceNodeItem item;
        public BreadcrumbObject (SourceNodeItem item) {
            Object ();
            this.item = item;
        }
    }

    public BreadcrumbSymbolOutlineNavigator (SourceView source_view) {
        Object (orientation: Gtk.Orientation.VERTICAL, spacing: 6);
        this.source_view = source_view;

        var header = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 4);

        back_button = new Gtk.ToggleButton ();
        back_button.icon_name = "go-previous-symbolic";
        back_button.active = false;
    }
}

```

```

back_button.add_css_class ("flat");
back_button.can_focus = false;

this.set_size_request (300, -1);

search_entry = new Gtk.SearchEntry ();
search_entry.hexpand = true;

header.append (back_button);
header.append (search_entry);
this.append (header);

ts_list_box = new Gtk.ListBox ();
ts_list_box.add_css_class ("navigation-sidebar");

lsp_list_box = new Gtk.ListBox ();
lsp_list_box.add_css_class ("navigation-sidebar");

spinner = new Gtk.Spinner ();
spinner.set_size_request (32, 32);

var loading_box = new Gtk.Box (Gtk.Orientation.VERTICAL, 12);
loading_box.valign = Gtk.Align.CENTER;
loading_box.halign = Gtk.Align.CENTER;

var loading_label = new Gtk.Label ("Loading Document Symbols...");
loading_label.add_css_class ("dim-label");

loading_box.append (spinner);
loading_box.append (loading_label);

stack = new Gtk.Stack ();

//
add_ts_symbols_recursively (source_view.get_full_outline (), 0);

//
ts_list_box.set_filter_func ((row) => {
    var text = search_entry.get_text ().down ();
    if (text == "")return true;

    var obj = row.get_data<BreadcrumbObject> ("item");
    var name = obj.item.name.down ();
    return name.contains (text);
});
search_entry.search_changed.connect (() => {

```

```

        ts_list_box.invalidate_filter ();
        refresh_state ();
    });

    ts_list_box.row_activated.connect (on_ts_row_activated);
    search_entry.activate.connect (() => {
        on_ts_row_activated (ts_list_box.get_selected_row ());
    });

    var ts_scroll = new Gtk.ScrolledWindow ();
    ts_scroll.propagate_natural_height = true;
    ts_scroll.set_max_content_height (400);
    ts_scroll.set_child (ts_list_box);
    stack.add_named (ts_scroll, "ts");

    stack.add_named (loading_box, "lsp_loading");

    var lsp_scroll = new Gtk.ScrolledWindow ();
    lsp_scroll.propagate_natural_height = true;
    lsp_scroll.set_max_content_height (400);
    lsp_scroll.set_child (lsp_list_box);
    stack.add_named (lsp_scroll, "lsp");

    stack.visible_child_name = "ts";

    this.append (stack);

    back_button.toggled.connect(toggle_mode);

    var key_controller = new Gtk.EventControllerKey ();
    key_controller.key_pressed.connect (on_key_pressed);
    search_entry.add_controller (key_controller);
    ts_list_box.set_selection_mode (Gtk.SelectionMode.SINGLE);
    lsp_list_box.set_selection_mode (Gtk.SelectionMode.SINGLE);

    this.refresh_state ();
}

private void toggle_mode() {
    if (!back_button.active) {
        spinner.stop ();
        stack.visible_child_name = "ts";
        return;
    }
}

spinner.start ();

```



```

        stack.visible_child_name = "lsp_loading";

        document_symbols.begin ();
    }

private async void document_symbols() {
    var symbols = yield IdeLspService.get_instance ().document_symbols (source_view.uri);
    spinner.stop ();
    stack.visible_child_name = "lsp";
}

private void select_ts_first_visible_row () {
    var row = ts_list_box.get_first_child ();
    while (row != null) {
        var list_row = row as Gtk.ListBoxRow;
        if (list_row.get_child_visible ()) {
            ts_list_box.select_row (list_row);
            break;
        }
        row = row.get_next_sibling ();
    }
}

private void refresh_state () {
    this.search_entry.grab_focus ();
    select_ts_first_visible_row ();
}

private bool on_key_pressed (Gtk.EventControllerKey controller, uint keyval, uint keycode) {
    if (keyval == Gdk.Key.Escape) {
        close_requested ();
        return true;
    } else if (keyval == Gdk.Key.Up || keyval == Gdk.Key.KP_Up) {
        move_ts_selection_up ();
        return true;
    } else if (keyval == Gdk.Key.Down || keyval == Gdk.Key.KP_Down) {
        move_ts_selection_down ();
        return true;
    }
    return false;
}

private void move_ts_selection_down () {
    var selected_row = ts_list_box.get_selected_row ();
    if (selected_row == null) {
        select_ts_first_visible_row ();
    }
}

```

```

        return;
    }

    var next_row_widget = selected_row.get_next_sibling ();
    while (next_row_widget != null) {
        var list_row = next_row_widget as Gtk.ListBoxRow;
        if (list_row.get_child_visible ()) {
            ts_list_box.select_row (list_row);
            return;
        }
        next_row_widget = next_row_widget.get_next_sibling ();
    }

    if (!selected_row.get_child_visible ())
        select_ts_first_visible_row ();
}

private void move_ts_selection_up () {
    var selected_row = ts_list_box.get_selected_row ();
    if (selected_row == null) {
        select_ts_first_visible_row ();
        return;
    }

    var prev_row_widget = selected_row.get_prev_sibling ();
    while (prev_row_widget != null) {
        var list_row = prev_row_widget as Gtk.ListBoxRow;
        if (list_row.get_child_visible ()) {
            ts_list_box.select_row (list_row);
            return;
        }
        prev_row_widget = prev_row_widget.get_prev_sibling ();
    }

    if (!selected_row.get_child_visible ())
        select_ts_first_visible_row ();
}

private void add_ts_symbols_recursively (Gee.List<SourceNodeItem?> symbols, int depth) {
    foreach (var sym in symbols) {
        var row = create_ts_symbol_row (sym, depth);
        ts_list_box.append (row);

        if (sym.children != null && sym.children.size > 0) {
            add_ts_symbols_recursively (sym.children, depth + 1);
        }
    }
}

```

```

    }
}

private Gtk.ListBoxRow create_ts_symbol_row (SourceNodeItem item, int depth) {
    var row = new Gtk.ListBoxRow ();
    var box = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 8);
    box.margin_start = 8 + (depth * 16); //
    box.margin_end = 8;
    box.margin_top = box.margin_bottom = 2;

    var icon = ImageFactory.create_for_ts (item.type);
    var label = new Gtk.Label (item.name);

    box.append (icon);
    box.append (label);
    row.set_child (box);

    //
    row.set_data ("item", new BreadcrumbObject (item));
    return row;
}

private void on_ts_row_activated (Gtk.ListBoxRow row) {
    var obj = row.get_data<BreadcrumbObject> ("item");
    this.source_view.goto ((int) obj.item.start_point.row,
                           (int) obj.item.start_point.column);
    this.close_requested ();
}
}

```

File: src/Widgets/TextView/TextView.vala

```

/*
 * textdocument.vala
 *
 * Copyright 2026 kai
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the

```

```

* GNU General Public License for more details.
*
* You should have received a copy of the GNU General Public License
* along with this program.  If not, see <https://www.gnu.org/licenses/>.
*
* SPDX-License-Identifier: GPL-3.0-or-later
*/

public class Iide.SaveDelegate : Panel.SaveDelegate {
    private Iide.TextView view;
    public SaveDelegate (Iide.TextView view) {
        Object ();
        this.view = view;

        var file = GLib.File.new_for_uri (view.uri);
        title = view.title;
        subtitle = file.get_path ();
    }

    public override bool save (GLib.Task task) {
        message ("Saving in delegate...");
        var result = view.save ();
        task.return_boolean (result);
        return result;
    }

    public override void close () {
        view.force_close ();
    }

    public override void discard () {
        view.force_close ();
    }
}

//
private struct Iide.ClickableIndicator {
    public Gdk.Rectangle rect;
    public int start_line;
}

public class Iide.TextView : Panel.Widget {
    public SourceView source_view;
    private Gtk.Overlay overlay;

```

```

private Gtk.DrawingArea indent_lines_canvas;
private Gee.ArrayList<ClickableIndicator?> visible_indicators = new Gee.ArrayList<Clicka

private GtkSource.Map source_map;
private FontZoomer font_zoomer;
private Iide.SettingsService settings;
private EditorStatusBar editor_status_bar;

public Window window;
public string uri { get; private set; }
public SourceView text_view { get { return source_view; } }

public bool is_modified { get { return ((GtkSource.Buffer) source_view.buffer).get_modifi

public signal void buffer_saved ();

public TextView (GLib.File file, GtkSource.Buffer buffer, Window window) {
    Object ();
    this.uri = file.get_uri ();
    this.window = window;
    this.settings = Iide.SettingsService.get_instance ();

    var adw_style_manager = Adw.StyleManager.get_default ();

    var style_manager = GtkSource.StyleSchemeManager.get_default ();
    if (adw_style_manager.color_scheme == Adw.ColorScheme.FORCE_LIGHT) {
        buffer.set_style_scheme (style_manager.get_scheme ("Adwaita"));
    } else {
        buffer.set_style_scheme (style_manager.get_scheme ("Adwaita-dark"));
    }

    adw_style_manager.notify["color-scheme"].connect (() => {
        if (adw_style_manager.color_scheme == Adw.ColorScheme.FORCE_LIGHT) {
            buffer.set_style_scheme (style_manager.get_scheme ("Adwaita"));
        } else {
            buffer.set_style_scheme (style_manager.get_scheme ("Adwaita-dark"));
        }
    });

    source_view = new SourceView (window, uri, buffer);
    source_view.bottom_margin = 400;

    // icon_name = source_view.icon_name;
    set_icon_name (ImageFactory.icon_name_for_file (file));
    font_zoomer = new FontZoomer (source_view);

```

```

// Connect to settings changes to apply to all open documents
settings.editor_setting_changed.connect ((key) => {
    switch (key) {
        case "editor-font-size":
            font_zoomer.set_zoom_level (settings.editor_font_size);
            break;
        case "show-minimap":
            toggle_minimap_visible (settings.show_minimap);
            break;
    }
});

source_map = new GtkSource.Map ();
// source_map.set_view (source_view);
source_map.add_css_class ("textview-map");
source_map.visible = settings.show_minimap;

font_zoomer.zoom_changed.connect ((level) => {
    source_view.mark_renderer.set_icons_size (FontSizeHelper.get_size_for_zoom_level (level));
    source_view.folding_gutter.set_icons_size (FontSizeHelper.get_size_for_zoom_level (level));
});

var scroll = new Gtk.ScrolledWindow ();
scroll.hexpand = true;
scroll.vexpand = true;

scroll.set_child (source_view);

//
//
// VAdjustment, ScrolledWindow
source_map.set_view (source_view);

// 3.
source_map.vexpand = true;

this.overlay = new Gtk.Overlay ();
this.overlay.set_child (scroll);
this.indent_lines_canvas = new Gtk.DrawingArea ();
this.indent_lines_canvas.can_target = false;
this.overlay.add_overlay (this.indent_lines_canvas);

var doc = this.source_view.document as Iide.TreeSitterDocument;
if (doc != null && doc.ts_highlighter != null) {
    this.indent_lines_canvas.set_draw_func (this.draw_indent_lines);
    doc.ts_highlighter.folding_structure_updated.connect ((blocks) => {
        //
        ->
    });
}

```

```

        this.indent_lines_canvas.queue_draw ();
    });
    buffer.notify["cursor-position"].connect (() => {
        //
        //
        this.indent_lines_canvas.queue_draw ();
    });
    this.source_view.folding_gutter.fold_state_changed.connect (() => {
        //
        // "...
        this.indent_lines_canvas.queue_draw ();
        this.indent_lines_canvas.queue_resize ();
    });

    //
    var click_gesture = new Gtk.GestureClick ();

    //
    // GTK 4:
    //
    // . CAPTURE
    //
    //
    click_gesture.set_propagation_phase (Gtk.PropagationPhase.CAPTURE);

    click_gesture.pressed.connect ((n_presses, x, y) => {
        foreach (var indicator in this.visible_indicators) {
            //
            // "...
            if (x >= indicator.rect.x && x <= (indicator.rect.x + indicator.rect.wid
                y >= indicator.rect.y && y <= (indicator.rect.y + indicator.rect.hei
                //
                // "... -
                !
                var folding_gutter = this.source_view.folding_gutter;

                Gtk.TextIter trigger_iter;
                this.source_view.get_buffer ().get_iter_at_line (out trigger_iter, 1,

                folding_gutter.activate (trigger_iter, Gdk.Rectangle (), 1, Gdk.Mod1

                //
                // "...
                this.indent_lines_canvas.queue_draw ();

                //
                // GTK,
                //
                click_gesture.set_state (Gtk.EventSequenceState.CLAIMED);
                return;
            }
        }

        //
        // "... -
        //

```

```

        //          CAPTURE          CLAIMED, GTK 4
        //          GtkSourceView.      ,      ,
        //          !
        click_gesture.set_state (Gtk.EventSequenceState.DENIED);
    });

    //
    this.source_view.add_controller (click_gesture);

}

this.source_view.get_vadjustment ().value_changed.connect (() => { this.indent_lines
this.source_view.get_hadjustment ().value_changed.connect (() => { this.indent_lines

var subbox = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 0);
subbox.homogeneous = false;
subbox.append (this.overlay);
subbox.append (source_map);

var font_map = Pango.CairoFontMap.get_default ();
source_map.set_font_map (font_map);

var box = new Gtk.Box (Gtk.Orientation.VERTICAL, 0);

box.append (subbox);

this.editor_status_bar = new EditorStatusBar (source_view);
box.append (this.editor_status_bar);
source_view.breadcrumbs_changed.connect (this.editor_status_bar.update_breadcrumbs);

child = box;

title = file.get_basename ();

save_delegate = new Iide.SaveDelegate (this);
modified = false;

buffer.modified_changed.connect_after (() => {
    modified = buffer.get_modified ();
});
buffer.notify["cursor-position"].connect (() => {
    Gtk.TextIter insert, selection;
    buffer.get_selection_bounds (out insert, out selection);

    // 1.
    int line = insert.get_line ();

```



```

        int col = insert.get_line_index (); //
        int sel_len = insert.get_offset () - selection.get_offset ();

        editor_status_bar.update_position (line, col, sel_len);
    });

    //      (Insert/Overwrite)
    source_view.notify["overwrite"].connect (() => {
        editor_status_bar.update_mode (source_view.overwrite);
    });

    var main_adj = scroll.get_vadjustment ();
    var map_adj = source_map.get_vadjustment ();

    // 1.      (    ),
    //      ,
    main_adj.value_changed.connect (() => {
        //
        double main_range = main_adj.upper - main_adj.page_size;
        double map_range = map_adj.upper - map_adj.page_size;

        //      ,
        if (main_range > 0) {
            // 1.      ( 0.0  1.0)
            double percentage = main_adj.value / main_range;

            // 2.
            double target_value = percentage * map_range;

            // 3.      (clamp),
            //
            map_adj.set_value (target_value.clamp (0, map_range));
        } else {
            //
            map_adj.set_value (0);
        }
    });
}

private void draw_rounded_rectangle (Cairo.Context cr, double x, double y, double width,
double degrees = Math.PI / 180.0;

cr.new_sub_path ();
cr.arc (x + width - radius, y + radius, radius, -90 * degrees, 0 * degrees);
cr.arc (x + width - radius, y + height - radius, radius, 0 * degrees, 90 * degrees);
cr.arc (x + radius, y + height - radius, radius, 90 * degrees, 180 * degrees);

```

```

        cr.arc (x + radius, y + radius, radius, 180 * degrees, 270 * degrees);
        cr.close_path ();
    }

private void draw_indent_lines (Gtk.DrawingArea drawing_area, Cairo.Context cr, int width)
{
    this.visible_indicators.clear ();

    var doc = this.source_view.document as Iide.TreeSitterDocument;
    if (doc == null || doc.ts_highlighter == null) return;

    var ts_blocks = doc.ts_highlighter.get_cached_indent_blocks ();
    var folding_gutter = this.source_view.folding_gutter;
    if (ts_blocks.size == 0) return;

    cr.set_source_rgba (0.5, 0.5, 0.5, 0.25);
    cr.set_line_width (1.0);

    double tab_width_px = this.calculate_tab_width_pixels ();
    int left_margin = this.source_view.get_left_margin ();
    double h_scroll = this.source_view.get_hadjustment ().get_value ();

    int gutter_width = 0;
    var left_gutter = this.source_view.get_gutter (Gtk.TextWindowType.LEFT);
    if (left_gutter != null) {
        gutter_width = left_gutter.get_width ();
    }

    foreach (var block in ts_blocks) {
        Gtk.TextIter start_iter;
        this.source_view.get_buffer ().get_iter_at_line (out start_iter, block.start_line);

        // 1.
        int start_y_buf, start_height;
        this.source_view.get_line_yrange (start_iter, out start_y_buf, out start_height);

        // ( ) -
        if (start_height <= 0) continue;

        // Y-
        int win_x, win_y_start;
        this.source_view.buffer_to_window_coords (Gtk.TextWindowType.TEXT, 0, start_y_buf,
            out win_x, out win_y_start);

        // Gutter-
        bool is_collapsed = folding_gutter.is_line_collapsed_by_number (block.start_line);

        // 1. ,

```

```

Gtk.TextIter cursor_iter;
var buffer = this.source_view.get_buffer ();
buffer.get_iter_at_mark (out cursor_iter, buffer.get_insert ());
int cursor_line = cursor_iter.get_line ();

// 2.
//                                     Gutter
var active_highlighter_block = folding_gutter.find_deepest_block_for_line (cursor_line);

if (is_collapsed) {
    // ===                                "... " ===

    Gtk.TextIter eol_iter = start_iter;
    eol_iter.forward_to_line_end ();

    Gdk.Rectangle eol_rect;
    this.source_view.get_iter_location (eol_iter, out eol_rect);

    int win_eol_x, win_eol_y;
    this.source_view.buffer_to_window_coords (Gtk.TextWindowType.TEXT, eol_rect, out win_eol_x, out win_eol_y);

    double line_h = (double) start_height;
    double btn_h = line_h * 0.65;
    double btn_w = btn_h * 1.7;
    double font_size = line_h * 0.45;

    //
    double btn_x = gutter_width + win_eol_x + (line_h * 0.3);
    double btn_y = win_eol_y + ((line_h - btn_h) / 2.0);

    // =====
    //      :                               btn_x (      ).
    //      ,                               !
    var click_zone = ClickableIndicator () {
        rect = Gdk.Rectangle () {
            x = (int) btn_x,
            y = (int) btn_y,
            width = (int) btn_w,
            height = (int) btn_h
        },
        start_line = block.start_line
    };
    this.visible_indicators.add (click_zone);
    // =====

    cr.save ();

```

```

//
cr.set_source_rgba (0.5, 0.5, 0.5, 0.15);
double radius = btn_h * 0.25;
this.draw_rounded_rectangle (cr, btn_x, btn_y, btn_w, btn_h, radius);
cr.fill ();

//
this.draw_rounded_rectangle (cr, btn_x, btn_y, btn_w, btn_h, radius);
cr.set_source_rgba (0.5, 0.5, 0.5, 0.4);
cr.set_line_width (1.0);
cr.stroke ();

//          "..."
cr.set_source_rgba (0.4, 0.4, 0.4, 1.0);
cr.select_font_face ("Monospace", Cairo.FontSlant.NORMAL, Cairo.FontWeight.NORMAL);
cr.set_font_size (font_size);

Cairo.TextExtents extents;
cr.text_extents ("...", out extents);
double text_x = btn_x + ((btn_w - extents.width) / 2.0) - extents.x_bearing;
double text_y = btn_y + ((btn_h - extents.height) / 2.0) - extents.y_bearing;

cr.move_to (text_x, text_y);
cr.show_text ("...");

cr.restore ();

continue;
}

// =====
//
// =====
Gtk.TextIter end_iter;
this.source_view.get_buffer ().get_iter_at_line (out end_iter, block.end_line);

int end_y_buf, end_height;
this.source_view.get_line_yrange (end_iter, out end_y_buf, out end_height);

//
//          -      -      .
//          -      -      .
if (end_height <= 0) continue;

int win_y_end;
this.source_view.buffer_to_window_coords (Gtk.TextWindowType.TEXT, 0, end_y_buf,

```

```

//      (      /      )
bool is_active_line = false;
if (active_highlighter_block != null) {
    if (block.start_line == active_highlighter_block.start_line && block.end_line == active_highlighter_block.end_line)
        is_active_line = true;
    } else if (cursor_line >= block.start_line && cursor_line <= block.end_line)
        is_active_line = true;
    }
}

if (is_active_line) {
    cr.set_source_rgba (0.2, 0.6, 1.0, 0.7);
    cr.set_line_width (1.2);
} else {
    cr.set_source_rgba (0.5, 0.5, 0.5, 0.25);
    cr.set_line_width (1.0);
}

double line_x = gutter_width + left_margin + (block.indent_level * tab_width_px);
if (line_x < gutter_width) continue;

if (win_y_start > height || (win_y_end + end_height) < 0) continue;

double draw_y_start = double.max (win_y_start, 0.0);
double draw_y_end = double.min (win_y_end + end_height, (double) height);

cr.move_to (line_x, draw_y_start);
cr.line_to (line_x, draw_y_end);
cr.stroke ();
}
}

private double calculate_tab_width_pixels () {
    // TODO: Implement real metrix char width...
    double char_width_px = 0.6f * FontSizeHelper.get_size_for_zoom_level (settings.editable_font_size);

    uint tab_width_chars = this.source_view.get_tab_width ();
    if (tab_width_chars == 0) tab_width_chars = 4;

    return char_width_px * tab_width_chars;
}

public override void size_allocate (int width, int height, int baseline) {
    base.size_allocate (width, height, baseline);
}

```

```

        source_map.visible = settings.show_minimap && width >= 600;
    }

    public void toggle_minimap_visible (bool visible) {
        source_map.visible = visible;
    }

    public void view_grab_focus () {
        source_view.grab_focus ();
    }

    public bool save () {
        try {
            var text = source_view.buffer.text;
            var file = GLib.File.new_for_uri (uri);
            file.replace_contents (text.data, null, false, GLib.FileCreateFlags.NONE, null);
            ((GtkSource.Buffer) source_view.buffer).set_modified (false);
            buffer_saved ();
        } catch (Error e) {
            critical (e.message);
        }
        return true;
    }

    public void update_diagnostics (Gee.ArrayList<LspDiagnostic> diagnostics) {
        var text_buffer = (Gtk.TextBuffer) source_view.buffer;

        text_buffer.begin_user_action ();

        LspDiagnosticsMark.clear_mark_attributes (source_view);

        int line_count = text_buffer.get_line_count ();

        int lsp_error_count = 0;
        int lsp_warning_count = 0;

        foreach (var diag in diagnostics) {
            if (diag.start_line >= line_count) {
                continue;
            }

            Gtk.TextIter start_iter;
            text_buffer.get_iter_at_line (out start_iter, diag.start_line);

            var mark = new LspDiagnosticsMark.from_lsp_diagnostic (diag);
            text_buffer.add_mark (mark, start_iter); //

```

```

        switch (diag.severity) {
        case 1:
            lsp_error_count++;
            break;
        case 2: case 3: case 4:
            lsp_warning_count++;
            break;
        }
    }

    text_buffer.end_user_action ();

    this.editor_status_bar.update_diagnostics (lsp_error_count, lsp_warning_count);
}

public void select_and_scroll (int line, int start_col, int end_col, bool is_new) {
    source_view.select_and_scroll (line, start_col, end_col, is_new);
}
}

```

File: src/application.vala

```

/* application.vala
 *
 * Copyright 2026 kai
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program. If not, see <https://www.gnu.org/licenses/>.
 *
 * SPDX-License-Identifier: GPL-3.0-or-later
 */

[CCode (cname = "gtk_style_context_add_provider_for_display", cheader_filename = "gtk/gtk.h")]
extern void add_provider_to_display (Gdk.Display display, Gtk.StyleProvider provider, uint p

```

```

public class Iide.Application : Adw.Application {
    private Iide.SettingsService settings;
    private Iide.AppActionsManager action_manager;
    private SimpleActionGroup simple_action_group = new SimpleActionGroup ();

    public signal void zoom_changed (int zoom_level);

    public Application () {
        Object (
            application_id: "org.github.kai66673.iide",
            flags: ApplicationFlags.DEFAULT_FLAGS,
            resource_base_path: "/org/github/kai66673/iide"
        );
    }

    construct {
        settings = Iide.SettingsService.get_instance ();
        action_manager = Iide.AppActionsManager.get_instance ();

        register_builitn_actions ();
    }

    private void register_builitn_actions () {
        action_manager.register_action (this, new SaveAllAction (this));
        action_manager.register_action (this, new OpenProjectAction (this));
        action_manager.register_action (this, new PreferencesAction (this));
        action_manager.register_action (this, new ToggleMinimapAction (this));
        action_manager.register_action (this, new FuzzyFinderAction (this));
        action_manager.register_action (this, new SearchSymbolAction (this));
        action_manager.register_action (this, new SearchInFilesAction (this));
        action_manager.register_action (this, new ZoomInAction ());
        action_manager.register_action (this, new ZoomOutAction ());
        action_manager.register_action (this, new ZoomResetAction ());
        action_manager.register_action (this, new ExpandSelectionAction ());
        action_manager.register_action (this, new ShrinkSelectionAction ());
        action_manager.register_action (this, new QuitAction ());
        action_manager.register_action (this, new NavigationBackAction (this));
        action_manager.register_action (this, new NavigationForwardAction (this));
        action_manager.register_action (this, new ShowLineNumbersAction (this));
        action_manager.register_action (this, new ShowDiagnosticsMarksAction (this));
        action_manager.register_action (this, new ShowFoldingAction (this));

        // Ins/Ovr toggle
        //
        var toggle_overwrite = new SimpleAction ("toggle-overwrite", null);
    }
}

```



```

toggle_overwrite.activate.connect (() => {
    // SourceView
    var win = active_window as Iide.Window;
    if (win != null) {
        var view = win.get_active_source_view ();
        if (view != null) {
            view.overwrite = !view.overwrite;
        }
    }
});
simple_action_group.add_action (toggle_overwrite);
set_accels_for_action ("editor.toggle-overwrite", { "Insert" });
}

public Iide.SettingsService get_settings () {
    return settings;
}

public void register_embedded_fonts () {
    // GResource
    // string font_path = "/org/github/kai66673/iide/fonts/SymbolsNerdFontMono-Regular.ttf";
    // var bytes = resources_lookup_data (font_path, ResourceLookupFlags.NONE);

    // GTK4 PangoCairo FontConfig
    // Linux/Arch -
    // Fontconfig .

    // GTK4 CSS ( ):
    string css = ""
        @font-face {
            font-family: "Symbols Nerd Font";
            src: url("resource:///org/github/kai66673/iide/fonts/SymbolsNerdFont-Regular.ttf");
        }
    "";
    var provider = new Gtk.CssProvider ();
    provider.load_from_bytes (new GLib.Bytes (css.data));
    add_provider_to_display (
        Gdk.Display.get_default (),
        provider,
        Gtk.STYLE_PROVIDER_PRIORITY_APPLICATION
    );
}

public override void activate () {
    base.activate ();
}

```

```

var icon_theme = Gtk.IconTheme.get_for_display (Gdk.Display.get_default ());
icon_theme.add_resource_path ("/org/github/kai66673/iide/icons");

var css_provider = new Gtk.CssProvider ();
css_provider.load_from_resource ("/org/github/kai66673/iide/style.css");
add_provider_to_display (
    Gdk.Display.get_default (),
    css_provider,
    Gtk.STYLE_PROVIDER_PRIORITY_APPLICATION
);

var css_symbol_provider = new Gtk.CssProvider ();
css_symbol_provider.load_from_resource ("/org/github/kai66673/iide/symbols/symbols.css");
add_provider_to_display (
    Gdk.Display.get_default (),
    css_symbol_provider,
    Gtk.STYLE_PROVIDER_PRIORITY_APPLICATION
);

register_embedded_fonts ();

var win = this.active_window ?? new Iide.Window (this);
win.present ();
}
}

private class Iide.SaveAllAction : Iide.AppAction {
    private weak Iide.Application app;

    public SaveAllAction (Iide.Application app) {
        this.app = app;
    }

    public override string id { get { return "save"; } }
    public override string name { get { return _("Save All"); } }
    public override string? description { get { return _("Save all open documents"); } }
    public override string? icon_name { get { return "document-save-symbolic"; } }
    public override string? category { get { return "File"; } }
    public override string? default_shortcut { get { return "<primary>s"; } }

    public override bool can_execute () {
        return app ? .active_window is Iide.Window;
    }

    public override void execute () {
        var win = app ? .active_window as Iide.Window;

```

```

        win?.save_modified ();
    }
}

private class Iide.OpenProjectAction : Iide.AppAction {
    private weak Iide.Application app;

    public OpenProjectAction (Iide.Application app) {
        this.app = app;
    }

    public override string id { get { return "open-project"; } }
    public override string name { get { return _("Open Project"); } }
    public override string? description { get { return _("Open a project folder"); } }
    public override string? icon_name { get { return "folder-open-symbolic"; } }
    public override string? category { get { return "File"; } }
    public override string? default_shortcut { get { return "<primary>o"; } }

    public override bool can_execute () {
        return true;
    }

    public override void execute () {
        var win = app ? .active_window as Iide.Window;
        win?.open_project_dialog ();
    }
}

private class Iide.PreferencesAction : Iide.AppAction {
    private weak Iide.Application app;

    public PreferencesAction (Iide.Application app) {
        this.app = app;
    }

    public override string id { get { return "preferences"; } }
    public override string name { get { return _("Preferences"); } }
    public override string? description { get { return _("Open preferences dialog"); } }
    public override string? icon_name { get { return "preferences-system-symbolic"; } }
    public override string? category { get { return "Application"; } }
    public override string? default_shortcut { get { return "<primary>comma"; } }

    public override bool can_execute () {
        return true;
    }
}

```

```

        public override void execute () {
            var dialog = new Iide.PreferencesDialog ();
            dialog.set_transient_for (app ? .active_window);
            dialog.present ();
        }
    }

private class Iide.ToggleMinimapAction : Iide.AppAction {
    private weak Iide.Application app;

    public ToggleMinimapAction (Iide.Application app) {
        this.app = app;
        this.state = Iide.SettingsService.get_instance ().show_minimap;
    }

    public override string id { get { return "show-minimap"; } }
    public override string name { get { return _("Toggle Minimap"); } }
    public override string? description { get { return _("Show or hide the minimap"); } }
    public override string? icon_name { get { return "view-fullscreen-symbolic"; } }
    public override string? category { get { return "View"; } }
    public override bool is_toggle { get { return true; } }
    public override string? default_shortcut { get { return "<primary>m"; } }

    public override bool can_execute () {
        return true;
    }

    public override void execute () {
        var settings = Iide.SettingsService.get_instance ();
        state = !state;
        settings.show_minimap = state;

        state_changed (state);
        // TODO: handle changing state from preferences dialog...
    }
}

private class Iide.ZoomInAction : Iide.AppAction {
    private Iide.SettingsService settings;

    public override string id { get { return "zoom-in"; } }
    public override string name { get { return _("Zoom In"); } }
    public override string? description { get { return _("Increase editor font size"); } }
    public override string? icon_name { get { return "zoom-in-symbolic"; } }
    public override string? category { get { return "View"; } }
    public override string? default_shortcut { get { return "<primary>plus"; } }
}

```

```

    public override bool can_execute () {
        settings = Iide.SettingsService.get_instance ();
        return settings.editor_font_size < Iide.FontSizeHelper.MAX_ZOOM_LEVEL;
    }

    public override void execute () {
        settings = Iide.SettingsService.get_instance ();
        var app = GLib.Application.get_default () as Iide.Application;
        settings.editor_font_size++;
        app?.zoom_changed (settings.editor_font_size);
    }
}

private class Iide.ZoomOutAction : Iide.AppAction {
    private Iide.SettingsService settings;

    public override string id { get { return "zoom-out"; } }
    public override string name { get { return _("Zoom Out"); } }
    public override string? description { get { return _("Decrease editor font size"); } }
    public override string? icon_name { get { return "zoom-out-symbolic"; } }
    public override string? category { get { return "View"; } }
    public override string? default_shortcut { get { return "<primary>minus"; } }

    public override bool can_execute () {
        settings = Iide.SettingsService.get_instance ();
        return settings.editor_font_size > Iide.FontSizeHelper.MIN_ZOOM_LEVEL;
    }

    public override void execute () {
        settings = Iide.SettingsService.get_instance ();
        var app = GLib.Application.get_default () as Iide.Application;
        settings.editor_font_size--;
        app?.zoom_changed (settings.editor_font_size);
    }
}

private class Iide.ZoomResetAction : Iide.AppAction {
    private Iide.SettingsService settings;

    public override string id { get { return "zoom-reset"; } }
    public override string name { get { return _("Zoom Reset"); } }
    public override string? description { get { return _("Reset editor font size to default"); } }
    public override string? icon_name { get { return "zoom-original-symbolic"; } }
    public override string? category { get { return "View"; } }
    public override string? default_shortcut { get { return "<primary>0"; } }
}

```

```

    public override bool can_execute () {
        return true;
    }

    public override void execute () {
        settings = Iide.SettingsService.get_instance ();
        var app = GLib.Application.get_default () as Iide.Application;
        settings.editor_font_size = Iide.FontSizeHelper.DEFAULT_ZOOM_LEVEL;
        app?.zoom_changed (settings.editor_font_size);
    }
}

private class Iide.ExpandSelectionAction : Iide.AppAction {
    public override string id { get { return "expand-selection"; } }
    public override string name { get { return _("Expand Selection"); } }
    public override string? description { get { return _("Expand the current selection"); } }
    public override string? icon_name { get { return "zoom-original-symbolic"; } }
    public override string? category { get { return "View"; } }
    public override string? default_shortcut { get { return "<primary>w"; } }

    public override bool can_execute () {
        return true;
    }

    public override void execute () {
        var app = GLib.Application.get_default () as Iide.Application;
        var win = app ? .active_window as Iide.Window;
        if (win != null) {
            win.get_active_source_view () ? .expand_selection ();
        }
    }
}

private class Iide.ShrinkSelectionAction : Iide.AppAction {
    public override string id { get { return "shrink-selection"; } }
    public override string name { get { return _("Shrink Selection"); } }
    public override string? description { get { return _("Shrink the current selection"); } }
    public override string? icon_name { get { return "zoom-original-symbolic"; } }
    public override string? category { get { return "View"; } }
    public override string? default_shortcut { get { return "<primary><shift>w"; } }

    public override bool can_execute () {
        return true;
    }
}

```

```

        public override void execute () {
            var app = GLib.Application.get_default () as Iide.Application;
            var win = app ? .active_window as Iide.Window;
            if (win != null) {
                win.get_active_source_view () ? .shrink_selection ();
            }
        }
    }

private class Iide.QuitAction : Iide.AppAction {
    public override string id { get { return "quit"; } }
    public override string name { get { return _("Quit"); } }
    public override string? description { get { return _("Quit the application"); } }
    public override string? icon_name { get { return "application-exit-symbolic"; } }
    public override string? category { get { return "Application"; } }
    public override string? default_shortcut { get { return "<primary>q"; } }

    public override bool can_execute () {
        return true;
    }

    public override void execute () {
        var app = GLib.Application.get_default () as Iide.Application;
        app?.quit ();
    }
}

private class Iide.FuzzyFinderAction : Iide.AppAction {
    private weak Iide.Application app;

    public FuzzyFinderAction (Iide.Application app) {
        this.app = app;
    }

    public override string id { get { return "fuzzy-finder"; } }
    public override string name { get { return _("Quick Open"); } }
    public override string? description { get { return _("Open a file quickly by name"); } }
    public override string? icon_name { get { return "system-search-symbolic"; } }
    public override string? category { get { return "File"; } }
    public override string? default_shortcut { get { return "<primary>p"; } }

    public override bool can_execute () {
        return app ? .active_window is Iide.Window;
    }

    public override void execute () {

```

```

        var win = app ? .active_window as Iide.Window;
        if (win != null) {
            var dialog = new Iide.SearchWindow (win, win.get_document_manager ());
            dialog.set_active_page ("files");
            dialog.present ();
        }
    }
}

private class Iide.SearchSymbolAction : Iide.AppAction {
    private weak Iide.Application app;

    public SearchSymbolAction (Iide.Application app) {
        this.app = app;
    }

    public override string id { get { return "search-symbol"; } }
    public override string name { get { return _("Search Symbol"); } }
    public override string? description { get { return _("Search Symbol in Project"); } }
    public override string? icon_name { get { return "system-search-symbolic"; } }
    public override string? category { get { return "Edit"; } }
    public override string? default_shortcut { get { return "<primary>t"; } }

    public override bool can_execute () {
        return app ? .active_window is Iide.Window;
    }

    public override void execute () {
        var win = app ? .active_window as Iide.Window;
        if (win != null) {
            var dialog = new Iide.SearchWindow (win, win.get_document_manager ());
            dialog.set_active_page ("symbols");
            dialog.present ();
        }
    }
}

private class Iide.SearchInFilesAction : Iide.AppAction {
    private weak Iide.Application app;

    public SearchInFilesAction (Iide.Application app) {
        this.app = app;
    }

    public override string id { get { return "search-in-files"; } }
    public override string name { get { return _("Search in Files"); } }

```



```

        public override string? description { get { return _("Search for text in all project files"); } }
        public override string? icon_name { get { return "edit-find-symbolic"; } }
        public override string? category { get { return "Edit"; } }
        public override string? default_shortcut { get { return "<primary><shift>f"; } }

        public override bool can_execute () {
            return app ? .active_window is Iide.Window;
        }

        public override void execute () {
            var win = app ? .active_window as Iide.Window;
            if (win != null) {
                var dialog = new Iide.SearchWindow (win, win.get_document_manager ());
                dialog.set_active_page ("text");
                dialog.present ();
            }
        }
    }

    private class Iide.NavigationBackAction : Iide.AppAction {
        private weak Iide.Application app;

        public NavigationBackAction (Iide.Application app) {
            this.app = app;
        }

        public override string id { get { return "navigation-back"; } }
        public override string name { get { return _("Navigation Back"); } }
        public override string? description { get { return _("Navigation Back"); } }
        public override string? icon_name { get { return "go-previous-symbolic"; } }
        public override string? category { get { return "Application"; } }
        public override string? default_shortcut { get { return "<Alt>Left"; } }

        public override bool can_execute () {
            return app ? .active_window is Iide.Window;
        }

        public override void execute () {
            var win = app ? .active_window as Iide.Window;
            if (win != null) {
                Iide.NavigationHistoryService.get_instance ().navigate_back ();
            }
        }
    }

    private class Iide.NavigationForwardAction : Iide.AppAction {

```

```

private weak Iide.Application app;

public NavigationForwardAction (Iide.Application app) {
    this.app = app;
}

public override string id { get { return "navigation-forward"; } }
public override string name { get { return _("Navigation Forward"); } }
public override string? description { get { return _("Navigation Forward"); } }
public override string? icon_name { get { return "go-next-symbolic"; } }
public override string? category { get { return "Application"; } }
public override string? default_shortcut { get { return "<Alt>Right"; } }

public override bool can_execute () {
    return app ? .active_window is Iide.Window;
}

public override void execute () {
    var win = app ? .active_window as Iide.Window;
    if (win != null) {
        Iide.NavigationHistoryService.get_instance ().navigate_forward ();
    }
}
}

private class Iide.ShowLineNumbersAction : Iide.AppAction {
    private weak Iide.Application app;

    public ShowLineNumbersAction (Iide.Application app) {
        this.app = app;
        this.state = Iide.SettingsService.get_instance ().show_minimap;
    }

    public override string id { get { return "show-line-numbers"; } }
    public override string name { get { return _("Show Line Numbers"); } }
    public override string? description { get { return _("Show or hide line numbers gutter"); } }
    public override string? icon_name { get { return "view-fullscreen-symbolic"; } }
    public override string? category { get { return "View"; } }
    public override bool is_toggle { get { return true; } }
    public override string? default_shortcut { get { return "<primary><Alt>n"; } }

    public override bool can_execute () {
        return true;
    }

    public override void execute () {

```

```

        var settings = Iide.SettingsService.get_instance ();
        state = !state;
        settings.show_line_numbers = state;

        state_changed (state);
        // TODO: handle changing state from preferences dialog...
    }
}

private class Iide.ShowDiagnosticsMarksAction : Iide.AppAction {
    private weak Iide.Application app;

    public ShowDiagnosticsMarksAction (Iide.Application app) {
        this.app = app;
        this.state = Iide.SettingsService.get_instance ().show_minimap;
    }

    public override string id { get { return "show-diagnostics-marks"; } }
    public override string name { get { return _("Show Diagnostics"); } }
    public override string? description { get { return _("Show or hide diagnostics marks gut"); } }
    public override string? icon_name { get { return "view-fullscreen-symbolic"; } }
    public override string? category { get { return "View"; } }
    public override bool is_toggle { get { return true; } }
    public override string? default_shortcut { get { return "<primary><Alt>m"; } }

    public override bool can_execute () {
        return true;
    }

    public override void execute () {
        var settings = Iide.SettingsService.get_instance ();
        state = !state;
        settings.show_diagnostics_marks = state;

        state_changed (state);
        // TODO: handle changing state from preferences dialog...
    }
}

private class Iide.ShowFoldingAction : Iide.AppAction {
    private weak Iide.Application app;

    public ShowFoldingAction (Iide.Application app) {
        this.app = app;
        this.state = Iide.SettingsService.get_instance ().show_minimap;
    }
}

```

```

public override string id { get { return "show-folding-gutter"; } }
public override string name { get { return _("Show Folding Gutter"); } }
public override string? description { get { return _("Show or hide folding gutter"); } }
public override string? icon_name { get { return "view-fullscreen-symbolic"; } }
public override string? category { get { return "View"; } }
public override bool is_toggle { get { return true; } }
public override string? default_shortcut { get { return "<primary><Alt>f"; } }

public override bool can_execute () {
    return true;
}

public override void execute () {
    var settings = Iide.SettingsService.get_instance ();
    state = !state;
    settings.show_folding_gutter = state;

    state_changed (state);
    // TODO: handle changing state from preferences dialog...
}
}

```

File: src/Widgets/TextView/SourceView.vala

```

public class Iide.WordRange {
    public int line;
    public int start_column;
    public int end_column;

    public WordRange (int line, int start_column, int end_column) {
        this.line = line;
        this.start_column = start_column;
        this.end_column = end_column;
    }

    public bool is_equal (WordRange? other) {
        if (other == null) {
            return false;
        }
        return line == other.line && start_column == other.start_column && end_column == other.end_column;
    }
}

public class Iide.LspTooltipWidget : Gtk.Box {

```

```

private Gtk.Label label;
private Gtk.Spinner spinner;

public LspTooltipWidget () {
    Object (
        orientation: Gtk.Orientation.HORIZONTAL,
        spacing: 6,
        margin_top: 8,
        margin_bottom: 8,
        margin_start: 8,
        margin_end: 8
    );

    spinner = new Gtk.Spinner ();
    label = new Gtk.Label ("    ...");
    label.use_markup = true;
    label.wrap = true;
    label.max_width_chars = 60;

    append (spinner);
    append (label);
    spinner.start ();
}

public void update_text (string? text, bool show_spinner) {
    if (show_spinner) {
        spinner.start ();
        spinner.show ();
    } else {
        spinner.stop ();
        spinner.hide ();
    }

    if (text != null && text != "") {
        label.set_markup (text);
    } else {
        label.set_text ("    ");
    }
}

}

public class Iide.SourceView : GtkSource.View {
    public Window window;
    public string uri { get; private set; }
    private Iide.TreeSitterManager ts_manager;
    public Iide.LineNumbersGutter line_numbers_gutter;

```

```

public GutterMarkRenderer mark_renderer;
public TreeSitterFoldingGutter folding_gutter;
private Gtk.TextIter? pending_scroll_iter = null;

private WordRange? last_hover_range = null;
private LspTooltipWidget tooltip_widget;
private string tooltip_separator = " ";

private LspDocumentClient lsp_document_client;
public SourceDocument document;

private int last_line = -1;
private NavigationHistoryService history;

public signal void breadcrumbs_changed (Gee.List<SourceNodeItem?> crumbs);

public SourceView (Window window, string uri, GtkSource.Buffer buffer) {
    Object (buffer : buffer);
    this.window = window;
    this.uri = uri;
    this.ts_manager = new TreeSitterManager ();
    this.history = NavigationHistoryService.get_instance ();

    this.tooltip_widget = new LspTooltipWidget ();

    // LSP-complete
    var completion = get_completion ();
    completion.select_on_show = true;
    completion.show_icons = true;
    completion.page_size = 18;
    completion.remember_info_visibility = false;
    var provider = new LspCompletionProvider (this);
    completion.add_provider (provider);

    // Build extra menu for context menu
    var zoom_section = new GLib.Menu ();
    zoom_section.append (_("Zoom In"), "app.zoom-in");
    zoom_section.append (_("Zoom Out"), "app.zoom-out");
    zoom_section.append (_("Reset Zoom"), "app.zoom-reset");

    var view_section = new GLib.Menu ();
    view_section.append (_("Minimap"), "app.show-minimap");
    view_section.append (_("Line Numbers"), "app.show-line-numbers");
    view_section.append (_("Diagnostics"), "app.show-diagnostics-marks");
    view_section.append (_("Folding"), "app.show-folding-gutter");

```

```

var extra_menu = new GLib.Menu ();
extra_menu.append_section (null, zoom_section);
extra_menu.append_section (null, view_section);
this.extra_menu = extra_menu;

var settings = Iide.SettingsService.get_instance ();

highlight_current_line = settings.highlight_current_line;
auto_indent = settings.auto_indent;
indent_on_tab = true;

// Marks gutter
set_show_line_marks (false);
var left_gutter = get_gutter (Gtk.TextWindowType.LEFT);
left_gutter.visible = true;

show_line_numbers = false;
this.line_numbers_gutter = new Iide.LineNumbersGutter ();
left_gutter.insert (this.line_numbers_gutter, 0); // 0 -

line_numbers_gutter.visible = settings.show_line_numbers;

mark_renderer = new GutterMarkRenderer ();
mark_renderer.set_icons_size (FontSizeHelper.get_size_for_zoom_level (settings.editor_zoom_level));
left_gutter.insert (mark_renderer, 10);

this.folding_gutter = new TreeSitterFoldingGutter ();
this.folding_gutter.set_icons_size (FontSizeHelper.get_size_for_zoom_level (settings.editor_zoom_level));
left_gutter.insert (this.folding_gutter, 20);

LspDiagnosticsMark.set_mark_attributes (this);

// Connect to settings changes to apply to all open documents
settings.editor_setting_changed.connect ((key) => {
    switch (key) {
        case "show-line-numbers" :
            line_numbers_gutter.visible = settings.show_line_numbers;
            break;
        case "show-diagnostics-marks":
            mark_renderer.visible = settings.show_diagnostics_marks;
            break;
        case "show-folding-gutter":
            folding_gutter.visible = settings.show_folding_gutter;
            break;
        case "highlight-current-line" :
            highlight_current_line = settings.highlight_current_line;

```

```

        break;
    case "auto-indent":
        auto_indent = settings.auto_indent;
        break;
    }
});

// SpaceDrawer
var space_drawer = get_space_drawer ();

// 2.
space_drawer.set_enable_matrix (true);
space_drawer.set_types_for_locations (
    GtkSource.SpaceLocationFlags.ALL,
    GtkSource.SpaceTypeFlags.NONE
);
space_drawer.set_types_for_locations (
    GtkSource.SpaceLocationFlags.LEADING | GtkSource.SpaceLocationFlags.TRAILING,
    GtkSource.SpaceTypeFlags.SPACE | GtkSource.SpaceTypeFlags.TAB
);

// LSP-tooltips
has_tooltip = true;
query_tooltip.connect (on_query_tooltip);

// Control-click controller (goto definition)
var click_gest = new Gtk.GestureClick ();
click_gest.set_button (1);
click_gest.released.connect (on_click_released);
add_controller (click_gest);

buffer.set_modified (false);

// 1.
var focus_controller = new Gtk.EventControllerFocus ();
focus_controller.enter.connect (() => {
    //
    handle_navigation_trigger (false);
});
add_controller (focus_controller);

// 2.
this.buffer.notify["cursor-position"].connect (() => {
    handle_navigation_trigger (true);
});

```



```

        create_document ();
    }

    private void create_document() {
        detect_language ();
        var ts_highlighter = ts_manager.get_ts_highlighter (this);
        if (ts_highlighter != null) {
            this.document = new TreeSitterDocument(this, ts_highlighter);
            ts_highlighter.folding_structure_updated.connect ((blocks) => {
                // Gutter renderer
                this.folding_gutter.update_blocks_data (blocks);
            });
            this.folding_gutter.update_blocks_data (ts_highlighter.get_cached_indent_blocks);
        } else {
            this.document = new SourceDocument (this);
            this.folding_gutter.visible = false;
            this.mark_renderer.visible = false;
        }

        this.lsp_document_client = new LspDocumentClient (this);
        this.document.document_changed.connect(this.lsp_document_client.add_change);
        this.document.breadcrumbs_changed.connect ((crumbs) => {
            this.breadcrumbs_changed(crumbs);
        });
    }

    public void expand_selection() {
        this.document.expand_selection ();
    }

    public void shrink_selection() {
        this.document.shrink_selection ();
    }

    public Gee.List<SourceNodeItem?> get_full_outline () {
        return this.document.get_full_outline ();
    }

    public async void lsp_sync_changes_async () {
        yield this.lsp_document_client.sync_changes_async ();
    }

    private void handle_navigation_trigger (bool check_distance) {
        Gtk.TextIter iter;
        this.buffer.get_iter_at_mark (out iter, this.buffer.get_insert ());
        int current_line = iter.get_line ();
    }

```

```

        if (!check_distance || (last_line == -1 || (current_line - last_line).abs () > 10))
            save_current_point (iter);
    }

    last_line = current_line;
}

private void save_current_point (Gtk.TextIter iter) {
    var file = File.new_for_uri (uri);
    if (file != null) {
        history.push_point (file, iter.get_line (), iter.get_line_offset ());
    }
}

//
public void bind_lsp_client (LspClient? client) {
    this.lsp_document_client.bind_lsp_client (client);
}

public GtkSource.Language? language {
    set {
        ((GtkSource.Buffer) buffer).language = value;
    }
    get {
        return ((GtkSource.Buffer) buffer).language;
    }
}

public void detect_language () {
    var manager = GtkSource.LanguageManager.get_default ();
    var file = GLib.File.new_for_uri (uri);

    string mime_type = mime_type_for_file (file);
    message ("MIME: _ " + mime_type);

    language = manager.guess_language (file.get_path (), mime_type);

    // Fake file type detection
    // "Not all files are equal"
    if (file.get_basename () == "CMakeLists.txt") {
        language = manager.get_language ("cmake");
    }
}

public WordRange ? get_word_range_under_cursor (Gtk.TextIter cursor_iter) {

```

```

// 2.      :
unichar c = cursor_iter.get_char ();
if (c.isspace () || c == '\\0') {
    return null;
}

// 3.
Gtk.TextIter end_iter = cursor_iter;

// 4.
//      , backward_word_start      false
//      ,
if (!cursor_iter.starts_word ()) {
    cursor_iter.backward_word_start ();
}

// 5.
if (!end_iter.ends_word ()) {
    end_iter.forward_word_end ();
}

return new WordRange (cursor_iter.get_line (), cursor_iter.get_line_offset (), end_i
}

private bool on_lsp_diagnostics_tooltip (GLib.SList<weak GtkSource.Mark> marks, Gtk.Tool
    var sb = new StringBuilder ();

    foreach (var mark in marks) {
        var lsp_mark = mark as LspDiagnosticsMark;
        if (lsp_mark == null) {
            continue;
        }

        string icon;
        string header_color;

        switch (lsp_mark.severity) {
            case 1 :
                icon = " ";
                header_color = "#F44336"; //
                break;
            case 2 :
                icon = " ";
                header_color = "#FF9800"; //
                break;
            case 3:

```

```

        case 4:
            icon = " ";
            header_color = "#2196F3"; //
            break;
        default:
            icon = " ";
            header_color = "#F44336"; //
            break;
    }

    if (sb.len > 0) {
        sb.append ("\n" + tooltip_separator + "\n");
    }

    //
    sb.append_printf ("%s <span font_weight='bold' foreground='%s'>%s</span>\n",
                      icon, header_color, lsp_mark.category.up ());
    sb.append_printf ("<span>%s</span>", GLib.Markup.escape_text (lsp_mark.diagnostics));
}

if (sb.len > 0) {
    tooltip_widget.update_text (sb.str, false);
    tooltip.set_custom (tooltip_widget);
    return true;
}

return false;
}

private bool on_lsp_hover_tooltip (Gtk.TextIter iter, Gtk.Tooltip tooltip) {
    WordRange? word_range = get_word_range_under_cursor (iter);
    if (word_range == null) {
        last_hover_range = null;
        return false;
    }

    tooltip.set_custom (tooltip_widget);

    //      ,      100
    if (word_range.is_equal (last_hover_range)) {
        return true;
    }
    tooltip_widget.update_text ("Loading...", true);
    last_hover_range = word_range;

    this.lsp_document_client.flush_changes ();
}

```

```

        fetch_lsp_hover_async.begin (word_range.line, word_range.start_column + 1);

        return true;
    }

private bool on_query_tooltip (int x, int y, bool keyboard_mode, Gtk.Tooltip tooltip) {
    Gtk.TextIter iter;

    //
    int buffer_x, buffer_y;
    window_to_buffer_coords (Gtk.TextWindowType.WIDGET, x, y, out buffer_x, out buffer_y);

    //
    if (!get_iter_at_location (out iter, buffer_x, buffer_y)) {
        return false;
    }

    //
    ( "error")
    var buffer = (GtkSource.Buffer) buffer;
    var marks = buffer.get_source_marks_at_line (iter.get_line (), null);

    if (marks.length () > 0) {
        return on_lsp_diagnostics_tooltip (marks, tooltip);
    }

    return on_lsp_hover_tooltip (iter, tooltip);
}

private async void fetch_lsp_hover_async (int line, int col) {
    var lsp_service = IdeLspService.get_instance ();
    string? markdown = yield lsp_service.request_hover (uri, line, col);

    tooltip_widget.update_text (escape_pango (markdown), false);
}

private void on_click_released (Gtk.GestureClick gesture, int n_press, double x, double y) {
    //
    var modifiers = gesture.get_current_event_state ();

    if ((modifiers & Gdk.ModifierType.CONTROL_MASK) != 0) {
        Gtk.TextIter iter;
        // GTK4
        //
        (
        int buf_x, buf_y;
        window_to_buffer_coords (Gtk.TextWindowType.WIDGET, (int) x, (int) y, out buf_x, out buf_y);
    }
}

```

```

        if (get_iter_at_location (out iter, buf_x, buf_y)) {
            get_buffer ().place_cursor (iter);

            //
            this.lsp_document_client.flush_changes ();
            handle_ctrl_click_async.begin (iter.get_line (), iter.get_line_offset ());
        }
    }

private async void handle_ctrl_click_async (int line, int col) {
    var lsp_service = IdeLspService.get_instance ();
    var locations = yield lsp_service.goto_definition (uri, line, col);

    if (locations == null || locations.size == 0) {
        LoggerService.get_instance ().warning ("LSP", "No locations found for goto definition");
        return;
    }

    var loc = locations.get (0);
    if (this.uri == loc.uri) {
        this.select_and_scroll (loc.start_line, loc.start_column, loc.end_column, false);
    } else {
        window.get_document_manager ().open_document_with_selection
            (File.new_for_uri (loc.uri), loc.start_line, loc.start_column, loc.end_column);
    }
}

public void select_and_scroll (int line, int start_col, int end_col, bool is_new) {
    if (line >= buffer.get_line_count ()) {
        return;
    }

    Gtk.TextIter start_iter;
    buffer.get_iter_at_line_offset (out start_iter, line, start_col);

    Gtk.TextIter end_iter;
    buffer.get_iter_at_line_offset (out end_iter, line, end_col);

    buffer.place_cursor (start_iter);
    buffer.select_range (start_iter, end_iter);
    scroll_to_iter (start_iter, 0.0, true, 0.5, 0.5);
    pending_scroll_iter = null;
    if (is_new) {
        pending_scroll_iter = start_iter;
    }
}

```

```

    }

    public override void size_allocate (int width, int height, int baseline) {
        base.size_allocate (width, height, baseline);
        if (pending_scroll_iter != null) {
            scroll_to_iter (pending_scroll_iter, 0.0, true, 0.5, 0.5);
            pending_scroll_iter = null;
        }
    }

    public void goto (int line, int column, bool save_to_navigation_history = true) {
        Gtk.TextIter iter;
        buffer.get_iter_at_line (out iter, line);
        iter.set_line_index (column);

        buffer.place_cursor (iter);
        scroll_to_iter (iter, 0.1, false, 0, 0.5);
        grab_focus ();
    }
}

```

File: src/meson.build

```

iide_sources = [
    'main.vala',
    'application.vala',
    'window.vala',
    'Widgets/TextView/TextView.vala',
    'Widgets/TextView/SourceView.vala',
    'Widgets/TextView/FontZoomer.vala',
    'Widgets/TextView/GutterMarkRenderer.vala',
    'Widgets/TextView/LineNumbersGutter.vala',
    'Widgets/TextView/BreadcrumbFileNavigator.vala',
    'Widgets/TextView/BreadcrumbTreeSitterNavigator.vala',
    'Widgets/TextView/BreadcrumbSymbolOutlineNavigator.vala',
    'Widgets/TextView/BreadcrumbsBar.vala',
    'Widgets/TextView/EditorStatusBar.vala',
    'Widgets/TextView/LspCompletionProvider.vala',
    'Widgets/TextView/TreeSitterFoldingGutter.vala',
    'Widgets/TextView/DiagnosticsPopover.vala',
    'Widgets/ToolViews/ProjectView.vala',
    'Widgets/ToolViews/TerminalView.vala',
    'Widgets/ToolViews/LogView.vala',
    'Widgets/Find/SearchResult.vala',
    'Widgets/Find/SearchResultsView.vala',

```

'Widgets/Find/SearchEngine.vala',
 'Widgets/Find/SearchPage.vala',
 'Widgets/Find/Engines/SymbolsSearchEngine.vala',
 'Widgets/Find/Engines/FzfSearchEngine.vala',
 'Widgets/Find/Engines/TextSearchEngine.vala',
 'Widgets/Find/SearchWindow.vala',
 'Widgets/PreferencesDialog.vala',
 'Widgets/Panels/BasePanel.vala',
 'Widgets/Panels/LogPanel.vala',
 'Widgets/Panels/TerminalPanel.vala',
 'Widgets/Panels/DiagnosticsPanel.vala',
 'Widgets/Panels/ProjectPanel.vala',
 'Services/Utils.vala',
 'Services/JsonUtils.vala',
 'Services/StyleManager.vala',
 'Services/LoggerService.vala',
 'Services/DocumentManager.vala',
 'Services/PendingChange.vala',
 'Services/LSP/LspDocumentClient.vala',
 'Services/SourceNodeItem.vala',
 'Services/TreeSitter/IndentBlock.vala',
 'Services/TreeSitter/TreeSitterDocument.vala',
 'Services/SourceDocument.vala',
 'Services/ImageFactory.vala',
 'Services/SymbIconProvider.vala',
 'Services/ProjectManager.vala',
 'Services/SettingsService.vala',
 'Services/PanelLayoutHelper.vala',
 'Services/NavigationHistoryService.vala',
 'Services/LSP/LspTypes.vala',
 'Services/LSP/config/LspConfig.vala',
 'Services/LSP/config/PythonLspConfig.vala',
 'Services/LSP/config/CppLspConfig.vala',
 'Services/LSP/config/LspRegistry.vala',
 'Services/LSP/index/ClangTidy.vala',
 'Services/LSP/LspClient.vala',
 'Services/LSP/LspService.vala',
 'Services/LSP/LspDiagnosticsMark.vala',
 'Services/LSP/DiagnosticsService.vala',
 'Services/TreeSitter/TreeSitterManager.vala',
 'Services/TreeSitter/model/AstItem.vala',
 'Services/TreeSitter/model/AstModel.vala',
 'Services/TreeSitter/BaseTreeSitterHighlighter.vala',
 'Services/TreeSitter/cpp/CppHighlighter.vala',
 'Services/TreeSitter/vala/ValaHighlighter.vala',
 'Services/TreeSitter/rust/RustHighlighter.vala',


```

        'Services/TreeSitter/python/PythonIndenter.vala',
        'Services/TreeSitter/python/PythonHighlighter.vala',
        'Services/Actions/AppAction.vala',
        'Services/Actions/AppActionsManager.vala',
    ]

    iide_deps = [
        config_dep,
        dependency('gtk4'),
        dependency('gio-2.0', version: '>= 2.50'),
        dependency('libadwaita-1', version: '>= 1.4'),
        dependency('libpanel-1', version: '>= 1.7'),
        dependency('gtksourceview-5'),
        dependency('vte-2.91-gtk4', version: '>= 0.75.0'),
        dependency('gee-0.8'),
        dependency('json-glib-1.0'),
        dependency('jsonrpc-glib-1.0'),
    ]

    iide_sources += gnome.compile_resources('iide-resources', 'iide.gresource.xml', c_name: 'iide')

    executable(
        'iide',
        iide_sources + [symbols_res],
        dependencies: [iide_deps, tree_sitter_dep],
        include_directories: config_inc,
        install: true,
    )

```

File: src/Services/TreeSitter/BaseTreeSitterHighlighter.vala

```

using TreeSitter;

public abstract class Iide.BaseTreeSitterHighlighter : Object {
    //      Tree-sitter
    protected Parser parser;
    protected TreeSitter.Tree? tree;
    protected Query? query;
    protected QueryCursor cursor;

    // GTK
    protected GtkSource.Buffer buffer;
    protected SourceView view;

    // private TreeSitter.Input ts_input;

```

```

//          : [capture_index, theme_index]
// theme_index: 1 - Light, 0 - Dark
private Gtk.TextTag ? [, ] capture_tags;

//          (          DocumentManager)
private int current_theme_index = 0;

//          Debounce
private uint highlight_timeout_id = 0;
private const uint DEBOUNCE_MS = 150;
private Gee.ArrayList<TreeSitter.Range?> pending_ui_ranges;
private ulong scroll_signal_id = 0;

//          Range      Tree-sitter
private Gee.ArrayQueue<TreeSitter.Range?> selection_stack = new Gee.ArrayQueue<TreeSitter.Range?> ();
private bool is_internal_selection_change = false;

private Gee.List<SourceNodeItem?> last_crumbs = null;
public signal void breadcrumbs_changed (Gee.List<SourceNodeItem?> crumbs);

//          ,          UI-          ,
public signal void folding_structure_updated (Gee.List<IndentBlock?> blocks);

//
protected Gee.List<IndentBlock?> cached_blocks;

private void set_color_theme () {
    var color_scheme = SettingsService.get_instance ().color_scheme;
    current_theme_index = color_scheme != ColorScheme.LIGHT ? 1 : 0;
    if (highlight_timeout_id > 0) {
        Source.remove (highlight_timeout_id);
    }
    initial_rehighlight ();
}

/* TODO:          ...          ...
private static string ? ts_read_callback (void* payload, uint32 byte_index, Point position) {
    var self = (BaseTreeSitterHighlighter) payload;
    var buffer = self.buffer;
    bytes_read = 0;

    Gtk.TextIter iter;
    buffer.get_start_iter (out iter);

    // 1.

```

```

//          get_iter_at_line          Point,
//          /          GTK,          .
//          ,          get_bytes_in_line
uint32 accumulated_bytes = 0;
while (accumulated_bytes + (uint32) iter.get_bytes_in_line () <= byte_index) {
    accumulated_bytes += (uint32) iter.get_bytes_in_line ();
    if (!iter.forward_line ()) {
        break; //
    }
}

//
// (
while (accumulated_bytes < byte_index) {
    int current_char_bytes = iter.get_bytes_in_line () - iter.get_line_index ();
    //
    if (current_char_bytes <= 0) break;

    //          (          !)
    if (!iter.forward_char ()) break;

    //          get_byte_offset_safe
    accumulated_bytes = get_byte_offset_safe (iter);
}

if (iter.is_end ()) return null;

// 2.          (          )
//          -          (          \n)
Gtk.TextIter end = iter;
int total_line_bytes = end.get_bytes_in_line ();
end.set_line_index (total_line_bytes); //

//          .          get_text (          get_slice)
//          ,          $FOLD_HIDE!
string chunk = buffer.get_text (iter, end, false);
bytes_read = (uint32) chunk.length; // Vala          UTF-8

return bytes_read > 0 ? chunk : null;
} */

protected BaseTreeSitterHighlighter (SourceView view) {
    view.set_insert_spaces_instead_of_tabs (true);
    view.set_tab_width (4);
    view.set_smart_backspace (true);

```

```

this.view = view;
this.buffer = (GtkSource.Buffer) view.get_buffer ();

//
// this.ts_input = {};
// this.ts_input.payload = (void*) this;
// this.ts_input.read = ts_read_callback;
// this.ts_input.encoding = TreeSitter.InputEncoding.UTF8;

this.parser = new Parser ();
this.cursor = new QueryCursor ();

this.cached_blocks = new Gee.ArrayList<IndentBlock?> ();
this.pending_ui_ranges = new Gee.ArrayList<TreeSitter.Range?> ();

//
unowned Language lang = get_ts_language ();
this.parser.set_language (lang);
//      Query
load_query (lang);
//                                Query
prepare_capture_mapping ();

//
set_color_theme ();
this.buffer.notify["style-scheme"].connect_after (set_color_theme);

buffer.notify["cursor-position"].connect (() => {
//                                expand/shrink -

    if (!is_internal_selection_change) {
        selection_stack.clear ();
    }
});

buffer.notify["cursor-position"].connect (update_breadcrumbs);

this.buffer.changed.connect (this.on_buffer_changed);
this.bind_highlighter_signals ();
}

private void bind_highlighter_signals () {
//      adjustment
var vadjustment = this.view.get_vadjustment ();

//                                ,

```

```

        if (this.scroll_signal_id != 0 && vadjustment != null) {
            vadjustment.disconnect (this.scroll_signal_id);
        }

        //
        this.scroll_signal_id = vadjustment.value_changed.connect (() => {
            //
            this.render_viewport_only ();
        });
    }

    public Gee.List<IndentBlock?> get_cached_indent_blocks () {
        return this.cached_blocks;
    }

    //                                     (Vala, Cpp  . .)
    protected abstract unowned Language get_ts_language ();
    protected abstract string query_source ();

    //                                     Breadcrumbs
    protected abstract bool is_container_node (string node_type);

    //
    protected void update_document_structure () {
        this.cached_blocks.clear ();

        //
        var root = this.tree.root_node ();
        if (root.is_null ()) return;

        //                                     0
        this.collect_foldable_blocks (root, 0);

        //                                     Gutter  Overlay
        this.folding_structure_updated (this.cached_blocks);
    }

    private void collect_foldable_blocks (TreeSitter.Node parent, int current_indent) {
        //
        for (uint32 i = 0; i < parent.named_child_count (); i++) {
            var child = parent.named_child (i);

            if (child.is_null ()) continue;

            string type = child.type ();
            bool is_foldable = false;

```

```

//
if (this.is_foldable_node_type (type)) {
    var start_point = this.body_start_point (child);
    var end_point = child.end_point ();

    //
    if (end_point.row > start_point.row) {
        var block = IndentBlock () {
            start_line = (int) start_point.row,
            end_line = (int) end_point.row,
            indent_level = current_indent
        };

        //
        this.cached_blocks.add (block);
        is_foldable = true;
    }
}

//
//
//
int next_indent = is_foldable ? current_indent + 1 : current_indent;

//
if (child.named_child_count () > 0) {
    this.collect_foldable_blocks (child, next_indent);
}
}

//
protected virtual bool is_foldable_node_type (string type) {
    //
    return type == "compound_statement" ||
           type == "function_definition" ||
           type == "class_definition" ||
           type == "block";
}

protected virtual TreeSitter.Point body_start_point(TreeSitter.Node foldable_node) {
    return foldable_node.start_point ();
}

//

```

```

public virtual GtkSource.Indenter? create_indenter() {
    return null;
}

public unowned TreeSitter.Tree? get_tree () {
    return tree;
}

private void load_query (Language lang) {
    string source = query_source ();

    uint32 error_offset;
    QueryError error_type;
    this.query = new Query (lang, source, (uint32) source.length, out error_offset, out

    if (error_type != QueryError.None) {
        warning ("TreeSitter Query Error at %u: %s", error_offset, error_type.to_string
    }
}

private void prepare_capture_mapping () {
    if (query == null) return;

    uint32 count = query.capture_count ();
    capture_tags = new Gtk.TextTag ? [count, 2];
    var style_service = StyleService.get_instance ();

    for (uint32 i = 0; i < count; i++) {
        uint32 name_len;
        string name = query.capture_name_for_id (i, out name_len);

        //
        capture_tags[i, 0] = style_service.get_tag (name, 0);
        capture_tags[i, 1] = style_service.get_tag (name, 1);

        //
        (      , @function.method -> @function)
        if (capture_tags[i, 0] == null && name.contains (".")) {
            string base_name = name.split (".")[0];
            capture_tags[i, 0] = style_service.get_tag (base_name, 0);
            capture_tags[i, 1] = style_service.get_tag (base_name, 1);
        }
    }
}

//
private void update_document_structure_silent () {

```

```

        this.cached_blocks.clear ();
        var root = this.tree.root_node ();
        if (!root.is_null ()) {
            this.collect_foldable_blocks (root, 0);
        }
    }

private void on_buffer_changed () {
    unowned TreeSitter.Tree? old_tree = this.tree;

    // 1. ( )
    Gtk.TextIter start, end;
    buffer.get_bounds (out start, out end);
    var new_tree = this.parser.parse_string (old_tree, buffer.get_text (start, end, true));
    //var new_tree = this.parser.parse (old_tree, this.ts_input);

    if (new_tree == null)
        return;
    this.tree = (owned) new_tree;

    var buffer = this.view.get_buffer ();
    var invisible_tag = buffer.get_tag_table ().lookup ("$FOLD_HIDE");

    //
    var folds_to_unfold = new Gee.ArrayList<FoldedMarkRange> ();

    var folding_gutter = this.view.folding_gutter;

    if (invisible_tag != null && folding_gutter != null) {
        //
        this.update_document_structure_silent ();

        //
        for (int i = folding_gutter.active_folds.size - 1; i >= 0; i--) {
            var fold = folding_gutter.active_folds[i];

            Gtk.TextIter current_start_iter;
            buffer.get_iter_at_mark (out current_start_iter, fold.start_mark);
            int current_header_line = current_start_iter.get_line () - 1;

            bool block_still_exists = false;
            foreach (var block in this.cached_blocks) {
                if (block.start_line == current_header_line) {
                    block_still_exists = true;
                    break;
                }
            }
        }
    }
}

```



```

    }

    //
    if (!block_still_exists) {
        folds_to_unfold.add (fold);
        folding_gutter.active_folds.remove_at (i);
    }
}

// =====
// 2. IDLE ( )
if (folds_to_unfold.size > 0) {
    GLib.Idle.add (() => {
        // 100% !
        buffer.begin_user_action ();

        foreach (var fold in folds_to_unfold) {
            Gtk.TextIter s, e;
            buffer.get_iter_at_mark (out s, fold.start_mark);
            buffer.get_iter_at_mark (out e, fold.end_mark);

            // -
            buffer.remove_tag (invisible_tag, s, e);
            fold.free_marks (buffer);
        }

        buffer.end_user_action ();

        //
        // var final_tree = this.parser.parse (this.tree, this.ts_input);
        Gtk.TextIter start1, end1;
        buffer.get_bounds (out start1, out end1);
        var final_tree = this.parser.parse_string (this.tree, buffer.get_text(start1, end1));
        this.tree = (owned) final_tree;

        //
        this.update_document_structure_silent ();
        this.view.queue_resize ();
        this.view.queue_draw ();

        //
        this.render_viewport_only ();

        return GLib.Source.REMOVE; //
    });
}

```

```

    }
    // =====

    // 3. -
    if (this.highlight_timeout_id > 0) {
        GLib.Source.remove (this.highlight_timeout_id);
    }

    this.highlight_timeout_id = GLib.Timeout.add (DEBOUNCE_MS, () => {
        this.sync_and_render ();
        this.highlight_timeout_id = 0;
        return GLib.Source.REMOVE;
    });
}

public void flush_changes() {
    if (highlight_timeout_id > 0) {
        Source.remove (highlight_timeout_id);
    }
    sync_and_render ();
    highlight_timeout_id = 0;
}

private void initial_rehighlight () {
    Gtk.TextIter start, end;
    buffer.get_bounds (out start, out end);

    //
    clear_highlighter_tags (start, end);
    // buffer.remove_all_tags (start, end);

    // this.tree = parser.parse (null, ts_input);
    this.tree = parser.parse_string (null, buffer.get_text (start, end, true).data);

    update_breadcrumbs ();
    update_document_structure ();

    this.cursor = new QueryCursor ();
    cursor.exec (query, tree.root_node ());
    render_matches ();
}

private void clear_highlighter_tags (Gtk.TextIter start, Gtk.TextIter end) {
    Gtk.TextBuffer buffer = this.view.get_buffer();

    //

```

```

    Gtk.TextIter it = start;

    //      :
    this.remove_syntax_tags_at_iter (buffer, it, start, end);

    //      , (Toggle Points)
    //
    while (it.forward_to_tag_toggle (null) && it.compare (end) < 0) {
        this.remove_syntax_tags_at_iter (buffer, it, start, end);
    }
}

private void remove_syntax_tags_at_iter (Gtk.TextBuffer buffer, Gtk.TextIter it, Gtk.Text
    //      ,
    var active_tags = it.get_tags();

    foreach (var tag in active_tags) {
        //      . Tree-sitter ( , "$"):
        if (tag.name != null && !tag.name.has_prefix ("$")) {

            //      ,
            Gtk.TextIter toggle_start = it;
            Gtk.TextIter toggle_end = it;

            //
            //      , range_start/range_end
            if (!toggle_start.has_tag (tag) || toggle_start.compare (range_start) < 0) {
                toggle_start = range_start;
            }

            //
            toggle_end.forward_to_tag_toggle (tag);
            if (toggle_end.compare (range_end) > 0) {
                toggle_end = range_end;
            }

            //
            buffer.remove_tag (tag, toggle_start, toggle_end);
        }
    }
}

private void sync_and_render () {
    // 1. ( )
    this.update_breadcrumbs ();
    this.update_document_structure ();
}

```

```

// 2.                                (Viewport)
Gdk.Rectangle visible_rect;
this.view.get_visible_rect (out visible_rect);

Gtk.TextIter visible_start_iter;
Gtk.TextIter visible_end_iter;

//
this.view.get_iter_at_location (out visible_start_iter, visible_rect.x, visible_rect.y);
this.view.get_iter_at_location (out visible_end_iter, visible_rect.x + visible_rect.width, visible_rect.y);

//
visible_start_iter.set_line_offset (0);
visible_end_iter.forward_to_line_end ();
if (!visible_end_iter.is_end ()) {
    visible_end_iter.forward_char ();
}

// 3.
//
this.clear_highlighter_tags (visible_start_iter, visible_end_iter);

// 4.                                ( set_byte_range!)
this.cursor = new QueryCursor ();

// set_byte_range. Tree-sitter
//
this.cursor.exec (this.query, this.tree.root_node ());

// 5.
//
//
uint32 viewport_start_byte = (uint32) get_byte_offset_safe (visible_start_iter);
uint32 viewport_end_byte = (uint32) get_byte_offset_safe (visible_end_iter);

this.render_matches_in_viewport (viewport_start_byte, viewport_end_byte);
}

private void render_matches () {
    QueryMatch match;
    while (cursor.next_match (out match)) {
        foreach (var capture in match.captures) {
            uint32 name_len;
            string capture_name = query.capture_name_for_id (capture.index, out name_len);

```

```

        Gtk.TextTag? tag = null;

        if (capture_name == "punctuation.bracket") {
            int lvl = (get_nesting_level (capture.node) % 5) + 1; //      5
            tag = StyleService.get_instance ().get_tag ("bracket.lvl" + lvl.to_string());
        } else {
            tag = capture_tags[capture.index, current_theme_index];
        }
        if (tag != null) {
            apply_tag_fast (capture.node, tag);
        }
    }
}

private void render_matches_in_viewport (uint32 viewport_start_byte, uint32 viewport_end_byte) {
    QueryMatch match;
    while (cursor.next_match (out match)) {
        foreach (var capture in match.captures) {
            // =====
            //                                VIEWPORT:
            //                                (0(1)  Tree-sitter)
            uint32 node_start = capture.node.start_byte ();
            uint32 node_end = capture.node.end_byte ();

            //
            //                                ,
            //                                GTK
            if (node_end < viewport_start_byte || node_start > viewport_end_byte) {
                continue;
            }
            // =====

            uint32 name_len;
            string capture_name = query.capture_name_for_id (capture.index, out name_len);

            Gtk.TextTag? tag = null;

            if (capture_name == "punctuation.bracket") {
                int lvl = (get_nesting_level (capture.node) % 5) + 1; //      5
                tag = StyleService.get_instance ().get_tag ("bracket.lvl" + lvl.to_string());
            } else {
                tag = capture_tags[capture.index, current_theme_index];
            }

            if (tag != null) {
                apply_tag_fast (capture.node, tag);
            }
        }
    }
}

```

```

    }
}

public void render_viewport_only () {
    //
    if (this.tree == null || this.tree.root_node ().is_null ()) return;

    // 1.
    Gdk.Rectangle visible_rect;
    this.view.get_visible_rect (out visible_rect);

    Gtk.TextIter visible_start_iter;
    Gtk.TextIter visible_end_iter;

    this.view.get_iter_at_location (out visible_start_iter, visible_rect.x, visible_rect.y);
    this.view.get_iter_at_location (out visible_end_iter, visible_rect.x + visible_rect.width, visible_rect.y);

    //
    visible_start_iter.set_line_offset (0);
    visible_end_iter.forward_to_line_end ();
    if (!visible_end_iter.is_end ()) {
        visible_end_iter.forward_char ();
    }

    // 2.
    uint32 viewport_start_byte = get_byte_offset_safe (visible_start_iter);
    uint32 viewport_end_byte = get_byte_offset_safe (visible_end_iter);

    // 3.
    this.clear_highlighter_tags (visible_start_iter, visible_end_iter);

    // 4.
    A (
    this.cursor = new QueryCursor ();
    this.cursor.exec (this.query, this.tree.root_node ());

    //
    this.render_matches_in_viewport (viewport_start_byte, viewport_end_byte);
}

private int get_nesting_level (TreeSitter.Node node) {
    int level = 0;
    TreeSitter.Node? parent = node.parent ();
    while (parent != null && !parent.is_null ()) {
        string type = parent.type ();
        //
        ,
        . .
    }
}

```

```

        if (type == "block" || type == "argument_list" || type == "parameters" || type == "statement")
            level++;
    }
    parent = parent.parent ();
}
return level;
}

private void apply_tag_fast (TreeSitter.Node node, Gtk.TextTag tag) {
    Gtk.TextIter start, end;

    //
    get_iters_from_ts_node (buffer, node, out start, out end);

    buffer.apply_tag (tag, start, end);
}

public static void get_iters_from_ts_node (Gtk.TextBuffer buffer, TreeSitter.Node node,
                                           out Gtk.TextIter start_iter, out Gtk.TextIter end_iter) {
    buffer.get_iter_at_line (out start_iter, (int) node.start_point ().row);
    start_iter.set_line_index ((int) node.start_point ().column);

    buffer.get_iter_at_line (out end_iter, (int) node.end_point ().row);
    end_iter.set_line_index ((int) node.end_point ().column);
}

//
private void get_iters_from_ts_node_coords (TreeSitter.Point start_p, TreeSitter.Point end_p,
                                           out Gtk.TextIter s, out Gtk.TextIter e) {
    buffer.get_iter_at_line (out s, (int) start_p.row);
    s.set_line_index ((int) start_p.column);

    buffer.get_iter_at_line (out e, (int) end_p.row);
    e.set_line_index ((int) end_p.column);
}

public bool handle_key_pressed(uint keyval, uint keycode, Gdk.ModifierType modifiers) {
    string opening = "";
    string closing = "";

    uint32 unicode = Gdk.keyval_to_unicode(keyval);

    //
    switch (unicode) {
        case '(': opening = "("; closing = ")"; break;
        case '{': opening = "{"; closing = "}"; break;
    }
}

```

```

        case '\\': opening = "\\"; closing = "\\"; break;
        case '\"': opening = "\""; closing = "\""; break;
        default: return false;
    }

    wrap_selection_or_insert_pair(opening, closing);
    return true;
}

private void wrap_selection_or_insert_pair(string op, string cl) {
    Gtk.TextIter start, end;

    // - Undo
    buffer.begin_user_action();

    if (buffer.get_selection_bounds(out start, out end)) {
        // :

        // 1. ( )
        int orig_start_offset = start.get_offset();
        int orig_end_offset = end.get_offset();

        // 2. (start )
        buffer.insert(ref start, op, -1);

        // 3. , 'end'
        // ( +1)
        buffer.get_iter_at_offset(out end, orig_end_offset + op.char_count());

        // 4.
        buffer.insert(ref end, cl, -1);

        // 5. :
        // 'op'
        // 'op' ( 'cl')
        Gtk.TextIter new_start, new_end;
        buffer.get_iter_at_offset(out new_start, orig_start_offset + op.char_count());
        buffer.get_iter_at_offset(out new_end, orig_end_offset + op.char_count());

        buffer.select_range(new_start, new_end);
    } else {
        // :
        Gtk.TextIter iter;
        buffer.get_iter_at_mark(out iter, buffer.get_insert());
        buffer.insert(ref iter, op + cl, -1);
    }
}

```



```

        //
        iter.backward_char();
        buffer.place_cursor(iter);
    }

    buffer.end_user_action();
}

public void on_insert_text (Gtk.TextIter iter, string text, int len_bytes) {
    uint32 start_byte = get_byte_offset_safe (iter);
    uint32 start_row = (uint32) iter.get_line ();
    uint32 start_col = (uint32) iter.get_line_index (); //

    uint32 lines_added = 0;
    uint32 last_line_bytes = 0;
    int last_nl_idx = -1;

    // UTF-8
    for (int i = 0; i < len_bytes; i++) {
        if (text[i] == '\n') {
            lines_added++;
            last_nl_idx = i;
        }
    }

    if (lines_added > 0) {
        // \n
        last_line_bytes = (uint32) (len_bytes - 1 - last_nl_idx);
    } else {
        // - len_bytes
        last_line_bytes = (uint32) len_bytes;
    }

    InputEdit edit = {};
    edit.start_byte = start_byte;
    edit.start_point = { start_row, start_col };

    //
    edit.old_end_byte = start_byte;
    edit.old_end_point = { start_row, start_col };

    // -
    edit.new_end_byte = start_byte + (uint32) len_bytes;
    edit.new_end_point = Point () {
        row = start_row + lines_added,
        // , 0 + .
    }
}

```

```

        //
        column = (lines_added > 0) ? last_line_bytes : start_col + last_line_bytes
    };

    this.tree.edit (edit);
}

public void on_delete_range (Gtk.TextIter start, Gtk.TextIter end) {
    uint32 start_byte = get_byte_offset_safe (start);
    uint32 old_end_byte = get_byte_offset_safe (end);

    InputEdit edit = {};
    //
    edit.start_byte = start_byte;
    edit.start_point = { (uint32) start.get_line (), (uint32) start.get_line_index () };

    //
    edit.old_end_byte = old_end_byte;
    edit.old_end_point = { (uint32) end.get_line (), (uint32) end.get_line_index () };

    //
    edit.new_end_byte = start_byte;
    edit.new_end_point = edit.start_point;

    this.tree.edit (edit);
}

public void expand_selection () {
    if (tree == null)
        return;

    Gtk.TextIter start_sel, end_sel;
    //
    buffer.get_selection_bounds (out start_sel, out end_sel);

    //
    Gtk.TextIter start_buf;
    buffer.get_start_iter (out start_buf);

    //
    uint32 start_byte = (uint32) buffer.get_slice (start_buf, start_sel, false).length;
    uint32 end_byte = (uint32) buffer.get_slice (start_buf, end_sel, false).length;

    //
    flush_changes ();
    TreeSitter.Node root = tree.root_node ();

```

```

TreeSitter.Node node = root.named_descendant_for_byte_range (start_byte, end_byte);

if (node.is_null ())return;

if (node.start_byte () == start_byte && node.end_byte () == end_byte) {
    var parent = node.parent ();
    if (!parent.is_null ())node = parent;
}

//
TreeSitter.Range current_range = {
    { (uint32) start_sel.get_line (), (uint32) start_sel.get_line_index () },
    { (uint32) end_sel.get_line (), (uint32) end_sel.get_line_index () },
    start_byte,
    end_byte
};

selection_stack.offer_head (current_range);

//
Gtk.TextIter new_start, new_end;
get_iters_from_ts_node (buffer, node, out new_start, out new_end);
is_internal_selection_change = true;
buffer.select_range (new_start, new_end);
is_internal_selection_change = false;
}

public void shrink_selection () {
    if (selection_stack.is_empty)
        return;

    //
    var last_range = selection_stack.poll ();

    if (last_range != null) {
        Gtk.TextIter s_iter, e_iter;

        //
        get_iters_from_ts_node_coords (last_range.start_point, last_range.end_point, out

        //
        is_internal_selection_change = true;
        buffer.select_range (s_iter, e_iter);
        is_internal_selection_change = false;
    }
}

```

```

private Gee.List<SourceNodeItem?> get_siblings_for_node (TreeSitter.Node parent_node) {
    var siblings = new Gee.ArrayList<SourceNodeItem?> ();
    if (parent_node.is_null ())return siblings;

    for (uint32 i = 0; i < parent_node.named_child_count (); i++) {
        var child = parent_node.named_child (i);
        if (is_container_node (child.type ())) {
            var name_node = find_name_node (child);
            if (name_node != null && !name_node.is_null ()) {
                Gtk.TextIter s, e;
                get_iters_from_ts_node (buffer, name_node, out s, out e);
                var start_point = child.start_point ();
                siblings.add (SourceNodeItem () {
                    name = buffer.get_text (s, e, false),
                    type = child.type (),
                    start_point = SourceNodePosition() {
                        row = start_point.row,
                        column = start_point.column
                    },
                });
            }
        }
    }
    return siblings;
}

public Gee.List<SourceNodeItem?> get_breadcrumbs_at_cursor () {
    var result = new Gee.ArrayList<SourceNodeItem?> ();
    if (tree == null)return result;

    Gtk.TextIter insert_iter;
    buffer.get_iter_at_mark (out insert_iter, buffer.get_insert ());

    //
    //
    Gtk.TextIter start_buf;
    buffer.get_start_iter (out start_buf);
    uint32 byte_offset = (uint32) buffer.get_slice (start_buf, insert_iter, false).length;

    //
    TreeSitter.Node node = tree.root_node ().named_descendant_for_byte_range (byte_offset,
    //
    while (!node.is_null ()) {
        //
        if (is_container_node (node.type ())) {

```

```

        TreeSitter.Node? name_node = find_name_node (node);
        if (name_node != null && !name_node.is_null ()) {
            Gtk.TextIter s, e;
            get_iters_from_ts_node (buffer, name_node, out s, out e);
            // ( )
            var siblings = get_siblings_for_node (node.parent ());
            var start_point = node.start_point ();
            result.insert (0, SourceNodeItem () {
                name = buffer.get_text (s, e, false),
                type = node.type (),
                start_point = SourceNodePosition() {
                    row = start_point.row,
                    column = start_point.column
                }, // (fn/class)
                siblings = siblings,
            });
        }
    }
    node = node.parent ();
}
return result;
}

private TreeSitter.Node? find_name_node (TreeSitter.Node node) {
    string f_name = "name";
    TreeSitter.Node name_node = node.child_by_field_name (f_name, (uint32) f_name.length);
    if (!name_node.is_null ())return name_node;

    // :
    for (uint32 i = 0; i < uint32.min (node.child_count (), 5); i++) {
        TreeSitter.Node child = node.child (i);
        if (child.type ().contains ("identifier"))return child;
    }
    return null;
}

private void update_breadcrumbs () {
    var new_crumbs = get_breadcrumbs_at_cursor ();
    if (last_crumbs != new_crumbs) {
        last_crumbs = new_crumbs;
        breadcrumbs_changed (last_crumbs);
    }
}

public Gee.List<SourceNodeItem?> get_full_outline () {
    var root = tree.root_node ();

```

```

        return collect_container_children (root);
    }

private Gee.List<SourceNodeItem?> collect_container_children (TreeSitter.Node parent) {
    var list = new Gee.ArrayList<SourceNodeItem?> ();

    for (uint32 i = 0; i < parent.named_child_count (); i++) {
        var child = parent.named_child (i);

        if (is_container_node (child.type ())) {
            var name_node = find_name_node (child);
            if (name_node != null && !name_node.is_null ()) {
                Gtk.TextIter s, e;
                get_iters_from_ts_node (buffer, name_node, out s, out e);
                var start_point = child.start_point ();
                var item = SourceNodeItem () {
                    name = buffer.get_text (s, e, false),
                    type = child.type (),
                    start_point = SourceNodePosition() {
                        row = start_point.row,
                        column = start_point.column
                    },
                    children = collect_container_children (child),
                };
                list.add (item);
            } else {
                //
                //
                list.add_all (collect_container_children (child));
            }
        } else {
            //
            // :
            //
            // ( , if namespace),
            //
            list.add_all (collect_container_children (child));
        }
    }
    return list;
}

~BaseTreeSitterHighlighter () {
    // free_functions VAPI
    this.parser = null;
    this.tree = null;
    this.query = null;
    this.cursor = null;
}

```

}
}